TYPESCRIPT · FULL-STACK · INTERVIEW PREP

TypeScript

The Complete Guide

Master TypeScript from the ground up — types, generics, classes, React, decorators, advanced patterns, and 100+ interview questions — all in one place.

17	100+	200+	550+
In-depth Chapters	Interview Q&As	Code Examples	Pages

WHAT'S INSIDE

01 Introduction to TypeScript
02 Type System Basics
03 Objects, Arrays & Tuples
04 Functions in TypeScript
05 Interfaces vs Type Aliases
06 Union, Intersection & Literal Types
07 Enums
08 Generics
09 Type Narrowing & Type Guards
10 Classes in TypeScript
11 Utility Types — Deep Dive
12 Advanced Types
13 Modules & Namespaces
14 Decorators
15 React + TypeScript
16 tsconfig.json Deep Dive
17 Interview Q&A — 100+ Questions

BEGINNER	INTERMEDIATE	ADVANCED
Types · Interfaces · Functions	Generics · Classes · Modules	Decorators · React · Interviews

Contents

17 Chapters · Real-World Code · 100+ Interview Questions

CH 01 **Introduction to TypeScript**

- What is TypeScript & why it matters
- TypeScript vs JavaScript — key differences
- Setting up the development environment
- Writing your first .ts file
- Compiling & running TypeScript code

CH 02 **Type System Basics**

- Primitive types — string, number, boolean
- Type inference — let TypeScript figure it out
- any, unknown, never, void — explained clearly
- Type annotations — when & how to use them
- Type aliases — naming your types

CH 03 **Objects, Arrays & Tuples**

- Typing object shapes with interfaces
- Optional (?) and readonly properties
- Index signatures for dynamic keys
- Typed arrays — string[], Array<number>
- Tuples — fixed-length typed arrays

CH 04 **Functions in TypeScript**

- Parameter types & return type annotations
- Optional & default parameters
- Rest parameters & spread types
- Function overloads — multiple signatures
- Typing async functions & Promises

CH 05 **Interfaces vs Type Aliases**

- Interface syntax & object typing
- Type alias syntax — more than just objects

- Extending interfaces vs intersecting types
- Declaration merging — interfaces only
- Choosing between interface and type

CH 06

Union, Intersection & Literal Types

- Union types (A | B) — one of many
- Intersection types (A & B) — combine all
- Literal types — exact string/number values
- Discriminated unions — type-safe variants
- Narrowing unions with type guards

CH 07

Enums

- Numeric enums — auto-incrementing values
- String enums — readable, debuggable
- Const enums — zero runtime cost
- Reverse mapping in numeric enums
- Enum vs union of literals — trade-offs

CH 08

Generics

- Generic functions — reusable typed logic
- Generic constraints with extends keyword
- Generic interfaces and classes
- Multiple type parameters — <T, K, V>
- Introduction to built-in utility types

CH 09

Type Narrowing & Type Guards

- typeof — narrow primitives safely
- instanceof — narrow class instances
- in operator — check property existence
- Custom type guard functions (x is Type)
- Control Flow Analysis (CFA) — how TS tracks types

CH 10

Classes in TypeScript

- Access modifiers — public, private, protected
- Readonly & static class members
- Inheritance, super() & method overriding

- Abstract classes — templates for subclasses
- Implementing interfaces & SOLID principles

CH 11

Utility Types — Deep Dive

- Partial<T>, Required<T>, Readonly<T>
- Pick<T,K>, Omit<T,K>, Record<K,V>
- Exclude<T,U>, Extract<T,U>, NonNullable<T>
- ReturnType<T>, Parameters<T>, Awaited<T>
- Composing utility types in real-world patterns

CH 12

Advanced Types

- Mapped types — transform every property
- Conditional types — if/else at type level
- The infer keyword — capture unknown types
- Template literal types — string manipulation
- Recursive types, Branded types & satisfies operator

CH 13

Modules & Namespaces

- ES Module syntax — named & default exports
- Barrel files (index.ts) — clean imports
- Module resolution — how TypeScript finds files
- Path aliases — @/ instead of ../../..
- Declaration files (.d.ts) & dynamic imports

CH 14

Decorators

- Class decorators — @Injectable, @Controller
- Method decorators — @Log, @Get, @Authorize
- Property decorators — @IsEmail, @Column
- Parameter decorators & Reflect Metadata
- NestJS / TypeORM patterns & Stage 3 decorators

CH 15

React + TypeScript

- Typing component props & children
- useState, useRef, useEffect — typed correctly
- Event handler types — onChange, onClick, etc.
- Generic components & custom typed hooks

- Context API, useReducer & forwardRef

CH 16

tsconfig.json — Deep Dive

- strict mode & all 8 sub-flags explained
- target, module & moduleResolution settings
- Path aliases — baseUrl + paths setup
- declaration, sourceMap, noEmit & more
- Project references, extends & config templates

CH 17

Interview Q&A — Rapid Fire

- 82+ interview questions with full answers
- Difficulty levels — Beginner to Advanced
- Code challenges — spot the error & fix it
- Predict the output — type inference exercises
- Final 30 rapid-fire concept check questions

Beginner → Intermediate → Advanced → Interview Ready

CHAPTER 1

Introduction to TypeScript

What • Why • Setup • TS vs JS

1. What is TypeScript?

TypeScript is a programming language created by Microsoft in 2012. It is a superset of JavaScript, which means any valid JavaScript code is also valid TypeScript code. TypeScript adds extra features on top of JavaScript — the most important one being Static Typing.

Simple Definition

TypeScript = JavaScript + Type System + Extra Features

Think of it like this: JavaScript is a car without a seatbelt. TypeScript is the same car but WITH a seatbelt. The car still works the same way, but now you are safer and can catch mistakes before they cause accidents.

1.1 The Core Idea — Static Typing

In JavaScript, you do not have to tell the language what type of value a variable holds. The type is figured out at runtime (when the code is actually running). This is called Dynamic Typing.

JavaScript Example (Dynamic Typing): — JavaScript does not complain even if you make a mistake.

```
// JavaScript — no type checking
let age = 25;
age = 'twenty five';  // No error! JS just allows this.

function add(a, b) {
  return a + b;
}
add(5, '10');  // Result: '510' (string concat, not addition!)
               // JavaScript does NOT warn you about this mistake.
```

TypeScript Example (Static Typing): — TypeScript catches the mistake BEFORE running.

```
// TypeScript — type checking happens at compile time
```

```
let age: number = 25;
age = 'twenty five';  // ERROR: Type 'string' is not assignable to type
'number'


function add(a: number, b: number): number {
  return a + b;
}
add(5, '10');  // ERROR: Argument of type 'string' is not assignable to
'number'
               // TypeScript catches this BEFORE you even run the code!
```

1.2 TypeScript is a Transpiled Language

Browsers and Node.js do NOT understand TypeScript directly. TypeScript code must be converted (compiled/transpiled) into plain JavaScript first. This conversion is done by the TypeScript Compiler (tsc).

How the Process Works

Step 1: You write TypeScript code (.ts file)

Step 2: TypeScript compiler (tsc) checks your code for errors

Step 3: If no errors, tsc converts it to JavaScript (.js file)

Step 4: JavaScript runs in browser / Node.js as usual

If there ARE errors in Step 2, tsc shows them to you and you fix them.

This is the whole point — catch bugs EARLY, not in production!

2. Why TypeScript? (The Problems it Solves)

To understand why TypeScript exists, we must first understand the problems JavaScript has at scale (i.e., in large projects with many developers).

2.1 Problems with JavaScript in Large Projects

Problem 1 — Runtime Errors are Discovered Too Late

In JavaScript, type mistakes only show up when you actually RUN the code — sometimes in production, affecting real users.

```
// JS — this code looks fine but crashes at runtime
function getUserName(user) {
  return user.profile.name;  // What if user is null? CRASH at runtime!
```

```
}
getUserName(null);  // TypeError: Cannot read property 'profile' of null
```

Problem 2 — No IDE Help / Poor Autocomplete

Because JavaScript has no types, your code editor cannot tell you what properties or methods are available on a variable. You have to remember everything or keep checking documentation.

Problem 3 — Refactoring is Risky

If you rename a function or change its parameters in a large JS codebase, there is no way to be sure you updated every place that calls it. TypeScript will immediately show you every place that breaks.

Problem 4 — Hard to Understand Other People's Code

```
// JavaScript — what does this function expect? No idea!
function processPayment(amount, method, options) { ... }

// TypeScript — crystal clear!
function processPayment(
  amount: number,
  method: 'credit' | 'debit' | 'upi',
  options: { retry: boolean; timeout: number }
): Promise<PaymentResult> { ... }
```

2.2 Benefits of TypeScript

Benefit	What it means for you
Early Error Detection	TypeScript finds bugs while you type — before running the code.
Better IDE Support	VS Code gives you autocomplete, hover-docs, and inline error highlighting.
Self-Documenting Code	Types act as documentation. New developers understand code faster.
Safe Refactoring	Rename a function? TS shows every broken reference instantly.
Works with JS	You can gradually add TypeScript to existing JavaScript projects.
Industry Standard	Angular, React, Vue, Node.js — all have excellent TS support.
Team Collaboration	Types enforce contracts, fewer integration mistakes between developers.

3. Setting Up TypeScript

3.1 Installation

```
# Install globally
npm install -g typescript
tsc --version   # Should print: Version 5.x.x

# OR install locally in your project (recommended)
npm install --save-dev typescript
npx tsc --version
```

3.2 Your First TypeScript File

```
// hello.ts
let message: string = 'Hello, TypeScript!';

function greet(name: string): string {
  return `Hello, ${name}! Welcome to TypeScript.`;
}

console.log(greet('Rahul'));  // Hello, Rahul! Welcome to TypeScript.
```

```
# Step 1: Compile
tsc hello.ts       # Creates hello.js

# Step 2: Run
node hello.js      # Output: Hello, Rahul! Welcome to TypeScript.
```

3.3 tsconfig.json — The Config File

```
# Generate tsconfig.json
tsc --init
```

```
{
  "compilerOptions": {
    "target": "ES6",          // What JS version to compile to
    "module": "commonjs",     // Module system
    "outDir": "./dist",       // Where compiled JS files go
    "rootDir": "./src",       // Where your .ts source files are
```

```
    "strict": true,           // Enable ALL strict type checks (ALWAYS do
this!)
    "esModuleInterop": true,  // Easier imports from CommonJS packages
    "sourceMap": true         // Debug TS source in browser
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

```
tsc          # Compile whole project
tsc --watch  # Watch mode — auto-recompile on save
```

4. TypeScript vs JavaScript

4.1 Side-by-Side Code Comparison

JavaScript Version:

```
// user.js
function createUser(name, age, email) {
  return { id: Math.random(), name, age, email, createdAt: new Date() };
}
const user = createUser('Rahul', '25', 'rahul@example.com'); // age is string!
No error.
```

TypeScript Version:

```
// user.ts
interface User {
  id: number;
  name: string;
  age: number;
  email: string;
  createdAt: Date;
}

function createUser(name: string, age: number, email: string): User {
  return { id: Math.random(), name, age, email, createdAt: new Date() };
}

// createUser('Rahul', '25', 'rahul@example.com');  // ✗ ERROR — age must be
```

```
number
const user = createUser('Rahul', 25, 'rahul@example.com');  // ✅ Correct
```

4.2 Comparison Table

Feature	JavaScript	TypeScript
Type System	Dynamic — runtime	Static — compile time
Error Detection	When code runs	Before code runs
Type Annotations	Not available	string, number, boolean, etc.
IDE Support	Basic	Full autocomplete + inline errors
Interfaces	No	Yes
Generics	No	Yes
Null Safety	No checks	strictNullChecks option
File Extension	.js	.ts or .tsx
Compilation	Runs directly	Must compile to JS first
Team Safety	Error-prone at scale	Types enforce contracts

5. Interview Questions — Chapter 1

Q: What is TypeScript and how is it different from JavaScript?

Answer: TypeScript is a statically typed superset of JavaScript developed by Microsoft. It adds a type system on top of JavaScript — we can declare types for variables, function parameters, and return values.

The biggest difference: JavaScript catches type errors at runtime (when app is running). TypeScript catches them at compile time (before the app runs). TypeScript compiles to plain JavaScript — browsers still run JS.

Q: What does it mean that TypeScript is a 'superset' of JavaScript?

Answer: Superset means TypeScript includes ALL of JavaScript plus additional features. Any valid JavaScript code is also valid TypeScript code — you can rename a .js file to .ts and it will work.

TypeScript adds: type annotations, interfaces, generics, enums, access modifiers, and more.

Q: What is static typing vs dynamic typing?

Answer: Dynamic typing (JavaScript) — variable types determined at runtime. You can assign any value of any type without errors.

Static typing (TypeScript) — types declared at compile time. The compiler checks type correctness before the program runs.

Example: In JS, let x = 5; x = 'hello'; works fine. In TS, assigning a string to a number variable causes a compile-time error.

Q: What is tsconfig.json and why is it important?

Answer: tsconfig.json is the configuration file for the TypeScript compiler. Key settings: target (JS version to output), strict (enable strict type checking), outDir (where to put compiled files), rootDir (where source files are).

Without it, you'd pass all options as command-line arguments every time. tsconfig.json keeps project configuration consistent and shareable.

Q: What is the 'strict' option in tsconfig.json?

Answer: strict: true enables ALL strict type-checking rules at once. It turns on: strictNullChecks (null/undefined must be explicit), noImplicitAny (no accidental any types), strictFunctionTypes, strictPropertyInitialization, and more.

Always set strict: true in new projects — it catches the most bugs.

Q: What is Type Inference in TypeScript?

Answer: TypeScript can automatically figure out a variable's type from the value you assign, without you explicitly writing the type annotation.

Example: let age = 25; — TypeScript infers age is number. If you later try age = 'hello', it gives an error even without :number annotation.

Rapid Fire — Chapter 1

Question	One-Line Answer
Who created TypeScript?	Microsoft (released 2012)
What compiles TypeScript?	tsc — the TypeScript Compiler

What initializes tsconfig.json?	tsc --init
.ts vs .tsx?	.ts for regular TS, .tsx for React components with JSX
Does TypeScript run in browser?	No — it compiles to JavaScript first
What is a transpiler?	Converts source code to source code (TS → JS)
Can JS libraries be used in TS?	Yes — via type definition files (.d.ts)
What is the TS playground?	typescriptlang.org/play — online TS editor

---

End of Chapter 1

CHAPTER 2

Type System Basics

Primitives • Type Inference • any • unknown • never • void

1. What is the Type System?

Before we learn individual types, let's understand what a 'Type System' even means.

Simple Analogy

Think of types like labels on containers in your kitchen.

A jar labeled 'Salt' — you know ONLY salt goes inside.

A jar labeled 'Honey' — you know ONLY honey goes inside.

If someone tries to pour juice into the 'Salt' jar, you immediately say:

'Hey! That's the wrong container!'

TypeScript does the SAME thing with your variables.

A variable labeled 'number' can only hold numbers.

If you try to put a string in it, TypeScript says: 'Wrong type!'

- **Primitive Types** — Basic building blocks: string, number, boolean, null, undefined, symbol, bigint
- **Complex Types** — Built from primitives: arrays, objects, functions, tuples, enums (Chapter 3+)

2. The string Type

Real-World Analogy: String is like a WhatsApp message — it is text. Names, emails, addresses, messages — anything made of characters is a string.

string

Real-World Use: Names, emails, phone numbers, addresses, messages — anything made of text.

Example: `let name: string = 'Rahul';`

```
let userName: string = 'Priya Sharma';
let email: string = 'priya@gmail.com';
let greeting: string = `Hello, ${userName}!`;

// TypeScript catches wrong assignments
let productName: string = 'iPhone 15';
productName = 42;        // ✗ ERROR: number is not assignable to string
productName = 'Pixel 8'; // ✓ Fine
```

> 🛒 **Real-World Scenario: E-Commerce App**
> In an Amazon-like app, product name, description, seller name, delivery address — all are strings. TypeScript ensures you never accidentally pass a product ID (number) where a product name (string) is expected.

```
// Real-world: User profile in a food delivery app (Swiggy / Zomato)
interface UserProfile {
  name: string;
  phone: string;     // Phone stored as string, NOT number!
  address: string;   // Because '9845012345' is text, not math
  city: string;
}

function sendOTP(phone: string): void {
  console.log(`Sending OTP to ${phone}`);
}

sendOTP('9845012345');  // ✓ Correct
sendOTP(9845012345);    // ✗ ERROR — TypeScript saves you from this bug!
```

3. The number Type

Real-World Analogy: Number is like a price tag. Price, quantity, age, rating, score — anything you do math with is a number. TypeScript has just ONE number type for both integers and decimals.

number

Real-World Use: Prices, ages, ratings, scores, quantities — anything you calculate with.
Example: `let price: number = 999.99;`

```
let age: number = 25;            // integer
let price: number = 1299.99;     // decimal
```

```
let rating: number = 4.5;
let temperature: number = -5;    // negative

// Real world: Shopping cart
let itemPrice: number = 599;
let quantity: number = 3;
let discount: number = 0.10;
let total: number = itemPrice * quantity * (1 - discount);  // 1617.3

// TypeScript catches wrong assignments
let salary: number = 50000;
salary = '50000';   // ✗ ERROR
salary = 55000;     // ✓ Fine
```

🏦 **Real-World Scenario: Banking / Payment App**

In GPay or PhonePe, transaction amount, account balance, interest rate are all numbers. TypeScript ensures you never accidentally pass a string '500' where a number 500 is needed — which would cause wrong calculations silently in plain JS.

Important: Phone numbers should be strings, not numbers!

✗ Wrong: let phone: number = 9845012345;

✓ Right: let phone: string = '9845012345';

Reason 1: Phone numbers can start with 0 — a number drops leading zeros.

Reason 2: Phone numbers can have '+91' prefix — that's not a math number.

Rule of thumb: If you won't do MATH with it, store it as a string!

4. The boolean Type

Real-World Analogy: Boolean is like a light switch — it's either ON or OFF. There is NO middle state. In code: true or false. That's it.

boolean

Real-World Use: Flags, toggles, permissions, status checks — yes/no, on/off decisions.
Example: `let isLoggedIn: boolean = false;`

```
let isLoggedIn: boolean = false;
let isPremiumUser: boolean = true;
let isEmailVerified: boolean = false;
```

```
// Real world: Netflix-like app settings
interface VideoSettings {
  isAutoPlayOn: boolean;
  isSubtitleEnabled: boolean;
  isHDAvailable: boolean;
}

// Boolean from comparisons
let age: number = 20;
let isAdult: boolean = age >= 18;   // true
let canDrink: boolean = age >= 21;  // false

// TypeScript is strict here
let isActive: boolean = true;
isActive = 'yes';  // ✕ ERROR (unlike JS, TS doesn't allow this!)
isActive = 1;      // ✕ ERROR
```

Real-World Scenario: User Authentication System

In any login system, isLoggedIn, isEmailVerified, isAdmin, isBanned — all are booleans. TypeScript ensures that when checking permissions, you compare actual true/false values, not accidentally use a number (1/0) like old JS patterns.

5. null and undefined

These two are the most misunderstood types in JavaScript. TypeScript makes them much cleaner.

5.1 What is the difference?

	undefined	null
Meaning	Variable declared but no value assigned yet	Variable intentionally set to 'no value'
Who sets it?	JavaScript sets it automatically	YOU (the developer) set it manually
Real-world analogy	A form field that hasn't been touched yet	A form field that was cleared on purpose
Example	let x; // x is undefined	let user = null; // no user logged in

```
// undefined — variable exists, value not yet assigned
let userName: string | undefined;
```

```
console.log(userName);  // undefined

// null — intentionally empty
let currentUser: { name: string } | null = null;
// No user is logged in right now. We deliberately set null.

// After login:
currentUser = { name: 'Rahul' };

// After logout:
currentUser = null;

// Real-world: Search results
function findUser(id: number): { name: string } | null {
  if (id === 1) return { name: 'Priya' };
  return null;  // No user found
}

const user = findUser(99);
if (user !== null) {
  console.log(user.name);  // Safe — checked first
}
```

5.2 strictNullChecks — The Safety Net

```
// WITH strictNullChecks (strict: true in tsconfig) — RECOMMENDED
let name: string = null;   // ✗ ERROR: null not assignable to string

// How to properly allow null
let name: string | null = null;   // ✓ Explicitly say: 'this CAN be null'

// Real-world: Profile picture
interface UserProfile {
  name: string;
  profilePicture: string | null;  // User may not have uploaded a pic
  bio: string | undefined;        // User may not have written a bio
}

function displayProfile(profile: UserProfile): void {
  const pic = profile.profilePicture ?? 'default-avatar.png';
  const bio = profile.bio ?? 'No bio provided.';
  console.log(`${profile.name}: ${bio}`);
```

```
}
```

Real-World Scenario: Social Media App (Instagram-like)

A user profile has name (always exists), profilePicture (might be null — not uploaded), and bio (might be undefined — field never filled). TypeScript with strictNullChecks forces you to handle these cases, preventing crashes for users with incomplete profiles.

6. Type Inference — TypeScript Reads Your Mind

Type inference is one of TypeScript's smartest features. You don't always have to write the type — TypeScript figures it out from the value you assign.

Simple Analogy

If you put a pizza in a box, anyone can look and say: 'That's pizza in there.'

TypeScript does the same — if you write: let age = 25;

TypeScript KNOWS age is a number without you writing :number.

It inferred the type from the value 25.

```
// Type Inference in action
let productName = 'iPhone 15 Pro'; // inferred: string
let price = 1299;                  // inferred: number
let isAvailable = true;            // inferred: boolean
let items = ['apple', 'banana'];   // inferred: string[]

productName = 999;  // ✗ ERROR even without :string annotation!

// Inference in functions
function double(n: number) {
  return n * 2;  // TS infers return type is 'number'
}

// When to write explicit types:
// 1. Function parameters — TS CANNOT infer these
function greet(name: string) { ... }  // ✓ Always annotate params

// 2. Declaring without assigning
let data: string;   // ✓ Must give a type

// 3. Complex return types — for readability
```

```
function getUser(): { name: string; age: number } { ... }
```

Rule of Thumb: When to write type annotations?

✅ ALWAYS write types for function parameters

✅ Write types when declaring a variable WITHOUT a value: let x: string;

✅ Write return types for important/public functions

⚡ SKIP types when TS can clearly infer from the assigned value

7. The 'any' Type — The Escape Hatch (Avoid!)

any completely disables TypeScript's type checking for that variable. It can hold any value, and TypeScript will not complain about how you use it.

Real-World Analogy

A box labeled 'ANYTHING GOES'. You can put salt, juice, shoes, a laptop — anything.

When you reach in to grab 'the juice', you might pull out a shoe!

That's exactly what 'any' does — removes ALL safety checks.

```
let data: any = 'Hello';
data = 42;           // No error
data = true;         // No error
data.toUpperCase();  // No error even if data is a number — DANGEROUS!

// The danger: silent bugs
function calculateTotal(price: any, qty: any) {
  return price * qty;
}
calculateTotal('500', 3);  // Returns NaN at runtime — SILENT BUG!

// What to use INSTEAD of any:
// 1. 'unknown' — if you truly don't know the type (explained next)
// 2. Union types — string | number  (Chapter 6)
// 3. Generics — <T>  (Chapter 8)
```

Enable noImplicitAny (via strict: true)

This errors when a variable implicitly becomes 'any':

```
function greet(name) { ... }        // ✗ ERROR: 'name' has implicit 'any'
function greet(name: string) { ... } // ✓ Must be explicit
```

8. The 'unknown' Type — The Safe Version of 'any'

unknown can hold any value — but unlike **any**, TypeScript forces you to CHECK the type before you USE the value. It's the safe alternative to any.

Real-World Analogy

A mystery box at a post office. You don't know what's inside.

With 'any': Reach in and grab it — no checking. Risky!

With 'unknown': You MUST inspect the box first, confirm what's inside, THEN use it.

```
let data: unknown = 'Hello';
data = 42;   // ✓ Can hold anything

// BUT you CANNOT use it without checking first
let result = data.toUpperCase();  // ✗ ERROR!

// How to use unknown safely — check type first
function processInput(input: unknown): string {
  if (typeof input === 'string') {
    return input.toUpperCase();  // ✓ Safe
  }
  if (typeof input === 'number') {
    return input.toFixed(2);    // ✓ Safe
  }
  return 'Unknown input';
}

// Real-world: API response
async function fetchUserData(): Promise<void> {
  const response = await fetch('/api/user');
  const data: unknown = await response.json();  // API data is unknown!
  // TypeScript forces you to validate before using
  if (typeof data === 'object' && data !== null && 'name' in data) {
    console.log((data as { name: string }).name);
  }
```

```
}
```

Feature	any	unknown
Can hold any value?	✅ Yes	✅ Yes
Type checking disabled?	✅ Completely OFF	❌ Checking stays ON
Can use value directly?	✅ Yes (dangerous)	❌ Must check type first
Safe to use?	❌ No	✅ Yes
When to use?	Migrating JS code only	API responses, user input
Recommended?	Avoid	Yes — prefer over any

9. The 'void' Type — Functions That Return Nothing

void means 'no value'. It is the return type of a function that does NOT return anything — it just does something (log, save to DB, send email) and finishes.

Real-World Analogy

Think of a delivery person. Their job is to DELIVER the package.

They don't hand you anything back — they just DO the action (deliver) and leave.

A void function does the same — it performs an action, returns nothing.

```
// Basic void usage
function logMessage(message: string): void {
  console.log(message);
  // No return statement — just doing an action
}

// Real-world examples of void functions
function saveOrder(orderId: number, total: number): void {
  console.log(`Saving order ${orderId} with total ₹${total}...`);
}

function sendConfirmationEmail(email: string, orderId: number): void {
  console.log(`Sending confirmation to ${email} for order #${orderId}`);
}

// React event handlers are always void
function handleButtonClick(event: MouseEvent): void {
```

```
  console.log('Button clicked!');
}

// Using return value of a void function
const result = logMessage('Hello');
console.log(result.toUpperCase()); // ✗ ERROR: void has no methods
```

10. The 'never' Type — The Function That Never Returns

never is for functions that NEVER successfully finish. Either they always throw an error, or they run forever. Different from void — void finishes but returns nothing. never NEVER finishes.

Real-World Analogy

Two workers:

Worker A (void): Does the job, then clocks out. Done.

Worker B (never): Gets stuck in an infinite loop, OR immediately triggers an emergency alarm.

Worker B never 'finishes normally' — that's what 'never' means!

```
// Case 1: Function that ALWAYS throws an error
function throwError(message: string): never {
  throw new Error(message);
  // This function NEVER returns — it always throws
}

// Case 2: Infinite loop
function startHeartbeatMonitor(): never {
  while (true) {
    checkServerHealth();  // Runs forever — never returns
  }
}

// Case 3: Exhaustive checking (ADVANCED — very useful)
type Status = 'active' | 'inactive' | 'banned';

function handleStatus(status: Status): string {
  switch (status) {
    case 'active':   return 'User is active';
    case 'inactive': return 'User is inactive';
    case 'banned':   return 'User is banned';
```

```
    default:
      const _check: never = status; // TS errors if a case is missed!
      throw new Error(`Unhandled: ${status}`);
  }
}
// If you add 'suspended' to Status but forget to handle it,
// TypeScript will ERROR at the default — brilliant safety!
```

Type	Returns?	Use Case
void	Yes — returns undefined (nothing useful)	Functions that DO things but give back no value
never	No — NEVER returns normally	Always-throw functions or infinite loops

11. All Primitive Types — Quick Reference

Type	Values it holds	Real-world Example	TypeScript Syntax
string	Text of any kind	Name, email, address	`let name: string = 'Rahul';`
number	Integers & decimals	Price, age, rating	`let price: number = 499.99;`
boolean	true or false only	isLoggedIn, isAdmin	`let active: boolean = true;`
null	Intentionally empty	No user logged in	`let user: string | null = null;`
undefined	Not yet assigned	Optional field not filled	`let bio: string | undefined;`
any	Anything (unsafe)	Migrating JS code (avoid!)	`let data: any = 'hello';`
unknown	Anything (safe)	API response, user input	`let input: unknown = getData();`
void	No return value	Save to DB, log message	`function log(): void { }`
never	Never returns	Always-throw, infinite loop	`function fail(): never { throw... }`
bigint	Very large integers	Crypto, govt IDs	`let big: bigint = 9999n;`
symbol	Unique identifier	Non-conflicting object keys	`let id: symbol = Symbol('id');`

12. Interview Questions — Chapter 2

Q: What are the primitive types in TypeScript?

Answer: TypeScript has 7 primitive types — string, number, boolean, null, undefined, symbol, and bigint.

- string: Text values like names, emails, addresses
- number: Both integers and decimals (no separate int/float)
- boolean: Only true or false
- null: Intentionally empty — you set this explicitly
- undefined: Variable declared but no value given yet
- symbol: Unique identifiers — for object keys
- bigint: Integers larger than Number.MAX_SAFE_INTEGER

Q: What is the difference between 'any' and 'unknown'?

Answer: Both can hold values of any type, but they behave very differently when you USE the value:

any: TypeScript completely disables type checking. You can use the value however you want. This is UNSAFE. A variable of type any can call .toUpperCase() even if it's a number — TS won't stop you.

unknown: TypeScript keeps checking ON. Before you can use the value, you MUST check its type first using typeof or instanceof. This is SAFE.

Rule: Prefer unknown over any whenever you have data whose type you genuinely don't know, like API responses.

Q: What is the difference between 'null' and 'undefined'?

Answer: Both represent 'no value', but with different meanings:

undefined: Variable exists but was never given a value. JavaScript sets this automatically.

null: Variable was deliberately set to 'empty' by the developer.

Real-world: bio might be undefined (user never visited that form field) vs profilePicture might be null (user deliberately removed their photo).

With strictNullChecks enabled, TypeScript treats them as separate types and forces you to handle both cases explicitly — preventing the famous 'Cannot read property of null' crash.

Q: What is Type Inference? When should you write explicit types?

Answer: Type inference means TypeScript automatically determines a variable's type from the assigned value — without you writing the annotation.

Example: let count = 0; — TypeScript infers count is number. Assigning count = 'hello' gives an error even without :number.

Write explicit types when: (1) function parameters — TS cannot infer these, (2) declaring without assigning: let data: string;, (3) complex return types for readability.

Skip them when the value clearly shows the type: let name = 'Rahul'; needs no :string.

Q: What is the 'void' return type? How is it different from 'never'?

Answer: void: The function DOES finish executing, but doesn't return a meaningful value. Use for functions that console.log(), save to DB, or send emails.

never: The function NEVER finishes normally. Either it always throws an Error, or it runs in an infinite loop. The function never returns to the caller.

Practical rule: Use void for 'action' functions. Use never for error-throwing utilities and exhaustive type checks.

Q: What does 'strictNullChecks' do? Why is it important?

Answer: strictNullChecks (enabled by strict: true) makes TypeScript treat null and undefined as separate types, not automatically assignable to other types.

Without it: let name: string = null; — no error! Dangerous.

With it: let name: string = null; — ERROR! Must write: let name: string | null = null;

This prevents 'TypeError: Cannot read property X of null' — the #1 most common JavaScript production error.

Rapid Fire — Chapter 2

Question	One-Line Answer
How many primitive types in TS?	7 — string, number, boolean, null, undefined, symbol, bigint
Separate int and float in TS?	No — both are just 'number'

'n' suffix means what?	Creates a bigint literal: let x = 9999n;
Can void functions return undefined?	Yes — they return undefined implicitly
Only value 'never' can be?	None — never represents impossibility
Assign null to string type?	Only with string | null, not with strict: true
What is noImplicitAny?	Errors when variable defaults to 'any' without annotation
Can unknown call methods directly?	No — must check type first with typeof

End of Chapter 2

CHAPTER 3

Objects, Arrays & Tuples

Typed Objects • Readonly • Optional Props • Arrays • Tuples

1. Typing Objects

In JavaScript, objects are everywhere. A user, a product, an order, a tweet — everything is an object. TypeScript lets you define exactly what shape an object must have.

Real-World Analogy

Think of an object type like a job application FORM.

The form has specific fields: Name (required), Age (required), Phone (optional).

If you submit a form with a field missing or the wrong type of data,

the HR department rejects it immediately.

TypeScript does the same — if your object is missing a required field,

or has a field of the wrong type, it errors right away.

1.1 Inline Object Type Annotation

```
// ── Basic object type ──────────────────────────────────────────
let user: { name: string; age: number; email: string } = {
  name: 'Rahul Sharma',
  age: 28,
  email: 'rahul@gmail.com',
};

// TypeScript checks every property
user.name = 42;           // ✗ ERROR: number not assignable to string
user.age = 'twenty';      // ✗ ERROR: string not assignable to number
user.name = 'Priya';      // ✓ Fine

// Accessing a non-existent property
console.log(user.phone);  // ✗ ERROR: 'phone' does not exist on this type
```

1.2 Optional Properties (?)

Not every property is required. Use `?` to mark a property as optional — it can be present OR absent.

```
// ── Optional properties with ? ──────────────────────────────
let userProfile: {
  name: string;
  age: number;
  bio?: string;           // Optional — may or may not exist
  profilePicture?: string; // Optional
} = {
  name: 'Priya',
  age: 25,
  // bio and profilePicture are not provided — that's fine!
};

// ── Real-world: Product in an e-commerce app ────────────────
let product: {
  id: number;
  name: string;
  price: number;
  description?: string;   // Not all products have descriptions
  discount?: number;      // Not all products are on sale
  imageUrl?: string;      // Not all have images yet
} = {
  id: 101,
  name: 'Wireless Earbuds',
  price: 1499,
  // description, discount, imageUrl are all optional — fine to skip
};

// When accessing optional properties, TypeScript warns you
console.log(product.description.toUpperCase());
// ✗ ERROR: 'description' is possibly undefined — must check first!

// ✓ Safe way:
if (product.description) {
  console.log(product.description.toUpperCase());
}
// OR using optional chaining:
console.log(product.description?.toUpperCase()); // undefined if missing
```

1.3 readonly Properties

Use `readonly` to prevent a property from being changed after the object is created. Perfect for IDs, creation timestamps, and other immutable data.

```
// ── readonly property ──────────────────────────────────────
let order: {
  readonly orderId: string;   // Cannot be changed after creation
  readonly createdAt: Date;   // Cannot be changed
  status: string;             // CAN be changed (order status updates)
  total: number;
} = {
  orderId: 'ORD-2024-001',
  createdAt: new Date(),
  status: 'pending',
  total: 2499,
};

// Trying to change a readonly property:
order.orderId = 'ORD-2024-002';  // ✗ ERROR: Cannot assign to 'orderId'
                                 // because it is a read-only property

// Changing a normal property:
order.status = 'shipped';        // ✓ Fine — status is not readonly
order.total = 2699;              // ✓ Fine
```

> 🛍️ **Real-World Scenario: Order Management System (Flipkart/Amazon)**
> An order's orderId and createdAt should NEVER change after creation — they're your permanent record. But status changes from 'pending' → 'processing' → 'shipped' → 'delivered'. TypeScript readonly enforces this business rule at the code level — no accidental overwriting of order IDs!

2. Type Aliases for Objects

Writing long object type annotations inline is messy and can't be reused. Type aliases let you give a name to any type — then use that name everywhere.

Real-World Analogy

Instead of writing 'a tall, double-shot, oat-milk, no-sugar latte' every time, you just say 'my usual'. Type alias is the same thing — give a complex type a name, then just use the name everywhere.

```
// ── Without type alias — messy and not reusable ────────────────
function createUser(data: { name: string; age: number; email: string }) {
  ...
}
function updateUser(user: { name: string; age: number; email: string }) {
  ...
}
// Same object shape written twice — bad!

// ── With type alias — clean and reusable ──────────────────────
type User = {
  name: string;
  age: number;
  email: string;
};

function createUser(data: User) { ... }
function updateUser(user: User) { ... }
function deleteUser(user: User) { ... }
// Now all three functions use the same type — DRY!

// ── Real-world: Full app types ────────────────────────────────
type Product = {
  id: number;
  name: string;
  price: number;
  category: string;
  inStock: boolean;
  imageUrl?: string;
};

type CartItem = {
  product: Product;   // Type inside type — composition!
  quantity: number;
  addedAt: Date;
};

type Order = {
  readonly orderId: string;
  items: CartItem[];  // Array of CartItem
  total: number;
  status: 'pending' | 'shipped' | 'delivered';
```

```
  deliveryAddress: string;
};
```

2.1 Extending Type Aliases with Intersection (&)

You can combine two type aliases into one using `&` . This is called an intersection type — the result must have ALL properties from both types.

```
type PersonalInfo = {
  name: string;
  age: number;
};

type ContactInfo = {
  email: string;
  phone: string;
};

// UserProfile must have ALL properties from BOTH types
type UserProfile = PersonalInfo & ContactInfo;

const user: UserProfile = {
  name: 'Rahul',       // from PersonalInfo
  age: 28,             // from PersonalInfo
  email: 'r@g.com',   // from ContactInfo
  phone: '9876543210', // from ContactInfo
  // Missing any one of these = ERROR
};

// Real-world: Employee = Person + Job details
type Person = { name: string; dob: Date };
type JobDetails = { role: string; department: string; salary: number };
type Employee = Person & JobDetails;
```

3. Arrays in TypeScript

Arrays are one of the most used data structures. TypeScript ensures every element in an array is of the correct type.

Real-World Analogy

Think of a typed array like a specific-purpose tray at a restaurant:

A DESSERT tray can only hold desserts — not biryani.
A DRINKS tray can only hold drinks — not cutlery.

A string[] array can only hold strings.
A number[] array can only hold numbers.
If you try to put the wrong thing in — TypeScript rejects it!

3.1 Array Type Syntax — Two Ways to Write

```
// ── Method 1: Type followed by []  (Most common — prefer this) ─
let names: string[] = ['Rahul', 'Priya', 'Ankit'];
let prices: number[] = [299, 599, 1299, 4999];
let flags: boolean[] = [true, false, true];

// ── Method 2: Array<Type>  (Generic syntax — same result) ──────
let names: Array<string> = ['Rahul', 'Priya', 'Ankit'];
let prices: Array<number> = [299, 599, 1299, 4999];

// Both are equivalent — Method 1 is more common in practice

// ── TypeScript enforces element types ────────────────────────
let scores: number[] = [95, 87, 92];
scores.push(88);      // ✅ Fine
scores.push('good');  // ❌ ERROR: string not assignable to number
scores[0] = 'A+';    // ❌ ERROR: string not assignable to number
```

3.2 Arrays of Objects

```
// ── Array of objects ────────────────────────────────────────
type Student = {
  name: string;
  grade: number;
  passed: boolean;
};

let students: Student[] = [
  { name: 'Rahul', grade: 88, passed: true },
  { name: 'Priya', grade: 92, passed: true },
  { name: 'Karan', grade: 45, passed: false },
];
```

```
// TypeScript knows every item in students[] is a Student
students.forEach(student => {
  // TypeScript gives autocomplete for .name, .grade, .passed
  console.log(`${student.name}: ${student.grade}`);
  // student.xyz   // ✗ ERROR: 'xyz' doesn't exist on Student
});

// ── Real-world: Product listing (Swiggy menu) ─────────────────
type MenuItem = {
  id: number;
  name: string;
  price: number;
  isVeg: boolean;
  rating: number;
};

let menu: MenuItem[] = [
  { id: 1, name: 'Paneer Butter Masala', price: 280, isVeg: true, rating:
4.5 },
  { id: 2, name: 'Chicken Biryani',      price: 350, isVeg: false, rating:
4.8 },
  { id: 3, name: 'Dal Makhani',          price: 220, isVeg: true, rating:
4.2 },
];

// Filter veg items — TypeScript knows item is MenuItem
const vegItems = menu.filter(item => item.isVeg);
const cheapItems = menu.filter(item => item.price < 300);
```

3.3 readonly Arrays

Use `readonly` to prevent modification of array contents after creation — useful for configuration data, constants, or any data that shouldn't change.

```
// ── readonly array ──────────────────────────────────────────
const allowedRoles: readonly string[] = ['admin', 'user', 'moderator'];

allowedRoles.push('superuser');    // ✗ ERROR: Cannot add to readonly array
allowedRoles[0] = 'superadmin';   // ✗ ERROR: Cannot modify readonly array
console.log(allowedRoles[0]);     // ✓ Reading is fine

// ── Real-world: Config data that must not change ──────────────
```

```
const SUPPORTED_CURRENCIES: readonly string[] = ['INR', 'USD', 'EUR', 'GBP'];
const TAX_RATES: readonly number[] = [0, 5, 12, 18, 28]; // GST slabs

// These are constants — nobody should accidentally modify them
```

3.4 Multi-dimensional Arrays

```
// ── 2D array (matrix / grid) ──────────────────────────────
let seatingChart: string[][] = [
  ['A1', 'A2', 'A3'],  // Row A
  ['B1', 'B2', 'B3'],  // Row B
  ['C1', 'C2', 'C3'],  // Row C
];

// Real-world: Chess board, spreadsheet cells, cinema seating
let movieSeats: boolean[][] = [
  [true,  false, true,  true ],  // Row 1: available/taken
  [false, false, true,  false],  // Row 2
  [true,  true,  false, true ],  // Row 3
];

// Access: movieSeats[rowIndex][seatIndex]
console.log(movieSeats[0][1]);  // false (seat taken)
```

4. Tuples — Fixed-Length, Fixed-Type Arrays

A Tuple is an array with a FIXED number of elements where EACH position has a specific type. Unlike regular arrays where all elements are the same type, each position in a tuple can have a different type.

Real-World Analogy

Think of a tuple like a FORM ENTRY in a spreadsheet:

Column 1 is always: Employee Name (string)
Column 2 is always: Employee ID (number)
Column 3 is always: Is Active (boolean)

Every row has EXACTLY 3 values, in EXACTLY that order, with EXACTLY those types.
That's a tuple — fixed positions, fixed types, fixed length.

4.1 Basic Tuple Syntax

```
// ── Basic tuple ──────────────────────────────────────────────
let employee: [string, number, boolean] = ['Rahul', 101, true];
//             position 0  position 1   position 2
//             string      number       boolean

// Access by index
console.log(employee[0]);  // 'Rahul'   — TypeScript knows it's a string
console.log(employee[1]);  // 101       — TypeScript knows it's a number
console.log(employee[2]);  // true      — TypeScript knows it's a boolean

// TypeScript enforces the types at each position
let point: [number, number] = [10, 20];  // x, y coordinates
point[0] = 15;       // ✅ Fine — number in position 0
point[0] = 'ten';   // ❌ ERROR — string not allowed in position 0

// Wrong length is also an error
let coord: [number, number] = [10, 20, 30]; // ❌ ERROR: too many elements
let coord2: [number, number] = [10];         // ❌ ERROR: too few elements
```

4.2 Real-World Tuple Examples

```
// ── RGB Color value ──────────────────────────────────────────
type RGB = [number, number, number];  // [red, green, blue]
let red: RGB   = [255, 0, 0];
let green: RGB = [0, 255, 0];
let white: RGB = [255, 255, 255];

// ── GPS Coordinates ──────────────────────────────────────────
type Coordinates = [number, number];  // [latitude, longitude]
let mumbai: Coordinates  = [19.0760, 72.8777];
let delhi: Coordinates   = [28.6139, 77.2090];
let bangalore: Coordinates = [12.9716, 77.5946];

// ── useState in React (returns a tuple!) ─────────────────────
// This is how React's useState hook works under the hood:
const [count, setCount]: [number, (n: number) => void] = useState(0);
//                        value   setter function

// ── HTTP Response ────────────────────────────────────────────
```

```
type ApiResponse = [number, string, boolean];
//                  status  message  success

function callApi(): ApiResponse {
  return [200, 'Data fetched successfully', true];
}

const [statusCode, message, isSuccess] = callApi(); // Destructuring
console.log(statusCode);  // 200 — TypeScript knows it's a number
console.log(message);     // 'Data fetched...' — knows it's string
```

4.3 Named Tuple Elements (TypeScript 4.0+)

You can label each position in a tuple for better readability. This does NOT change how the tuple works — it just makes the code easier to understand.

```
// ── Without labels — hard to remember what each position means
type OldCoords = [number, number];  // which is lat? which is lng?

// ── With labels — crystal clear!
type GeoPoint = [latitude: number, longitude: number];
let office: GeoPoint = [28.6139, 77.2090];

// ── Employee record with labels
type EmployeeRecord = [
  name: string,
  employeeId: number,
  department: string,
  isActive: boolean
];

const emp: EmployeeRecord = ['Priya Sharma', 1042, 'Engineering', true];
```

4.4 Optional Tuple Elements

```
// ── Optional elements with ? at the end ──────────────────────
type UserLog = [userId: number, action: string, timestamp?: Date];

// Both are valid:
const log1: UserLog = [1, 'login'];                  // without timestamp
const log2: UserLog = [1, 'login', new Date()];     // with timestamp
```

```
// ── Rest elements in tuples ──────────────────────────────
type MinOneString = [string, ...string[]]; // at least one string
const tags1: MinOneString = ['typescript'];            // ✅
const tags2: MinOneString = ['typescript', 'react', 'nodejs'];  // ✅
const tags3: MinOneString = [];  // ❌ ERROR — need at least one
```

4.5 Tuples vs Arrays — When to Use Which

	Array	Tuple
Length	Variable — can grow/shrink	Fixed — always same length
Element Types	All elements same type	Each position has its own type
Use When	List of similar items	Fixed structure with mixed types
Example	List of product names	RGB color, GPS coordinate, useState
Syntax	string[] or Array<string>	[string, number, boolean]

5. Putting It All Together — Real-World Example

Let's build a complete mini type system for a food delivery app like Swiggy — combining objects, arrays, and tuples.

```
// ── Complete Food Delivery App Type System ────────────────

// 1. Basic types
type Address = {
  street: string;
  city: string;
  pincode: string;
  landmark?: string;       // Optional
};

type MenuItem = {
  readonly id: number;     // Cannot change after creation
  name: string;
  price: number;
  isVeg: boolean;
  rating: number;
  imageUrl?: string;       // Optional
};
```

```
type Restaurant = {
  readonly restaurantId: string;
  name: string;
  address: Address;        // Nested object type
  menu: MenuItem[];        // Array of MenuItems
  rating: number;
  isOpen: boolean;
  cuisines: string[];      // e.g. ['North Indian', 'Chinese']
};

type CartItem = {
  item: MenuItem;
  quantity: number;
  addedAt: Date;
};

type Order = {
  readonly orderId: string;
  readonly createdAt: Date;
  customer: {
    name: string;
    phone: string;
    deliveryAddress: Address;
  };
  items: CartItem[];
  total: number;
  status: 'placed' | 'confirmed' | 'preparing' | 'out_for_delivery' |
'delivered';
};

// ── Tuple: Delivery time estimate ──────────────────────────────
type TimeEstimate = [minMinutes: number, maxMinutes: number];
const estimate: TimeEstimate = [25, 35];  // 25-35 minutes

// ── Using our types ────────────────────────────────────────────
const restaurant: Restaurant = {
  restaurantId: 'REST-001',
  name: 'Punjabi Tadka',
  address: { street: 'MG Road', city: 'Bangalore', pincode: '560001' },
  menu: [
    { id: 1, name: 'Dal Makhani', price: 220, isVeg: true, rating: 4.5 },
    { id: 2, name: 'Butter Chicken', price: 320, isVeg: false, rating: 4.8 },
```

```
  ],
  rating: 4.6,
  isOpen: true,
  cuisines: ['North Indian', 'Punjabi'],
};

// TypeScript gives you autocomplete and safety throughout:
console.log(restaurant.menu[0].name);  // 'Dal Makhani'
restaurant.restaurantId = 'NEW';       // ✕ ERROR: readonly!
```

6. Interview Questions — Chapter 3

Q: How do you define the type of an object in TypeScript?

Answer: There are two main ways:

1. Inline annotation: let user: { name: string; age: number } = { name: 'Rahul', age: 25 };

2. Type alias: type User = { name: string; age: number }; — then use User as the type anywhere.

Type aliases are preferred because they're reusable, readable, and you can reference the type name in multiple places without repeating the structure.

Q: What are optional properties in TypeScript objects?

Answer: Optional properties are marked with ? after the property name. They can be present or absent — TypeScript won't error if they're missing.

Example: type Product = { name: string; description?: string }

Here, name is required. description is optional — you can create a Product without it.

When you ACCESS an optional property, TypeScript warns you it might be undefined — you must check before using it: product.description?.toUpperCase() or use an if check.

Q: What does 'readonly' do in TypeScript? Where would you use it?

Answer: readonly prevents a property from being modified after the object is created. It's like const but for object properties.

Example: type Order = { readonly orderId: string; status: string }

orderId cannot be changed after creation, but status can be updated.

Use it for: IDs (userId, orderId, productId), creation timestamps, configuration values, and any data that represents an immutable fact about an entity.

Q: What are the two ways to type arrays in TypeScript?

Answer: Two syntaxes — both do exactly the same thing:

1. Type[]: let names: string[] — most common, preferred
2. Array<Type>: let names: Array<string> — generic syntax

For simple types, string[] is preferred for readability. For complex or generic types, Array<T> can sometimes be clearer.

Q: What is a Tuple in TypeScript? How is it different from an Array?

Answer: A Tuple is a fixed-length array where each position has a specific, potentially different type.

Array: Variable length, all elements same type. Example: number[] can hold any number of numbers.

Tuple: Fixed length, each position has its own type. Example: [string, number] always has exactly 2 elements — a string at position 0 and a number at position 1.

Real-world use cases: RGB colors [number, number, number], GPS coordinates [latitude: number, longitude: number], React's useState which returns [value, setter] — a tuple!

Q: What is a Type Alias? Why should you use it?

Answer: A type alias gives a name to any type using the 'type' keyword. Example: type User = { name: string; age: number }

Reasons to use type aliases:

1. Reusability — define once, use everywhere
2. Readability — User is clearer than { name: string; age: number } inline
3. Single source of truth — change the type in one place, updates everywhere
4. Composability — combine types using & (intersection) or | (union)

Q: What is the difference between 'readonly' array and a regular array?

Answer: A readonly array cannot be mutated — you cannot push, pop, splice, or modify elements after creation.

Regular: let arr: number[] = [1,2,3]; — arr.push(4) is fine.

Readonly: const arr: readonly number[] = [1,2,3]; — arr.push(4) is an ERROR.

Use readonly arrays for: configuration constants, lookup tables, any data that must remain stable (like a list of supported currencies, allowed roles, or tax rates).

Q: How do you type an array of objects in TypeScript?

Answer: First define a type alias for the object shape, then use it with [] for an array.

Example:

```
type Product = { id: number; name: string; price: number };
let products: Product[] = [
  { id: 1, name: 'Phone', price: 9999 },
  { id: 2, name: 'Laptop', price: 49999 },
];
```

TypeScript then gives you full autocomplete when iterating: products.forEach(p => p.) shows id, name, price. And it errors if you try to access a non-existent property.

Rapid Fire — Chapter 3

Question	One-Line Answer
How to mark a property optional?	Use ? after property name: name?: string
How to make a property immutable?	Use readonly: readonly id: number
Two ways to type arrays?	string[] OR Array<string>
What is a tuple?	Fixed-length array where each position has a specific type
How to type a 2D array?	string[][] — array of string arrays
What is a type alias?	A named type defined with: type Name = { ... }
How to combine two type aliases?	Use & intersection: type C = A & B
Difference: array vs tuple length?	Array: variable length. Tuple: fixed length.

Chapter 3 Key Takeaways

Objects: Use type aliases for reusable object shapes.

Optional (?): Properties that may or may not exist — always check before using.

Readonly: Properties that must never change after creation (IDs, timestamps).

Arrays: string[] or Array<string> — all elements must be the same type.

Readonly Arrays: readonly string[] — contents cannot be modified.

Tuples: [string, number, boolean] — fixed positions, fixed types, fixed length.

Compose types: type Emp = Person & JobDetails — combine with & intersection.

Arrays of objects: type Product = {...}; then Product[] for a list.

End of Chapter 3

CHAPTER 4

Functions in TypeScript

Typed Params & Returns • Optional & Default • Rest • Overloads • Arrow Functions • Callbacks

1. Why Type Your Functions?

Functions are the backbone of any program. In JavaScript, functions are completely untyped — you can pass anything and get anything back. TypeScript changes this — every function can clearly declare what it expects and what it returns.

Real-World Analogy

Think of a function like a VENDING MACHINE.

✅ A good vending machine has labeled slots: Insert ₹10 coin → Get a biscuit.
You know EXACTLY what to put in and what you'll get out.

❌ A bad vending machine accepts anything — coins, paper, your phone.
And gives back random things. Nobody wants this!

TypeScript makes every function a 'labeled vending machine'.
You know exactly what goes IN (parameter types) and what comes OUT (return type).

```
// ── JavaScript function — no idea what goes in or comes out ──
function calculateDiscount(price, discountPercent) {
  return price - (price * discountPercent / 100);
}

// These bugs are invisible in JS:
calculateDiscount('500', 10);     // price is a string! Returns NaN silently
calculateDiscount(500, '10%');    // discountPercent has % sign! NaN again

// ── TypeScript function — crystal clear ──────────────────────
function calculateDiscount(price: number, discountPercent: number): number {
  return price - (price * discountPercent / 100);
}
```

```
calculateDiscount('500', 10);   // ✗ ERROR caught immediately!
calculateDiscount(500, '10%');  // ✗ ERROR caught immediately!
calculateDiscount(500, 10);     // ✓ Returns 450 — correct!
```

2. Parameter Types & Return Types

2.1 Typing Parameters

Every parameter should have a type. If you don't add a type and strict mode is on, TypeScript gives an error: 'Parameter has an implicit any type'.

```
// ── Basic typed parameters ──────────────────────────────────
function greet(name: string, age: number): string {
//             ^^^^^^^^^^^  ^^^^^^^^^^^   ^^^^^^
//             param type   param type    return type
  return `Hi ${name}, you are ${age} years old!`;
}

greet('Rahul', 25);       // ✓ 'Hi Rahul, you are 25 years old!'
greet('Priya', '25');     // ✗ ERROR: '25' is string, not number
greet(25, 'Rahul');       // ✗ ERROR: wrong order of types

// ── Real-world: Payment function ────────────────────────────
function processPayment(
  userId: number,
  amount: number,
  currency: string,
  paymentMethod: string
): boolean {
  console.log(`Processing ₹${amount} for user ${userId}`);
  return true;  // Returns true if payment succeeded
}
```

2.2 Return Types

Always specify what your function returns. If a function returns nothing, use void. TypeScript can often infer return types, but being explicit is good practice — especially for public functions.

```
// ── Explicit return types ───────────────────────────────────
function add(a: number, b: number): number {
  return a + b;           // ✓ Must return a number
```

```
}

function getFullName(first: string, last: string): string {
  return `${first} ${last}`;
}

function isAdult(age: number): boolean {
  return age >= 18;
}

function logError(message: string): void {
  console.error(message); // Returns nothing
}

// ── TypeScript catches wrong return types ──────────────────────
function getAge(): number {
  return 'twenty five'; // ✗ ERROR: string not assignable to number
}

// ── Returning an object ────────────────────────────────────────
type User = { id: number; name: string; email: string };

function createUser(name: string, email: string): User {
  return {
    id: Math.floor(Math.random() * 10000),
    name,
    email,
  };
}

const user = createUser('Rahul', 'rahul@gmail.com');
console.log(user.name);   // ✓ TypeScript knows .name exists
console.log(user.phone);  // ✗ ERROR: 'phone' does not exist on User
```

3. Optional Parameters (?)

Just like optional properties in objects, function parameters can also be optional using `?` . An optional parameter can be omitted when calling the function — its value will be `undefined` if not provided.

Real-World Analogy

Think of ordering a pizza.

Required: Size and crust type — you MUST choose.

Optional: Extra cheese, extra toppings — you can skip these.

Optional function parameters work the same way.

Some are required. Some you can skip — and the function handles it.

```
// ── Optional parameter with ? ──────────────────────────────
function sendEmail(
  to: string,
  subject: string,
  body?: string        // Optional — can be skipped
): void {
  console.log(`To: ${to}`);
  console.log(`Subject: ${subject}`);
  if (body) {
    console.log(`Body: ${body}`);
  } else {
    console.log('Body: (empty)');
  }
}

sendEmail('rahul@gmail.com', 'Hello');           // ✅ body is undefined
sendEmail('priya@gmail.com', 'Hi', 'How are you?'); // ✅ body provided

// ── IMPORTANT: Optional params must come LAST ──────────────
function greet(name: string, greeting?: string, age: number) { }
// ❌ ERROR: Required param 'age' cannot follow optional param 'greeting'

function greet(name: string, age: number, greeting?: string) { }
// ✅ Correct — optional comes last

// ── Real-world: Search function ────────────────────────────
function searchProducts(
  query: string,
  category?: string,   // Optional: filter by category
  maxPrice?: number,   // Optional: filter by max price
  sortBy?: string      // Optional: sort order
): void {
  console.log(`Searching for: ${query}`);
  if (category) console.log(`Category: ${category}`);
```

```
  if (maxPrice) console.log(`Max price: ₹${maxPrice}`);
}

searchProducts('laptop');                      // Just search
searchProducts('laptop', 'Electronics');       // With category
searchProducts('laptop', 'Electronics', 50000); // With price limit
```

4. Default Parameters

Default parameters let you specify a fallback value for a parameter when it's not provided. They're similar to optional parameters but smarter — instead of getting undefined, the parameter gets a default value automatically.

Real-World Analogy

Think of a coffee shop order form.

Size: defaults to 'Medium' if you don't specify.

Milk: defaults to 'regular' if you don't choose oat/almond.

Sugar: defaults to 1 if you don't say how many spoons.

You only need to specify when you want something DIFFERENT from the default.

```
// ── Default parameter syntax ──────────────────────────────
function createOrder(
  item: string,
  quantity: number = 1,          // Default: 1
  priority: string = 'normal',   // Default: 'normal'
): void {
  console.log(`Order: ${quantity}x ${item} [${priority}]`);
}

createOrder('Burger');                 // Order: 1x Burger [normal]
createOrder('Pizza', 2);              // Order: 2x Pizza [normal]
createOrder('Biryani', 3, 'urgent'); // Order: 3x Biryani [urgent]

// ── Real-world: Pagination ────────────────────────────────
function getProducts(
  page: number = 1,
  limit: number = 10,
  sortBy: string = 'createdAt',
  order: string = 'desc'
```

```
): void {
  console.log(`Page ${page}, ${limit} items, sorted by ${sortBy} ${order}`);
}

getProducts();              // Page 1, 10 items, sorted by createdAt desc
getProducts(2);             // Page 2, 10 items, sorted by createdAt desc
getProducts(1, 20);        // Page 1, 20 items, sorted by createdAt desc
getProducts(1, 20, 'price', 'asc'); // Page 1, 20 items, sorted by price asc

// ── Default vs Optional: Key Difference ────────────────────
function optionalExample(name: string, greeting?: string) {
  // greeting is undefined if not passed — must check before use
  console.log(greeting?.toUpperCase()); // might be undefined
}

function defaultExample(name: string, greeting: string = 'Hello') {
  // greeting is ALWAYS a string — 'Hello' if not passed
  console.log(greeting.toUpperCase()); // ALWAYS safe — never undefined
}
```

Feature	Optional Parameter (?)	Default Parameter (= value)
Value when not provided	undefined	The default value you set
Must check before use?	Yes — might be undefined	No — always has a value
Syntax	param?: string	param: string = 'Hello'
Shows in call signature?	Can be omitted	Can be omitted
Use when	Parameter is truly optional	You have a sensible fallback

5. Rest Parameters

Rest parameters let a function accept any number of arguments. They collect all remaining arguments into an **array**. Use `...` before the parameter name.

Real-World Analogy

Think of a WhatsApp group message.

You send one message but it goes to MULTIPLE people — 2, 5, 50 — any number.

A rest parameter works the same way:
'sendMessage(message, ...recipients)' — one message, any number of recipients.

```
// ── Basic rest parameter ──────────────────────────────────
function sum(...numbers: number[]): number {
  return numbers.reduce((total, n) => total + n, 0);
}

sum(1, 2);           // 3
sum(10, 20, 30);     // 60
sum(1, 2, 3, 4, 5); // 15
// Any number of arguments — all collected into numbers[]

// ── Rest parameter with regular parameters ────────────────
function sendNotification(
  message: string,          // Required: the message
  ...userIds: number[]      // Rest: any number of user IDs
): void {
  console.log(`Sending '${message}' to ${userIds.length} users`);
  userIds.forEach(id => console.log(`  -> User ${id}`));
}

sendNotification('Sale is live!', 101, 102, 103, 104);
// Sending 'Sale is live!' to 4 users
//   -> User 101
//   -> User 102 ...

// ── Real-world: Cart total calculator ─────────────────────
type Product = { name: string; price: number };

function calculateCartTotal(
  taxPercent: number,
  ...products: Product[]
): number {
  const subtotal = products.reduce((sum, p) => sum + p.price, 0);
  const tax = subtotal * taxPercent / 100;
  return subtotal + tax;
}

const total = calculateCartTotal(
  18,  // 18% GST
  { name: 'Shirt', price: 799 },
```

```
  { name: 'Jeans', price: 1499 },
  { name: 'Shoes', price: 2499 },
);
console.log(`Total with tax: ₹${total}`);  // ₹5,651.06

// ── RULE: Rest param must always be the LAST parameter ────────
function wrong(...nums: number[], extra: string) { }  // ✗ ERROR
function right(extra: string, ...nums: number[]) { }  // ✓ Fine
```

6. Arrow Functions

Arrow functions are the shorter syntax for writing functions. TypeScript fully supports arrow functions — you can add types to parameters and return types just like regular functions.

```
// ── Regular function vs Arrow function ──────────────────────

// Regular function:
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function — same thing, shorter syntax:
const add = (a: number, b: number): number => {
  return a + b;
};

// Even shorter — single expression, implicit return:
const add = (a: number, b: number): number => a + b;

// ── Arrow functions with various return types ────────────────
const greet = (name: string): string => `Hello, ${name}!`;

const isEven = (n: number): boolean => n % 2 === 0;

const logMessage = (msg: string): void => console.log(msg);

// ── Arrow function returning an object ───────────────────────
// Wrap the object in parentheses to avoid confusion with {}
const createUser = (name: string, age: number): object =>
  ({ name, age, createdAt: new Date() });
//  ^                                  ^
```

```
//  These parens are needed so TS knows it's an object, not a code block

// ── Real-world: Array methods with arrow functions ────────────
type Product = { name: string; price: number; inStock: boolean };

const products: Product[] = [
  { name: 'Phone', price: 9999, inStock: true },
  { name: 'Laptop', price: 49999, inStock: false },
  { name: 'Tablet', price: 19999, inStock: true },
];

// filter — TypeScript knows p is Product
const availableProducts = products.filter(
  (p: Product): boolean => p.inStock
);

// map — TypeScript knows we get string[]
const productNames: string[] = products.map(
  (p: Product): string => p.name
);

// find — TypeScript knows result is Product | undefined
const cheapest = products.find(
  (p: Product): boolean => p.price < 15000
);

// sort by price ascending
const sorted = [...products].sort(
  (a: Product, b: Product): number => a.price - b.price
);
```

7. Function Types

In TypeScript, functions are values — you can store them in variables, pass them as parameters, and return them from other functions. Function types let you describe the shape of a function.

Real-World Analogy

Think of a function type like a JOB DESCRIPTION.

'Needs someone who takes an order number and returns a delivery status.'

Input: number. Output: string.

Anyone who can do that job qualifies — regardless of their name or how they do it.
Function types describe the 'job' a function must be able to do.

```
// ── Function type syntax ──────────────────────────────────
// (param: type, param: type) => returnType

// Storing a function in a variable with its type:
const multiply: (a: number, b: number) => number = (a, b) => a * b;

// Using a type alias for a function type (much cleaner!):
type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;
const subtract: MathOperation = (a, b) => a - b;
const divide: MathOperation = (a, b) => a / b;

// ── Function as a parameter (callback) ────────────────────
function applyOperation(
  a: number,
  b: number,
  operation: MathOperation   // Function as a parameter!
): number {
  return operation(a, b);
}

applyOperation(10, 5, add);       // 15
applyOperation(10, 5, subtract);  // 5
applyOperation(10, 5, multiply);  // 50

// ── Real-world: Event handler types ───────────────────────
type ClickHandler = (event: MouseEvent) => void;
type ChangeHandler = (value: string) => void;
type SubmitHandler = (formData: Record<string, string>) => Promise<void>;

// ── Real-world: Callback in async operations ──────────────
type OnSuccess = (data: User) => void;
type OnError = (error: Error) => void;

type User = { id: number; name: string };
```

```
function fetchUser(
  userId: number,
  onSuccess: OnSuccess,
  onError: OnError
): void {
  // Simulating API call
  if (userId > 0) {
    onSuccess({ id: userId, name: 'Rahul' });
  } else {
    onError(new Error('Invalid user ID'));
  }
}

fetchUser(
  101,
  (user) => console.log(`Welcome, ${user.name}!`),
  (err)  => console.error(`Error: ${err.message}`)
);
```

8. Function Overloads

Function overloads let you define MULTIPLE call signatures for the same function. This is useful when a function behaves differently depending on what types of arguments you pass to it.

Real-World Analogy

Think of a universal remote control.

Press POWER on a TV → turns TV on/off
Press POWER on an AC → turns AC on/off
Press POWER on a speaker → turns speaker on/off

Same button (function name), different behavior based on context (input type).

Function overloading lets you describe all possible call signatures clearly.

```
// ── Function Overload Syntax ──────────────────────────────
// Step 1: Write overload signatures (no body)
function formatInput(input: string): string;
function formatInput(input: number): string;
function formatInput(input: boolean): string;
```

```
// Step 2: Write the implementation signature (with body)
// Implementation must handle ALL overload cases
function formatInput(input: string | number | boolean): string {
  if (typeof input === 'string')  return input.trim().toUpperCase();
  if (typeof input === 'number')  return input.toFixed(2);
  if (typeof input === 'boolean') return input ? 'Yes' : 'No';
  return '';
}

formatInput('  hello  ');  // 'HELLO'
formatInput(3.14159);      // '3.14'
formatInput(true);         // 'Yes'

// ── Real-world: Date formatting ──────────────────────────────
function formatDate(date: Date): string;
function formatDate(timestamp: number): string;
function formatDate(dateString: string): string;

function formatDate(input: Date | number | string): string {
  const date = input instanceof Date
    ? input
    : typeof input === 'number'
      ? new Date(input)
      : new Date(input);

  return date.toLocaleDateString('en-IN', {
    day: '2-digit',
    month: 'short',
    year: 'numeric',
  });
}

formatDate(new Date());           // '15 Dec 2024'
formatDate(1734000000000);        // '12 Dec 2024'
formatDate('2024-12-15');         // '15 Dec 2024'

// ── Real-world: getElementById-like function ─────────────────
// Returns different types based on what selector is passed
function getElement(id: string): HTMLElement | null;
function getElement(index: number): HTMLElement | null;

function getElement(selector: string | number): HTMLElement | null {
```

```
  if (typeof selector === 'string') {
    return document.getElementById(selector);
  }
  return document.querySelectorAll('*')[selector] as HTMLElement ?? null;
}
```

9. async Functions & Promise Return Types

Modern TypeScript code uses async/await everywhere. Async functions always return a Promise — TypeScript lets you type exactly what the Promise resolves to.

Real-World Analogy

A Promise is like ordering food online.

You place the order now (call the async function).
You get a 'tracking number' immediately (the Promise object).
Food arrives later (Promise resolves with the data).
Sometimes the order fails (Promise rejects with an error).

Promise<User> means: 'This will eventually give you a User object.'

```
// ── Basic async function type ──────────────────────────────
type User = { id: number; name: string; email: string };

// Returns a Promise that resolves to a User
async function fetchUser(userId: number): Promise<User> {
  const response = await fetch(`/api/users/${userId}`);
  const data = await response.json();
  return data as User;
}

// Returns a Promise that resolves to User[]
async function fetchAllUsers(): Promise<User[]> {
  const response = await fetch('/api/users');
  return response.json();
}

// Returns a Promise that resolves to nothing
async function deleteUser(userId: number): Promise<void> {
```

```
  await fetch(`/api/users/${userId}`, { method: 'DELETE' });
  // No return value
}

// ── Real-world: Complete API service ──────────────────────
type Product = { id: number; name: string; price: number };
type ApiError = { code: number; message: string };

// Returns Product on success, throws ApiError on failure
async function getProductById(id: number): Promise<Product> {
  try {
    const response = await fetch(`/api/products/${id}`);
    if (!response.ok) {
      const err: ApiError = await response.json();
      throw new Error(err.message);
    }
    return response.json();
  } catch (error) {
    throw error;
  }
}

// ── Using async functions ─────────────────────────────────
async function displayProduct(id: number): Promise<void> {
  try {
    const product = await getProductById(id);
    // TypeScript KNOWS product is type Product
    console.log(`${product.name}: ₹${product.price}`);
  } catch (error) {
    console.error('Failed to load product');
  }
}
```

10. The 'never' in Functions — Exhaustive Checks Revisited

Remember never from Chapter 2? Here's a very practical function pattern using never to ensure you handle ALL cases in a switch statement. This is called an exhaustive check.

```
// ── Exhaustive check pattern ──────────────────────────────
type PaymentStatus = 'pending' | 'completed' | 'failed' | 'refunded';
```

```
// This helper function takes 'never' — if execution ever reaches here,
// TypeScript knows you missed a case.
function assertNever(value: never, message: string): never {
  throw new Error(`${message}: ${JSON.stringify(value)}`);
}

function getStatusMessage(status: PaymentStatus): string {
  switch (status) {
    case 'pending':   return 'Your payment is being processed...';
    case 'completed': return 'Payment successful! 🎉';
    case 'failed':    return 'Payment failed. Please try again.';
    case 'refunded':  return 'Your refund has been initiated.';
    default:
      return assertNever(status, 'Unhandled payment status');
      // ↑ If you add 'cancelled' to PaymentStatus but forget to add
      // a case here, TypeScript ERRORS — 'cancelled' is not assignable
      // to type 'never'. You cannot miss a case silently!
  }
}
```

11. Function Best Practices

TypeScript Function Best Practices

1. Always type function parameters — never rely on implicit 'any'.

2. Write return types for public/exported functions — it's self-documentation.

3. Use default parameters instead of optional when you have a sensible default.

4. Keep rest params (...args) as the last parameter always.

5. Use type aliases for function types: type Handler = (e: Event) => void

6. Prefer arrow functions for callbacks and short inline functions.

7. Use async/await with explicit Promise<T> return types.

8. For overloads: write clear overload signatures, then a single implementation.

9. Use the exhaustive check pattern with never in switch statements.

10. Keep functions small and focused — one function, one responsibility.

12. Complete Real-World Example — E-Commerce Utility Functions

Let's see all function concepts together in one realistic example — a product service for an online store.

```
// ── types ──────────────────────────────────────────────────
type Product = {
  id: number;
  name: string;
  price: number;
  category: string;
  rating: number;
  inStock: boolean;
};

type SortOption = 'price_asc' | 'price_desc' | 'rating' | 'name';
type FilterResult = { products: Product[]; total: number; page: number };

// ── Function with optional + default params ──────────────────
function filterProducts(
  products: Product[],
  category?: string,          // Optional filter
  maxPrice: number = Infinity, // Default: no price limit
  minRating: number = 0,       // Default: any rating
  inStockOnly: boolean = false  // Default: show all
): Product[] {
  return products.filter(p => {
    if (category && p.category !== category) return false;
    if (p.price > maxPrice) return false;
    if (p.rating < minRating) return false;
    if (inStockOnly && !p.inStock) return false;
    return true;
```

```
  });
}

// ── Arrow function for sorting ──────────────────────────────
const sortProducts = (products: Product[], by: SortOption): Product[] => {
  return [...products].sort((a, b) => {
    switch (by) {
      case 'price_asc':  return a.price - b.price;
      case 'price_desc': return b.price - a.price;
      case 'rating':     return b.rating - a.rating;
      case 'name':       return a.name.localeCompare(b.name);
    }
  });
};

// ── Rest parameters for bulk operations ─────────────────────
function applyDiscounts(
  discountPercent: number,
  ...productIds: number[]
): void {
  console.log(`Applying ${discountPercent}% discount to ${productIds.length}
products`);
}

// ── Overload: getPrice accepts ID or name ───────────────────
function getPrice(productId: number): number | null;
function getPrice(productName: string): number | null;
function getPrice(input: number | string): number | null {
  // Implementation...
  return null;
}

// ── Async function with typed Promise ───────────────────────
async function fetchPagedProducts(
  page: number = 1,
  limit: number = 20
): Promise<FilterResult> {
  const res = await fetch(`/api/products?page=${page}&limit=${limit}`);
  return res.json();
}

// ── Usage ───────────────────────────────────────────────────
const products: Product[] = []; // imagine this is filled from API
```

```
const mobiles = filterProducts(products, 'Electronics', 30000, 4.0, true);
const sorted  = sortProducts(mobiles, 'price_asc');
applyDiscounts(20, 101, 102, 103, 201);
const price = getPrice('iPhone 15');  // overload — string version
const price2 = getPrice(101);          // overload — number version
```

13. Interview Questions — Chapter 4

Q: How do you type function parameters and return types in TypeScript?

Answer: You add a type annotation after each parameter name using a colon, and specify the return type after the closing parenthesis with a colon.

Example: function add(a: number, b: number): number { return a + b; }

If the function returns nothing, use void as the return type.

TypeScript can often infer return types, but writing them explicitly is better practice for public functions — it serves as documentation and TypeScript will error if your implementation returns the wrong type.

Q: What is the difference between optional and default parameters?

Answer: Optional parameters (marked with ?) can be omitted — their value is undefined when not provided. You must check before using them.

Default parameters have a fallback value — if not provided, they use the default. They are never undefined.

Example:

Optional: function greet(name: string, greeting?: string) — greeting may be undefined

Default: function greet(name: string, greeting: string = 'Hello') — greeting is always a string

Rule: Use default when you have a sensible fallback. Use optional when the value is truly not needed.

Q: What are rest parameters? When would you use them?

Answer: Rest parameters (using ...) allow a function to accept any number of arguments. They collect all extra arguments into an array.

Syntax: function sum(...numbers: number[]): number

Use them when: you don't know in advance how many arguments will be passed.

Examples: logging function that accepts multiple values, a notification sender that takes any number of user IDs, or a math utility that adds any number of numbers.

Important rule: Rest parameters must always be the LAST parameter.

Q: What is a function type in TypeScript? How do you define one?

Answer: A function type describes the shape of a function — what parameters it accepts and what it returns. You can define function types inline or using type aliases.

Inline: const add: (a: number, b: number) => number = (a, b) => a + b;

Type alias (preferred): type MathOp = (a: number, b: number) => number;

Function types are useful when passing functions as parameters (callbacks), storing functions in variables, or defining an array/object of functions.

Q: What are function overloads? When do you need them?

Answer: Function overloads let you define multiple call signatures for the same function — different input types give different output types (or behaviors).

You write the overload signatures first (without body), then write one implementation that handles all cases.

Use overloads when: a function can accept fundamentally different types of input and the return type changes accordingly. Example: a formatDate function that can accept a Date, a timestamp number, or a date string.

Important: The implementation signature is NOT directly callable — only the overload signatures are visible to callers.

Q: What is the return type of an async function?

Answer: Async functions always return a Promise. The TypeScript return type is Promise<T>, where T is the type of the value the Promise resolves to.

Examples:

```
async function getUser(): Promise<User> — resolves to a User object
async function getAll(): Promise<User[]> — resolves to an array of Users
async function deleteItem(): Promise<void> — resolves to nothing
```

This is important because TypeScript then knows what type you'll receive when you await the function — giving you full type safety in async code.

Q: Can you pass a function as a parameter in TypeScript?

Answer: Yes! Functions are first-class values in TypeScript (and JavaScript). You can pass them as parameters, return them, and store them in variables.

To type a function parameter, use a function type:

```
function run(callback: (value: number) => void): void { callback(42); }
```

Or use a type alias: `type Callback = (value: number) => void;`

This is used extensively for event handlers, array methods (map, filter, sort), and callback-based APIs.

Q: What is the difference between function declaration and function expression in TypeScript?

Answer: Both work the same way in terms of types, but have some differences:

Function declaration: `function add(a: number, b: number): number { return a + b; }`

- Hoisted — can be called before its definition in the file
- Named — easier to see in stack traces

Function expression: `const add = (a: number, b: number): number => a + b;`

- Not hoisted — must be defined before use
- Can be stored in variables, passed around
- Arrow functions also have no 'this' binding (important in classes)

In practice: use regular functions for module-level utilities, arrow functions for callbacks and class methods.

Rapid Fire — Chapter 4

Question	One-Line Answer
Where must optional params go?	After all required params: fn(req, opt?)
Where must rest params go?	Last: fn(a: string, ...rest: number[])
Difference: void vs never return?	void = finishes but no value. never = never finishes.
Return type of async function?	Promise<T> — e.g. Promise<User>
How many rest params per function?	Only one, and it must be last
Can arrow functions be overloaded?	No — only regular function declarations can be overloaded
What is a callback?	A function passed as an argument to another function
How to type a void callback?	type Cb = () => void

Chapter 4 Key Takeaways

Always type parameters and return types — never rely on implicit any.

Optional params (?): might be undefined — check before use.

Default params (= value): always has a value — never undefined.

Rest params (...arr): collects extra args into array — always last.

Arrow functions: shorter syntax, same typing rules as regular functions.

Function types: (a: number) => string — describe a function's signature.

Callbacks: functions passed as parameters — use function types to type them.

Overloads: multiple signatures, one implementation — for versatile functions.

async functions: always return Promise<T> — fully type-safe async code.

Exhaustive switch + never: ensures you handle every possible case.

End of Chapter 4

CHAPTER 5

Interfaces vs Type Aliases

Differences • Extending • Declaration Merging • When to Use What

1. The Big Picture — Two Ways to Name a Type

TypeScript gives you TWO main tools to describe the shape of an object or a type: interface and type. At first glance they look almost identical — and for most daily use they ARE interchangeable. But they have important differences that you MUST know for interviews.

Simple Analogy — Two Architects, Same Blueprint

Imagine you need a blueprint for a building.

Architect A (interface) uses a special 'official blueprint form'.
It can be extended later — other architects can ADD floors to the same blueprint.
Multiple blueprints with the same name automatically MERGE together.

Architect B (type alias) writes a custom specification document.
More flexible — can describe buildings, vehicles, even abstract concepts.
But once written, it cannot be re-opened and extended with the same name.

Both produce a valid building description.
But they have different rules for how they can be modified and combined.

```
// ── interface ──────────────────────────────────────────────
interface User {
  name: string;
  age: number;
  email: string;
}

// ── type alias ─────────────────────────────────────────────
type User = {
  name: string;
  age: number;
  email: string;
};
```

```
// For simple objects like this — they work IDENTICALLY.
// The variable below works with EITHER definition above:
const user: User = { name: 'Rahul', age: 28, email: 'r@gmail.com' };
```

2. interface — Deep Dive

2.1 Basic Interface Syntax

```
// ── Defining an interface ──────────────────────────────────
interface Product {
  id: number;
  name: string;
  price: number;
  inStock: boolean;
  description?: string;      // Optional property
  readonly createdAt: Date;  // Readonly property
}

// Using the interface
const phone: Product = {
  id: 1,
  name: 'iPhone 15',
  price: 79999,
  inStock: true,
  createdAt: new Date(),
};

// Interface for a function signature
interface Formatter {
  (value: number, currency: string): string;
}

const formatPrice: Formatter = (value, currency) =>
  `${currency}${value.toLocaleString()}`;

console.log(formatPrice(79999, '₹'));  // ₹79,999
```

2.2 Extending Interfaces — The 'extends' Keyword

Interfaces support **inheritance** — one interface can extend another, inheriting all its properties and adding new ones. This is perfect for building type hierarchies.

Real-World Analogy

Think of job roles in a company:

Every Employee has: name, id, department
A Manager is an Employee + has: teamSize, budget
A Director is a Manager + has: division, reportingTo

Each level INHERITS everything from above and ADDS more.
That's exactly how interface extends works!

```
// ── Base interface ──────────────────────────────────────────
interface Animal {
  name: string;
  age: number;
  makeSound(): void;
}

// ── Extending a single interface ────────────────────────────
interface Dog extends Animal {
  breed: string;
  isVaccinated: boolean;
}

// Dog now has: name, age, makeSound (from Animal) + breed, isVaccinated
const myDog: Dog = {
  name: 'Bruno',
  age: 3,
  makeSound() { console.log('Woof!'); },
  breed: 'Labrador',
  isVaccinated: true,
};

// ── Extending MULTIPLE interfaces ───────────────────────────
interface Flyable {
  maxAltitude: number;
  fly(): void;
}
```

```
interface Swimmable {
  maxDepth: number;
  swim(): void;
}

// Duck can fly AND swim
interface Duck extends Animal, Flyable, Swimmable {
  color: string;
}

// Duck now has ALL properties from Animal + Flyable + Swimmable + its own
const duck: Duck = {
  name: 'Donald',
  age: 2,
  makeSound() { console.log('Quack!'); },
  maxAltitude: 100,
  fly() { console.log('Flying!'); },
  maxDepth: 5,
  swim() { console.log('Swimming!'); },
  color: 'white',
};
```

Real-World Scenario: Real-World: Company Employee Hierarchy

In a large org like Infosys or TCS, Employee is the base. Engineer extends Employee with skills[]. SeniorEngineer extends Engineer with mentoring. TeamLead extends SeniorEngineer with teamMembers[]. Each level adds fields without rewriting the common ones.

```
// ── Real-world: Employee hierarchy ──────────────────────────
interface Employee {
  readonly employeeId: string;
  name: string;
  email: string;
  department: string;
  salary: number;
  joinDate: Date;
}

interface Engineer extends Employee {
  skills: string[];
  githubProfile?: string;
  currentProject: string;
}
```

```
interface TeamLead extends Engineer {
  teamSize: number;
  teamMembers: string[];  // array of employeeIds
  conductsReviews: boolean;
}

interface Manager extends Employee {
  directReports: string[];
  budget: number;
  okrTargets: string[];
}

// TeamLead has everything from Employee + Engineer + its own fields
const lead: TeamLead = {
  employeeId: 'EMP-1042',
  name: 'Priya Sharma',
  email: 'priya@company.com',
  department: 'Engineering',
  salary: 180000,
  joinDate: new Date('2020-06-01'),
  skills: ['TypeScript', 'React', 'Node.js'],
  currentProject: 'Payment Gateway v2',
  teamSize: 6,
  teamMembers: ['EMP-1043', 'EMP-1044', 'EMP-1045'],
  conductsReviews: true,
};
```

2.3 Declaration Merging — The Superpower of interface

This is something **ONLY interfaces** can do — and it's a key interview topic. If you declare an interface with the same name TWICE, TypeScript automatically MERGES them into one combined interface.

Real-World Analogy

Imagine a shared Google Doc for project requirements.

Team A writes: 'The user must have a name and email.'

Team B later opens the SAME document and adds: 'The user must also have a phone.'

Result: The document now has name, email, AND phone — it merged both contributions.

That's declaration merging — multiple interface declarations with the same name

are automatically combined into one by TypeScript.

```
// ── Declaration Merging ──────────────────────────────────────
// First declaration
interface Window {
  title: string;
}

// Second declaration — SAME name!
interface Window {
  location: string;
  history: string[];
}

// TypeScript MERGES them:
// interface Window {
//   title: string;       (from first declaration)
//   location: string;    (from second declaration)
//   history: string[];   (from second declaration)
// }

const w: Window = {
  title: 'Home Page',
  location: '/home',
  history: ['/login', '/dashboard'],
};

// ── REAL WORLD USE: Extending third-party library types ───────
// Express.js defines Request like this internally:
// interface Request { url: string; method: string; ... }

// You can ADD your own fields to it without modifying the library!
interface Request {
  user?: { id: number; role: string };  // Added by your auth middleware
  requestId?: string;                    // Added by your logging middleware
}

// Now everywhere in your app, req.user is available with TypeScript support!
// This is called 'module augmentation' — super useful in real projects.

// ── type alias CANNOT do this ────────────────────────────────
type Config = { host: string };
```

```
type Config = { port: number };  // ✗ ERROR: Duplicate identifier 'Config'
```

3. type Alias — Deep Dive

3.1 type Can Do Things interface Cannot

While interface is great for objects, **type alias is more flexible** — it can name ANY type: primitives, unions, intersections, tuples, and more. interface can only describe object shapes.

```
// ── type alias for primitives ──────────────────────────────
type ID = string;           // Just a renamed string
type Age = number;          // Just a renamed number
type Status = string;

// ── type alias for union types ─────────────────────────────
// interface CANNOT do this!
type StringOrNumber = string | number;
type PaymentMethod = 'cash' | 'card' | 'upi' | 'netbanking';
type NullableUser = User | null;
type ID = string | number;   // ID can be either

// ── type alias for intersection ────────────────────────────
type PersonalInfo = { name: string; age: number };
type ContactInfo  = { email: string; phone: string };

// Combined type — must have ALL properties
type FullProfile = PersonalInfo & ContactInfo;

const profile: FullProfile = {
  name: 'Rahul',
  age: 28,
  email: 'rahul@gmail.com',
  phone: '9876543210',
};

// ── type alias for tuples ──────────────────────────────────
// interface CANNOT do this!
type Coordinate = [latitude: number, longitude: number];
type RGBColor    = [r: number, g: number, b: number];
type APIResponse = [statusCode: number, data: unknown, error: string | null];

// ── type alias for functions ───────────────────────────────
```

```
type EventHandler  = (event: MouseEvent) => void;
type Comparator<T> = (a: T, b: T) => number;

// ── type alias for mapped/conditional types ──────────────────
// (Advanced — we'll cover these in Chapter 12)
type Optional<T> = { [K in keyof T]?: T[K] };
type ReadonlyUser = Readonly<User>;
```

3.2 Extending Type Aliases — Using Intersection (&)

type aliases can be **extended** using the `&` (intersection) operator. The result must have ALL properties from all intersected types.

```
// ── Extending with & ──────────────────────────────────────────
type Vehicle = {
  make: string;
  model: string;
  year: number;
};

type ElectricVehicle = Vehicle & {
  batteryCapacity: number;  // kWh
  chargingTime: number;     // hours
  rangeKm: number;
};

const tesla: ElectricVehicle = {
  make: 'Tesla',
  model: 'Model 3',
  year: 2024,
  batteryCapacity: 75,
  chargingTime: 8,
  rangeKm: 560,
};

// ── Chaining multiple intersections ──────────────────────────
type Auditable = {
  createdAt: Date;
  updatedAt: Date;
  createdBy: string;
}

type SoftDeletable = {
```

```
  isDeleted: boolean;
  deletedAt?: Date;
};

// A full database entity has all three:
type DbEntity<T> = T & Auditable & SoftDeletable;

type UserEntity = DbEntity<{
  id: number;
  name: string;
  email: string;
}>;

// UserEntity now has: id, name, email, createdAt, updatedAt,
// createdBy, isDeleted, deletedAt
```

4. interface vs type — Side-by-Side Comparison

Feature	interface	type alias
Describe object shapes	✅ Yes — primary use case	✅ Yes
Primitive types	❌ No	✅ Yes — type Age = number
Union types	❌ No	✅ Yes — string | number
Intersection / combine	✅ Yes — via extends	✅ Yes — via &
Extend another type	✅ extends keyword	✅ & intersection
Extend an interface	✅ interface A extends B	✅ type A = B & {...}
Extend a type alias	✅ interface A extends B {}	✅ type A = B & {...}
Declaration merging	✅ Yes — unique to interface!	❌ No — duplicate name = error
Tuples	❌ No	✅ Yes — [string, number]
Mapped types	❌ No	✅ Yes — { [K in T]: ... }
Conditional types	❌ No	✅ Yes — T extends U ? A : B
Function types	✅ Yes (call signature)	✅ Yes (arrow syntax)
Implement in a class	✅ class Dog implements Animal	✅ class Dog implements Animal
Error messages	Shows interface name clearly	Shows expanded type (verbose)

Performance	Slightly faster for large types	Can be slower with complex types

5. When to Use What — The Decision Guide

5.1 Use interface when…

```
// ✅ 1. Describing the shape of an OBJECT or CLASS ──────────────
interface UserRepository {
  findById(id: number): Promise<User | null>;
  findAll(): Promise<User[]>;
  save(user: User): Promise<User>;
  delete(id: number): Promise<void>;
}

// ✅ 2. When you need declaration merging (library augmentation) —
// Augmenting Express Request with your custom fields
interface Request {
  currentUser?: { id: number; role: 'admin' | 'user' };
}

// ✅ 3. Defining contracts a CLASS must implement ─────────────
interface Serializable {
  serialize(): string;
  deserialize(data: string): void;
}

class UserModel implements Serializable {
  serialize(): string {
    return JSON.stringify(this);
  }
  deserialize(data: string): void {
    Object.assign(this, JSON.parse(data));
  }
}

// ✅ 4. Building type hierarchies with clear inheritance ─────────
interface Shape {
  color: string;
  area(): number;
```

```
}
interface Circle extends Shape {
  radius: number;
}
interface Rectangle extends Shape {
  width: number;
  height: number;
}
```

5.2 Use type alias when…

```
// ✅ 1. Union types — interface CANNOT do this ─────────────────
type Theme = 'light' | 'dark' | 'system';
type InputType = 'text' | 'email' | 'password' | 'number' | 'tel';
type OrderStatus = 'placed' | 'confirmed' | 'shipped' | 'delivered' |
'cancelled';

// ✅ 2. Tuples ──────────────────────────────────────────────────
type LatLng = [number, number];
type DateRange = [startDate: Date, endDate: Date];

// ✅ 3. Renaming primitives for semantic clarity ────────────────
type UserID    = string;   // Now 'UserID' is more meaningful than 'string'
type ProductID = number;
type Email     = string;
type Currency  = 'INR' | 'USD' | 'EUR';

function transferMoney(
  from: UserID,
  to: UserID,
  amount: number,
  currency: Currency
): void { }
// Much clearer than: function transferMoney(from: string, to: string, ...)

// ✅ 4. Complex intersections / utility type compositions ──────
type CreateProductDto = Omit<Product, 'id' | 'createdAt'>;
type UpdateProductDto = Partial<Pick<Product, 'name' | 'price' | 'inStock'>>;

// ✅ 5. Conditional types and mapped types ─────────────────────
type NonNullable<T> = T extends null | undefined ? never : T;
type Readonly<T> = { readonly [P in keyof T]: T[P] };
```

```
// ✅ 6. Function types ────────────────────────────────────────
type RequestHandler = (req: Request, res: Response) => void;
type Middleware = (req: Request, res: Response, next: () => void) => void;
```

The Simple Rule — When in Doubt

Use INTERFACE when:

→ You are describing an object or class contract

→ You need extends / implements

→ You are writing a library/SDK others will use (declaration merging)

Use TYPE ALIAS when:

→ You need union types: string | number

→ You need tuples: [string, number]

→ You need to rename a primitive for clarity: type ID = string

→ You are using utility types: Partial<T>, Pick<T,K>, Omit<T,K>

→ You are doing mapped / conditional types

For MOST everyday object shapes:

Both work — pick one and be CONSISTENT in your codebase.

Many teams use type for everything. Many use interface for objects.

The official TS docs now lean towards interface for objects.

6. extends vs & — The Key Difference

Both interface extends and type & achieve extension — but they behave differently when there are conflicting property types.

```
// ── Conflict handling: extends vs & ──────────────────────────

// ── With interface extends: CONFLICT = ERROR ─────────────────
interface A {
  value: string;
}

interface B extends A {
  value: number;  // ✗ ERROR: 'value' property types are incompatible
}               //  string cannot be overridden with number
```

```
// interface explicitly checks for conflicts and REJECTS them.

// ── With type &: CONFLICT = never ──────────────────────────
type C = {
  value: string;
};

type D = C & {
  value: number;  // No immediate error from TypeScript here
};

// But what happens to D.value?
// value: string & number = never  (impossible type!)

const d: D = { value: 'hello' }; // ✗ ERROR: string is not assignable to never
const d2: D = { value: 42 };     // ✗ ERROR: number is not assignable to never
// You can NEVER create a valid D — the type is broken

// ── LESSON: ────────────────────────────────────────────────
// interface extends fails LOUDLY at the point of conflict
// type & fails SILENTLY by creating 'never' — harder to debug!
// interface extends is SAFER when you might have conflicts.
```

🏢 Real-World Scenario: Real-World: API Response Types

In a REST API client, you might extend a base ApiResponse for each endpoint. Using interface extends for response types catches conflicts immediately if two API designers accidentally define the same field with different types — instead of silently creating a 'never' type that explodes at runtime.

7. Declaration Merging in Detail

Let's look at declaration merging more deeply — this is a powerful real-world pattern used in many popular libraries.

7.1 Merging Interfaces in Your Own Code

```
// ── Gradually building up an interface ────────────────────
// In config.ts:
interface AppConfig {
  appName: string;
```

```
  version: string;
}

// In database.ts (same project):
interface AppConfig {
  dbHost: string;
  dbPort: number;
  dbName: string;
}

// In auth.ts (same project):
interface AppConfig {
  jwtSecret: string;
  tokenExpiry: number;
}

// TypeScript merges ALL of these automatically:
// interface AppConfig {
//   appName: string;   version: string;
//   dbHost: string;    dbPort: number;    dbName: string;
//   jwtSecret: string; tokenExpiry: number;
// }

const config: AppConfig = {
  appName: 'ShopApp',
  version: '2.0.1',
  dbHost: 'localhost',
  dbPort: 5432,
  dbName: 'shopdb',
  jwtSecret: 'super-secret',
  tokenExpiry: 3600,
};
```

7.2 Module Augmentation — Extending Library Types

This is the most powerful real-world use of declaration merging — adding your own properties to types from third-party libraries.

```
// ── Real-world: Extending Express.js types ──────────────────────

// Express defines this internally:
// namespace Express {
//   interface Request { ... }
```

```
// }

// In your types/express.d.ts file, you can ADD to it:
declare namespace Express {
  interface Request {
    // Your auth middleware adds these:
    user?: {
      id: number;
      email: string;
      role: 'admin' | 'user' | 'moderator';
    };
    // Your request ID middleware adds this:
    requestId?: string;
    // Your rate limiting middleware adds this:
    rateLimit?: {
      remaining: number;
      resetTime: Date;
    };
  }
}

// Now in your route handlers, you get full TypeScript support:
app.get('/profile', (req, res) => {
  if (!req.user) {
    return res.status(401).json({ error: 'Not authenticated' });
  }
  // TypeScript KNOWS req.user exists here and has id, email, role
  res.json({ userId: req.user.id, role: req.user.role });
});

// ── Real-world: Extending Window object ────────────────────
// Adding Google Analytics to the window object:
interface Window {
  gtag: (command: string, ...args: unknown[]) => void;
  dataLayer: unknown[];
}

// Now window.gtag is typed — no more 'any'!
window.gtag('event', 'purchase', { value: 1299 });
```

8. Implementing Interfaces in Classes

Classes can **implement** interfaces — this means the class MUST have all the properties and methods defined in the interface. It's like signing a contract.

Real-World Analogy

A 'Certified Restaurant' must meet certain standards:

→ Must have a kitchen

→ Must have food safety certification

→ Must have a menu

Any restaurant that 'implements' this certification MUST have all of these.

Similarly, any class that 'implements' an interface MUST have all its members.

```
// ── Interface as a contract ──────────────────────────────
interface PaymentGateway {
  name: string;
  processPayment(amount: number, currency: string): Promise<boolean>;
  refund(transactionId: string, amount: number): Promise<boolean>;
  getTransactionStatus(transactionId: string): Promise<string>;
}

// ── Razorpay implements the contract ────────────────────────
class RazorpayGateway implements PaymentGateway {
  name = 'Razorpay';

  async processPayment(amount: number, currency: string): Promise<boolean> {
    console.log(`Razorpay: Processing ₹${amount}`);
    return true;
  }

  async refund(txId: string, amount: number): Promise<boolean> {
    console.log(`Razorpay: Refunding ₹${amount} for ${txId}`);
    return true;
  }

  async getTransactionStatus(txId: string): Promise<string> {
    return 'completed';
  }
}

// ── Stripe implements the SAME contract ─────────────────────
class StripeGateway implements PaymentGateway {
```

```
  name = 'Stripe';

  async processPayment(amount: number, currency: string): Promise<boolean> {
    console.log(`Stripe: Processing $${amount} in ${currency}`);
    return true;
  }

  async refund(txId: string, amount: number): Promise<boolean> {
    console.log(`Stripe: Refunding $${amount} for ${txId}`);
    return true;
  }

  async getTransactionStatus(txId: string): Promise<string> {
    return 'succeeded';
  }
}

// ── The beauty: use the interface type everywhere ────────────
function checkout(gateway: PaymentGateway, amount: number): void {
  // TypeScript only knows 'gateway' is a PaymentGateway
  // Works with BOTH RazorpayGateway and StripeGateway!
  gateway.processPayment(amount, 'INR');
}

checkout(new RazorpayGateway(), 999);  // ✅
checkout(new StripeGateway(), 999);    // ✅

// If you forget a required method in your class:
class BrokenGateway implements PaymentGateway {
  name = 'Broken';
  // missing processPayment, refund, getTransactionStatus
  // ❌ ERROR: Class does not implement interface PaymentGateway
}
```

9. Complete Real-World Example — E-Commerce Type System

Let's build a complete type system using both interface and type alias correctly — showing when to use each.

```
// ═══ Use 'type' for unions, primitives, utilities ═══════════
```

```
type ProductID  = number;
type UserID     = string;
type Currency   = 'INR' | 'USD' | 'EUR';
type OrderStatus = 'placed' | 'confirmed' | 'packed'
                 | 'shipped' | 'delivered' | 'cancelled';
type PaymentMethod = 'upi' | 'card' | 'netbanking' | 'cod' | 'wallet';
type SortOrder  = 'asc' | 'desc';
type Coordinate = [latitude: number, longitude: number];

// == Use 'interface' for objects and class contracts ==========
interface Address {
  line1: string;
  line2?: string;
  city: string;
  state: string;
  pincode: string;
  coordinates?: Coordinate;  // using type alias inside interface
}

interface User {
  readonly id: UserID;
  name: string;
  email: string;
  phone: string;
  addresses: Address[];
  isPremium: boolean;
}

interface Product {
  readonly id: ProductID;
  name: string;
  price: number;
  originalPrice?: number;
  category: string;
  rating: number;
  reviewCount: number;
  inStock: boolean;
  images: string[];
}

interface CartItem {
  product: Product;
  quantity: number;
```

```
  selectedSize?: string;
  selectedColor?: string;
}

interface Order {
  readonly orderId: string;
  readonly placedAt: Date;
  user: User;
  items: CartItem[];
  deliveryAddress: Address;
  status: OrderStatus;         // using type alias
  paymentMethod: PaymentMethod; // using type alias
  total: number;
  currency: Currency;           // using type alias
}

// == interface for service contracts ========================
interface ProductService {
  getById(id: ProductID): Promise<Product | null>;
  getAll(page: number, limit: number): Promise<Product[]>;
  search(query: string, category?: string): Promise<Product[]>;
  create(data: Omit<Product, 'id'>): Promise<Product>;
  update(id: ProductID, data: Partial<Product>): Promise<Product>;
  delete(id: ProductID): Promise<void>;
}

// == Extending interfaces ===================================
interface AdminUser extends User {
  permissions: string[];
  lastLoginAt: Date;
  canDeleteOrders: boolean;
}

// == Type alias for utility types ============================
type CreateOrderDto = Omit<Order, 'orderId' | 'placedAt' | 'status'>;
type UpdateOrderStatus = Pick<Order, 'orderId' | 'status'>;
type ProductSummary = Pick<Product, 'id' | 'name' | 'price' | 'rating'>;
```

10. Interview Questions — Chapter 5

Q: What is the difference between interface and type alias in TypeScript?

Answer: Both can describe object shapes and are interchangeable for most use cases, but they have key differences:

interface is ONLY for object shapes and class contracts. type alias can describe ANYTHING — unions, tuples, primitives, intersections, mapped types, etc.

interface supports declaration merging — two interface declarations with the same name are automatically merged. type alias does NOT support this.

interface uses 'extends' for inheritance. type alias uses '&' (intersection) for combining types.

For error messages: interface shows the interface name. type alias shows the expanded structure (can be verbose for complex types).

Q: What is declaration merging? When is it useful?

Answer: Declaration merging is when TypeScript automatically combines multiple interface declarations with the same name into one single interface.

It's useful in two main scenarios:

1. Module augmentation — adding your own properties to types from third-party libraries. For example, adding req.user to Express's Request type from your auth middleware.

2. Gradually building up complex types across multiple files in your project.

type alias CANNOT do this — declaring two type aliases with the same name is an error.

Q: How do you extend an interface in TypeScript?

Answer: Use the 'extends' keyword. An interface can extend one or multiple interfaces.

Single: interface Admin extends User { permissions: string[]; }

Multiple: interface Duck extends Animal, Flyable, Swimmable { color: string; }

The extending interface inherits ALL properties from the base interfaces and can add its own. If there's a property conflict (same name, different types), TypeScript throws an error immediately — which is safer than type intersection which silently creates 'never'.

Q: What is the difference between interface extends and type intersection (&)?

Answer: Both combine types, but they handle conflicts differently:

interface extends: If the base and child have the same property with incompatible types, TypeScript ERRORS immediately with a clear message. It's strict.

type & intersection: If two types have the same property with incompatible types, TypeScript doesn't immediately error — instead, that property becomes type 'never' (impossible). This can cause confusing errors later when you try to use the type.

interface extends is safer for type hierarchies because conflicts are caught early.

Q: Can a class implement a type alias?

Answer: Yes! A class can implement both an interface and a type alias (as long as the type alias describes an object shape — not a union or primitive).

Example:

```
type Printable = { print(): void };
class Document implements Printable { print() { ... } }
```

However, interface is the conventional and recommended choice for defining class contracts. Type aliases work, but using interface for 'implements' is more expressive and is the standard pattern.

Q: When would you choose interface over type, and vice versa?

Answer: Use interface when:

- Describing the shape of an object or class
- You need 'extends' for inheritance hierarchies
- You are building a library and want users to be able to merge types (declaration merging)
- Defining contracts for classes to implement

Use type alias when:

- You need union types: 'admin' | 'user' | 'guest'
- You need tuple types: [string, number]
- You are naming a primitive for clarity: type ID = string
- You are using utility types: Partial<T>, Pick<T,K>, Omit<T,K>
- You need mapped or conditional types

When in doubt for plain objects: both work. Pick one style and be consistent.

Q: What does 'implements' mean in TypeScript?

Answer: When a class 'implements' an interface, it's making a promise that it will have ALL the properties and methods defined in that interface.

If the class is missing any required member, TypeScript throws a compile-time error: 'Class X does not implement interface Y. Property Z is missing.'

This is powerful for dependency injection and polymorphism — you can write code that depends on the interface type, and swap implementations freely. For example, a PaymentGateway interface implemented by both RazorpayGateway and StripeGateway.

Q: Can an interface extend a type alias?

Answer: Yes — an interface CAN extend a type alias, as long as the type alias describes an object shape (not a union or primitive).

Example:

```
type PersonBase = { name: string; age: number };
interface Employee extends PersonBase { employeeId: string; }
```

And a type alias CAN extend an interface using &:

```
interface Animal { name: string }
type Pet = Animal & { owner: string }
```

They can cross-reference each other freely — except that unions/primitives cannot be used with 'extends'.

Rapid Fire — Chapter 5

Question	One-Line Answer
Can type alias do unions?	✅ Yes — string | number. interface cannot.
Can interface do declaration merging?	✅ Yes. type alias cannot — duplicate = error.
Which keyword to extend interface?	extends: interface B extends A {}
Which operator to extend type alias?	& intersection: type B = A & { ... }
Can a class implement both?	Yes — class C implements InterfaceOrType {}

What happens on property conflict in &?	The property becomes type 'never'
What happens on conflict in extends?	Immediate compile-time error — safer!
Which is better for unions?	Always use type alias for unions
Which is better for objects in OOP?	interface — cleaner extends/implements
Can interface describe a tuple?	No — use type alias: type T = [string, number]

Chapter 5 Key Takeaways

interface and type alias both describe object shapes — mostly interchangeable.

interface ONLY describes objects. type alias describes ANYTHING.

Declaration merging: only interface supports it — critical for library augmentation.

Extend interface: use 'extends'. Extend type: use '&' intersection.

Conflict in extends → immediate ERROR (safe). Conflict in & → silent 'never' (dangerous).

classes 'implement' interfaces to guarantee they have required members.

Use type for: unions, tuples, primitives, utility types, mapped/conditional types.

Use interface for: objects, class contracts, library augmentation, OOP hierarchies.

Consistency is key — pick a convention and stick to it in your codebase.

End of Chapter 5

CHAPTER 6

Union, Intersection & Literal Types

Union (|) • Intersection (&) • Literal Types • Discriminated Unions • Type Narrowing

1. The Big Picture — Combining Types

So far we've learned about individual types — string, number, boolean, objects. But real-world data is rarely that simple. A user ID might be a string OR a number. An API response might succeed OR fail with an error. A button might be 'primary', 'secondary', or 'danger' — and ONLY those three values.

TypeScript gives you three powerful tools to handle this complexity:

- **Union Types (|)** — This OR that. A value can be one of several types.
- **Intersection Types (&)** — This AND that. A value must satisfy multiple types at once.
- **Literal Types** — Not just 'string', but the EXACT string 'success' or 'error'.

The Food Court Analogy

Imagine a food court order system:

Union (|): Payment can be CASH or CARD or UPI — any one of them.
You choose ONE option at a time.

Intersection (&): A Premium Meal must have MainCourse AND Drink AND Dessert.
It must satisfy ALL of them together.

Literal: The table number is not just 'any number' — it's specifically
1, 2, 3, 4, or 5. Only exact allowed values.

2. Union Types (|) — This OR That

A union type means a value can be **one of several types**. You write it with the | (pipe) symbol between types.

Real-World Analogy

Think of a form field for 'Contact Preference'.

A user can enter: a phone number (number)
OR an email address (string)
OR a WhatsApp number (number)

The field accepts STRING or NUMBER — not both at the same time,
but either one is valid. That's a union type.

2.1 Basic Union Syntax

```
// ── Union between primitive types ──────────────────────────
let contactInfo: string | number;

contactInfo = 'rahul@gmail.com'; // ✅ string is fine
contactInfo = 9876543210;        // ✅ number is fine
contactInfo = true;              // ❌ ERROR: boolean not in union

// ── Union in function parameters ───────────────────────────
function displayId(id: string | number): void {
  console.log(`ID: ${id}`);
}

displayId('USR-101');   // ✅ string
displayId(101);         // ✅ number
displayId(true);        // ❌ ERROR: boolean not assignable to string | number

// ── Union with null / undefined (very common!) ─────────────
let userName: string | null = null;    // No user logged in yet
userName = 'Rahul';                    // User logged in
userName = null;                       // User logged out

let userAge: number | undefined;       // Age not provided yet
userAge = 25;                          // Age provided
```

2.2 Union with Object Types

```
// ── Different object shapes in a union ─────────────────────
type SuccessResponse = {
  status: 'success';
  data: { userId: number; name: string };
```

```
};

type ErrorResponse = {
  status: 'error';
  errorCode: number;
  message: string;
};

// API can return EITHER a success OR an error response
type ApiResponse = SuccessResponse | ErrorResponse;

function handleResponse(response: ApiResponse): void {
  // TypeScript knows response is one of the two types
  // But we must narrow it down before accessing specific fields
  if (response.status === 'success') {
    // Here TypeScript KNOWS it's SuccessResponse
    console.log('Welcome,', response.data.name);
  } else {
    // Here TypeScript KNOWS it's ErrorResponse
    console.log('Error:', response.message);
  }
}

// ── Real-world: Search result can be different types ─────────
type UserResult    = { type: 'user';    id: number; name: string; email:
string };
type ProductResult = { type: 'product'; id: number; name: string; price:
number };
type OrderResult   = { type: 'order';   id: number; total: number; date:
Date };

type SearchResult = UserResult | ProductResult | OrderResult;

function renderSearchResult(result: SearchResult): string {
  switch (result.type) {
    case 'user':    return `👤 ${result.name} (${result.email})`;
    case 'product': return `📦 ${result.name} — ₹${result.price}`;
    case 'order':   return `□ Order #${result.id} — ₹${result.total}`;
  }
}
```

Real-World Scenario: OTT Platform (Netflix/Hotstar)

A media item can be a Movie (has duration, director) OR a TVShow (has seasons, episodes)

OR a Podcast (has host, episodeCount). Union types let you create MediaItem = Movie | TVShow | Podcast — one type that handles all three correctly, with TypeScript guiding you to check which kind before accessing specific properties.

3. Literal Types — Exact Values Only

Literal types let you specify **exact values** a variable can hold — not just 'any string' but the specific strings 'north', 'south', 'east', 'west'. Not just 'any number' but only 1, 2, or 3.

Real-World Analogy

Imagine a traffic light controller.

Without literal types: color: string → someone could pass 'purple' ✗

With literal types: color: 'red' | 'yellow' | 'green'

→ ONLY those 3 exact values are allowed ✓

Literal types are like dropdown menus — only the listed options are valid.

3.1 String Literal Types

```
// ── String literals — only exact values allowed ──────────────
type Direction = 'north' | 'south' | 'east' | 'west';

function move(direction: Direction, steps: number): void {
  console.log(`Moving ${steps} steps ${direction}`);
}

move('north', 3);    // ✓
move('south', 5);    // ✓
move('up', 2);       // ✗ ERROR: 'up' is not assignable to Direction
move('NORTH', 1);    // ✗ ERROR: case-sensitive — 'NORTH' ≠ 'north'

// ── Real-world: HTTP methods ─────────────────────────────────
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE';

function makeRequest(url: string, method: HttpMethod): void {
  console.log(`${method} ${url}`);
}
```

```
makeRequest('/api/users', 'GET');     // ✅
makeRequest('/api/users', 'FETCH');   // ❌ Not a valid HTTP method

// ── Real-world: User roles ──────────────────────────────
type UserRole = 'admin' | 'moderator' | 'user' | 'guest';

interface User {
  name: string;
  role: UserRole;  // Only these 4 values are valid
}

const admin: User = { name: 'Rahul', role: 'admin' };     // ✅
const hacker: User = { name: 'Bad Actor', role: 'superuser' }; // ❌ ERROR

// ── Real-world: Button variants ─────────────────────────
type ButtonVariant = 'primary' | 'secondary' | 'danger' | 'ghost' | 'link';
type ButtonSize    = 'xs' | 'sm' | 'md' | 'lg' | 'xl';

interface ButtonProps {
  label: string;
  variant: ButtonVariant;
  size: ButtonSize;
  disabled?: boolean;
}
```

3.2 Number Literal Types

```
// ── Number literals — only specific numbers allowed ──────
type DiceValue = 1 | 2 | 3 | 4 | 5 | 6;

function rollDice(): DiceValue {
  return Math.ceil(Math.random() * 6) as DiceValue;
}

// ── Real-world: HTTP status codes ────────────────────────
type SuccessCode  = 200 | 201 | 204;
type ClientError  = 400 | 401 | 403 | 404 | 422;
type ServerError  = 500 | 502 | 503;
type StatusCode   = SuccessCode | ClientError | ServerError;

function sendResponse(code: StatusCode, message: string): void {
```

```
  console.log(`${code}: ${message}`);
}

sendResponse(200, 'OK');          // ✅
sendResponse(404, 'Not Found');   // ✅
sendResponse(999, 'Custom');      // ❌ ERROR: 999 not in StatusCode

// ── Real-world: Rating system ──────────────────────────────
type StarRating = 1 | 2 | 3 | 4 | 5;

interface Review {
  productId: number;
  rating: StarRating;  // Only 1-5 stars, no 0 or 6
  comment: string;
}
```

3.3 Boolean Literal Types

```
// ── Boolean literals are less common but exist ──────────────
type AlwaysTrue  = true;
type AlwaysFalse = false;

// More useful in conditional types (Chapter 12)
// But here's a practical example:

// A function that returns EXACTLY true or false based on condition:
function isEmailValid(email: string): boolean {
  return email.includes('@') && email.includes('.');
}

// With literal return type — guarantees exact value:
type ValidationResult<T extends boolean> = {
  valid: T;
  message: string;
};
```

3.4 Literal Types + Union = Powerful Combinations

```
// ── The real power: literal unions for state machines ────────
type OrderStatus =
  | 'pending'
  | 'payment_confirmed'
```

```
  | 'processing'
  | 'packed'
  | 'out_for_delivery'
  | 'delivered'
  | 'cancelled'
  | 'refund_initiated'
  | 'refunded';

// With this type:
// 1. No typos possible — 'deliverd' would be a compile error
// 2. Autocomplete works — IDE shows all valid values
// 3. Switch statements can be exhaustively checked

function getOrderStatusLabel(status: OrderStatus): string {
  switch (status) {
    case 'pending':             return '⏳ Order Placed';
    case 'payment_confirmed':   return '✅ Payment Confirmed';
    case 'processing':          return '🔧 Being Processed';
    case 'packed':              return '📦 Packed';
    case 'out_for_delivery':    return '🚚 On the Way';
    case 'delivered':           return '🎉 Delivered';
    case 'cancelled':           return '❌ Cancelled';
    case 'refund_initiated':    return '💸 Refund Initiated';
    case 'refunded':            return '💰 Refunded';
  }
}

// ── Real-world: Notification types ──────────────────────────
type NotificationType =
  | 'order_update'
  | 'promotional'
  | 'payment_reminder'
  | 'review_request'
  | 'account_security';

type NotificationChannel = 'push' | 'email' | 'sms' | 'whatsapp';

interface Notification {
  type: NotificationType;
  channel: NotificationChannel;
  message: string;
  sentAt: Date;
```

```
}
```

4. Discriminated Unions — The Most Useful Pattern

Discriminated unions (also called **tagged unions**) are a pattern where each type in a union has a common **discriminant property** — a literal type field that uniquely identifies which variant it is. TypeScript uses this property to narrow the type automatically.

Real-World Analogy

Think of a parcel sorting system at a courier company.

Every parcel has a label: 'FRAGILE', 'PERISHABLE', or 'STANDARD'.

If label = 'FRAGILE' → special handling rules apply
If label = 'PERISHABLE' → refrigeration rules apply
If label = 'STANDARD' → normal handling

The 'label' is the DISCRIMINANT — one property that tells you exactly which type of parcel it is, and what rules apply.

4.1 The Classic Pattern

```
// ── Each shape has a 'kind' discriminant ──────────────────────
type Circle = {
  kind: 'circle';      // Discriminant — always 'circle'
  radius: number;
};

type Rectangle = {
  kind: 'rectangle';  // Discriminant — always 'rectangle'
  width: number;
  height: number;
};

type Triangle = {
  kind: 'triangle';   // Discriminant — always 'triangle'
  base: number;
  height: number;
};
```

```
type Shape = Circle | Rectangle | Triangle;

function calculateArea(shape: Shape): number {
  switch (shape.kind) {
    case 'circle':
      // TypeScript KNOWS shape is Circle here
      return Math.PI * shape.radius ** 2;

    case 'rectangle':
      // TypeScript KNOWS shape is Rectangle here
      return shape.width * shape.height;

    case 'triangle':
      // TypeScript KNOWS shape is Triangle here
      return 0.5 * shape.base * shape.height;
  }
}

console.log(calculateArea({ kind: 'circle', radius: 5 }));        // 78.54
console.log(calculateArea({ kind: 'rectangle', width: 4, height: 6 })); // 24
```

4.2 Real-World: Payment System

```
// ── Discriminated union for payment methods ──────────────────
type CashPayment = {
  method: 'cash';
  amountTendered: number;
};

type CardPayment = {
  method: 'card';
  cardNumber: string;   // last 4 digits
  cardType: 'visa' | 'mastercard' | 'rupay' | 'amex';
  isContactless: boolean;
};

type UpiPayment = {
  method: 'upi';
  upiId: string;        // e.g. 'rahul@paytm'
  app: 'gpay' | 'phonepe' | 'paytm' | 'bhim';
};
```

```
type NetbankingPayment = {
  method: 'netbanking';
  bankCode: string;
  referenceNumber: string;
};

type Payment = CashPayment | CardPayment | UpiPayment | NetbankingPayment;

function processPayment(payment: Payment, amount: number): string {
  switch (payment.method) {
    case 'cash':
      const change = payment.amountTendered - amount;
      return `Cash received. Change: ₹${change}`;

    case 'card':
      return `Card ${payment.cardType} ending ${payment.cardNumber} charged
₹${amount}`;

    case 'upi':
      return `UPI payment via ${payment.app} from ${payment.upiId}`;

    case 'netbanking':
      return `Netbanking (${payment.bankCode}) ref:
${payment.referenceNumber}`;
  }
}

// Usage:
processPayment({ method: 'upi', upiId: 'rahul@paytm', app: 'paytm' }, 599);
// 'UPI payment via paytm from rahul@paytm'
```

4.3 Real-World: Notification System

```
// ── Discriminated union for different notification types ──────
type EmailNotification = {
  channel: 'email';
  to: string;
  subject: string;
  body: string;
  cc?: string[];
};
```

```
type SmsNotification = {
  channel: 'sms';
  to: string;        // phone number
  message: string;   // max 160 chars
};

type PushNotification = {
  channel: 'push';
  deviceToken: string;
  title: string;
  body: string;
  imageUrl?: string;
  deepLink?: string;
};

type Notification = EmailNotification | SmsNotification | PushNotification;

async function sendNotification(notification: Notification): Promise<void> {
  switch (notification.channel) {
    case 'email':
      // TypeScript knows all email-specific fields are available
      await emailService.send(notification.to, notification.subject,
notification.body);
      break;
    case 'sms':
      await smsService.send(notification.to, notification.message);
      break;
    case 'push':
      await pushService.send(notification.deviceToken, notification.title,
notification.body);
      break;
  }
}
```

5. Intersection Types (&) — This AND That

An intersection type combines MULTIPLE types into ONE. The resulting type must have **ALL properties** from ALL the combined types. Use `&` between types.

Real-World Analogy
Imagine a job requirement:

'The candidate must be a Software Engineer AND a Project Manager'

They need ALL the skills of a Software Engineer
AND ALL the skills of a Project Manager.

Neither set alone is enough — they need BOTH at the same time.
That's an intersection type — satisfy ALL requirements simultaneously.

5.1 Basic Intersection Syntax

```
// ── Basic intersection ──────────────────────────────────────
type HasTimestamps = {
  createdAt: Date;
  updatedAt: Date;
};

type HasSoftDelete = {
  isDeleted: boolean;
  deletedAt?: Date;
};

type Product = {
  id: number;
  name: string;
  price: number;
};

// A database product has product fields + timestamps + soft delete
type DbProduct = Product & HasTimestamps & HasSoftDelete;

const product: DbProduct = {
  id: 1,
  name: 'iPhone 15',
  price: 79999,
  createdAt: new Date('2024-01-01'),
  updatedAt: new Date('2024-06-01'),
  isDeleted: false,
  // deletedAt is optional, so we can skip it
};
```

```
// ── Reusable mixins via intersection ──────────────────────────
type Serializable = {
  serialize(): string;
};

type Loggable = {
  log(level: 'info' | 'warn' | 'error'): void;
};

type Cacheable = {
  cacheKey: string;
  ttl: number;
};

// A full service entity needs all three capabilities:
type ServiceEntity = Product & Serializable & Loggable & Cacheable;
```

5.2 Real-World: Role-Based Access Control

```
// ── Building complex roles via intersection ────────────────────
type CanReadPosts = {
  readPost(id: number): Promise<Post>;
  listPosts(): Promise<Post[]>;
};

type CanWritePosts = {
  createPost(data: Partial<Post>): Promise<Post>;
  updatePost(id: number, data: Partial<Post>): Promise<Post>;
};

type CanDeletePosts = {
  deletePost(id: number): Promise<void>;
};

type CanManageUsers = {
  banUser(userId: number): Promise<void>;
  promoteUser(userId: number, role: string): Promise<void>;
};

// Regular user: can only READ
type GuestRepository = CanReadPosts;
```

```
// Author: can read and write
type AuthorRepository = CanReadPosts & CanWritePosts;

// Moderator: can read, write, and delete
type ModeratorRepository = CanReadPosts & CanWritePosts & CanDeletePosts;

// Admin: can do everything
type AdminRepository = CanReadPosts & CanWritePosts & CanDeletePosts &
CanManageUsers;

// This function only accepts admins:
function banSpammer(repo: AdminRepository, userId: number): void {
  repo.banUser(userId);
}

// This function works for any reader:
function displayPosts(repo: CanReadPosts): void {
  repo.listPosts().then(posts => posts.forEach(p => console.log(p)));
}
```

6. Type Narrowing — Using Unions Safely

When you have a union type, TypeScript **doesn't know** which specific type it is until you narrow it down. Type narrowing is the process of telling TypeScript: 'In THIS block, the type is definitely X'.

Real-World Analogy

Imagine you receive a sealed envelope.

It could contain: a cheque, a letter, or a legal notice.

You CANNOT sign it as a cheque until you open it and confirm it's a cheque.

Signing randomly would be dangerous!

Type narrowing is the 'opening and confirming' step.

TypeScript makes you check what's inside before you use it as a specific type.

6.1 typeof Narrowing

```
// ── typeof narrows primitive unions ──────────────────────────
function formatValue(value: string | number | boolean): string {
```

```
  if (typeof value === 'string') {
    // TypeScript KNOWS: value is string here
    return value.toUpperCase();
  }
  if (typeof value === 'number') {
    // TypeScript KNOWS: value is number here
    return value.toFixed(2);
  }
  // TypeScript KNOWS: value must be boolean here
  return value ? 'Yes' : 'No';
}

formatValue('hello');   // 'HELLO'
formatValue(3.14);      // '3.14'
formatValue(true);      // 'Yes'

// ── Real-world: Flexible ID input ──────────────────────────
function findUser(id: string | number): void {
  if (typeof id === 'string') {
    // String ID — maybe a UUID or username
    console.log(`Searching by string ID: ${id.trim()}`);
  } else {
    // Number ID — numeric primary key
    console.log(`Searching by numeric ID: ${id}`);
  }
}
```

6.2 Equality Narrowing

```
// ── Checking exact values to narrow ────────────────────────
type TrafficLight = 'red' | 'yellow' | 'green';

function getAction(light: TrafficLight): string {
  if (light === 'red') {
    return 'STOP — Do not cross';
  } else if (light === 'yellow') {
    return 'SLOW DOWN — Prepare to stop';
  } else {
    // TypeScript knows: if not red, not yellow → must be green
    return 'GO — Safe to cross';
  }
}
```

```
// ── Narrowing with null checks ──────────────────────────────
function greetUser(name: string | null): string {
  if (name === null) {
    return 'Hello, Guest!';
  }
  // TypeScript KNOWS name is string here (not null)
  return `Hello, ${name.toUpperCase()}!`;
}
```

6.3 instanceof Narrowing

```
// ── instanceof narrows class instances ──────────────────────
class NetworkError extends Error {
  statusCode: number;
  constructor(message: string, statusCode: number) {
    super(message);
    this.statusCode = statusCode;
  }
}

class ValidationError extends Error {
  fields: string[];
  constructor(message: string, fields: string[]) {
    super(message);
    this.fields = fields;
  }
}

function handleError(error: NetworkError | ValidationError): void {
  if (error instanceof NetworkError) {
    // TypeScript KNOWS: error is NetworkError
    console.log(`Network error ${error.statusCode}: ${error.message}`);
    if (error.statusCode === 401) logout();
  } else {
    // TypeScript KNOWS: error is ValidationError
    console.log(`Validation failed on: ${error.fields.join(', ')}`);
  }
}

// ── Real-world: API error handling ──────────────────────────
async function fetchData(url: string): Promise<void> {
  try {
    const res = await fetch(url);
```

```
    if (!res.ok) throw new NetworkError('Request failed', res.status);
  } catch (error) {
    if (error instanceof NetworkError) {
      console.log(`HTTP ${error.statusCode}`);
    } else if (error instanceof Error) {
      console.log(`Unknown error: ${error.message}`);
    }
  }
}
```

6.4 'in' Operator Narrowing

```
// ── 'in' checks if a property exists in an object ────────────
type Cat = { type: 'cat'; meow(): void; indoor: boolean };
type Dog = { type: 'dog'; bark(): void; breed: string };
type Bird = { type: 'bird'; sing(): void; canFly: boolean };

type Pet = Cat | Dog | Bird;

function makeSound(pet: Pet): void {
  if ('meow' in pet) {
    pet.meow();  // TypeScript KNOWS it's a Cat
  } else if ('bark' in pet) {
    pet.bark();  // TypeScript KNOWS it's a Dog
  } else {
    pet.sing();  // TypeScript KNOWS it's a Bird
  }
}

// ── Real-world: Different response shapes ────────────────────
type SuccessResult = { data: unknown; total: number };
type ErrorResult   = { error: string; code: number };

function processResult(result: SuccessResult | ErrorResult): void {
  if ('data' in result) {
    // TypeScript KNOWS: result is SuccessResult
    console.log(`Got ${result.total} records`);
  } else {
    // TypeScript KNOWS: result is ErrorResult
    console.log(`Error ${result.code}: ${result.error}`);
  }
}
```

6.5 Custom Type Guard Functions

```
// ── Type predicate: 'param is Type' ──────────────────────────
// A function that RETURNS a type predicate to narrow types

type User    = { kind: 'user';    name: string; email: string };
type Bot     = { kind: 'bot';     botId: string; owner: string };
type Guest   = { kind: 'guest';   sessionId: string };

type Actor = User | Bot | Guest;

// Custom type guard — returns 'actor is User' not just boolean
function isUser(actor: Actor): actor is User {
  return actor.kind === 'user';
}

function isBot(actor: Actor): actor is Bot {
  return actor.kind === 'bot';
}

function processActor(actor: Actor): void {
  if (isUser(actor)) {
    // TypeScript KNOWS actor is User
    sendWelcomeEmail(actor.email);
  } else if (isBot(actor)) {
    // TypeScript KNOWS actor is Bot
    logBotActivity(actor.botId);
  } else {
    // TypeScript KNOWS actor is Guest
    trackAnonymousSession(actor.sessionId);
  }
}

// ── Real-world: isAdmin guard ────────────────────────────────
interface RegularUser { role: 'user'; name: string }
interface AdminUser   { role: 'admin'; name: string; permissions: string[] }

type AppUser = RegularUser | AdminUser;

function isAdmin(user: AppUser): user is AdminUser {
  return user.role === 'admin';
}
```

```
function deletePost(user: AppUser, postId: number): void {
  if (!isAdmin(user)) {
    throw new Error('Only admins can delete posts!');
  }
  // TypeScript KNOWS user is AdminUser here
  console.log(`Admin ${user.name} deleting post ${postId}`);
  console.log(`Permissions: ${user.permissions.join(', ')}`);
}
```

7. Union vs Intersection — The Core Difference

	Union (|)	Intersection (&)
Meaning	Value is ONE OF the types	Value satisfies ALL the types
Analogy	A or B or C	A and B and C simultaneously
Properties	Only COMMON properties guaranteed	ALL properties from all types
Width	Wider — more possible values	Narrower — fewer valid values
Example	string | number	Person & Employee
Use case	Multiple valid shapes, pick one	Combining capabilities/mixins
Real-world	API response: success OR error	DbUser = User & Timestamps & SoftDelete

```
// ── Union: A value that could be EITHER shape ─────────────────
type Cat = { name: string; meow(): void };
type Dog = { name: string; bark(): void };

type Pet = Cat | Dog;

// With union, only COMMON properties are safe to access without narrowing
function getPetName(pet: Pet): string {
  return pet.name;     // ✅ name exists in BOTH Cat and Dog
  // pet.meow();       // ❌ meow might not exist (could be a Dog)
  // pet.bark();       // ❌ bark might not exist (could be a Cat)
}

// ── Intersection: A value that satisfies ALL shapes ────────────
type Swimmer = { swim(): void; swimSpeed: number };
type Runner  = { run(): void;  runSpeed: number };
```

```
type Triathlete = Swimmer & Runner; // Can do BOTH

const athlete: Triathlete = {
  swim()      { console.log('Swimming!'); },
  swimSpeed:  3,
  run()       { console.log('Running!'); },
  runSpeed:   12,
  // Must have ALL properties from BOTH types
};

// With intersection, ALL properties are accessible directly
athlete.swim();           // ✅
athlete.run();            // ✅
console.log(athlete.swimSpeed, athlete.runSpeed); // ✅
```

8. Complete Real-World Example — Ride-Sharing App

Let's build a complete type system for a ride-sharing app (like Ola or Uber) using union, intersection, literal, and discriminated union types together.

```
// ═══ Literal types for controlled values ═══════════════
type RideStatus =
  | 'searching'
  | 'driver_assigned'
  | 'driver_arriving'
  | 'ride_started'
  | 'ride_completed'
  | 'cancelled';

type VehicleType = 'auto' | 'mini' | 'sedan' | 'suv' | 'bike';
type PaymentMode = 'cash' | 'upi' | 'card' | 'wallet';
type CancelReason = 'driver_late' | 'changed_plans' | 'wrong_location' |
'other';
type StarRating = 1 | 2 | 3 | 4 | 5;

// ═══ Intersection: Reusable mixins ═══════════════════
type HasLocation = {
  latitude: number;
  longitude: number;
  address: string;
};
```

```
type HasTimestamp = {
  createdAt: Date;
  updatedAt: Date;
};

// ═══ Interfaces for core entities ═══════════════════════
interface Driver {
  driverId: string;
  name: string;
  phone: string;
  rating: number;
  vehicleType: VehicleType;
  vehicleNumber: string;
  currentLocation: HasLocation;
  isOnline: boolean;
}

interface Rider {
  riderId: string;
  name: string;
  phone: string;
  savedPlaces: Array<HasLocation & { label: string }>;
}

// ═══ Discriminated union for ride events ═══════════════
type RideSearching = {
  status: 'searching';
  pickup: HasLocation;
  drop: HasLocation;
  vehicleType: VehicleType;
  estimatedFare: number;
};

type RideConfirmed = {
  status: 'driver_assigned';
  pickup: HasLocation;
  drop: HasLocation;
  driver: Driver;
  otp: string;
  estimatedArrival: number; // minutes
};

type RideInProgress = {
```

```
  status: 'ride_started';
  driver: Driver;
  startTime: Date;
  startLocation: HasLocation;
};

type RideCompleted = {
  status: 'ride_completed';
  driver: Driver;
  startTime: Date;
  endTime: Date;
  fare: number;
  distance: number; // km
  paymentMode: PaymentMode;
  rating?: StarRating;
};

type RideCancelled = {
  status: 'cancelled';
  cancelledBy: 'rider' | 'driver' | 'system';
  reason: CancelReason;
  cancellationFee?: number;
};

// Union of all ride states:
type Ride = RideSearching | RideConfirmed | RideInProgress | RideCompleted |
RideCancelled;

// ═══ Using the discriminated union ═══════════════════════
function getRideStatusMessage(ride: Ride): string {
  switch (ride.status) {
    case 'searching':
      return `🔍 Finding ${ride.vehicleType} near you...`;

    case 'driver_assigned':
      return `🚗 ${ride.driver.name} arriving in ~${ride.estimatedArrival}
mins. OTP: ${ride.otp}`;

    case 'ride_started':
      return `🚀 Ride started with ${ride.driver.name}`;

    case 'ride_completed':
      return `✅ Arrived! Fare: ₹${ride.fare} (${ride.distance}km)`;
```

```
    case 'cancelled':
      const fee = ride.cancellationFee ? ` Fee: ₹${ride.cancellationFee}` :
'';
      return `✕ Ride cancelled. Reason: ${ride.reason}.${fee}`;
  }
}
```

9. Interview Questions — Chapter 6

Q: What is a Union type in TypeScript? Give a real-world example.

Answer: A union type means a variable can hold one of several types. You write it with the | operator.

Example: type ID = string | number — an ID can be either a string like 'USR-101' or a number like 101.

Real-world use: A search result can be a User | Product | Order. An API response can be SuccessResponse | ErrorResponse. A form field value can be string | number | null.

When working with union types, TypeScript only lets you access properties that exist in ALL types of the union without narrowing first.

Q: What is a Literal Type? Why is it useful?

Answer: A literal type restricts a variable to a specific exact value, not just a broad type like string or number.

Example: type Direction = 'north' | 'south' | 'east' | 'west' — only those 4 strings are valid.

Why useful: 1) Prevents typos — 'nroth' is a compile error. 2) Enables autocomplete — IDE shows only valid values. 3) Makes switch statements exhaustively checkable. 4) Serves as self-documenting code — the type itself tells you what values are allowed.

Used heavily for: status codes, user roles, button variants, HTTP methods, theme names, and any fixed set of values.

Q: What is a Discriminated Union? When should you use it?

Answer: A discriminated union is a pattern where each type in a union has a common literal type field (the discriminant) that uniquely identifies which variant it is.

Example: type Shape = Circle | Rectangle | Triangle where each has a 'kind' field: 'circle', 'rectangle', 'triangle'.

TypeScript uses the discriminant to automatically narrow the type in switch/if statements — inside case 'circle', it knows the type is Circle and gives you access to Circle-specific properties.

Use it when: you have a union of object types where each variant has a unique identifier field. It's perfect for state machines, API responses, event systems, and payment methods.

Q: What is the difference between Union and Intersection types?

Answer: Union (|): A value can be ONE of the listed types. Only properties common to ALL types are accessible without narrowing.

Intersection (&): A value must satisfy ALL the listed types simultaneously. ALL properties from all types are accessible.

Example:

```
type A = { x: number };  type B = { y: string };
type Union = A | B        // Has x OR y — only common props accessible
type Inter = A & B        // Has x AND y — all props accessible
```

Use union for: 'this value could be one of these shapes'.

Use intersection for: 'this value must have all these capabilities at once'.

Q: What are the different ways to narrow a union type?

Answer: TypeScript has several narrowing techniques:

1. typeof — for primitives: if (typeof x === 'string')
2. Equality check — if (x === 'specific-value')
3. instanceof — for class instances: if (x instanceof NetworkError)
4. 'in' operator — checks property existence: if ('meow' in pet)
5. Discriminant property — switch(x.kind) with literal type discriminant
6. Custom type guard — function isUser(x): x is User { return x.kind === 'user' }

TypeScript tracks which narrowing you've done and automatically updates the type in each branch.

Q: What is a custom Type Guard?

Answer: A custom type guard is a function with a special return type called a type predicate: 'param is Type'. When this function returns true, TypeScript narrows the type of the parameter in the calling code.

Syntax: function isAdmin(user: AppUser): user is AdminUser { return user.role === 'admin'; }

After calling if (isAdmin(user)), TypeScript knows user is AdminUser inside the if block and user is RegularUser outside it.

Use custom type guards when: the narrowing logic is complex enough to extract into a reusable function, or when you want to give semantic meaning to the check (isAdmin is more readable than user.role === 'admin' scattered everywhere).

Q: What happens when you access a property that doesn't exist on all types in a union?

Answer: TypeScript gives a compile-time error.

Example: type Pet = Cat | Dog where Cat has 'meow' and Dog has 'bark'.

If you try pet.meow() directly, TypeScript errors: 'Property meow does not exist on type Dog'.

You MUST first narrow the type:

if ('meow' in pet) { pet.meow(); } // Safe

OR use the discriminant: if (pet.kind === 'cat') { pet.meow(); }

Only properties that exist in ALL types of the union can be accessed without narrowing first.

Rapid Fire — Chapter 6

Question	One-Line Answer
Symbol for union?	| — type A = string | number
Symbol for intersection?	& — type B = TypeX & TypeY
What is a discriminant?	A shared literal-type property that uniquely identifies each union member
Can you access union-only props without narrowing?	No — only common properties are accessible

What does typeof narrow?	Primitive types: string, number, boolean, etc.
What does instanceof narrow?	Class instances
What does 'in' narrow?	Object types based on whether a property exists
Return type of a type guard function?	type is SomeType — e.g. x is AdminUser
What is type widening?	When TS infers a broad type (string) instead of a literal ('north')
How to prevent widening?	Use 'as const': const dir = 'north' as const

Chapter 6 Key Takeaways

Union (|): value is ONE of the types. Only common properties accessible without narrowing.

Literal types: exact values only — 'red' | 'green' | 'blue', not just 'string'.

Literal + union = powerful enums without the overhead. Use for status, roles, variants.

Discriminated unions: add a 'kind' or 'type' discriminant to each union member.
TypeScript narrows automatically in switch/if blocks.

Intersection (&): value must satisfy ALL types. ALL properties are accessible directly.

Type Narrowing techniques: typeof, equality (===), instanceof, 'in', discriminants, custom guards.

Custom type guards: function isX(v: Union): v is X { ... } — reusable, semantic narrowing.

Discriminated unions = the most important union pattern in real-world TypeScript code.

End of Chapter 6

CHAPTER 7

Enums

Numeric Enums • String Enums • Const Enums • Reverse Mapping • Pros & Cons • vs Literal Types

1. What is an Enum?

Enum (short for Enumeration) is a special TypeScript feature that lets you define a set of named constants. Instead of using magic numbers or random strings scattered throughout your code, enums give them meaningful names that are grouped together.

Real-World Analogy — Days of the Week

Without enum, a calendar app might store days as numbers:

0 = Sunday, 1 = Monday, 2 = Tuesday ... 6 = Saturday

This is a nightmare! When you see 'day === 3' you think:

'Is that Wednesday? Or is it 0-indexed? Or 1-indexed?'

With an enum:

Day.WEDNESDAY — instantly clear, no guessing!

Day.MONDAY, Day.FRIDAY — self-documenting, impossible to confuse

Enum = giving meaningful NAMES to a fixed set of related values.

```
// ── Without enum — confusing magic numbers ──────────────────
function getDiscount(userType: number): number {
  if (userType === 1) return 0;      // What is 1? Regular?
  if (userType === 2) return 0.10;   // What is 2? Premium?
  if (userType === 3) return 0.20;   // What is 3? VIP?
  return 0;
}

getDiscount(2); // What does 2 mean here? Nobody knows without context!

// ── With enum — crystal clear ────────────────────────────────
enum UserType {
  Regular = 1,
```

```
  Premium = 2,
  VIP     = 3,
}

function getDiscount(userType: UserType): number {
  if (userType === UserType.Regular) return 0;
  if (userType === UserType.Premium) return 0.10;
  if (userType === UserType.VIP)     return 0.20;
  return 0;
}

getDiscount(UserType.Premium); // Crystal clear — 10% discount for Premium!
```

2. Numeric Enums

Numeric enums are the default. Each member gets a numeric value — by default starting from 0 and auto-incrementing. You can also set custom starting values or manually assign values to each member.

2.1 Auto-Incrementing (Default Behaviour)

```
// ── Default numeric enum — starts at 0, auto-increments ──────
enum Direction {
  Up,     // 0
  Down,   // 1
  Left,   // 2
  Right,  // 3
}

console.log(Direction.Up);     // 0
console.log(Direction.Down);   // 1
console.log(Direction.Left);   // 2
console.log(Direction.Right);  // 3

// Using in a function
function move(dir: Direction): void {
  switch (dir) {
    case Direction.Up:    console.log('Moving up');    break;
    case Direction.Down:  console.log('Moving down');  break;
    case Direction.Left:  console.log('Moving left');  break;
    case Direction.Right: console.log('Moving right'); break;
  }
```

```
}

move(Direction.Left);   // 'Moving left'
move(2);                // Also works! 2 maps to Direction.Left
                        // (this is why numeric enums can be risky)
```

2.2 Custom Starting Value

```
// ── Custom start value — useful for matching DB values ────────
enum Priority {
  Low    = 1,   // Start at 1 instead of 0
  Medium = 2,   // auto would also give 2
  High   = 3,
  Urgent = 4,
}

// ── Mixed — some manually set, rest auto-increment ────────────
enum StatusCode {
  OK           = 200,
  Created      = 201,
  BadRequest   = 400,
  Unauthorized = 401,
  NotFound     = 404,
  ServerError  = 500,
}

function handleStatusCode(code: StatusCode): string {
  switch (code) {
    case StatusCode.OK:           return 'Request successful';
    case StatusCode.Created:      return 'Resource created';
    case StatusCode.BadRequest:   return 'Invalid request data';
    case StatusCode.Unauthorized: return 'Please log in';
    case StatusCode.NotFound:     return 'Resource not found';
    case StatusCode.ServerError:  return 'Server error, try later';
  }
}
```

2.3 Reverse Mapping — A Unique Feature of Numeric Enums

Numeric enums have a special feature called **reverse mapping** — you can look up the **name** of an enum member from its **value**. This does NOT work with string enums.

Real-World Analogy — Two-Way Dictionary

A normal dictionary: word → meaning

A reverse-mapping dictionary: meaning → word (also works!)

Numeric enum reverse mapping works the same way:

Direction.Up → 0 (name to value)

Direction[0] → 'Up' (value to name — reverse!)

```
// ── Reverse mapping demo ──────────────────────────────────
enum Direction {
  Up    = 0,
  Down  = 1,
  Left  = 2,
  Right = 3,
}

// Forward: name → value
console.log(Direction.Up);      // 0
console.log(Direction['Left']); // 2

// Reverse: value → name
console.log(Direction[0]);   // 'Up'
console.log(Direction[2]);   // 'Left'
console.log(Direction[99]);  // undefined (no member with value 99)

// ── Practical use: displaying enum names from stored DB values ─
enum OrderStatus {
  Pending   = 1,
  Confirmed = 2,
  Shipped   = 3,
  Delivered = 4,
  Cancelled = 5,
}

// Stored in DB as number (e.g. 3)
const dbStatusValue: number = 3;

// Convert to human-readable name using reverse mapping
const statusName = OrderStatus[dbStatusValue];
console.log(statusName);  // 'Shipped'
console.log(`Your order is: ${statusName}`); // 'Your order is: Shipped'
```

```
// ── What the compiled JS looks like (explains how it works) ──
// TypeScript generates this JS for numeric enums:
// var OrderStatus;
// (function (OrderStatus) {
//   OrderStatus[OrderStatus['Pending']   = 1] = 'Pending';
//   OrderStatus[OrderStatus['Confirmed'] = 2] = 'Confirmed';
//   ...  adds BOTH directions to the object
// })(OrderStatus || (OrderStatus = {}));
```

> 🗄 **Real-World Scenario: Database Integration (Backend API)**
> In a backend API, order status is stored as a number (1,2,3,4,5) in the database for efficiency. But when sending the response to the frontend or logging, you need the human-readable name. Reverse mapping lets you convert DB number 3 to 'Shipped' instantly without a separate lookup table.

3. String Enums

String enums give each member an explicit **string value**. Unlike numeric enums, there is **NO auto-increment** and **NO reverse mapping** — every member must be assigned a string value manually. They are much more debuggable because the values are readable strings.

Why String Enums are Usually Better

Numeric enum value in a log file: status = 3

→ What does 3 mean? You have to look it up.

String enum value in a log file: status = 'shipped'

→ Instantly clear! No lookup needed.

String enums are self-documenting in logs, APIs, databases, and error messages.

3.1 Basic String Enum Syntax

```
// ── String enum — every member needs an explicit value ────────
enum OrderStatus {
  Pending          = 'PENDING',
  PaymentConfirmed = 'PAYMENT_CONFIRMED',
  Processing       = 'PROCESSING',
  Packed           = 'PACKED',
  Shipped          = 'SHIPPED',
```

```
  OutForDelivery   = 'OUT_FOR_DELIVERY',
  Delivered        = 'DELIVERED',
  Cancelled        = 'CANCELLED',
  Refunded         = 'REFUNDED',
}

// Usage
let currentStatus: OrderStatus = OrderStatus.Pending;
console.log(currentStatus);  // 'PENDING' — readable!

// Update status
currentStatus = OrderStatus.Shipped;
console.log(currentStatus);  // 'SHIPPED'

// Type safety
currentStatus = 'SHIPPED';           // ✗ ERROR: string not assignable to
OrderStatus
currentStatus = OrderStatus.Shipped; // ✓ Must use the enum member

// ── String enum in a function ──────────────────────────────
function getStatusMessage(status: OrderStatus): string {
  switch (status) {
    case OrderStatus.Pending:          return '⏳ Order placed, awaiting
payment';
    case OrderStatus.PaymentConfirmed: return '✓ Payment confirmed';
    case OrderStatus.Shipped:          return '🚚 Order shipped';
    case OrderStatus.Delivered:        return '🎉 Order delivered!';
    case OrderStatus.Cancelled:        return '✗ Order cancelled';
    default:                           return `Status: ${status}`;
  }
}

getStatusMessage(OrderStatus.Shipped); // '🚚 Order shipped'
```

3.2 Real-World String Enums

```
// ── Environment enum ──────────────────────────────────────
enum Environment {
  Development = 'development',
  Staging     = 'staging',
  Production  = 'production',
}
```

```
const currentEnv: Environment = Environment.Production;

if (currentEnv === Environment.Production) {
  console.log('Running in PROD — be careful!');
}

// ── Log levels ──────────────────────────────────────────────
enum LogLevel {
  Debug   = 'DEBUG',
  Info    = 'INFO',
  Warning = 'WARNING',
  Error   = 'ERROR',
  Fatal   = 'FATAL',
}

function log(level: LogLevel, message: string): void {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] [${level}] ${message}`);
  // Example output: [2024-12-15T10:30:00Z] [ERROR] Database connection failed
}

log(LogLevel.Error, 'Database connection failed');
log(LogLevel.Info,  'Server started on port 3000');

// ── User roles ──────────────────────────────────────────────
enum UserRole {
  Guest     = 'GUEST',
  User      = 'USER',
  Moderator = 'MODERATOR',
  Admin     = 'ADMIN',
  SuperAdmin= 'SUPER_ADMIN',
}

interface AppUser {
  id: number;
  name: string;
  role: UserRole;
}

function canDeleteContent(user: AppUser): boolean {
  return user.role === UserRole.Admin || user.role === UserRole.SuperAdmin;
}
```

```
function canModerateComments(user: AppUser): boolean {
  return [UserRole.Moderator, UserRole.Admin, UserRole.SuperAdmin]
    .includes(user.role);
}

// ── API endpoints enum ──────────────────────────────────────
enum ApiEndpoint {
  Users    = '/api/v1/users',
  Products = '/api/v1/products',
  Orders   = '/api/v1/orders',
  Auth     = '/api/v1/auth',
  Payment  = '/api/v1/payment',
}

async function fetchData(endpoint: ApiEndpoint): Promise<unknown> {
  const response = await fetch(endpoint);
  return response.json();
}

fetchData(ApiEndpoint.Orders);  // fetches '/api/v1/orders'
```

4. Heterogeneous Enums (Mixed)

TypeScript technically allows mixing string and numeric values in the same enum. However, this is **strongly discouraged** in practice because it leads to confusing and hard-to-maintain code.

```
// ── Heterogeneous enum — AVOID in practice ──────────────────
enum MixedEnum {
  Yes  = 'YES',   // string
  No   = 0,       // number
  Maybe = 1,      // number
}

// This is confusing — are we dealing with strings or numbers?
// Stick to one type within an enum.

// ✅ Better: Use all strings OR all numbers, never mix.
enum BoolChoice {
  Yes   = 'YES',
  No    = 'NO',
```

```
  Maybe = 'MAYBE',
}
```

5. Const Enums — Maximum Performance

Const enums are declared with the `const` keyword before `enum`. They are **completely inlined at compile time** — TypeScript replaces every usage with the actual value and removes the enum object entirely from the compiled JavaScript. This produces zero runtime overhead.

Real-World Analogy — Macros vs Runtime Lookup

Regular enum: Like a dictionary book that ships with your app.

Every time you need a value, you open the book and look it up.

The book itself takes up space and time.

Const enum: Like a find-and-replace before printing the book.

All lookups are replaced with actual values at print time.

The book has no dictionary — values are already written in place.

No space, no lookup time — just the values.

5.1 Const Enum Syntax and Inlining

```
// ── Regular enum (creates a JS object at runtime) ──────────────
enum RegularDirection {
  Up    = 0,
  Down  = 1,
  Left  = 2,
  Right = 3,
}

const dir = RegularDirection.Up;
// Compiled JS: const dir = RegularDirection.Up;
// (still references the enum object at runtime)

// ── Const enum (inlined at compile time, NO runtime object) ───
const enum ConstDirection {
  Up    = 0,
  Down  = 1,
  Left  = 2,
  Right = 3,
```

```
}

const dir = ConstDirection.Up;
// Compiled JS: const dir = 0;  ← value directly inlined!
// The ConstDirection object does NOT exist in compiled JS at all!

// ── Performance impact example ──────────────────────────────
const enum HttpMethod {
  Get    = 'GET',
  Post   = 'POST',
  Put    = 'PUT',
  Delete = 'DELETE',
}

// Your code:
if (request.method === HttpMethod.Get) {
  serveCache();
}

// Compiled JS becomes:
if (request.method === 'GET') {  // Directly inlined — no object lookup
  serveCache();
}
```

5.2 Const Enum Limitations

```
// ── Const enums CANNOT be used in some ways regular enums can ─

const enum Status {
  Active   = 'ACTIVE',
  Inactive = 'INACTIVE',
}

// ✗ Cannot iterate over const enum values
// Object.values(Status)  — ERROR: const enum cannot be used here

// ✗ Cannot use reverse mapping on const enum
// Status['ACTIVE']  — will be undefined

// ✗ Cannot pass const enum as a runtime value to a function
// Object.keys(Status) — ERROR
```

```
// ✅ Can only use members as direct values (they are inlined)
const s: Status = Status.Active;   // ✅ 'ACTIVE' — inlined
if (s === Status.Active) { ... }   // ✅ Compiled to: if (s === 'ACTIVE')

// ── When to use const enum ──────────────────────────────────
// Use const enum when:
//  1. You only need to COMPARE values (not iterate or reverse-map)
//  2. Performance is critical (large loops, hot code paths)
//  3. You want zero runtime overhead
//  4. Values are used as constants/flags

// Avoid const enum when:
//  1. You need to iterate: Object.values(MyEnum)
//  2. You need reverse mapping: MyEnum[value]
//  3. You are building a library (inlining can cause issues for consumers)
```

6. Enum Member Types & Computed Members

6.1 Computed Enum Members

```
// ── Computed members — value calculated from an expression ────
enum FileAccess {
  // Constant members
  None  = 0,
  Read  = 1 << 0,   // 1  (bit shift — common for flags)
  Write = 1 << 1,   // 2
  ReadWrite = Read | Write,  // 3 (bitwise OR)
  // Computed member
  G = '123'.length,  // 3
}

console.log(FileAccess.Read);       // 1
console.log(FileAccess.Write);      // 2
console.log(FileAccess.ReadWrite);  // 3

// ── Real-world: Permission flags (bit flags pattern) ──────────
enum Permission {
  None    = 0,        // 0000
  Read    = 1 << 0,  // 0001 = 1
  Write   = 1 << 1,  // 0010 = 2
  Execute = 1 << 2,  // 0100 = 4
```

```
  Admin   = 1 << 3,  // 1000 = 8
}

// Combine permissions with bitwise OR
const userPermissions = Permission.Read | Permission.Write;  // 3 = 0011

// Check if a permission is granted with bitwise AND
function hasPermission(userPerms: number, perm: Permission): boolean {
  return (userPerms & perm) === perm;
}

console.log(hasPermission(userPermissions, Permission.Read));    // true
console.log(hasPermission(userPermissions, Permission.Execute)); // false
console.log(hasPermission(userPermissions, Permission.Admin));   // false
```

7. Enum vs Literal Union Types — The Big Debate

This is one of the most discussed topics in TypeScript. Both enums and literal union types can represent a fixed set of values. Which should you use? Let's settle this debate with a full comparison.

7.1 Side-by-Side Comparison

```
// ── The same thing with enum vs literal union ─────────────────

// Approach 1: Enum
enum Direction {
  Up    = 'UP',
  Down  = 'DOWN',
  Left  = 'LEFT',
  Right = 'RIGHT',
}

function moveWithEnum(dir: Direction): void {
  console.log(`Moving: ${dir}`);
}

moveWithEnum(Direction.Up);    // ✅
moveWithEnum('UP');             // ❌ ERROR: string not assignable to Direction

// Approach 2: Literal Union
type DirectionType = 'UP' | 'DOWN' | 'LEFT' | 'RIGHT';
```

```
function moveWithLiteral(dir: DirectionType): void {
  console.log(`Moving: ${dir}`);
}

moveWithLiteral('UP');            // ✅ string literals work directly!
moveWithLiteral(Direction.Up);    // ✅ also works (it IS 'UP')
moveWithLiteral('DIAGONAL');      // ❌ ERROR: not in the union
```

Feature	Enum	Literal Union Type
Syntax	enum Status { Active = 'active' }	type Status = 'active' | 'inactive'
Compile output	Creates a JS object at runtime	Compiles away — zero JS output
Bundle size impact	Adds code to bundle	Zero — no runtime code generated
Reverse mapping	✅ Numeric enums only	❌ Not supported
Pass raw strings	❌ Must use Enum.Member	✅ Can pass 'active' directly
Declaration merging	❌ Not supported	✅ Via type alias extend
Iterate values	✅ Object.values(Enum)	❌ No built-in iteration
Tree-shaking	❌ Harder — object always included	✅ Fully tree-shakeable
IDE autocomplete	✅ Shows enum members	✅ Shows literal values
Exhaustive switch check	✅ With never default	✅ With never default
Works across modules	Must import enum	Just import the type (erased)
JSON serialization	String enum value serializes cleanly	Strings serialize cleanly
Recommended by TS team?	No strong preference	Increasingly preferred

7.2 When to Choose Enum vs Literal Union

Choose ENUM when:

- ✅ You need to iterate over all values: Object.values(Status)
- ✅ You need reverse mapping from value to name: Status[3] → 'Shipped'
- ✅ You want to group related constants under one named object
- ✅ Values are used in many files — enum provides a single import

✅ Working with a framework that expects enums (e.g. some ORMs)

✅ You need computed/bitwise flag values: Permission.Read | Permission.Write

✅ Consistency with team convention that already uses enums

Choose LITERAL UNION when:

✅ You want zero runtime overhead (no JS object generated)

✅ You want to accept raw strings: fn('success') not fn(Status.Success)

✅ Tree-shaking matters (frontend bundles)

✅ Simple use cases — no need for a full enum object

✅ You prefer the modern TypeScript style

✅ The values will be in API responses (no need to import an enum)

✅ You already came from Chapter 6 and love literal unions 😄

The Honest Truth

In modern TypeScript (2021+), many developers and the TypeScript team itself prefer LITERAL UNION TYPES over enums for most use cases.

Reasons: enums compile to runtime objects (bundle bloat), numeric enums allow passing raw numbers which bypasses type safety, and const enums have limitations when used in libraries.

However, string enums are still GREAT and widely used in real projects. Many codebases (Angular, NestJS, enterprise apps) use enums heavily.

Bottom line: Know BOTH well. Use the one your team prefers consistently. For new projects with no preference — lean towards literal unions for objects and string enums when you need iteration or a grouped namespace.

8. Enums in Real-World Projects

8.1 Enums in a REST API (NestJS / Express)

```
// ── A complete NestJS-style API using enums ──────────────────


// enums/order.enum.ts
```

```
export enum OrderStatus {
  Pending          = 'PENDING',
  PaymentConfirmed = 'PAYMENT_CONFIRMED',
  Processing       = 'PROCESSING',
  Shipped          = 'SHIPPED',
  Delivered        = 'DELIVERED',
  Cancelled        = 'CANCELLED',
}

export enum PaymentMethod {
  Cash       = 'CASH',
  Card       = 'CARD',
  UPI        = 'UPI',
  NetBanking = 'NET_BANKING',
  Wallet     = 'WALLET',
}

export enum SortOrder {
  Asc  = 'ASC',
  Desc = 'DESC',
}

// order.entity.ts
interface Order {
  id: number;
  status: OrderStatus;
  paymentMethod: PaymentMethod;
  total: number;
}

// order.service.ts
function updateOrderStatus(
  orderId: number,
  newStatus: OrderStatus
): void {
  // Only valid OrderStatus values are accepted
  console.log(`Order ${orderId} status: ${newStatus}`);
}

// Valid transitions only:
updateOrderStatus(1, OrderStatus.Shipped);    // ✅
updateOrderStatus(1, 'SHIPPED');               // ❌ ERROR: string ≠
OrderStatus
```

```
updateOrderStatus(1, OrderStatus.Pending);     // ✅
```

8.2 Enums with Database (TypeORM / Prisma)

```
// ── Using enums with a database ORM ──────────────────────────

enum UserStatus {
  Active    = 'ACTIVE',
  Inactive  = 'INACTIVE',
  Suspended = 'SUSPENDED',
  Deleted   = 'DELETED',
}

enum SubscriptionTier {
  Free       = 'FREE',
  Basic      = 'BASIC',
  Pro        = 'PRO',
  Enterprise = 'ENTERPRISE',
}

// TypeORM entity (example)
// @Entity()
// class User {
//   @Column({ type: 'enum', enum: UserStatus, default: UserStatus.Active })
//   status: UserStatus;
//
//   @Column({ type: 'enum', enum: SubscriptionTier, default:
SubscriptionTier.Free })
//   tier: SubscriptionTier;
// }

// Getting all users on Pro or Enterprise tier:
const premiumTiers = [SubscriptionTier.Pro, SubscriptionTier.Enterprise];

function isPremiumUser(tier: SubscriptionTier): boolean {
  return premiumTiers.includes(tier);
}

isPremiumUser(SubscriptionTier.Pro);    // true
isPremiumUser(SubscriptionTier.Free);   // false

// Iterating over all enum values (not possible with literal unions!)
const allStatuses = Object.values(UserStatus);
```

```
console.log(allStatuses);
// ['ACTIVE', 'INACTIVE', 'SUSPENDED', 'DELETED']
```

8.3 Enum with Exhaustive Switch (Best Practice)

```
// ── Exhaustive switch ensures all cases are handled ──────────
enum NotificationType {
  OrderPlaced     = 'ORDER_PLACED',
  OrderShipped    = 'ORDER_SHIPPED',
  OrderDelivered  = 'ORDER_DELIVERED',
  PaymentFailed   = 'PAYMENT_FAILED',
  AccountAlert    = 'ACCOUNT_ALERT',
}

// Helper that catches missing cases at compile time
function assertNever(value: never): never {
  throw new Error(`Unhandled notification type: ${value}`);
}

function getNotificationIcon(type: NotificationType): string {
  switch (type) {
    case NotificationType.OrderPlaced:    return '📦';
    case NotificationType.OrderShipped:   return '🚚';
    case NotificationType.OrderDelivered: return '🎉';
    case NotificationType.PaymentFailed:  return '⚠';
    case NotificationType.AccountAlert:   return '🔐';
    default:
      return assertNever(type);
      // If you add a new enum member but forget to add a case here,
      // TypeScript ERRORS at compile time — you CANNOT miss it!
  }
}
```

9. Common Enum Pitfalls

Pitfall 1 — Numeric Enums Accept Any Number

```
// ── Numeric enums are NOT fully type-safe ────────────────────
enum Direction {
  Up    = 0,
  Down  = 1,
```

```
  Left  = 2,
  Right = 3,
}

function move(dir: Direction): void {
  console.log(dir);
}

move(Direction.Up);  // ✅ Expected
move(0);             // ✅ TypeScript ALLOWS this — 0 is a number
move(999);           // ✅ TypeScript ALLOWS this — 999 is a number
// TypeScript does NOT error on raw numbers with numeric enums!
// This is a known weakness of numeric enums.

// ✅ Fix: Use STRING enums instead — they DO reject raw strings:
enum StringDirection {
  Up    = 'UP',
  Down  = 'DOWN',
}

function moveStr(dir: StringDirection): void { }
moveStr(StringDirection.Up);  // ✅
moveStr('UP');                // ❌ ERROR — string ≠ StringDirection
moveStr('anything');          // ❌ ERROR — properly rejected
```

Pitfall 2 — Enums Produce Runtime JavaScript

```
// ── Regular enum compiles to a JS object ──────────────────
enum Color {
  Red   = 'RED',
  Green = 'GREEN',
  Blue  = 'BLUE',
}

// Compiled JavaScript output:
// var Color;
// (function (Color) {
//   Color['Red']   = 'RED';
//   Color['Green'] = 'GREEN';
//   Color['Blue']  = 'BLUE';
// })(Color || (Color = {}));
```

```
// This code EXISTS in your production bundle — it takes space!
// For large apps with many enums, this adds up.

// ✅ Alternatives that produce ZERO runtime code:
// Option 1: const enum (inlined, zero output)
const enum ConstColor { Red = 'RED', Green = 'GREEN', Blue = 'BLUE' }

// Option 2: literal union type (zero output)
type ColorType = 'RED' | 'GREEN' | 'BLUE';
```

Pitfall 3 — Don't Use Enums as Namespaces

```
// ── BAD: Misusing enum as a namespace for functions ──────────
enum StringUtils {
  // ❌ Enums are for VALUES, not functions or logic
  // This pattern is an anti-pattern
}

// ✅ Use a module/namespace or plain object for grouping functions:
const StringUtils = {
  toUpperCase: (s: string) => s.toUpperCase(),
  trim: (s: string) => s.trim(),
} as const;

// ── BAD: Creating enums just to avoid 'magic strings' ────────
// If you never need to iterate or group — just use literal types
enum Colours {  // ❌ Overkill if you only compare values
  Red = 'red',
  Blue = 'blue',
}

// ✅ Better for simple cases:
type Colour = 'red' | 'blue';  // Cleaner, zero runtime overhead
```

10. Complete Real-World Example — Job Portal App

Let's build a complete type system for a job portal (like LinkedIn or Naukri) using enums throughout.

```
// ═══ All enums for the job portal ═══════════════════════
```

```
enum JobType {
  FullTime   = 'FULL_TIME',
  PartTime   = 'PART_TIME',
  Contract   = 'CONTRACT',
  Freelance  = 'FREELANCE',
  Internship = 'INTERNSHIP',
}

enum WorkMode {
  OnSite  = 'ON_SITE',
  Remote  = 'REMOTE',
  Hybrid  = 'HYBRID',
}

enum ExperienceLevel {
  Fresher     = 'FRESHER',
  Junior      = 'JUNIOR',      // 1-3 years
  Mid         = 'MID',         // 3-6 years
  Senior      = 'SENIOR',      // 6-10 years
  Lead        = 'LEAD',        // 10+ years
  Executive   = 'EXECUTIVE',
}

enum ApplicationStatus {
  Applied     = 'APPLIED',
  Shortlisted = 'SHORTLISTED',
  Interviewing = 'INTERVIEWING',
  Offered     = 'OFFERED',
  Hired       = 'HIRED',
  Rejected    = 'REJECTED',
  Withdrawn   = 'WITHDRAWN',
}

enum SalaryPeriod {
  Monthly  = 'MONTHLY',
  Annual   = 'ANNUAL',
}

// ═══ Interfaces using the enums ═══════════════════════
interface JobListing {
  readonly jobId: string;
  title: string;
  company: string;
```

```
  location: string;
  type: JobType;
  workMode: WorkMode;
  level: ExperienceLevel;
  skills: string[];
  salary: { min: number; max: number; period: SalaryPeriod };
  postedAt: Date;
  isActive: boolean;
}

interface Application {
  readonly applicationId: string;
  jobId: string;
  applicantId: string;
  status: ApplicationStatus;
  appliedAt: Date;
  updatedAt: Date;
  coverLetter?: string;
}

// ==== Functions using the enums ==========================

// Filter jobs by type and work mode
function filterJobs(
  jobs: JobListing[],
  type?: JobType,
  workMode?: WorkMode,
  level?: ExperienceLevel
): JobListing[] {
  return jobs.filter(job => {
    if (type     && job.type     !== type)     return false;
    if (workMode && job.workMode !== workMode) return false;
    if (level    && job.level    !== level)    return false;
    return job.isActive;
  });
}

// Get application status label
function getStatusBadge(status: ApplicationStatus): string {
  switch (status) {
    case ApplicationStatus.Applied:      return '📤 Applied';
    case ApplicationStatus.Shortlisted:  return '⭐ Shortlisted';
    case ApplicationStatus.Interviewing: return '🎤 Interviewing';
```

```
    case ApplicationStatus.Offered:      return '🎉 Offer Received!';
    case ApplicationStatus.Hired:        return '✅ Hired!';
    case ApplicationStatus.Rejected:     return '❌ Not Selected';
    case ApplicationStatus.Withdrawn:    return '↩ Withdrawn';
  }
}

// Get all remote jobs for senior engineers:
const seniorRemoteJobs = filterJobs(
  allJobs,
  JobType.FullTime,
  WorkMode.Remote,
  ExperienceLevel.Senior
);

// Iterate over all job types for a dropdown:
const jobTypeOptions = Object.values(JobType);
// ['FULL_TIME', 'PART_TIME', 'CONTRACT', 'FREELANCE', 'INTERNSHIP']
```

11. Interview Questions — Chapter 7

Q: What is an enum in TypeScript? Why would you use one?

Answer: An enum (enumeration) is a way to define a set of named constants. Instead of using magic numbers or raw strings scattered throughout the code, enums give them meaningful names grouped together under a single identifier.

You would use an enum when you have a fixed set of related values: order statuses, user roles, HTTP methods, directions, log levels etc.

Benefits: Self-documenting code (OrderStatus.Shipped is clearer than 3), prevents typos and invalid values, enables IDE autocomplete, and makes refactoring easier — change the value in one place.

Q: What is the difference between numeric enums and string enums?

Answer: Numeric enums assign number values (0, 1, 2... by default). String enums require you to manually assign a string to each member.

Key differences:

Numeric: auto-increments from 0, supports reverse mapping (Enum[0] → 'Name'), but accepts ANY number — less type-safe

String: must assign every value manually, NO reverse mapping, NO auto-increment, but safer — rejects raw strings

String enums are generally preferred in modern TypeScript because they produce readable values in logs, APIs, and databases, and are more type-safe.

Q: What is reverse mapping in TypeScript enums?

Answer: Reverse mapping is a feature of NUMERIC enums where you can look up the name of an enum member from its value.

Example: enum Status { Active = 1 }

Status.Active → 1 (forward: name to value)

Status[1] → 'Active' (reverse: value to name)

This is useful when you store enum values as numbers in a database and need to display the human-readable name.

String enums do NOT support reverse mapping — Status['ACTIVE'] returns undefined.

Q: What is a const enum? How is it different from a regular enum?

Answer: A const enum (declared with 'const enum') is completely inlined at compile time. TypeScript replaces every usage with the actual value and the enum object does NOT exist in the compiled JavaScript.

Regular enum: const dir = Direction.Up; → compiles to: const dir = Direction.Up; (references the runtime object)

Const enum: const dir = Direction.Up; → compiles to: const dir = 0; (value inlined directly)

Benefits: Zero runtime overhead, smaller bundle size.

Limitations: Cannot iterate (no Object.values), no reverse mapping, not suitable for libraries.

Use const enums when you only need to compare values and performance matters.

Q: What is the main weakness of numeric enums regarding type safety?

Answer: Numeric enums accept ANY number as a valid value — not just the defined members. TypeScript does not reject raw numbers passed where a numeric enum is expected.

Example: enum Dir { Up = 0 } function move(d: Dir) {}

```
move(Direction.Up) → ✅ expected
move(0)            → ✅ TypeScript allows this
move(999)          → ✅ TypeScript also allows this! Bug!
```

String enums fix this — they reject raw strings, so move('UP') gives an error.

This is one of the main reasons modern TypeScript code prefers string enums or literal unions over numeric enums.

Q: When would you choose an enum over a literal union type?

Answer: Choose enum over literal union when:

1. You need to ITERATE over all values: Object.values(Status) — not possible with literal unions
2. You need REVERSE MAPPING: Status[3] → 'Shipped'
3. You want a GROUPED NAMESPACE: Status.Active instead of 'active' scattered everywhere
4. Working with an ORM that expects enums (TypeORM, Prisma)
5. You need bitwise FLAG operations: Permission.Read | Permission.Write

Choose literal union when you want zero runtime overhead and simpler code for read-only comparisons.

Q: Can you use Object.values() on an enum?

Answer: Yes — on regular string and numeric enums, Object.values() works.

For a string enum like: enum Color { Red = 'RED', Blue = 'BLUE' }

Object.values(Color) → ['RED', 'BLUE']

For a numeric enum like: enum Dir { Up = 0, Down = 1 }

Object.values(Dir) → [0, 1, 'Up', 'Down'] — returns BOTH values AND names due to reverse mapping!

For const enums — Object.values() does NOT work because const enums are erased at compile time and have no runtime object.

Rapid Fire — Chapter 7

Question	One-Line Answer
Default start value of numeric enum?	0 — then auto-increments by 1
Do string enums support reverse mapping?	No — only numeric enums do
What does 'const enum' do?	Inlines values at compile time — zero runtime object
Can numeric enums be bypassed with raw numbers?	Yes — TypeScript allows passing raw numbers (weakness)
Can string enums be bypassed with raw strings?	No — raw strings are rejected (safer)
Can you iterate a const enum?	No — it has no runtime object
Enum vs literal union — which has zero JS output?	Literal union types — enums generate JS code
How to do exhaustive enum switch?	Use 'default: assertNever(value)' pattern
Can enum extend another enum?	No — enums cannot extend other enums in TypeScript
What is a heterogeneous enum?	Mixing string and number values — possible but discouraged

Chapter 7 Key Takeaways

Enum = named constants for a fixed set of values. Eliminates magic numbers/strings.

Numeric enums: auto-increment from 0. Support reverse mapping. Accept raw numbers (weakness).

String enums: explicit string values. No reverse mapping. Reject raw strings (safer). Prefer these!

const enum: inlined at compile time. Zero runtime overhead. No iteration/reverse mapping.

Reverse mapping: numeric only. Status[3] → 'Shipped'. Useful for DB value → name conversion.

Enum vs Literal Union: Use enum for iteration, grouping, ORM integration, bit flags.

Use literal union for zero overhead, simple comparisons, modern TS style.

String enums are more type-safe than numeric — always prefer string enums.

Exhaustive switch + assertNever: ensures all enum cases are handled — compile-time safety.

End of Chapter 7

CHAPTER 8

Generics

Generic Functions • Generic Interfaces • Generic Classes • Constraints • Utility Types

1. What Are Generics? — The Core Idea

Generics are one of the most powerful and most misunderstood features of TypeScript. They let you write code that works with MULTIPLE types while still being type-safe — without giving up the benefits of TypeScript's type system.

The Problem Generics Solve

Imagine you want to write a function that returns the first item of an array.

Without generics, you have two bad options:

Option 1: Use 'any' — works with all arrays but loses ALL type safety

```
function first(arr: any[]): any { return arr[0]; }
const name = first(['Rahul', 'Priya']); // name is type 'any' — useless
```

Option 2: Write one function per type — works but repeats code for every type

```
function firstString(arr: string[]): string { return arr[0]; }
function firstNumber(arr: number[]): number { return arr[0]; }
function firstUser(arr: User[]): User { return arr[0]; }
... write a new function for every type you need? TERRIBLE!
```

Option 3: Generics — ONE function, works for ALL types, FULLY type-safe!

```
function first<T>(arr: T[]): T { return arr[0]; }
const name = first(['Rahul', 'Priya']); // name is 'string' ✅
const num  = first([10, 20, 30]);       // num is 'number'  ✅
const user = first(users);              // user is 'User'   ✅
```

Real-World Analogy — A Reusable Gift Box

Think of a generic as a REUSABLE GIFT BOX with a label slot.

Box<T> = A box that can hold ANY item of type T.

Box<Book> — a box that holds a Book. When you open it, you get a Book.
Box<Laptop> — a box that holds a Laptop. Open it, you get a Laptop.
Box<Phone> — a box that holds a Phone. Open it, you get a Phone.

One box design. Works for ALL item types. And you always know what's inside.
No surprises. No 'any'. Just whatever type you put on the label.

2. Generic Functions

2.1 Basic Syntax — The <T> Type Parameter

The `<T>` is a **type parameter** — a placeholder for a type that will be provided later. `T` is just a convention; you can name it anything, but `T`, `U`, `V`, `K`, `V` are the common conventions.

```
// ── Generic function syntax ──────────────────────────────
//         type param
//            ↓
function identity<T>(value: T): T {
//                    ↑      ↑
//            param type  return type
  return value;
}

// TypeScript INFERS T from the argument:
const str    = identity('Hello');    // T = string,  str    is string
const num    = identity(42);         // T = number,  num    is number
const bool   = identity(true);       // T = boolean, bool   is boolean
const arr    = identity([1, 2, 3]);  // T = number[], arr   is number[]

// You can also provide T explicitly (useful when inference fails):
const str2 = identity<string>('Hello');  // T explicitly = string

// ── The first function example from the intro ────────────────
function first<T>(arr: T[]): T {
  return arr[0];
}
```

```
const firstName = first(['Rahul', 'Priya', 'Ankit']); // string
const firstNum  = first([10, 20, 30, 40]);             // number
const firstUser = first(users);                         // User

// TypeScript knows the TYPE of the result:
firstName.toUpperCase(); // ✅ string method available
firstNum.toFixed(2);     // ✅ number method available
firstUser.email;         // ✅ User properties available
```

2.2 Multiple Type Parameters

```
// ── Two type parameters ──────────────────────────────────────
function pair<T, U>(first: T, second: U): [T, U] {
  return [first, second];
}

const p1 = pair('Rahul', 28);           // [string, number]
const p2 = pair(true, 'active');         // [boolean, string]
const p3 = pair({ id: 1 }, [1, 2, 3]); // [{ id: number }, number[]]

// ── Real-world: Key-Value pair ───────────────────────────────
function createEntry<K, V>(key: K, value: V): { key: K; value: V } {
  return { key, value };
}

createEntry('name', 'Rahul');   // { key: string; value: string }
createEntry('age', 25);         // { key: string; value: number }
createEntry(1, { id: 1, name: 'Priya' }); // { key: number; value: User }

// ── Real-world: API wrapper ──────────────────────────────────
function wrapResponse<T>(data: T, success: boolean): {
  data: T;
  success: boolean;
  timestamp: Date;
} {
  return { data, success, timestamp: new Date() };
}

const userResponse = wrapResponse({ id: 1, name: 'Rahul' }, true);
// userResponse.data is { id: number; name: string }

const listResponse = wrapResponse([1, 2, 3], true);
```

```
// listResponse.data is number[]
```

2.3 Real-World Generic Functions

```
// ── Generic filter function ──────────────────────────────
function filterBy<T>(
  items: T[],
  predicate: (item: T) => boolean
): T[] {
  return items.filter(predicate);
}

// Works for ANY array type:
const adults = filterBy(users, u => u.age >= 18);     // User[]
const evens  = filterBy([1,2,3,4,5], n => n % 2===0); // number[]
const active = filterBy(products, p => p.inStock);     // Product[]

// ── Generic find function ────────────────────────────────
function findById<T extends { id: number }>(items: T[], id: number): T |
undefined {
  return items.find(item => item.id === id);
}

const user    = findById(users, 42);     // User | undefined
const product = findById(products, 5);   // Product | undefined
const order   = findById(orders, 99);    // Order | undefined

// ── Generic map function ─────────────────────────────────
function mapItems<T, U>(items: T[], transform: (item: T) => U): U[] {
  return items.map(transform);
}

const names    = mapItems(users, u => u.name);              // string[]
const prices   = mapItems(products, p => p.price);          // number[]
const summaries = mapItems(orders, o => ({                   // summary[]
  id: o.id,
  total: o.total,
}));

// ── Generic groupBy ──────────────────────────────────────
function groupBy<T, K extends string | number>(
  items: T[],
  keyFn: (item: T) => K
```

```
): Record<K, T[]> {
  return items.reduce((acc, item) => {
    const key = keyFn(item);
    if (!acc[key]) acc[key] = [];
    acc[key].push(item);
    return acc;
  }, {} as Record<K, T[]>);
}

// Group products by category:
const byCategory = groupBy(products, p => p.category);
// { Electronics: Product[], Clothing: Product[], ... }

// Group users by role:
const byRole = groupBy(users, u => u.role);
// { admin: User[], user: User[], guest: User[] }
```

3. Generic Constraints — <T extends Something>

Sometimes you want `<T>` to accept ANY type — but sometimes you need T to have **certain properties** or **extend a specific type**. That's what constraints are for. You use `extends` to constrain what T can be.

Real-World Analogy — Job Requirements

Imagine a function that accepts a 'CEO of a company'.

Not just ANYONE — specifically someone with a 'companyName' and 'employees'.

Without constraint: function hire<T>(person: T) — accepts a dog, a rock, anything!

With constraint: function hire<T extends HasCompany>(person: T)

→ T must AT LEAST have companyName and employees.

→ Could be a StartupCEO, or a MNCChief — both qualify.

→ But a random person without those fields? Rejected.

3.1 Basic Constraint Syntax

```
// ── Without constraint — too permissive ──────────────────────
function getLength<T>(item: T): number {
  return item.length; // ✗ ERROR: 'length' doesn't exist on T
                      // T could be a number, boolean, anything
```

```
}

// ── With constraint — T must have a 'length' property ────────
function getLength<T extends { length: number }>(item: T): number {
  return item.length; // ✅ Safe — T is guaranteed to have .length
}

getLength('Hello');        // ✅ string has .length → 5
getLength([1, 2, 3]);     // ✅ array has .length → 3
getLength({ length: 10 }); // ✅ object has .length → 10
getLength(42);             // ❌ ERROR: number has no .length
getLength(true);           // ❌ ERROR: boolean has no .length

// ── Constraint with an interface ──────────────────────────────
interface HasId {
  id: number;
}

function findById<T extends HasId>(items: T[], id: number): T | undefined {
  return items.find(item => item.id === id);
}

// Works for ANY object that has an 'id: number' property:
findById(users, 1);     // User must have id: number ✅
findById(products, 5);  // Product must have id: number ✅
findById(orders, 99);   // Order must have id: number ✅

// ── keyof constraint — very powerful ──────────────────────────
// K must be a key that exists in T
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { name: 'Rahul', age: 28, email: 'r@g.com' };

getProperty(user, 'name');   // ✅ returns string ('Rahul')
getProperty(user, 'age');    // ✅ returns number (28)
getProperty(user, 'email');  // ✅ returns string ('r@g.com')
getProperty(user, 'phone');  // ❌ ERROR: 'phone' is not a key of user!
```

3.2 Real-World Constraints

```
// ── Constraint: T must be serializable to JSON ────────────────
type JsonPrimitive = string | number | boolean | null;
type JsonValue = JsonPrimitive | JsonValue[] | { [key: string]: JsonValue };

function saveToLocalStorage<T extends JsonValue>(key: string, value: T): void
{
  localStorage.setItem(key, JSON.stringify(value));
}

// ── Constraint: T must have createdAt ──────────────────────────
interface HasTimestamp {
  createdAt: Date;
}

function sortByDate<T extends HasTimestamp>(items: T[]): T[] {
  return [...items].sort(
    (a, b) => b.createdAt.getTime() - a.createdAt.getTime()
  );
}

// Works for Orders, Users, Products — anything with createdAt:
const latestOrders  = sortByDate(orders);   // Order[]
const latestUsers   = sortByDate(users);    // User[]
const latestReviews = sortByDate(reviews);  // Review[]

// ── Constraint: T must be a constructor ────────────────────────
type Constructor<T> = new (...args: unknown[]) => T;

function createInstance<T>(cls: Constructor<T>): T {
  return new cls();
}

// ── Real-world: Generic repository ─────────────────────────────
interface Entity {
  id: number;
  createdAt: Date;
  updatedAt: Date;
}

class Repository<T extends Entity> {
  private items: T[] = [];
```

```
  findById(id: number): T | undefined {
    return this.items.find(item => item.id === id);
  }

  findAll(): T[] {
    return [...this.items];
  }

  save(item: T): T {
    const existing = this.findById(item.id);
    if (existing) {
      Object.assign(existing, item);
    } else {
      this.items.push(item);
    }
    return item;
  }

  delete(id: number): boolean {
    const idx = this.items.findIndex(i => i.id === id);
    if (idx === -1) return false;
    this.items.splice(idx, 1);
    return true;
  }
}

// One Repository class works for ALL entity types:
const userRepo    = new Repository<User>();
const productRepo = new Repository<Product>();
const orderRepo   = new Repository<Order>();

userRepo.findById(1);   // returns User | undefined
productRepo.findAll();  // returns Product[]
```

4. Generic Interfaces

Interfaces can also be generic — you can define an interface shape that works for any type by using type parameters.

```
// ── Generic interface syntax ──────────────────────────────
```

```
interface Box<T> {
  value: T;
  label: string;
}

const nameBox: Box<string>  = { value: 'Rahul', label: 'Name' };
const ageBox:  Box<number>  = { value: 28, label: 'Age' };
const userBox: Box<User>    = { value: { id: 1, name: 'Priya' }, label:
'User' };

// ── Generic API Response interface ──────────────────────────
interface ApiResponse<T> {
  success: boolean;
  data: T;
  message: string;
  timestamp: Date;
  statusCode: number;
}

// Reuse for ANY API endpoint:
type UserResponse    = ApiResponse<User>;
type UserListResponse = ApiResponse<User[]>;
type ProductResponse = ApiResponse<Product>;
type OrderResponse   = ApiResponse<Order>;

async function fetchUser(id: number): Promise<ApiResponse<User>> {
  const res = await fetch(`/api/users/${id}`);
  return res.json();
}

const result = await fetchUser(1);
// result.data is User — TypeScript knows!
console.log(result.data.name);       // ✅
console.log(result.data.email);      // ✅
console.log(result.data.nonExistent); // ❌ ERROR

// ── Generic Paginated Response ──────────────────────────────
interface PaginatedResponse<T> {
  items: T[];
  total: number;
  page: number;
  limit: number;
  totalPages: number;
```

```
  hasNextPage: boolean;
  hasPrevPage: boolean;
}

// Works for any paginated list:
type UserListPage    = PaginatedResponse<User>;
type ProductListPage = PaginatedResponse<Product>;
type OrderListPage   = PaginatedResponse<Order>;

async function getProducts(page: number): Promise<PaginatedResponse<Product>>
{
  const res = await fetch(`/api/products?page=${page}`);
  return res.json();
}

const page = await getProducts(1);
page.items.forEach(product => {
  console.log(product.name, product.price); // ✅ TypeScript knows each item
is Product
});
```

Real-World Scenario: E-Commerce API (Flipkart/Amazon Scale)

Every list endpoint returns paginated data. With a generic PaginatedResponse<T>, your entire API client is typed with ONE interface. Products page returns PaginatedResponse<Product>. Orders page returns PaginatedResponse<Order>. You write the interface once and it works for all 50+ API endpoints.

5. Generic Classes

Classes can also have type parameters, making them reusable containers and data structures for any type.

Real-World Analogy — A Generic Stack

A stack data structure is like a stack of plates.

→ Push a plate on top

→ Pop a plate from top

→ Peek at the top plate

A BOOK stack holds books. A NUMBER stack holds numbers. A USER stack holds users.

Same logic, different types. That's exactly what a generic class solves.

```
// ── Generic Stack class ──────────────────────────────────────
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }

  get size(): number {
    return this.items.length;
  }
}

// A stack of strings:
const browserHistory = new Stack<string>();
browserHistory.push('/home');
browserHistory.push('/products');
browserHistory.push('/cart');
console.log(browserHistory.peek()); // '/cart'
browserHistory.pop();
console.log(browserHistory.peek()); // '/products' — back button!

// A stack of numbers:
const undoStack = new Stack<number>();
undoStack.push(1);
undoStack.push(2);

// TypeScript prevents mixing types:
browserHistory.push(42);     // ✗ ERROR: number not assignable to string
undoStack.push('hello');     // ✗ ERROR: string not assignable to number
```

```
// ── Generic Queue class ──────────────────────────────────────
class Queue<T> {
  private items: T[] = [];

  enqueue(item: T): void {
    this.items.push(item);
  }

  dequeue(): T | undefined {
    return this.items.shift();
  }

  front(): T | undefined {
    return this.items[0];
  }

  get size(): number { return this.items.length; }
}

// Real-world: Order processing queue
const orderQueue = new Queue<Order>();
orderQueue.enqueue(order1);
orderQueue.enqueue(order2);
const nextOrder = orderQueue.dequeue(); // Order | undefined
```

5.2 Real-World Generic Class — Event Emitter

```
// ── Generic Event Emitter ────────────────────────────────────
type EventMap = Record<string, unknown>;

class TypedEventEmitter<Events extends EventMap> {
  private listeners: {
    [K in keyof Events]?: Array<(data: Events[K]) => void>;
  } = {};

  on<K extends keyof Events>(
    event: K,
    listener: (data: Events[K]) => void
  ): void {
    if (!this.listeners[event]) this.listeners[event] = [];
    this.listeners[event]!.push(listener);
```

```
  }

  emit<K extends keyof Events>(event: K, data: Events[K]): void {
    this.listeners[event]?.forEach(fn => fn(data));
  }
}

// Define all events and their payload types:
interface AppEvents {
  userLogin:    { userId: number; timestamp: Date };
  orderPlaced:  { orderId: string; total: number };
  pageView:     { page: string; userId?: number };
  error:        { message: string; code: number };
}

const emitter = new TypedEventEmitter<AppEvents>();

// TypeScript knows EXACTLY what data each event carries:
emitter.on('userLogin', ({ userId, timestamp }) => {
  console.log(`User ${userId} logged in at ${timestamp}`); // ✅ typed!
});

emitter.on('orderPlaced', ({ orderId, total }) => {
  console.log(`Order ${orderId} placed for ₹${total}`);   // ✅ typed!
});

emitter.emit('userLogin', { userId: 1, timestamp: new Date() }); // ✅
emitter.emit('orderPlaced', { orderId: 'ORD-1', total: 1299 });  // ✅
emitter.emit('userLogin', { userId: 1, wrong: true });            // ❌ ERROR
```

6. Built-in Generic Utility Types

TypeScript ships with many built-in utility types that use generics under the hood. These are extremely useful in real-world code and are frequently asked in interviews.

What are Utility Types?

Utility types are pre-built generic type transformations that come with TypeScript.

They transform an existing type into a new one.

Think of them like JavaScript array methods but for TYPES:

Array has .map(), .filter(), .find() — transforms arrays
TypeScript has Partial<T>, Omit<T,K>, Pick<T,K> — transforms types

6.1 Partial<T> — Make All Properties Optional

```
// ── Partial<T>: makes every property optional ─────────────────
interface User {
  id: number;
  name: string;
  email: string;
  phone: string;
  age: number;
}

// Partial<User> = { id?: number; name?: string; email?: string; ... }
// All properties become optional

// ── Real-world: Update/PATCH endpoint ────────────────────────
function updateUser(userId: number, updates: Partial<User>): User {
  // updates can have ANY subset of User properties
  // You don't need to send all fields — just what changed
  return { ...getExistingUser(userId), ...updates };
}

// Valid — only send fields that changed:
updateUser(1, { name: 'New Name' });         // ✅ just name
updateUser(1, { email: 'new@gmail.com' });   // ✅ just email
updateUser(1, { name: 'x', age: 30 });       // ✅ name + age
// Don't need to send id, phone etc. — they stay unchanged
```

6.2 Required<T> — Make All Properties Required

```
// ── Required<T>: makes every optional property required ───────
interface UserDraft {
  name?: string;
  email?: string;
  phone?: string;
}

// Required<UserDraft> = { name: string; email: string; phone: string }
// All optional properties become required
```

```
// ── Real-world: Form validation ──────────────────────────────
function submitUserForm(data: Required<UserDraft>): void {
  // Before saving, ensure ALL fields are filled
  console.log(`Saving: ${data.name}, ${data.email}, ${data.phone}`);
}

submitUserForm({ name: 'Rahul', email: 'r@g.com', phone: '98765' }); // ✅
submitUserForm({ name: 'Rahul', email: 'r@g.com' }); // ❌ ERROR: phone missing
```

6.3 Readonly<T> — Make All Properties Readonly

```
// ── Readonly<T>: makes every property readonly ────────────────
interface Config {
  apiUrl: string;
  timeout: number;
  retries: number;
}

const config: Readonly<Config> = {
  apiUrl: 'https://api.myapp.com',
  timeout: 5000,
  retries: 3,
};

config.apiUrl = 'hacked';  // ❌ ERROR: Cannot assign to readonly property

// ── Real-world: Immutable app config / frozen state ──────────
function createImmutableConfig(input: Config): Readonly<Config> {
  return Object.freeze({ ...input });
}
```

6.4 Pick<T, K> — Select Specific Properties

```
// ── Pick<T, K>: select only the properties you want ──────────
interface User {
  id: number;
  name: string;
  email: string;
  password: string;   // sensitive!
  role: string;
  createdAt: Date;
}
```

```
// Pick only safe fields to return in API response (never send password!)
type PublicUser = Pick<User, 'id' | 'name' | 'email' | 'role'>;
// { id: number; name: string; email: string; role: string }

function getPublicProfile(user: User): PublicUser {
  return {
    id: user.id,
    name: user.name,
    email: user.email,
    role: user.role,
    // password is NOT included — type won't even allow it!
  };
}

// ── Real-world: Card summary vs full product ─────────────────
interface Product {
  id: number; name: string; price: number;
  description: string; specs: object;
  images: string[]; stock: number;
}

// For listing page — only need summary
type ProductCard = Pick<Product, 'id' | 'name' | 'price' | 'images'>;

// For detail page — need everything (use full Product)
```

6.5 Omit<T, K> — Exclude Specific Properties

```
// ── Omit<T, K>: exclude the properties you don't want ────────
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  createdAt: Date;
}

// Omit sensitive or auto-generated fields:
type CreateUserDto = Omit<User, 'id' | 'createdAt'>;
// { name: string; email: string; password: string }
// User provides name, email, password — server generates id + createdAt
```

```
type SafeUser = Omit<User, 'password'>;
// { id: number; name: string; email: string; createdAt: Date }

// ── Pick vs Omit — which to use? ──────────────────────────
// Use Pick when you want a FEW specific fields from a large type
// Use Omit when you want MOST fields minus a few specific ones

// Picking 2 from 10 fields → use Pick
// Removing 2 from 10 fields → use Omit

// ── Real-world: DTO pattern ───────────────────────────────
interface Order {
  readonly orderId: string;  // server generates
  readonly createdAt: Date;  // server sets
  readonly updatedAt: Date;  // server manages
  userId: number;
  items: CartItem[];
  total: number;
  status: string;
}

// What the client sends to CREATE an order:
type CreateOrderDto = Omit<Order, 'orderId' | 'createdAt' | 'updatedAt' |
'status'>;
// { userId: number; items: CartItem[]; total: number }

// What the client sends to UPDATE status:
type UpdateOrderDto = Pick<Order, 'orderId' | 'status'>;
// { orderId: string; status: string }
```

6.6 Record<K, V> — Create an Object Type

```
// ── Record<K, V>: creates an object type with keys K and values V ─
// Equivalent to: { [key in K]: V }

// Map of string keys to numbers:
const wordCount: Record<string, number> = {};
wordCount['hello'] = 5;
wordCount['world'] = 3;

// Map of specific literal keys to values:
```

```
type UserRoles = Record<'admin' | 'user' | 'guest', string[]>;

const permissions: UserRoles = {
  admin: ['read', 'write', 'delete'],
  user:  ['read', 'write'],
  guest: ['read'],
};

// ── Real-world: Translation / i18n ──────────────────────────
type Language = 'en' | 'hi' | 'ta' | 'bn';

const translations: Record<Language, Record<string, string>> = {
  en: { greeting: 'Hello', farewell: 'Goodbye' },
  hi: { greeting: 'Namaste', farewell: 'Alvida' },
  ta: { greeting: 'Vanakkam', farewell: 'Poittuvaren' },
  bn: { greeting: 'Namaskar', farewell: 'Abar Dekha Hobe' },
};

// ── Real-world: Caching layer ───────────────────────────────
const cache: Record<string, { data: unknown; expiresAt: number }> = {};

function setCache(key: string, data: unknown, ttlMs: number): void {
  cache[key] = { data, expiresAt: Date.now() + ttlMs };
}
```

6.7 ReturnType<T> and Parameters<T>

```
// ── ReturnType<T>: extracts the return type of a function ─────
function createUser(name: string, email: string) {
  return { id: Math.random(), name, email, createdAt: new Date() };
}

type CreatedUser = ReturnType<typeof createUser>;
// { id: number; name: string; email: string; createdAt: Date }

// Useful when you don't want to define the return type separately:
const user: CreatedUser = createUser('Rahul', 'r@g.com');
console.log(user.createdAt); // ✅ TypeScript knows this field exists

// ── Parameters<T>: extracts parameter types as a tuple ────────
function sendEmail(to: string, subject: string, body: string): void {}
```

```
type EmailParams = Parameters<typeof sendEmail>;
// [to: string, subject: string, body: string]

const args: EmailParams = ['r@g.com', 'Hello', 'Body text'];
sendEmail(...args); // ✅ Spread the tuple as arguments

// ── Real-world: Mocking / testing ──────────────────────────
// When writing tests, you can use ReturnType to type your mocks:
type UserServiceReturn = ReturnType<typeof UserService.prototype.getUser>;
// Now your mock must return the same shape as the real service
```

6.8 Quick Reference — All Built-in Utility Types

Utility Type	What it does	Real-world use
`Partial<T>`	All props → optional	PATCH/update endpoints
`Required<T>`	All props → required	Form submission validation
`Readonly<T>`	All props → readonly	Immutable config, frozen state
`Pick<T, K>`	Keep only specified keys	Public API response (hide password)
`Omit<T, K>`	Remove specified keys	Create DTOs without auto fields
`Record<K, V>`	Object with keys K and values V	Lookup tables, caches, i18n
`Exclude<T, U>`	Remove U from union T	Filter out 'null' from nullable type
`Extract<T, U>`	Keep only U from union T	Keep only string types from a union
`NonNullable<T>`	Remove null and undefined from T	Ensure a value is never null
`ReturnType<T>`	Extract return type of function T	Type the result of a function call
`Parameters<T>`	Extract params of function T as tuple	Mocking, forwarding function calls
`Awaited<T>`	Unwrap Promise<T> to T	Get resolved type of async function

7. Complete Real-World Example — Type-Safe API Client

Let's build a complete, fully type-safe API client using all the generic concepts covered in this chapter.

```
// ═══ Generic API client — production-grade pattern ═══════

// Generic response wrapper (used for every endpoint)
```

```typescript
interface ApiResponse<T> {
  data: T;
  success: boolean;
  message: string;
  statusCode: number;
}

// Generic paginated response
interface PagedResponse<T> {
  items: T[];
  total: number;
  page: number;
  totalPages: number;
}

// Generic base HTTP client
class HttpClient {
  private baseUrl: string;

  constructor(baseUrl: string) {
    this.baseUrl = baseUrl;
  }

  async get<T>(path: string): Promise<ApiResponse<T>> {
    const res = await fetch(`${this.baseUrl}${path}`);
    return res.json();
  }

  async post<TBody, TResponse>(
    path: string,
    body: TBody
  ): Promise<ApiResponse<TResponse>> {
    const res = await fetch(`${this.baseUrl}${path}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body),
    });
    return res.json();
  }

  async put<TBody, TResponse>(
    path: string,
    body: TBody
```

```
  ): Promise<ApiResponse<TResponse>> {
    const res = await fetch(`${this.baseUrl}${path}`, {
      method: 'PUT',
      body: JSON.stringify(body),
    });
    return res.json();
  }

  async delete<T>(path: string): Promise<ApiResponse<T>> {
    const res = await fetch(`${this.baseUrl}${path}`, { method: 'DELETE' });
    return res.json();
  }
}

// Domain types
interface User { id: number; name: string; email: string; role: string }
interface Product { id: number; name: string; price: number; inStock:
boolean }

// DTOs using utility types
type CreateUserDto   = Omit<User, 'id'>;
type UpdateUserDto   = Partial<Omit<User, 'id'>>;
type CreateProductDto = Omit<Product, 'id'>;

// Specific service using the generic client
class UserService {
  constructor(private client: HttpClient) {}

  async getAll(page = 1): Promise<PagedResponse<User>> {
    const res = await
this.client.get<PagedResponse<User>>(`/users?page=${page}`);
    return res.data;
  }

  async getById(id: number): Promise<User> {
    const res = await this.client.get<User>(`/users/${id}`);
    return res.data;
  }

  async create(dto: CreateUserDto): Promise<User> {
    const res = await this.client.post<CreateUserDto, User>('/users', dto);
    return res.data;
  }
```

```
  async update(id: number, dto: UpdateUserDto): Promise<User> {
    const res = await this.client.put<UpdateUserDto, User>(`/users/${id}`,
dto);
    return res.data;
  }
}

// Usage — fully type-safe throughout:
const http = new HttpClient('https://api.myapp.com');
const userService = new UserService(http);

const users = await userService.getAll();
users.items.forEach(u => console.log(u.name));  // ✅ TypeScript knows u is
User

const newUser = await userService.create({ name: 'Rahul', email: 'r@g.com',
role: 'user' });
console.log(newUser.id);  // ✅ TypeScript knows newUser is User
```

8. Interview Questions — Chapter 8

Q: What are Generics in TypeScript? Why do we need them?

Answer: Generics let you write functions, classes, and interfaces that work with multiple types while remaining type-safe. Instead of using 'any' (which loses type information), generics use a type parameter like <T> that TypeScript fills in based on usage.

We need them to avoid two bad alternatives: writing the same function for every type (code duplication), or using 'any' which destroys type safety.

Example: function first<T>(arr: T[]): T { return arr[0]; }

This one function works for string[], number[], User[] — and TypeScript knows the return type matches the input.

Q: What is a generic constraint? Give an example.

Answer: A constraint limits what types <T> can be. You use 'extends' to say 'T must be at least this type'.

Without constraint: <T> — T can be literally ANYTHING: number, null, Date, etc.

With constraint: <T extends { id: number }> — T must have an id property.

Example: function findById<T extends { id: number }>(items: T[], id: number): T | undefined

This works for User, Product, Order — any object with id — but not for strings or numbers.

Another common constraint: <K extends keyof T> — K must be a key of T.

function get<T, K extends keyof T>(obj: T, key: K): T[K]

This ensures the key you pass actually exists on the object.

Q: What is the difference between Partial<T> and Required<T>?

Answer: Partial<T> makes ALL properties of T optional — every property gets a '?' added.

Required<T> does the opposite — removes '?' from all properties, making them all required.

Partial<T> use case: PATCH/update endpoints where you only send changed fields. function updateUser(id: number, changes: Partial<User>)

Required<T> use case: Form submission or final validation where all fields must be filled before saving.

Q: What is the difference between Pick<T,K> and Omit<T,K>?

Answer: Both create a new type from an existing one by selecting properties.

Pick<T, K>: Creates a type with ONLY the specified keys K from T.

Example: Pick<User, 'id' | 'name'> → { id: number; name: string }

Omit<T, K>: Creates a type with all keys of T EXCEPT the specified K.

Example: Omit<User, 'password'> → all User fields except password

Rule of thumb: If you want a FEW fields from a large type → Pick.

If you want MOST fields minus a few → Omit.

Q: What does Record<K, V> do?

Answer: Record<K, V> creates an object type where keys are of type K and values are of type V.

Record<string, number> = { [key: string]: number } — any string keys, number values.

Record<'admin' | 'user', string[]> = { admin: string[]; user: string[] }

Common use cases: lookup tables, caches (Record<string, CachedData>), permission maps (Record<UserRole, Permission[]>), translation maps (Record<Language, string>).

Q: What is ReturnType<T> and when would you use it?

Answer: ReturnType<T> extracts the return type of a function type T.

Example: function createUser() { return { id: 1, name: 'Rahul' } }

type User = ReturnType<typeof createUser> → { id: number; name: string }

Useful when: you don't want to define a separate interface for the return type, you're working with a function from a library and want to type its return value, or you're writing tests and need to type mocks to match the real function's return.

Q: Can you use multiple type parameters in a generic function?

Answer: Yes. You can have as many type parameters as needed.

function pair<T, U>(a: T, b: U): [T, U] { return [a, b]; }

pair('hello', 42) → [string, number]

pair(true, { id: 1 }) → [boolean, { id: number }]

Common naming conventions: T, U, V for general types. K for keys, V for values. E for elements. R for return types.

Q: What is the difference between using 'any' vs Generics?

Answer: Both accept multiple types, but they behave very differently:

any: Throws away ALL type information. TypeScript stops checking.

function first(arr: any[]): any — the return type is 'any', useless.

result.toUpperCase() — TypeScript allows this even if result is a number!

Generics: Preserve type information. TypeScript tracks the relationship between input and output.

function first<T>(arr: T[]): T — if you pass string[], you get back string.

result.toUpperCase() — TypeScript knows this is valid because result IS a string.

Generics give you the flexibility of 'any' with the safety of explicit types.

Rapid Fire — Chapter 8

Question	One-Line Answer
What symbol denotes a generic type parameter?	<T> — angle brackets around a letter
What does <T extends string> mean?	T can only be string or a subtype of string
What does keyof T return?	A union of all keys (property names) of T
Partial<User> — what changes?	All User properties become optional (?)
Omit<User, 'password'> — what changes?	Returns User type without the 'password' field
Pick<User, 'id'|'name'> — what do you get?	A type with only id and name from User
What does Record<string, number> create?	An object with string keys and number values
ReturnType<typeof fn> gives you?	The return type of function fn
Can interfaces be generic?	Yes — interface Box<T> { value: T }
Can classes be generic?	Yes — class Stack<T> { items: T[] }

Chapter 8 Key Takeaways

Generics = type parameters that let code work with any type while staying type-safe.

<T> is a placeholder. TypeScript infers T from usage, or you can specify it explicitly.

Generic constraint: <T extends Something> — T must satisfy a minimum shape.

keyof T — gives you a union of all keys of T. Use with constraints for safe property access.

Generic interfaces: interface ApiResponse<T> { data: T } — reuse for every endpoint.

Generic classes: class Stack<T> — one class works for all types of stacks.

Utility types — memorise these 6 core ones:

Partial<T> = all optional | Required<T> = all required | Readonly<T> = all readonly

Pick<T,K> = keep keys | Omit<T,K> = remove keys | Record<K,V> = key→value object

ReturnType<typeof fn> — extract return type. Parameters<typeof fn> — extract param types.

Generics > any — always. any loses types. Generics preserve and track them.

End of Chapter 8

CHAPTER 9

Type Narrowing & Type Guards

typeof • instanceof • in • Equality • Truthiness • Custom Guards • Assertion Functions

1. What is Type Narrowing?

TypeScript is very smart — it constantly analyses your code and tracks which type a variable CAN be at every single point in your program. Type Narrowing is the process where TypeScript automatically narrows down a broad type (like string | number | null) to a more specific type inside a particular code block.

The Core Idea — Sherlock Holmes

Think of TypeScript as Sherlock Holmes.

You walk into his office. He doesn't know who you are yet.

You: 'I am either a doctor OR a lawyer OR a teacher.'

Sherlock sees a stethoscope around your neck.

Sherlock NARROWS down: 'Now I know you're a doctor — not a lawyer or teacher.'

TypeScript does the same. You have: type Person = Doctor | Lawyer | Teacher

TypeScript checks your code for CLUES (typeof, instanceof, if checks).

After each clue, TypeScript NARROWS the type in that code block.

Inside the 'stethoscope' check — TypeScript knows you're a Doctor.

So it gives you Doctor methods. Not Lawyer methods. Not Teacher methods.

```
// ── Without narrowing — TypeScript is cautious ──────────────
function printId(id: string | number) {
  // TypeScript knows id is 'string | number'
  // It won't let you call string OR number specific methods
  // because it doesn't know WHICH one it is yet
  console.log(id.toUpperCase()); // ✗ ERROR: toUpperCase doesn't exist on number
  console.log(id.toFixed(2));    // ✗ ERROR: toFixed doesn't exist on string
}
```

```
// ── With narrowing — TypeScript knows exactly what you have ──
function printId(id: string | number) {
  if (typeof id === 'string') {
    // HERE: TypeScript KNOWS id is string
    console.log(id.toUpperCase()); // ✅ Safe — string method
    console.log(id.trim());        // ✅ Safe — string method
  } else {
    // HERE: TypeScript KNOWS id is number (not string, so must be number)
    console.log(id.toFixed(2));    // ✅ Safe — number method
    console.log(id.toString());    // ✅ Safe
  }
}
```

Why This Matters in Production

Without narrowing, you must use 'any' or 'as' type casting everywhere — dangerous.

With proper narrowing, TypeScript guides you to write safe code.

The most common JavaScript runtime error is:

'TypeError: Cannot read properties of null (reading something)'

Narrowing eliminates this class of bug at COMPILE TIME.

You handle null/undefined/wrong-types BEFORE they reach users.

2. typeof Narrowing

The `typeof` operator is the most common narrowing technique. It works with the 7 primitive types: `string`, `number`, `boolean`, `bigint`, `symbol`, `undefined`, and `object` (note: `null` also returns `'object'` — a famous JS quirk!).

2.1 Basic typeof Narrowing

```
// ── typeof with all primitive types ──────────────────────────
function processValue(value: string | number | boolean | null) {
  if (typeof value === 'string') {
    // value: string
    return `String: ${value.toUpperCase()}`;
  }
  if (typeof value === 'number') {
    // value: number
```

```
    return `Number: ${value.toFixed(2)}`;
  }
  if (typeof value === 'boolean') {
    // value: boolean
    return `Boolean: ${value ? 'Yes' : 'No'}`;
  }
  // value: null (only remaining type)
  return 'Null value';
}

// ── Real-world: Flexible search input ──────────────────────
function searchProduct(query: string | number): void {
  if (typeof query === 'string') {
    // User typed a product name
    console.log(`Searching by name: '${query.trim()}'`);
  } else {
    // User typed a product ID
    console.log(`Searching by ID: ${query}`);
  }
}

searchProduct('iPhone');  // Searching by name: 'iPhone'
searchProduct(1042);      // Searching by ID: 1042
```

2.2 The null Trap — typeof null === 'object'

```
// ── Famous JS quirk: typeof null === 'object' ────────────────
console.log(typeof null);      // 'object' — NOT 'null' !!
console.log(typeof {});        // 'object'
console.log(typeof []);        // 'object'
console.log(typeof undefined); // 'undefined'

// This means typeof alone is NOT enough to narrow out null:
function processData(data: string | null) {
  if (typeof data === 'object') {
    // data could be null OR an object — dangerous!
    data.toUpperCase(); // ✗ Still could crash if data is null
  }
}

// ── Correct way to handle null ───────────────────────────────
function processData(data: string | null) {
  if (data === null) {
```

```
    return 'No data provided';
  }
  // HERE: TypeScript KNOWS data is string (null is excluded)
  return data.toUpperCase(); // ✅ Safe
}

// ── Or use truthiness narrowing ──────────────────────────
function processData(data: string | null) {
  if (data) {
    // data is truthy → it's a non-empty string
    return data.toUpperCase(); // ✅ Safe
  }
  return 'Empty or null';
}
```

3. Truthiness Narrowing

JavaScript treats certain values as **falsy**: `false` , `0` , `''` (empty string), `null` , `undefined` , `NaN` . Everything else is truthy. TypeScript uses this to narrow types in if statements.

Real-World Analogy — The Empty Box Check

Before taking a gift out of a box, you shake it.

If it feels empty (falsy) → you say 'Nothing here, skip it'

If it has something (truthy) → you open it and use it

Truthiness narrowing does the same — check if the value 'has something' before trying to use it. Eliminates null, undefined, and empty string.

```
// ── Truthiness narrows out null and undefined ────────────
function greetUser(name: string | null | undefined): string {
  if (name) {
    // HERE: name is string (null and undefined are falsy — excluded)
    // BUT: empty string '' is also falsy — so name is a non-empty string
    return `Hello, ${name.toUpperCase()}!`;
  }
  return 'Hello, Guest!';
}

greetUser('Rahul');     // 'Hello, RAHUL!'
greetUser(null);        // 'Hello, Guest!'
```

```
greetUser(undefined);  // 'Hello, Guest!'
greetUser('');         // 'Hello, Guest!' (empty string is falsy too)

// ── Truthiness in real-world: optional config ─────────────────
interface Config {
  apiKey?: string;
  timeout?: number;
  retries?: number;
}

function initClient(config: Config): void {
  if (config.apiKey) {
    // apiKey is string here (not undefined)
    console.log(`API Key: ${config.apiKey.slice(0, 4)}****`);
  }
  const timeout = config.timeout ?? 5000; // nullish coalescing
  const retries = config.retries ?? 3;
  console.log(`Timeout: ${timeout}ms, Retries: ${retries}`);
}

// ── Double negation !! — convert to boolean ──────────────────
const hasUser = !!user;       // true if user is not null/undefined
const hasItems = !!items.length; // true if array is non-empty

// ── Array truthiness trap ────────────────────────────────────
const arr: string[] = [];
if (arr) {
  // ⚠ This is ALWAYS truthy even if array is empty!
  // An empty array [] is truthy in JavaScript
}
// ✔ Correct way to check for non-empty array:
if (arr.length > 0) { ... }
if (arr.length)     { ... }  // also works
```

4. Equality Narrowing

TypeScript uses `===`, `!==`, `==`, and `!=` checks to narrow types. This works especially well with **discriminated unions** and **literal types**.

```
// ── Strict equality narrows literal types ────────────────────
type Direction = 'north' | 'south' | 'east' | 'west';
```

```
function describeDirection(dir: Direction): string {
  if (dir === 'north') {
    // dir: 'north' — TypeScript narrows to exactly this literal
    return 'Heading towards the mountains';
  }
  if (dir === 'south') {
    return 'Heading towards the ocean';
  }
  // TypeScript knows: dir is 'east' | 'west' here
  return `Going ${dir}`;
}

// ── Narrowing out null and undefined with === ─────────────────
function processUser(user: User | null | undefined): string {
  if (user === null) {
    return 'No user (explicitly null)';
  }
  if (user === undefined) {
    return 'User not provided';
  }
  // user: User — both null and undefined excluded
  return `Welcome, ${user.name}`;
}

// ── Loose equality == narrows both null AND undefined ─────────
function processUser(user: User | null | undefined): string {
  if (user == null) {
    // user == null is TRUE for BOTH null and undefined
    // Narrows out both in one check
    return 'No user';
  }
  // user: User (neither null nor undefined)
  return `Welcome, ${user.name}`;
}

// ── Real-world: Status-based routing ──────────────────────────
type OrderStatus = 'pending' | 'processing' | 'shipped' | 'delivered' |
'cancelled';

function getNextAction(status: OrderStatus): string {
  if (status === 'pending') return 'Confirm payment';
  if (status === 'processing') return 'Prepare for shipping';
```

```
  if (status === 'shipped') return 'Track delivery';
  if (status === 'delivered') return 'Request review';
  // status: 'cancelled' (only one left)
  return 'Process refund if applicable';
}
```

5. instanceof Narrowing

`instanceof` narrows the type when working with **class instances**. After `x instanceof MyClass`, TypeScript knows `x` is of type `MyClass` in that block.

Real-World Analogy — ID Card Check

At a club entrance, the bouncer checks your ID card.

If you show a Student ID → you're a student (narrowed to Student)
If you show a Staff Badge → you're staff (narrowed to Staff)
If you show a VIP Pass → you're VIP (narrowed to VIP)

instanceof is the ID card check — it confirms WHICH class the object belongs to.

```
// ── Basic instanceof narrowing ──────────────────────────────
class Dog {
  name: string;
  constructor(name: string) { this.name = name; }
  bark(): void { console.log('Woof!'); }
}

class Cat {
  name: string;
  constructor(name: string) { this.name = name; }
  meow(): void { console.log('Meow!'); }
}

type Pet = Dog | Cat;

function makeSound(pet: Pet): void {
  if (pet instanceof Dog) {
    // pet: Dog
    pet.bark(); // ✅ Safe
```

```
  } else {
    // pet: Cat
    pet.meow(); // ✅ Safe
  }
}

// ── Real-world: Error handling ──────────────────────────────
class ApiError extends Error {
  constructor(public statusCode: number, message: string) {
    super(message);
    this.name = 'ApiError';
  }
}

class ValidationError extends Error {
  constructor(public fields: string[], message: string) {
    super(message);
    this.name = 'ValidationError';
  }
}

class NetworkError extends Error {
  constructor(public retryAfter: number, message: string) {
    super(message);
    this.name = 'NetworkError';
  }
}

function handleError(error: unknown): string {
  if (error instanceof ApiError) {
    // error: ApiError — statusCode is available
    if (error.statusCode === 401) return 'Please log in again';
    if (error.statusCode === 403) return 'You do not have permission';
    if (error.statusCode === 404) return 'Resource not found';
    return `API Error ${error.statusCode}: ${error.message}`;
  }
  if (error instanceof ValidationError) {
    // error: ValidationError — fields is available
    return `Validation failed: ${error.fields.join(', ')}`;
  }
  if (error instanceof NetworkError) {
    // error: NetworkError — retryAfter is available
    return `Network error. Retry in ${error.retryAfter}s`;
```

```
  }
  if (error instanceof Error) {
    // Generic Error — only .message is available
    return `Error: ${error.message}`;
  }
  // error: unknown — could be anything (string, number thrown as error)
  return 'An unexpected error occurred';
}
```

> **Real-World Scenario: Real API Error Handling (Axios / Fetch)**
> In production apps, errors can come from many sources — 401 auth failures, 422 validation errors, 503 network timeouts, or unexpected crashes. Using instanceof with custom error classes lets you handle each case precisely and show the right message to the user, instead of one generic 'Something went wrong'.

6. The 'in' Operator Narrowing

The `'property' in object` check narrows types based on whether a property EXISTS on an object. TypeScript uses this to distinguish between union member types that have different properties.

```
// ── Basic 'in' narrowing ──────────────────────────────────
type Admin = {
  name: string;
  role: 'admin';
  permissions: string[];  // Only Admin has permissions
};

type RegularUser = {
  name: string;
  role: 'user';
  subscriptionTier: string;  // Only User has subscriptionTier
};

type AppUser = Admin | RegularUser;

function getUserInfo(user: AppUser): string {
  if ('permissions' in user) {
    // user: Admin — permissions property exists
    return `Admin ${user.name} — Permissions: ${user.permissions.join(', ')}`;
  }
  // user: RegularUser — subscriptionTier exists
```

```
  return `User ${user.name} — Plan: ${user.subscriptionTier}`;
}

// ── Real-world: Different payment response shapes ──────────────
type SuccessPayment = {
  transactionId: string;
  amount: number;
  paidAt: Date;
};

type FailedPayment = {
  errorCode: string;
  reason: string;
  canRetry: boolean;
};

type PaymentResult = SuccessPayment | FailedPayment;

function handlePaymentResult(result: PaymentResult): void {
  if ('transactionId' in result) {
    // result: SuccessPayment
    console.log(`✅ Payment success! Txn: ${result.transactionId}`);
    console.log(`Amount: ₹${result.amount} at ${result.paidAt}`);
  } else {
    // result: FailedPayment
    console.log(`❌ Payment failed: ${result.reason}`);
    if (result.canRetry) console.log('Please try again.');
  }
}

// ── Real-world: Media player ───────────────────────────────────
type Video = { title: string; duration: number; resolution: string };
type Audio = { title: string; duration: number; bitrate: number };
type Image = { title: string; width: number; height: number };

type Media = Video | Audio | Image;

function describeMedia(media: Media): string {
  if ('resolution' in media) {
    // media: Video
    return `Video: ${media.title} (${media.duration}s, ${media.resolution})`;
  }
```

```
  if ('bitrate' in media) {
    // media: Audio
    return `Audio: ${media.title} (${media.bitrate}kbps)`;
  }
  // media: Image
  return `Image: ${media.title} (${media.width}x${media.height})`;
}
```

7. Discriminant Property Narrowing

This is the **most powerful and most used** narrowing technique in real-world TypeScript. When a union type has a **shared literal-type property** (discriminant), TypeScript uses it to narrow the type in switch/if statements. We covered the pattern in Chapter 6 — now let's deep-dive into how the narrowing actually works.

Why Discriminant Narrowing is the Best

It's explicit — the 'kind' or 'type' field tells you exactly what you have.

It's exhaustive — TypeScript checks if you handled all cases.

It's readable — code is self-documenting.

It scales — works perfectly as your union grows to 10, 20 variants.

This is the RECOMMENDED pattern for any union of object types.

```
// ── Discriminant narrowing with switch ──────────────────────
type LoadingState = { status: 'loading' };
type SuccessState = { status: 'success'; data: User[] };
type ErrorState   = { status: 'error';   error: string; retryable: boolean };
type EmptyState   = { status: 'empty';   message: string };

type DataState = LoadingState | SuccessState | ErrorState | EmptyState;

function renderState(state: DataState): string {
  switch (state.status) {
    case 'loading':
      // state: LoadingState
      return '⏳ Loading...';

    case 'success':
      // state: SuccessState — state.data is User[]
      return `✅ Loaded ${state.data.length} users`;
```

```
    case 'error':
      // state: ErrorState — state.error and state.retryable are available
      const retry = state.retryable ? ' (click to retry)' : '';
      return `✖ ${state.error}${retry}`;

    case 'empty':
      // state: EmptyState — state.message is available
      return `📭 ${state.message}`;
  }
}

// ── Real-world: Redux-style action narrowing ──────────────────
type LoginAction    = { type: 'LOGIN';    payload: { userId: number; token:
string } };
type LogoutAction   = { type: 'LOGOUT' };
type UpdateAction   = { type: 'UPDATE';   payload: Partial<User> };
type SetErrorAction = { type: 'SET_ERROR'; error: string };

type AppAction = LoginAction | LogoutAction | UpdateAction | SetErrorAction;

function reducer(state: AppState, action: AppAction): AppState {
  switch (action.type) {
    case 'LOGIN':
      // action: LoginAction — payload.userId and payload.token available
      return { ...state, user: action.payload, isLoggedIn: true };

    case 'LOGOUT':
      // action: LogoutAction — no payload
      return { ...state, user: null, isLoggedIn: false };

    case 'UPDATE':
      // action: UpdateAction — payload is Partial<User>
      return { ...state, user: { ...state.user!, ...action.payload } };

    case 'SET_ERROR':
      // action: SetErrorAction — error is string
      return { ...state, error: action.error };
  }
}
```

8. Custom Type Guards

Sometimes the built-in narrowing techniques aren't enough — your logic is too complex for a simple `typeof` or `instanceof` check. Custom type guards let you write a function that tells TypeScript: 'If this function returns true, the argument IS this specific type.'

The Magic: 'param is Type' Return Annotation

A normal function returns boolean: true or false.

A type guard function also returns boolean — but with a PROMISE to TypeScript:

```
function isUser(x: unknown): x is User { ... }
```

When this returns true, TypeScript BELIEVES you and narrows x to User.

When it returns false, TypeScript narrows x to everything EXCEPT User.

You are making a TYPE PROMISE. Keep it accurate or you'll have runtime bugs!

8.1 Basic Custom Type Guard

```
// ── Type guard syntax: function name(arg): arg is Type ────────
interface User {
  kind: 'user';
  id: number;
  name: string;
  email: string;
}

interface Bot {
  kind: 'bot';
  botId: string;
  apiKey: string;
}

type Actor = User | Bot;

// Custom type guard — returns 'actor is User' not just boolean
function isUser(actor: Actor): actor is User {
  return actor.kind === 'user';
  // When true → TypeScript knows actor is User
  // When false → TypeScript knows actor is NOT User (so it's Bot)
```

```
}

function isBot(actor: Actor): actor is Bot {
  return actor.kind === 'bot';
}

function processActor(actor: Actor): void {
  if (isUser(actor)) {
    // actor: User
    console.log(`User: ${actor.name} (${actor.email})`);
    sendWelcomeEmail(actor.email);
  } else {
    // actor: Bot
    console.log(`Bot: ${actor.botId}`);
    logBotActivity(actor.apiKey);
  }
}
```

8.2 Type Guards for unknown Type

```
// ── Validating unknown data (e.g. from API) ──────────────────
interface Product {
  id: number;
  name: string;
  price: number;
  inStock: boolean;
}

// Type guard to validate that unknown data has the Product shape
function isProduct(data: unknown): data is Product {
  return (
    typeof data === 'object' &&
    data !== null &&
    'id'      in data && typeof (data as Product).id      === 'number' &&
    'name'    in data && typeof (data as Product).name    === 'string' &&
    'price'   in data && typeof (data as Product).price   === 'number' &&
    'inStock' in data && typeof (data as Product).inStock === 'boolean'
  );
}

// Using it with real API data:
async function fetchProduct(id: number): Promise<Product> {
  const response = await fetch(`/api/products/${id}`);
```

```
  const data: unknown = await response.json();

  if (!isProduct(data)) {
    throw new Error('Invalid product data received from API');
  }

  // data: Product — guaranteed by our type guard
  return data;
}

// ── Array type guard ──────────────────────────────────────
function isStringArray(value: unknown): value is string[] {
  return (
    Array.isArray(value) &&
    value.every(item => typeof item === 'string')
  );
}

const tags: unknown = ['typescript', 'react', 'nodejs'];
if (isStringArray(tags)) {
  // tags: string[]
  tags.forEach(tag => console.log(tag.toUpperCase())); // ✅
}
```

8.3 Real-World Custom Type Guards

```
// ── isAdmin guard ─────────────────────────────────────────
interface RegularUser { role: 'user';  name: string; email: string }
interface AdminUser   { role: 'admin'; name: string; permissions: string[] }
type AppUser = RegularUser | AdminUser;

function isAdmin(user: AppUser): user is AdminUser {
  return user.role === 'admin';
}

function deleteResource(user: AppUser, resourceId: number): void {
  if (!isAdmin(user)) {
    throw new Error(`${user.name} is not authorized to delete`);
  }
  // user: AdminUser — permissions available
  if (!user.permissions.includes('delete')) {
    throw new Error('Missing delete permission');
```

```
  }
  console.log(`Admin ${user.name} deleted resource ${resourceId}`);
}

// ── isDefined guard — remove undefined ──────────────────────
function isDefined<T>(value: T | undefined | null): value is T {
  return value !== undefined && value !== null;
}

const values = [1, null, 2, undefined, 3, null, 4];
const definedValues = values.filter(isDefined);
// definedValues: number[] — null and undefined removed!
// Without this guard, filter would return (number | null | undefined)[]

// ── hasProp guard — check optional property ─────────────────
function hasProp<T extends object, K extends string>(
  obj: T,
  key: K
): obj is T & Record<K, unknown> {
  return key in obj;
}

interface UserBase { name: string }
interface UserWithEmail extends UserBase { email: string }

function sendEmail(user: UserBase): void {
  if (hasProp(user, 'email')) {
    // user: UserBase & { email: unknown }
    console.log(`Sending to: ${user.email}`);
  }
}
```

9. Assertion Functions

Assertion functions are a variant of type guards. Instead of returning `boolean`, they **throw an error** if the condition fails. TypeScript narrows the type AFTER the assertion call — if the function didn't throw, the value must be what the assertion says it is.

```
// ── Basic assertion function syntax ──────────────────────
// Return type: 'asserts condition' or 'asserts param is Type'

function assertIsString(value: unknown): asserts value is string {
```

```
  if (typeof value !== 'string') {
    throw new TypeError(`Expected string, got ${typeof value}`);
  }
  // If we reach here without throwing, value IS a string
}

function processInput(input: unknown): void {
  assertIsString(input);
  // HERE: input is narrowed to string — TypeScript trusts the assertion
  console.log(input.toUpperCase()); // ✅ Safe
  console.log(input.trim());        // ✅ Safe
}

// ── Real-world: assertDefined ──────────────────────────────
function assertDefined<T>(
  value: T | null | undefined,
  message = 'Value must be defined'
): asserts value is T {
  if (value === null || value === undefined) {
    throw new Error(message);
  }
}

function processOrder(orderId: string | null): void {
  assertDefined(orderId, 'Order ID is required');
  // orderId: string — null is excluded after assertion
  console.log(`Processing order: ${orderId.toUpperCase()}`); // ✅
}

// ── Real-world: assertIsUser ───────────────────────────────
function assertIsUser(data: unknown): asserts data is User {
  if (!isUser(data)) {
    throw new Error('Data is not a valid User');
  }
}

// After this call, data is narrowed to User:
assertIsUser(apiResponse.data);
console.log(apiResponse.data.name);  // ✅ TypeScript knows it's User
```

10. Control Flow Analysis

TypeScript's Control Flow Analysis (CFA) is what powers all narrowing. TypeScript tracks the type of every variable at every point in your code — including after returns, throws, and loops. This is automatic and often magical.

```
// ── TypeScript tracks narrowing through the entire function ──
function processValue(value: string | number | null): string {
  // Here: value is string | number | null

  if (value === null) {
    return 'null';
    // After return, TypeScript knows null is handled
  }
  // Here: value is string | number (null is excluded by the early return)

  if (typeof value === 'string') {
    return value.toUpperCase();
    // After return, TypeScript knows string is handled
  }
  // Here: value is number (string and null both excluded)

  return value.toFixed(2); // ✅ Only number methods available here
}

// ── CFA through assignments ──────────────────────────────────
let x: string | number = 'hello';
// x: string | number

x = 42;
// x: number — TypeScript KNOWS x is now a number
console.log(x.toFixed(2)); // ✅

x = 'world';
// x: string — TypeScript KNOWS x is now a string
console.log(x.toUpperCase()); // ✅

// ── CFA through loops ────────────────────────────────────────
function findFirstString(items: (string | number)[]): string | undefined {
  for (const item of items) {
    if (typeof item === 'string') {
      return item; // item: string here
    }
  }
  return undefined;
```

```
}

// ── The 'never' type in CFA — exhaustive check ──────────────
type Shape = 'circle' | 'square' | 'triangle';

function getArea(shape: Shape, size: number): number {
  if (shape === 'circle')   return Math.PI * size * size;
  if (shape === 'square')   return size * size;
  if (shape === 'triangle') return (size * size) / 2;

  // TypeScript knows: if we reach here, shape is 'never'
  // (all cases handled above)
  const _exhaustive: never = shape;
  throw new Error(`Unknown shape: ${shape}`);
}
// Add 'pentagon' to Shape → TypeScript ERRORS at _exhaustive line
// Forcing you to handle the new case. Can't miss it!
```

11. All Narrowing Techniques — Quick Reference

Technique	Syntax	Best Used For	Example
typeof	typeof x === 'string'	Primitive types	string, number, boolean, undefined
instanceof	x instanceof MyClass	Class instances	Error, Date, custom classes
in operator	'prop' in obj	Objects with diff shapes	SuccessResult vs ErrorResult
Equality	x === 'value'	Literal types & null	status === 'shipped', x === null
Truthiness	if (x)	null/undefined removal	if (user), if (config.key)
Discriminant	switch(x.kind)	Union of objects	'circle'|'rect'|'triangle'
Custom guard	fn(x): x is Type	Complex validation	isAdmin(), isProduct()
Assertion fn	asserts x is Type	Throw if wrong type	assertDefined(), assertIsUser()
Assignment	x = 'hello'	After reassignment	TypeScript re-infers the type
Control flow	Early return/throw	After impossible paths	Exclude handled cases

12. Complete Real-World Example — Form Validation System

Let's build a complete form validation system that uses every narrowing technique we've learned.

```
// ═══ Types ═══════════════════════════════════════
type FieldType = 'text' | 'email' | 'number' | 'phone' | 'date';

interface TextField    { type: 'text';   value: string;  minLength?: number;
maxLength?: number }
interface EmailField   { type: 'email';  value: string }
interface NumberField  { type: 'number'; value: number;  min?: number; max?:
number }
interface PhoneField   { type: 'phone';  value: string;  countryCode: string }
interface DateField    { type: 'date';   value: Date;    minDate?: Date;
maxDate?: Date }

type FormField = TextField | EmailField | NumberField | PhoneField |
DateField;

interface ValidationResult {
  isValid: boolean;
  errors: string[];
}

// ═══ Custom type guards ═══════════════════════════
function isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function isValidPhone(phone: string): boolean {
  return /^[6-9]\d{9}$/.test(phone);  // Indian phone validation
}

// ═══ Main validator using discriminant narrowing ═══
function validateField(field: FormField): ValidationResult {
  const errors: string[] = [];

  switch (field.type) {
    case 'text':
      // field: TextField
      if (!field.value.trim()) {
        errors.push('Text field cannot be empty');
```

```
      }
      if (field.minLength && field.value.length < field.minLength) {
        errors.push(`Minimum ${field.minLength} characters required`);
      }
      if (field.maxLength && field.value.length > field.maxLength) {
        errors.push(`Maximum ${field.maxLength} characters allowed`);
      }
      break;

    case 'email':
      // field: EmailField
      if (!field.value) {
        errors.push('Email is required');
      } else if (!isValidEmail(field.value)) {
        errors.push('Invalid email format');
      }
      break;

    case 'number':
      // field: NumberField
      if (typeof field.value !== 'number' || isNaN(field.value)) {
        errors.push('Must be a valid number');
      } else {
        if (field.min !== undefined && field.value < field.min) {
          errors.push(`Minimum value is ${field.min}`);
        }
        if (field.max !== undefined && field.value > field.max) {
          errors.push(`Maximum value is ${field.max}`);
        }
      }
      break;

    case 'phone':
      // field: PhoneField
      if (!isValidPhone(field.value)) {
        errors.push(`Invalid phone number for ${field.countryCode}`);
      }
      break;

    case 'date':
      // field: DateField
      if (!(field.value instanceof Date)) {
        errors.push('Must be a valid date');
```

```
      } else {
        if (field.minDate && field.value < field.minDate) {
          errors.push(`Date must be after ${field.minDate.toDateString()}`);
        }
        if (field.maxDate && field.value > field.maxDate) {
          errors.push(`Date must be before ${field.maxDate.toDateString()}`);
        }
      }
      break;
  }

  return { isValid: errors.length === 0, errors };
}

// ═══ Usage ═══════════════════════════════════════════
const nameResult = validateField({
  type: 'text', value: 'Rahul', minLength: 2, maxLength: 50
});
// { isValid: true, errors: [] }

const emailResult = validateField({
  type: 'email', value: 'not-an-email'
});
// { isValid: false, errors: ['Invalid email format'] }

const ageResult = validateField({
  type: 'number', value: 15, min: 18, max: 100
});
// { isValid: false, errors: ['Minimum value is 18'] }
```

13. Interview Questions — Chapter 9

Q: What is Type Narrowing in TypeScript?

Answer: Type narrowing is when TypeScript automatically refines a broad type (like string | number | null) to a more specific type inside a particular code block, based on conditions you write.

For example, if you write: if (typeof x === 'string') — inside that block, TypeScript KNOWS x is a string, not a number or null. It narrows the type from string | number | null to just string.

TypeScript does this through Control Flow Analysis (CFA) — it tracks which types are possible at each point in your code based on checks you've done.

Q: What are the different ways to narrow types in TypeScript?

Answer: There are 8+ narrowing techniques:

1. typeof — for primitives: typeof x === 'string'
2. instanceof — for class instances: x instanceof Error
3. 'in' operator — for property existence: 'email' in user
4. Equality checks — x === null, x === 'specific-value'
5. Truthiness — if (x) — removes null, undefined, 0, empty string
6. Discriminant property — switch(x.kind) on a shared literal field
7. Custom type guards — function isUser(x): x is User
8. Assertion functions — function assertDefined(x): asserts x is T

TypeScript tracks narrowing through assignments, loops, returns, and throws.

Q: What is a custom type guard? How does the 'is' keyword work?

Answer: A custom type guard is a function with a special return type called a type predicate: 'parameter is Type'.

Syntax: function isUser(x: unknown): x is User { return ... }

When this function returns true, TypeScript NARROWS the type of x to User in the calling code. When it returns false, TypeScript knows x is NOT User.

The 'is' keyword is what makes it a type predicate — it tells TypeScript: 'If this function says true, you can trust that the argument is this specific type.'

Use custom type guards when built-in narrowing isn't enough — complex validation, API response checking, or reusable named checks like isAdmin(), isDefined().

Q: What is the difference between a type guard and an assertion function?

Answer: Both narrow types, but differently:

Type guard: returns boolean. If true → type is narrowed. If false → type is NOT narrowed (you handle it).

```
function isString(x): x is string { return typeof x === 'string'; }
if (isString(x)) { /* x is string here */ }
```

Assertion function: returns void but THROWS if the condition fails. After the call, TypeScript assumes the type is narrowed (because if it wasn't, an error would have been thrown).

```
function assertIsString(x): asserts x is string { if (typeof x !== 'string') throw new Error(); }
assertIsString(x);
// x is string here — TypeScript trusts it because no error was thrown
```

Q: Why is typeof null === 'object' a problem? How do you handle it?

Answer: typeof null returns 'object' — this is a famous JavaScript bug from the early days. This means typeof alone cannot distinguish null from a real object.

Problem: if (typeof x === 'object') — this matches null AND objects.

Solutions:

1. Always check for null explicitly: if (x !== null && typeof x === 'object')
2. Use truthiness: if (x) — null is falsy, so this works
3. Use loose equality: if (x != null) — narrows out both null AND undefined
4. Use optional chaining: x?.property instead of x && x.property

Q: What is Control Flow Analysis (CFA) in TypeScript?

Answer: CFA is the mechanism TypeScript uses to track what types are possible at each point in your code. It analyses every conditional, assignment, return, throw, and loop to build a picture of what a variable's type can be.

Example: After an early return that handles null, TypeScript removes null from the type for all code below that return. After x = 42, TypeScript knows x is number even if its declared type was string | number.

CFA also enables the 'never' exhaustive check pattern — after handling all cases in a switch, the remaining type is 'never'. If TypeScript can reach your default case with a non-never type, it means you missed a case.

Q: What is the 'never' type used for in narrowing?

Answer: 'never' appears in narrowing when TypeScript determines that a code path is UNREACHABLE — all possible types have been handled.

The exhaustive check pattern uses this: at the end of a switch that handles ALL union members, you can write:

```
const _check: never = value;
```

If all cases are handled, TypeScript sees _check: never = never — OK.

If you ADD a new union member but forget to handle it, TypeScript errors because _check: never = 'newValue' — 'newValue' is not assignable to never.

This is a compile-time guarantee that you've handled every case.

Rapid Fire — Chapter 9

Question	One-Line Answer
What does typeof null return?	'object' — a famous JS quirk, handle with === null check
What narrowing does 'if (x)' do?	Removes null, undefined, 0, '', NaN, false from the type
What does 'x is Type' in a function return mean?	It's a type predicate — TypeScript narrows x to Type when the function returns true
What is CFA?	Control Flow Analysis — TS tracks types at every code point
When does TypeScript see 'never' in a switch?	When all union cases are handled — remaining type is impossible
Can truthiness narrow out empty string?	Yes — '' is falsy in JavaScript
What does assertDefined(x) do?	Throws if x is null/undefined, otherwise x is narrowed to NonNullable<T>
Can 'in' narrow class instances?	It works but instanceof is preferred for class instances
What happens to type after an early return?	TypeScript removes the handled case from the type below
Which narrowing is best for discriminated unions?	Discriminant property narrowing — switch(x.kind)

Chapter 9 Key Takeaways

Type narrowing = TypeScript tracking which type a variable ACTUALLY is at each code point.

typeof: for primitives. Remember typeof null === 'object' — handle null separately!

instanceof: for class instances. Best for Error subclasses and your own classes.

'in' operator: narrows by checking property existence — great for union of object shapes.

Equality (===): narrows literals and removes null/undefined precisely.

Truthiness (if x): removes null, undefined, 0, empty string — convenient but broad.

Discriminant narrowing: switch(x.kind) — the BEST pattern for union of objects.

Custom type guard: function fn(x): x is Type — reusable, semantic, powerful.

Assertion function: function fn(x): asserts x is Type — throws instead of returning false.

never exhaustive check: after handling all union cases, assign to never to catch missed cases.

CFA: TypeScript AUTOMATICALLY tracks narrowing through if/switch/return/throw/assignments.

End of Chapter 9

CHAPTER 10

Classes in TypeScript

Class Basics • Access Modifiers • Inheritance • Abstract Classes • Interfaces • Decorators • SOLID

1. What are Classes? — The Big Picture

A class is a blueprint for creating objects. It bundles data (properties) and behaviour (methods) into a single reusable unit. TypeScript builds on JavaScript's ES6 classes by adding access modifiers, type annotations, abstract classes, and more — making them far more powerful and safe.

Real-World Analogy — The Car Factory

A car factory has a BLUEPRINT (class) for a car model.

Every car built from that blueprint is an INSTANCE (object).

Blueprint (class) defines:

→ Properties: color, model, engineSize, fuelType

→ Methods: start(), accelerate(), brake(), refuel()

From ONE blueprint you can build THOUSANDS of cars.

Each car has its own color, model, mileage — but all follow the same blueprint.

TypeScript classes are exactly this — a blueprint for objects,

with type safety baked in at every step.

1.1 JavaScript Class vs TypeScript Class

```
// ── JavaScript class (no type safety) ──────────────────────
class UserJS {
  constructor(name, email, age) {  // No types — anything goes
    this.name  = name;
    this.email = email;
    this.age   = age;
  }
}
```

```
const u = new UserJS(42, true, 'oops');  // JS allows this — disaster!

// ── TypeScript class (fully type-safe) ──────────────────────
class User {
  name:  string;
  email: string;
  age:   number;

  constructor(name: string, email: string, age: number) {
    this.name  = name;
    this.email = email;
    this.age   = age;
  }

  greet(): string {
    return `Hi, I'm ${this.name} (${this.age})`;
  }

  isAdult(): boolean {
    return this.age >= 18;
  }
}

const u = new User(42, true, 'oops');     // ✗ ERROR at compile time
const u = new User('Rahul', 'r@g.com', 28); // ✓ Correct

console.log(u.greet());    // 'Hi, I'm Rahul (28)'
console.log(u.isAdult());  // true
console.log(u.nonExistent); // ✗ ERROR — property doesn't exist
```

2. Access Modifiers

TypeScript adds three access modifiers that control who can access a class member: `public`, `private`, and `protected`. Understanding these is critical for writing well-encapsulated, maintainable code.

Real-World Analogy — The Hospital

Think of a hospital building with different access levels:

PUBLIC → Reception, cafeteria — anyone can enter

PRIVATE → Operating theatre, medicine storage — only doctors/staff of THAT dept.
PROTECTED→ Staff corridors — doctors AND their students (subclasses) can enter

public = anyone can access (default)
private = only THIS class can access
protected = this class AND any class that extends it

2.1 public (Default)

```
// ── public — accessible from anywhere ──────────────────────
class Product {
  public name: string;     // explicit public
  public price: number;
  inStock: boolean;        // public by default (no modifier = public)

  constructor(name: string, price: number, inStock: boolean) {
    this.name    = name;
    this.price   = price;
    this.inStock = inStock;
  }

  public getDisplayPrice(): string {
    return `₹${this.price.toLocaleString('en-IN')}`;
  }
}

const phone = new Product('iPhone 15', 79999, true);

// All public — accessible everywhere:
console.log(phone.name);             // ✅ 'iPhone 15'
console.log(phone.price);            // ✅ 79999
console.log(phone.getDisplayPrice()); // ✅ '₹79,999'
phone.price = 74999;                  // ✅ Can change public property
```

2.2 private — Encapsulation

```
// ── private — only accessible inside THIS class ──────────────
class BankAccount {
  private balance: number;          // Only BankAccount can touch this
  private readonly accountNumber: string; // Private + readonly
  public  owner: string;
```

```
  constructor(owner: string, initialBalance: number) {
    this.owner         = owner;
    this.balance       = initialBalance;
    this.accountNumber = this.generateAccountNumber();
  }

  private generateAccountNumber(): string {
    return 'ACC' + Math.random().toString(36).slice(2, 10).toUpperCase();
  }

  // Public API — controlled access to private balance
  public deposit(amount: number): void {
    if (amount <= 0) throw new Error('Amount must be positive');
    this.balance += amount;
    console.log(`Deposited ₹${amount}. New balance: ₹${this.balance}`);
  }

  public withdraw(amount: number): void {
    if (amount > this.balance) throw new Error('Insufficient funds');
    this.balance -= amount;
    console.log(`Withdrawn ₹${amount}. New balance: ₹${this.balance}`);
  }

  public getBalance(): number {
    return this.balance;  // Controlled read access
  }
}

const acc = new BankAccount('Rahul', 10000);

acc.deposit(5000);          // ✅ Public method — allowed
acc.withdraw(3000);         // ✅ Public method — allowed
console.log(acc.getBalance()); // ✅ 12000

acc.balance = 1000000;      // ❌ ERROR: 'balance' is private
acc.generateAccountNumber(); // ❌ ERROR: method is private
console.log(acc.accountNumber); // ❌ ERROR: 'accountNumber' is private
```

2.3 protected — Inheritance-Friendly

```
// ── protected — accessible in this class AND subclasses ────────
```

```
class Animal {
  protected name: string;     // Subclasses CAN access
  private   dna: string;      // Subclasses CANNOT access
  public    species: string;  // Everyone can access

  constructor(name: string, species: string) {
    this.name    = name;
    this.species = species;
    this.dna     = 'classified';
  }

  protected makeSound(sound: string): void {
    console.log(`${this.name} says: ${sound}`);
  }
}

class Dog extends Animal {
  private breed: string;

  constructor(name: string, breed: string) {
    super(name, 'Canis lupus familiaris');
    this.breed = breed;
  }

  bark(): void {
    this.makeSound('Woof!');     // ✅ Can access protected method
    console.log(this.name);      // ✅ Can access protected property
    // console.log(this.dna);   // ❌ ERROR: dna is private to Animal
  }

  introduce(): string {
    return `I'm ${this.name}, a ${this.breed}`;  // ✅ name is protected
  }
}

const dog = new Dog('Bruno', 'Labrador');
dog.bark();          // ✅ 'Bruno says: Woof!'
dog.introduce();     // ✅
console.log(dog.species); // ✅ Public — accessible
console.log(dog.name);    // ❌ ERROR: 'name' is protected — not accessible
outside
```

2.4 readonly — Immutable Properties

```
// ── readonly — can only be set once (in constructor) ──────────
class Employee {
  readonly employeeId: string;   // Set once, never changed
  readonly joinDate: Date;
  public   name: string;
  public   salary: number;

  constructor(name: string, salary: number) {
    this.name       = name;
    this.salary     = salary;
    this.employeeId = 'EMP' + Date.now();
    this.joinDate   = new Date();
  }
}

const emp = new Employee('Priya', 80000);
emp.name   = 'Priya Sharma';  // ✅ name is mutable
emp.salary = 90000;           // ✅ salary is mutable
emp.employeeId = 'HACKED';    // ❌ ERROR: employeeId is readonly
emp.joinDate   = new Date();  // ❌ ERROR: joinDate is readonly
```

2.5 Access Modifier Shorthand in Constructor

```
// ── Shorthand: declare + assign in one step ────────────────────
// Long way (repetitive):
class ProductLong {
  public  name: string;
  private price: number;
  readonly id: number;

  constructor(name: string, price: number, id: number) {
    this.name  = name;
    this.price = price;
    this.id    = id;
  }
}

// ✅ Shorthand way — MUCH cleaner (same result):
class Product {
  constructor(
    public  name:    string,
```

```
    private price:   number,
    readonly id:     number,
    protected category: string = 'General'  // with default
  ) {}
  // TypeScript automatically creates properties and assigns them!
}

const p = new Product('iPhone', 79999, 1);
console.log(p.name);     // ✅ 'iPhone'
console.log(p.id);       // ✅ 1
// console.log(p.price); // ❌ private

// ── Real-world example with shorthand ──────────────────────
class OrderItem {
  constructor(
    public  readonly productId:   number,
    public  readonly productName: string,
    public  quantity:             number,
    public  readonly unitPrice:   number,
  ) {}

  get total(): number {
    return this.quantity * this.unitPrice;
  }
}

const item = new OrderItem(1, 'iPhone 15', 2, 79999);
console.log(item.total); // 159998
item.quantity = 3;       // ✅ quantity is mutable
item.unitPrice = 0;      // ❌ readonly
```

3. Getters and Setters

Getters (`get`) and setters (`set`) let you access a property like a field while running custom logic. Getters are used for computed properties and controlled reads; setters are used for validation before assignment.

```
// ── Getters and setters ────────────────────────────────────
class Temperature {
  private _celsius: number;
```

```
  constructor(celsius: number) {
    this._celsius = celsius;
  }

  // Getter — read like a property: temp.fahrenheit
  get fahrenheit(): number {
    return this._celsius * 9/5 + 32;
  }

  // Setter — set like a property: temp.celsius = 30
  get celsius(): number {
    return this._celsius;
  }

  set celsius(value: number) {
    if (value < -273.15) {
      throw new Error('Temperature below absolute zero is not possible');
    }
    this._celsius = value;
  }
}

const temp = new Temperature(100);
console.log(temp.fahrenheit); // 212 — getter: no () needed
temp.celsius = 37;            // setter: validated before setting
temp.celsius = -300;          // ✕ throws Error

// ── Real-world: User with password hashing ──────────────────
class User {
  private _passwordHash: string = '';
  constructor(
    public readonly id: number,
    public name: string,
    public email: string,
  ) {}

  // Write-only setter — can SET but not GET raw password
  set password(raw: string) {
    if (raw.length < 8) throw new Error('Password too short (min 8 chars)');
    this._passwordHash = this.hash(raw);
  }

  get passwordHash(): string {
```

```
    return this._passwordHash;
  }

  private hash(value: string): string {
    return Buffer.from(value).toString('base64');
  }

  get displayName(): string {
    return `${this.name} <${this.email}>`;
  }
}

const user = new User(1, 'Rahul', 'r@g.com');
user.password = 'secret123';     // ✅ setter validates + hashes
user.password = 'weak';          // ❌ throws — too short
console.log(user.displayName);   // 'Rahul <r@g.com>'
console.log(user.passwordHash);  // hashed value — safe to log
```

4. Static Members

Static properties and methods belong to the **class itself**, not to instances. You access them via the class name, not via an object. Perfect for shared state, factory methods, counters, and utility functions.

```
// ── Static properties and methods ──────────────────────────
class Counter {
  private static instanceCount: number = 0;  // Shared across ALL instances
  public  readonly id: number;

  constructor() {
    Counter.instanceCount++;
    this.id = Counter.instanceCount;
  }

  static getCount(): number {
    return Counter.instanceCount;
  }

  static reset(): void {
    Counter.instanceCount = 0;
  }
```

```
}

const c1 = new Counter(); // id = 1
const c2 = new Counter(); // id = 2
const c3 = new Counter(); // id = 3

console.log(Counter.getCount()); // 3 — accessed via CLASS not instance
Counter.reset();
console.log(Counter.getCount()); // 0

// ── Static factory methods — common pattern ──────────────────
class Temperature {
  private constructor(private readonly celsius: number) {}
  // private constructor prevents: new Temperature(37) — must use factory

  static fromCelsius(c: number): Temperature {
    return new Temperature(c);
  }

  static fromFahrenheit(f: number): Temperature {
    return new Temperature((f - 32) * 5 / 9);
  }

  static fromKelvin(k: number): Temperature {
    return new Temperature(k - 273.15);
  }

  get fahrenheit(): number { return this.celsius * 9/5 + 32; }
  get kelvin(): number     { return this.celsius + 273.15; }
  toString(): string       { return `${this.celsius}°C`; }
}

const bodyTemp    = Temperature.fromCelsius(37);
const boilingPoint = Temperature.fromFahrenheit(212);
const absoluteZero = Temperature.fromKelvin(0);

console.log(bodyTemp.toString());    // '37°C'
console.log(boilingPoint.toString()); // '100°C'

// ── Real-world: Singleton pattern ────────────────────────────
class DatabaseConnection {
  private static instance: DatabaseConnection | null = null;
```

```
  private isConnected: boolean = false;

  private constructor(private readonly url: string) {}

  static getInstance(url: string): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection(url);
    }
    return DatabaseConnection.instance;
  }

  connect(): void {
    if (!this.isConnected) {
      console.log(`Connected to ${this.url}`);
      this.isConnected = true;
    }
  }
}

const db1 = DatabaseConnection.getInstance('mongodb://localhost');
const db2 = DatabaseConnection.getInstance('mongodb://localhost');
console.log(db1 === db2); // true — same instance!
```

5. Inheritance — extends

Inheritance lets a class **inherit properties and methods** from a parent class. The child class gets everything from the parent and can add its own properties and override parent methods. You use the `extends` keyword.

Real-World Analogy — Company Hierarchy

All employees share common traits: name, employeeId, joinDate, getPayslip()

Engineer extends Employee → adds: techStack, codeReview()
Manager extends Employee → adds: teamSize, conductReview()
Designer extends Employee → adds: tools, createMockup()

Each inherits the common Employee base — no code duplication.
Each adds their own specialisation on top.

```
// ── Base class ──────────────────────────────────────────────
class Employee {
  readonly employeeId: string;
  readonly joinDate: Date;

  constructor(
    public name:       string,
    public email:      string,
    public department: string,
    protected salary:  number,
  ) {
    this.employeeId = 'EMP' + Date.now();
    this.joinDate   = new Date();
  }

  getPayslip(): string {
    return `${this.name} | Dept: ${this.department} | Salary:
₹${this.salary}`;
  }

  // Can be overridden in subclasses
  describe(): string {
    return `Employee: ${this.name}`;
  }
}

// ── Subclass 1 ──────────────────────────────────────────────
class Engineer extends Employee {
  constructor(
    name: string,
    email: string,
    salary: number,
    public techStack: string[],
    public level: 'junior' | 'mid' | 'senior' | 'lead',
  ) {
    // super() must be called FIRST in subclass constructor
    super(name, email, 'Engineering', salary);
  }

  codeReview(pr: string): string {
    return `${this.name} reviewed PR: ${pr}`;
  }
```

```
  // Override parent method
  override describe(): string {
    return `Engineer (${this.level}): ${this.name} — ${this.techStack.join(',
')}`;
  }

  // Access protected salary from parent
  getBonus(): number {
    return this.salary * (this.level === 'senior' ? 0.2 : 0.1);
  }
}

// ── Subclass 2 ──────────────────────────────────────────────
class Manager extends Employee {
  constructor(
    name: string,
    email: string,
    salary: number,
    public teamSize: number,
    public reports: Employee[],
  ) {
    super(name, email, 'Management', salary);
  }

  conductReview(employee: Employee): string {
    return `${this.name} reviewed ${employee.name}`;
  }

  override describe(): string {
    return `Manager: ${this.name} — team of ${this.teamSize}`;
  }
}

// ── Using the hierarchy ─────────────────────────────────────
const eng = new Engineer('Rahul', 'r@g.com', 90000, ['TypeScript', 'React'],
'senior');
const mgr = new Manager('Priya', 'p@g.com', 150000, 8, [eng]);

console.log(eng.describe());     // 'Engineer (senior): Rahul — TypeScript,
React'
console.log(mgr.describe());     // 'Manager: Priya — team of 8'
console.log(eng.getPayslip());   // inherited from Employee
console.log(eng.getBonus());     // 18000 (20% for senior)
console.log(mgr.conductReview(eng)); // 'Priya reviewed Rahul'
```

```
// instanceof checks work with inheritance:
console.log(eng instanceof Engineer);  // true
console.log(eng instanceof Employee);  // true — Engineer IS-A Employee
console.log(mgr instanceof Engineer);  // false
```

6. Abstract Classes

An abstract class is a **partially implemented base class** that **cannot be instantiated directly**. It defines a contract that all subclasses must follow — some methods are fully implemented, others are declared `abstract` (meaning 'subclasses MUST implement this').

Real-World Analogy — A Legal Contract Template

A law firm has a CONTRACT TEMPLATE that says:

→ Sections 1-3 are pre-filled (fully implemented in abstract class)

→ Section 4 (Specific Obligations) MUST be filled by each client (abstract method)

You CANNOT submit the template directly — it's incomplete.

Every client fills in Section 4 differently.

Abstract class = a template with some blanks that MUST be filled by subclasses.

```
// ── Abstract class syntax ──────────────────────────────────
abstract class Shape {
  constructor(protected color: string) {}

  // Abstract method — NO implementation here
  // Every subclass MUST provide their own implementation
  abstract calculateArea(): number;
  abstract calculatePerimeter(): number;

  // Concrete method — shared implementation for all subclasses
  describe(): string {
    return `A ${this.color} shape with area
${this.calculateArea().toFixed(2)}`;
  }
}

// new Shape('red'); // ✗ ERROR: Cannot instantiate abstract class
```

```
class Circle extends Shape {
  constructor(color: string, private radius: number) {
    super(color);
  }

  calculateArea(): number {
    return Math.PI * this.radius ** 2;  // MUST implement abstract method
  }

  calculatePerimeter(): number {
    return 2 * Math.PI * this.radius;
  }
}

class Rectangle extends Shape {
  constructor(color: string, private width: number, private height: number) {
    super(color);
  }

  calculateArea(): number      { return this.width * this.height; }
  calculatePerimeter(): number { return 2 * (this.width + this.height); }
}

const shapes: Shape[] = [
  new Circle('red', 5),
  new Rectangle('blue', 4, 6),
];

shapes.forEach(s => console.log(s.describe()));
// 'A red shape with area 78.54'
// 'A blue shape with area 24.00'
```

6.2 Real-World Abstract Class — Payment Gateway

```
// ── Abstract payment gateway ──────────────────────────────
abstract class PaymentGateway {
  protected readonly merchantId: string;
  protected readonly currency: string;

  constructor(merchantId: string, currency: string = 'INR') {
    this.merchantId = merchantId;
```

```
    this.currency   = currency;
  }

  // Abstract — each gateway has different implementation
  abstract initiatePayment(amount: number, orderId: string): Promise<string>;
  abstract verifyPayment(paymentId: string): Promise<boolean>;
  abstract refundPayment(paymentId: string, amount: number): Promise<boolean>;

  // Concrete — shared logic for all gateways
  protected formatAmount(amount: number): string {
    return `${this.currency} ${amount.toFixed(2)}`;
  }

  protected log(message: string): void {
    console.log(`[${this.constructor.name}] ${message}`);
  }

  async processOrder(amount: number, orderId: string): Promise<void> {
    this.log(`Processing ${this.formatAmount(amount)} for order ${orderId}`);
    const paymentId = await this.initiatePayment(amount, orderId);
    const isVerified = await this.verifyPayment(paymentId);
    if (!isVerified) throw new Error('Payment verification failed');
    this.log(`Order ${orderId} paid successfully`);
  }
}

// ── Concrete implementation 1: Razorpay ──────────────────────
class RazorpayGateway extends PaymentGateway {
  constructor(merchantId: string, private apiKey: string) {
    super(merchantId, 'INR');
  }

  async initiatePayment(amount: number, orderId: string): Promise<string> {
    this.log(`Creating Razorpay order for ${this.formatAmount(amount)}`);
    return `rzp_${orderId}_${Date.now()}`;
  }

  async verifyPayment(paymentId: string): Promise<boolean> {
    this.log(`Verifying Razorpay payment ${paymentId}`);
    return true; // Call Razorpay verify API
  }

  async refundPayment(paymentId: string, amount: number): Promise<boolean> {
```

```
    this.log(`Refunding ${this.formatAmount(amount)} for ${paymentId}`);
    return true;
  }
}

// ── Concrete implementation 2: Stripe ──────────────────────
class StripeGateway extends PaymentGateway {
  constructor(merchantId: string, private secretKey: string) {
    super(merchantId, 'USD');
  }

  async initiatePayment(amount: number, orderId: string): Promise<string> {
    this.log(`Creating Stripe PaymentIntent for
${this.formatAmount(amount)}`);
    return `pi_${orderId}_${Date.now()}`;
  }

  async verifyPayment(paymentId: string): Promise<boolean> { return true; }
  async refundPayment(paymentId: string, amount: number): Promise<boolean>
{ return true; }
}

// ── Usage — polymorphism in action ──────────────────────────
function getGateway(provider: 'razorpay' | 'stripe'): PaymentGateway {
  if (provider === 'razorpay')
    return new RazorpayGateway('MERCH_001', 'rzp_key_xxx');
  return new StripeGateway('MERCH_001', 'sk_live_xxx');
}

const gateway = getGateway('razorpay');
await gateway.processOrder(1299, 'ORD-001'); // Works for ANY gateway
```

7. Implementing Interfaces

A class can `implement` one or more interfaces — promising that it will provide all the required properties and methods. This is how TypeScript enforces contracts and enables polymorphism without inheritance.

```
// ── Interface as a contract ──────────────────────────────
interface Serializable {
  serialize(): string;
  deserialize(data: string): void;
```

```
}

interface Cacheable {
  readonly cacheKey: string;
  ttl: number;  // time-to-live in seconds
}

interface Loggable {
  log(level: 'info' | 'warn' | 'error', message: string): void;
}

// ── Class implementing multiple interfaces ──────────────────────
class UserSession implements Serializable, Cacheable, Loggable {
  readonly cacheKey: string;
  ttl: number = 3600;

  constructor(
    public readonly userId: number,
    public readonly token: string,
  ) {
    this.cacheKey = `session:${userId}`;
  }

  serialize(): string {
    return JSON.stringify({ userId: this.userId, token: this.token });
  }

  deserialize(data: string): void {
    const parsed = JSON.parse(data);
    Object.assign(this, parsed);
  }

  log(level: 'info' | 'warn' | 'error', message: string): void {
    console.log(`[${level.toUpperCase()}][Session:${this.userId}]
${message}`);
  }
}

const session = new UserSession(42, 'jwt_token_xyz');
const serialized = session.serialize();
session.log('info', 'Session created');
console.log(session.cacheKey); // 'session:42'
```

```
// ── Interface polymorphism ──────────────────────────────────
// Any class implementing Serializable can be used here:
function saveToCache(item: Cacheable & Serializable): void {
  redis.set(item.cacheKey, item.serialize(), item.ttl);
}

saveToCache(session); // Works because UserSession implements both
```

8. Abstract Class vs Interface — When to Use Which

	Abstract Class	Interface
Can have implementation?	✅ Yes — concrete methods	❌ No — only declarations (pre TS 4.9)
Can have constructor?	✅ Yes	❌ No
Access modifiers?	✅ public/private/protected	❌ No — all implicitly public
Multiple inheritance?	❌ Only ONE extends	✅ implements multiple interfaces
Generates JS code?	✅ Yes — a real JS class	❌ No — erased at compile time
Use for?	Shared base with some logic	Pure contracts / shapes
Real-world example	PaymentGateway, Animal	Serializable, Cacheable, Loggable

The Golden Rule

Use INTERFACE when: You only want to define a contract (what the class must do).
No shared implementation needed.
Multiple types should implement it (not related by inheritance).

Use ABSTRACT CLASS when: You have SHARED implementation some subclasses should inherit.
There's a real IS-A relationship (Dog IS-A Animal).
You want some methods to be optional (concrete) and some required (abstract).

9. Class with Generics

```
// ── Generic class — builds on Chapter 8 ──────────────────────
```

```typescript
class GenericRepository<T extends { id: number }> {
  private items: Map<number, T> = new Map();

  add(item: T): void {
    this.items.set(item.id, item);
  }

  findById(id: number): T | undefined {
    return this.items.get(id);
  }

  findAll(): T[] {
    return Array.from(this.items.values());
  }

  update(id: number, changes: Partial<T>): T | undefined {
    const item = this.items.get(id);
    if (!item) return undefined;
    const updated = { ...item, ...changes };
    this.items.set(id, updated);
    return updated;
  }

  remove(id: number): boolean {
    return this.items.delete(id);
  }

  get count(): number {
    return this.items.size;
  }
}

// One class, THREE different typed repositories:
interface User    { id: number; name: string; email: string }
interface Product { id: number; name: string; price: number }
interface Order   { id: number; total: number; status: string }

const userRepo    = new GenericRepository<User>();
const productRepo = new GenericRepository<Product>();
const orderRepo   = new GenericRepository<Order>();

userRepo.add({ id: 1, name: 'Rahul', email: 'r@g.com' });
```

```
userRepo.add({ id: 2, name: 'Priya', email: 'p@g.com' });

const user = userRepo.findById(1);  // User | undefined — fully typed!
user?.name;  // ✅ TypeScript knows this is User

productRepo.add({ id: 1, name: 'iPhone', price: 79999 });
productRepo.update(1, { price: 74999 }); // Partial<Product>
console.log(productRepo.count); // 1
```

10. SOLID Principles in TypeScript Classes

SOLID is a set of five design principles for writing maintainable object-oriented code. TypeScript's class system is perfectly suited for applying all five.

S — Single Responsibility Principle

```
// ❌ BAD — UserService does too many things
class UserService {
  createUser(data: CreateUserDto): User { ... }
  sendWelcomeEmail(user: User): void { ... }  // Email responsibility
  saveToDatabase(user: User): void { ... }    // DB responsibility
  validateInput(data: CreateUserDto): void { ... } // Validation
responsibility
}

// ✅ GOOD — each class has one responsibility
class UserValidator {
  validate(data: CreateUserDto): void { ... }
}

class UserRepository {
  save(user: User): void { ... }
  findById(id: number): User | undefined { ... }
}

class EmailService {
  sendWelcome(user: User): void { ... }
}

class UserService {
  constructor(
```

```
    private validator: UserValidator,
    private repo: UserRepository,
    private email: EmailService,
  ) {}

  createUser(data: CreateUserDto): User {
    this.validator.validate(data);
    const user = buildUser(data);
    this.repo.save(user);
    this.email.sendWelcome(user);
    return user;
  }
}
```

O — Open/Closed Principle

```
// Open for EXTENSION, Closed for MODIFICATION
// Use abstract class / interface instead of modifying existing code

abstract class DiscountStrategy {
  abstract calculate(price: number): number;
}

class NoDiscount extends DiscountStrategy {
  calculate(price: number): number { return price; }
}

class PercentDiscount extends DiscountStrategy {
  constructor(private percent: number) { super(); }
  calculate(price: number): number { return price * (1 - this.percent /
100); }
}

class FlatDiscount extends DiscountStrategy {
  constructor(private amount: number) { super(); }
  calculate(price: number): number { return Math.max(0, price -
this.amount); }
}

// Adding NEW discount types doesn't modify existing code:
class BuyOneGetOne extends DiscountStrategy {
  calculate(price: number): number { return price / 2; }
}
```

```
class Order {
  constructor(
    private price: number,
    private discount: DiscountStrategy
  ) {}

  getFinalPrice(): number {
    return this.discount.calculate(this.price);
  }
}
```

D — Dependency Inversion Principle

```
// Depend on ABSTRACTIONS, not concrete implementations

// ✗ BAD — tightly coupled to a specific implementation
class OrderService {
  private db = new MySQLDatabase();  // Directly depends on MySQL
  // If you switch to Postgres, you must change OrderService!
}

// ✓ GOOD — depends on an abstraction
interface Database {
  query<T>(sql: string, params: unknown[]): Promise<T[]>;
  insert(table: string, data: unknown): Promise<number>;
}

class OrderService {
  constructor(private db: Database) {}  // Depends on interface
  // Works with MySQL, Postgres, SQLite, MongoDB — anything that implements
Database
}

class MySQLDatabase implements Database {
  async query<T>(sql: string, params: unknown[]): Promise<T[]> { ... }
  async insert(table: string, data: unknown): Promise<number>  { ... }
}

class MockDatabase implements Database {
  async query<T>(): Promise<T[]> { return []; }    // For testing!
  async insert(): Promise<number> { return 1; }
```

```
}

// Real usage:
const orderService = new OrderService(new MySQLDatabase());
// Test usage:
const testService  = new OrderService(new MockDatabase());
```

11. Complete Real-World Example — E-Commerce Class Hierarchy

```
// ═══ Abstract base class ═══════════════════════════════
abstract class BaseEntity {
  readonly id: string;
  readonly createdAt: Date;
  updatedAt: Date;

  constructor() {
    this.id        = crypto.randomUUID();
    this.createdAt = new Date();
    this.updatedAt = new Date();
  }

  touch(): void { this.updatedAt = new Date(); }
  abstract toJSON(): Record<string, unknown>;
}

// ═══ Product class ═══════════════════════════════════════
class Product extends BaseEntity {
  private _stock: number;

  constructor(
    public readonly sku:  string,
    public  name:         string,
    private _price:       number,
    initialStock:         number,
    public  category:     string,
  ) {
    super();
    this._stock = initialStock;
  }
```

```
  get price(): number  { return this._price; }
  get stock(): number  { return this._stock; }
  get inStock(): boolean { return this._stock > 0; }

  set price(value: number) {
    if (value < 0) throw new Error('Price cannot be negative');
    this._price = value;
    this.touch();
  }

  reserve(qty: number): void {
    if (qty > this._stock) throw new Error(`Only ${this._stock} in stock`);
    this._stock -= qty;
    this.touch();
  }

  restock(qty: number): void {
    this._stock += qty;
    this.touch();
  }

  toJSON(): Record<string, unknown> {
    return { id: this.id, sku: this.sku, name: this.name,
             price: this._price, stock: this._stock };
  }
}

// ==== Cart class ========================================
interface CartLine {
  product:   Product;
  quantity:  number;
}

class ShoppingCart extends BaseEntity {
  private lines: Map<string, CartLine> = new Map();

  constructor(public readonly userId: string) { super(); }

  addItem(product: Product, qty: number = 1): void {
    if (!product.inStock) throw new Error(`${product.name} is out of stock`);
    const existing = this.lines.get(product.sku);
    if (existing) {
      existing.quantity += qty;
```

```
    } else {
      this.lines.set(product.sku, { product, quantity: qty });
    }
    this.touch();
  }

  removeItem(sku: string): void {
    this.lines.delete(sku);
    this.touch();
  }

  get subtotal(): number {
    let total = 0;
    this.lines.forEach(line => {
      total += line.product.price * line.quantity;
    });
    return total;
  }

  get itemCount(): number {
    let count = 0;
    this.lines.forEach(l => count += l.quantity);
    return count;
  }

  get isEmpty(): boolean { return this.lines.size === 0; }

  toJSON(): Record<string, unknown> {
    return { id: this.id, userId: this.userId,
             subtotal: this.subtotal, itemCount: this.itemCount };
  }
}

// ═══ Usage ═══════════════════════════════════════════════
const iphone = new Product('SKU-001', 'iPhone 15', 79999, 50, 'Electronics');
const airpods = new Product('SKU-002', 'AirPods Pro', 24999, 100,
'Electronics');

const cart = new ShoppingCart('user-123');
cart.addItem(iphone, 1);
cart.addItem(airpods, 2);

console.log(cart.subtotal);   // 79999 + 49998 = 129997
```

```
console.log(cart.itemCount);  // 3
console.log(iphone.inStock);  // true

iphone.price = 74999;          // Setter validates and touches updatedAt
console.log(cart.subtotal);    // 74999 + 49998 = 124997
```

12. Interview Questions — Chapter 10

Q: What are access modifiers in TypeScript classes?

Answer: Access modifiers control who can read or modify a class member.

public (default): Accessible from anywhere — inside the class, outside, and in subclasses.

private: Only accessible inside THIS class. Subclasses cannot access it.

protected: Accessible inside this class AND in any subclass. Not accessible from outside.

readonly: Can only be set in the constructor, then becomes immutable.

The shorthand constructor syntax (constructor(public name: string)) declares AND assigns the property in one step — eliminating repetitive this.name = name boilerplate.

Q: What is the difference between private and protected?

Answer: private members are accessible ONLY inside the class that defines them. Even subclasses cannot access them.

protected members are accessible inside the class AND in any class that extends it (subclasses).

Example: If Animal has private 'dna' and protected 'name',

class Dog extends Animal can use this.name (protected) ✅

class Dog extends Animal CANNOT use this.dna (private) ❌

Use private for true encapsulation. Use protected when subclasses legitimately need access.

Q: What is an abstract class? How is it different from an interface?

Answer: An abstract class is a partially implemented class that CANNOT be instantiated directly. It can have both concrete methods (fully implemented) and abstract methods (must be implemented by subclasses).

Key differences from interface:

Abstract class: can have implementation, constructor, access modifiers, and generates JS code.

Interface: only declarations, no constructor, no access modifiers, erased at compile time.

An abstract class can only be extended by ONE parent (no multiple inheritance).

A class can implement MULTIPLE interfaces.

Use abstract class for an IS-A relationship with shared behaviour (Animal → Dog).

Use interface for a CAN-DO contract with no shared implementation (Serializable, Cacheable).

Q: What are static class members? When should you use them?

Answer: Static members belong to the CLASS itself, not to instances. You access them via ClassName.member, not via this.

Use static for: shared state across all instances (Counter.instanceCount), factory methods (Temperature.fromCelsius()), Singleton pattern, utility/helper methods that don't need instance data.

Example: class Counter { static instanceCount = 0; } — every Counter shares the SAME instanceCount.

A private constructor + static getInstance() is the classic Singleton pattern — ensures only one instance of a class can exist (e.g. DatabaseConnection).

Q: What are getters and setters? When should you use them?

Answer: Getters (get) and setters (set) let you access a class property like a field while running custom logic.

Getter: Computed or derived properties — accessed without () like a field: temp.fahrenheit

Setter: Validation before assignment — validates data before allowing a property to be set.

Use getters for: computed properties (total = price * qty), derived values (displayName = first + last), lazy-loaded values.

Use setters for: validation (throw if value invalid), transformation (hash password on set), triggering side effects on change.

Q: Explain the SOLID principles with a TypeScript example.

Answer: SOLID is a set of five OOP design principles:

S — Single Responsibility: Each class should have one reason to change. Split UserService into UserValidator, UserRepository, EmailService.

O — Open/Closed: Classes should be open for extension but closed for modification. Use abstract DiscountStrategy and add new discount types without changing Order.

L — Liskov Substitution: Subclasses should work wherever the parent class is used. Every Shape subclass can be used where Shape is expected.

I — Interface Segregation: Many specific interfaces are better than one large one. Serializable, Cacheable, Loggable instead of one mega interface.

D — Dependency Inversion: Depend on abstractions (interfaces), not concrete implementations. OrderService takes Database interface, not MySQLDatabase.

Q: What is the 'override' keyword in TypeScript?

Answer: The 'override' keyword (TypeScript 4.3+) explicitly marks a method as overriding a parent class method.

Why useful: Without it, if you make a typo in the method name (describ instead of describe), TypeScript might silently create a NEW method instead of overriding. With override, TypeScript ERRORS if no parent method with that name exists.

Example: override describe(): string { return 'Child'; }

If the parent class renames 'describe' to 'toString', TypeScript errors on the override keyword — forcing you to update the child class too.

This is a safety feature — always use override when intentionally overriding a parent method.

Rapid Fire — Chapter 10

Question	One-Line Answer
Default access modifier in TS classes?	public — members are public unless otherwise specified
Can you instantiate an abstract class?	No — abstract classes cannot be used with 'new'
Can a class extend multiple abstract	No — only one 'extends' allowed; use interfaces for

classes?	multiple
What does 'super()' do?	Calls the parent class constructor — required first in subclass constructor
Can private members be accessed in subclasses?	No — use protected for subclass-accessible members
What does readonly do in a class?	Property can only be assigned in constructor, then immutable
Static vs instance members?	Static = on the class; instance = on each object created
Getter return type annotation?	TypeScript infers it; or specify: get name(): string
Can a class implement multiple interfaces?	Yes — class User implements Serializable, Cacheable, Loggable
What is the Singleton pattern?	Private constructor + static getInstance() ensuring one instance

Chapter 10 Key Takeaways

Classes = blueprints for objects. TypeScript adds types, access modifiers, and contracts.

Access modifiers: public (anywhere), private (this class only), protected (this + subclasses).
readonly: set once in constructor, immutable after. Constructor shorthand saves repetition.

Getters: computed properties accessed without (). Setters: validate before assignment.

Static members: belong to the class, not instances. Factory methods & Singleton pattern.

extends: inherit properties and methods. super() must be called first in child constructor.
override keyword: explicit method override — TypeScript errors if parent method doesn't exist.

Abstract class: cannot be instantiated. Mix of concrete and abstract methods.
Use for: shared logic + enforced contract. IS-A relationship (Dog IS-A Animal).

implements: class fulfils an interface contract. A class can implement MULTIPLE interfaces.

Abstract class vs Interface: abstract for shared implementation; interface for pure contract.

SOLID: S (one job), O (extend don't modify), L (liskov), I (small interfaces), D (depend on abstractions).

End of Chapter 10

CHAPTER 11

Utility Types — Deep Dive

Partial • Required • Readonly • Pick • Omit • Record • Exclude • Extract • NonNullable • Mapped Types

1. What are Utility Types? — The Full Picture

Utility types are TypeScript's built-in generic type transformers. They take an existing type and produce a new, modified type from it — without you having to rewrite the type from scratch. Think of them as built-in power tools for reshaping types.

Real-World Analogy — A Swiss Army Knife for Types

Imagine you have a MASTER USER TYPE with 10 properties.

Different situations need different 'versions' of User:

Creating a user → need all fields except auto-generated 'id' and 'createdAt'
Updating a user → all fields optional (only send what changed)
Public API resp. → all fields except 'password' and 'secretToken'
Config object → all fields immutable (readonly)

Without utility types: define 4 separate interfaces, manually maintain them in sync.
With utility types: derive all 4 from ONE source of truth — automatically in sync!

```
type CreateUserDto = Omit<User, 'id' | 'createdAt'>
type UpdateUserDto = Partial<Omit<User, 'id' | 'createdAt'>>
type PublicUser    = Omit<User, 'password' | 'secretToken'>
type FrozenUser    = Readonly<User>
```

Chapter 8 vs Chapter 11

Chapter 8 introduced utility types briefly as part of Generics.
THIS chapter goes DEEP — every utility type with:

→ What it actually does internally (the mapped type behind it)
→ Multiple real-world use cases
→ Common mistakes and how to avoid them

→ Combinations of utility types together

2. Partial<T> — Make All Properties Optional

Creates a new type where ALL properties of T become optional. Equivalent to adding `?` to every property.

```
// ── What Partial<T> does internally ──────────────────────────
// TypeScript's implementation:
// type Partial<T> = { [P in keyof T]?: T[P] }
// Reads: For every key P in T, make it optional and preserve its type

interface User {
  id:        number;    // required
  name:      string;    // required
  email:     string;    // required
  phone:     string;    // required
  address:   string;    // required
  bio:       string;    // required
}

// Partial<User> becomes:
// {
//   id?:      number;
//   name?:    string;
//   email?:   string;
//   phone?:   string;
//   address?: string;
//   bio?:     string;
// }

// ── Use Case 1: PATCH endpoint (update only changed fields) ───
async function patchUser(id: number, changes: Partial<User>): Promise<User> {
  const existing = await db.findUser(id);
  const updated  = { ...existing, ...changes };
  return db.saveUser(updated);
}

// Only send the fields that changed:
patchUser(1, { name: 'New Name' });             // ✅ just name
patchUser(1, { email: 'new@g.com', phone: '+91-...' }); // ✅ two fields
```

```
patchUser(1, {});                              // ✅ no changes (valid)
patchUser(1, { nonExistent: 'x' });            // ❌ ERROR: invalid key

// ── Use Case 2: Object builder pattern ──────────────────────
class UserBuilder {
  private data: Partial<User> = {};

  setName(name: string):    this { this.data.name = name;    return this; }
  setEmail(email: string):  this { this.data.email = email;  return this; }
  setPhone(phone: string):  this { this.data.phone = phone;  return this; }
  setBio(bio: string):      this { this.data.bio = bio;      return this; }

  build(): User {
    if (!this.data.name || !this.data.email) {
      throw new Error('name and email are required');
    }
    return this.data as User;
  }
}

const user = new UserBuilder()
  .setName('Rahul')
  .setEmail('r@g.com')
  .build();

// ── Use Case 3: Default config merging ──────────────────────
interface AppConfig {
  theme:      'light' | 'dark';
  language:   string;
  fontSize:   number;
  animations: boolean;
}

const DEFAULT_CONFIG: AppConfig = {
  theme: 'light', language: 'en', fontSize: 14, animations: true
};

// User only provides overrides — rest uses defaults:
function createConfig(overrides: Partial<AppConfig>): AppConfig {
  return { ...DEFAULT_CONFIG, ...overrides };
}
```

```
createConfig({ theme: 'dark' });         // Only override theme
createConfig({ fontSize: 18, language: 'hi' }); // Two overrides
```

3. Required<T> — Make All Properties Mandatory

The opposite of Partial. Creates a new type where ALL optional properties become required. Removes all `?` marks.

```
// ── What Required<T> does internally ──────────────────────
// type Required<T> = { [P in keyof T]-?: T[P] }
// The '-?' removes the optional modifier from every key

interface UserDraft {
  name?:    string;
  email?:   string;
  phone?:   string;
  address?: string;
}

// Required<UserDraft> becomes:
// { name: string; email: string; phone: string; address: string }
// Every ? is removed — all are required

// ── Use Case 1: Final form submission validation ──────────────
function submitRegistration(data: Required<UserDraft>): void {
  // By using Required, we guarantee ALL fields are present
  // No need to check for undefined inside
  console.log(`Registering: ${data.name} (${data.email})`);
  saveToDatabase(data);
}

submitRegistration({ name: 'Rahul', email: 'r@g.com', phone: '9876', address:
'Delhi' }); // ✔
submitRegistration({ name: 'Rahul' }); // ✖ ERROR: email, phone, address
missing

// ── Use Case 2: Configuration finalisation ────────────────────
interface ServerConfig {
  host?:     string;
  port?:     number;
  timeout?:  number;
  maxConn?:  number;
```

```
}

// After resolving defaults, all values are present:
function resolveConfig(input: ServerConfig): Required<ServerConfig> {
  return {
    host:    input.host    ?? 'localhost',
    port:    input.port    ?? 3000,
    timeout: input.timeout ?? 5000,
    maxConn: input.maxConn ?? 100,
  };
}

const config = resolveConfig({ port: 8080 });
// config.host is string (not string | undefined) — Required guarantees it
console.log(config.host.toUpperCase()); // ✅ Safe — no undefined
```

4. Readonly<T> — Immutable Types

Creates a type where ALL properties are `readonly` — they cannot be reassigned after the object is created. TypeScript will error at compile time if you try to mutate a Readonly property.

```
// ── What Readonly<T> does internally ──────────────────────
// type Readonly<T> = { readonly [P in keyof T]: T[P] }
// Adds 'readonly' modifier to every key

interface Config {
  apiUrl:   string;
  timeout:  number;
  maxRetry: number;
}

// ── Use Case 1: Immutable app configuration ───────────────
const CONFIG: Readonly<Config> = {
  apiUrl:   'https://api.myapp.com',
  timeout:  5000,
  maxRetry: 3,
};

CONFIG.apiUrl = 'hacked.com'; // ❌ ERROR: Cannot assign to readonly property
CONFIG.timeout++;             // ❌ ERROR: Cannot assign to readonly property
```

```
// ── Use Case 2: Preventing accidental mutation in functions ───
function displayUser(user: Readonly<User>): void {
  // This function CANNOT modify the user — guaranteed by type
  console.log(user.name);
  user.name = 'Changed'; // ✗ ERROR — prevents accidental mutation
}

// ── Use Case 3: React-style props (immutable component input) —
interface ButtonProps {
  label:    string;
  variant:  'primary' | 'secondary' | 'danger';
  onClick:  () => void;
}

function renderButton(props: Readonly<ButtonProps>): void {
  // Props should never be mutated — Readonly enforces this
  console.log(`Button: ${props.label} (${props.variant})`);
  props.label = 'Changed'; // ✗ ERROR
}

// ── Important: Readonly is SHALLOW ──────────────────────────
interface Order {
  id:    number;
  items: string[];  // array inside
}

const order: Readonly<Order> = { id: 1, items: ['apple'] };
order.id = 2;           // ✗ ERROR — id is readonly
order.items = [];       // ✗ ERROR — items ref is readonly
order.items.push('banana'); // ⚠ TypeScript ALLOWS this — Readonly is
shallow!
// The REFERENCE to items is readonly, but the array contents are not.
// Use ReadonlyArray<string> or readonly string[] for deep immutability.

interface OrderDeep {
  id:    number;
  items: ReadonlyArray<string>; // Or: readonly string[]
}
const o2: Readonly<OrderDeep> = { id: 1, items: ['apple'] };
o2.items.push('banana'); // ✗ ERROR — now truly immutable!
```

5. Pick<T, K> — Select Specific Properties

Creates a new type containing ONLY the specified keys K from type T. Use when you want **a few specific fields** from a larger type.

```
// ── What Pick<T, K> does internally ──────────────────────
// type Pick<T, K extends keyof T> = { [P in K]: T[P] }
// Only keeps keys that are in K, with their original types

interface User {
  id:          number;
  name:        string;
  email:       string;
  password:    string;    // sensitive!
  phone:       string;
  address:     string;
  role:        string;
  createdAt:   Date;
  lastLoginAt: Date;
}

// ── Use Case 1: Public API response — hide sensitive fields ───
type PublicProfile = Pick<User, 'id' | 'name' | 'role'>;
// { id: number; name: string; role: string }
// password, email, phone etc. are NOT included

function getPublicProfile(user: User): PublicProfile {
  return { id: user.id, name: user.name, role: user.role };
}

// ── Use Case 2: Login form — only need these fields ────────
type LoginCredentials = Pick<User, 'email' | 'password'>;
// { email: string; password: string }

function login(creds: LoginCredentials): Promise<string> {
  return authenticateUser(creds.email, creds.password);
}

// ── Use Case 3: Product listing card vs detail page ────────
interface Product {
  id:          number;
  name:        string;
  price:       number;
```

```
  description: string;   // long text — not needed on listing
  specs:       object;   // detailed specs — not needed on listing
  images:      string[];
  stock:       number;
  category:    string;
  sellerId:    number;
}

// Product card on listing page — only summary needed
type ProductCard = Pick<Product, 'id' | 'name' | 'price' | 'images' |
'stock'>;

// Search result — even less data
type ProductSearchResult = Pick<Product, 'id' | 'name' | 'price' |
'category'>;

// Compare page — a bit more
type ProductCompare = Pick<Product, 'id' | 'name' | 'price' | 'specs'>;

// ── Use Case 4: Update form for admin ──────────────────────────
// Admin can only update name, price, and stock (not id, sellerId)
type AdminProductUpdate = Pick<Product, 'name' | 'price' | 'stock'>;

function adminUpdateProduct(id: number, data: AdminProductUpdate): void {
  db.update('products', id, data);
}
```

6. Omit<T, K> — Exclude Specific Properties

The complement of Pick. Creates a new type with ALL properties of T EXCEPT the ones in K. Use when you want **most fields minus a few specific ones**.

```
// ── What Omit<T, K> does internally ──────────────────────────
// type Omit<T, K extends keyof T> = Pick<T, Exclude<keyof T, K>>
// Excludes keys in K, keeps everything else

interface User {
  id:        number;
  name:      string;
  email:     string;
  password:  string;
  createdAt: Date;
```

```
  updatedAt: Date;
}

// ── Use Case 1: DTO for creating a resource (no auto-fields) ──
// Server generates: id, createdAt, updatedAt
// Client provides: name, email, password
type CreateUserDto = Omit<User, 'id' | 'createdAt' | 'updatedAt'>;
// { name: string; email: string; password: string }

async function createUser(dto: CreateUserDto): Promise<User> {
  return db.insert('users', {
    ...dto,
    id:        generateId(),
    createdAt: new Date(),
    updatedAt: new Date(),
  });
}

// Client sends ONLY what's needed — no id or timestamps:
createUser({ name: 'Rahul', email: 'r@g.com', password: 'hashed' });

// ── Use Case 2: Safe user — no password ──────────────────────
type SafeUser = Omit<User, 'password'>;
// { id: number; name: string; email: string; createdAt: Date; updatedAt:
Date }

function getCurrentUser(req: Request): SafeUser {
  const user = getUserFromToken(req);
  const { password, ...safeUser } = user;  // Runtime: remove password
  return safeUser;                          // Type: SafeUser
}

// ── Use Case 3: Event without auto-assigned fields ────────────
interface CalendarEvent {
  readonly eventId:  string;   // server assigns
  title:   string;
  date:    Date;
  time:    string;
  location?: string;
  readonly createdBy: string;  // server assigns from auth token
}

type CreateEventDto = Omit<CalendarEvent, 'eventId' | 'createdBy'>;
```

```
// { title: string; date: Date; time: string; location?: string }

// ── Pick vs Omit — when to choose which ──────────────────────
// Interface has 10 fields. You need 2 → use Pick   (easier to list 2)
// Interface has 10 fields. You need 8 → use Omit   (easier to exclude 2)
```

7. Record<K, V> — Key-Value Object Types

Creates an object type with keys of type K and values of type V. A cleaner alternative to `{ [key: string]: Value }` index signatures, especially when keys are a specific union of literals.

```
// ── What Record<K, V> does internally ────────────────────────
// type Record<K extends keyof any, T> = { [P in K]: T }

// ── Use Case 1: Lookup table from literal union ──────────────
type UserRole = 'admin' | 'moderator' | 'user' | 'guest';

const rolePermissions: Record<UserRole, string[]> = {
  admin:     ['read', 'write', 'delete', 'manage_users'],
  moderator: ['read', 'write', 'delete'],
  user:      ['read', 'write'],
  guest:     ['read'],
};

// TypeScript ensures ALL roles are covered — no missing keys!
// If you add 'superadmin' to UserRole and forget the Record entry → ERROR

// ── Use Case 2: Translation / i18n map ───────────────────────
type SupportedLang = 'en' | 'hi' | 'ta' | 'te' | 'bn' | 'mr';
type TranslationKey = 'welcome' | 'logout' | 'settings' | 'help';

const translations: Record<SupportedLang, Record<TranslationKey, string>> = {
  en: { welcome: 'Welcome',  logout: 'Logout',  settings: 'Settings', help:
'Help' },
  hi: { welcome: 'Namaste', logout: 'Nikalein', settings: 'Sttings',  help:
'Madad' },
  ta: { welcome: 'Vanakkam',logout: 'Logout',   settings: 'Amaippugal', help:
'Udavi' },
  te: { welcome: 'Swaagatam',logout: 'Logout',  settings: 'Settings', help:
'Sahayam' },
  bn: { welcome: 'Swagat',  logout: 'Logout',   settings: 'Settings', help:
'Sahajyo' },
  mr: { welcome: 'Swagat',  logout: 'Logout',   settings: 'Settings', help:
```

```
'Maddat' },
};

function t(lang: SupportedLang, key: TranslationKey): string {
  return translations[lang][key];
}

console.log(t('hi', 'welcome')); // 'Namaste'

// ── Use Case 3: HTTP status code labels ──────────────────────
type HttpStatus = 200 | 201 | 400 | 401 | 403 | 404 | 500;

const statusLabels: Record<HttpStatus, string> = {
  200: 'OK',
  201: 'Created',
  400: 'Bad Request',
  401: 'Unauthorized',
  403: 'Forbidden',
  404: 'Not Found',
  500: 'Internal Server Error',
};

// ── Use Case 4: Cache / state store ──────────────────────────
interface CacheEntry<T> {
  value:     T;
  expiresAt: number;  // timestamp
}

class Cache<T> {
  private store: Record<string, CacheEntry<T>> = {};

  set(key: string, value: T, ttlMs: number): void {
    this.store[key] = { value, expiresAt: Date.now() + ttlMs };
  }

  get(key: string): T | null {
    const entry = this.store[key];
    if (!entry || Date.now() > entry.expiresAt) return null;
    return entry.value;
  }
}

const userCache = new Cache<User>();
```

```
userCache.set('user:1', { id: 1, name: 'Rahul' } as User, 60000);
```

8. Exclude<T, U> — Remove from Union

Removes all union members of T that are assignable to U. Works on **union types**, not object types (that's Omit's job).

```
// ── What Exclude<T, U> does internally ──────────────────────
// type Exclude<T, U> = T extends U ? never : T
// For each member of T: if it extends U, replace with never (remove it)

// ── Basic usage ─────────────────────────────────────────────
type AllColors = 'red' | 'green' | 'blue' | 'yellow' | 'purple';
type WarmColors = 'red' | 'yellow';

type CoolColors = Exclude<AllColors, WarmColors>;
// 'green' | 'blue' | 'purple'  — warm colors removed

// ── Use Case 1: Remove null/undefined from union ────────────
type MaybeString = string | null | undefined;
type DefiniteString = Exclude<MaybeString, null | undefined>;
// string — null and undefined removed

// ── Use Case 2: Filter out specific event types ─────────────
type DomEvent = 'click' | 'mousedown' | 'mouseup' | 'keydown' | 'keyup' |
'scroll' | 'resize';
type MouseEvent = 'click' | 'mousedown' | 'mouseup';

type KeyboardAndOtherEvents = Exclude<DomEvent, MouseEvent>;
// 'keydown' | 'keyup' | 'scroll' | 'resize'

// ── Use Case 3: Remove 'unknown' from union for type safety ──
type ApiField = string | number | boolean | null | unknown[];
type SafeApiField = Exclude<ApiField, unknown[]>;
// string | number | boolean | null

// ── Use Case 4: Feature flags — remove deprecated features ──
type AllFeatures = 'darkMode' | 'betaEditor' | 'legacyLayout' |
'newDashboard';
type DeprecatedFeatures = 'legacyLayout';

type ActiveFeatures = Exclude<AllFeatures, DeprecatedFeatures>;
```

```
// 'darkMode' | 'betaEditor' | 'newDashboard'

function enableFeature(feature: ActiveFeatures): void {
  console.log(`Enabling: ${feature}`);
}

enableFeature('darkMode');       // ✅
enableFeature('legacyLayout');   // ❌ ERROR — deprecated/excluded
```

9. Extract<T, U> — Keep Matching Union Members

The complement of Exclude. Keeps ONLY the union members of T that are assignable to U.

```
// ── What Extract<T, U> does internally ──────────────────────
// type Extract<T, U> = T extends U ? T : never
// Opposite of Exclude — keeps what Exclude would remove

// ── Basic usage ─────────────────────────────────────────────
type AllValues  = string | number | boolean | null | undefined | object;
type Primitives = Extract<AllValues, string | number | boolean>;
// string | number | boolean — only primitive types kept

// ── Use Case 1: Extract string types from a union ────────────
type Mixed = 'success' | 'error' | 404 | 200 | boolean;
type StringsOnly = Extract<Mixed, string>;
// 'success' | 'error'

type NumbersOnly = Extract<Mixed, number>;
// 404 | 200

// ── Use Case 2: Get only function types from a union ─────────
type MixedTypes = string | number | (() => void) | ((x: number) => string);
type FunctionTypes = Extract<MixedTypes, Function>;
// (() => void) | ((x: number) => string)

// ── Use Case 3: Extract specific literal types ───────────────
type AllRoutes = '/home' | '/about' | '/products' | '/cart' | '/admin' |
'/admin/users';
type AdminRoutes = Extract<AllRoutes, `/admin${string}`>;
// '/admin' | '/admin/users'  — template literal matching!
```

```
// ── Use Case 4: Payment methods — extract online-only ──────────
type AllPaymentMethods = 'cash' | 'card' | 'upi' | 'netbanking' | 'cod' |
'wallet';
type OnlinePayments    = Extract<AllPaymentMethods, 'card' | 'upi' |
'netbanking' | 'wallet'>;
// 'card' | 'upi' | 'netbanking' | 'wallet'

function processOnlinePayment(method: OnlinePayments): void {
  console.log(`Processing online: ${method}`);
}

processOnlinePayment('upi');   // ✅
processOnlinePayment('cash');  // ❌ ERROR — cash is not online
processOnlinePayment('cod');   // ❌ ERROR — COD is not online
```

10. NonNullable<T> — Remove null and undefined

Removes `null` and `undefined` from a type. Shorthand for `Exclude<T, null | undefined>` .

```
// ── What NonNullable<T> does internally ──────────────────────
// type NonNullable<T> = T extends null | undefined ? never : T

type MaybeUser = User | null | undefined;
type DefiniteUser = NonNullable<MaybeUser>;
// User — null and undefined removed

// ── Use Case 1: Function that guarantees non-null output ──────
function getUser(id: number): User | null {
  return db.find(id);
}

function getUserOrThrow(id: number): NonNullable<ReturnType<typeof getUser>> {
  const user = getUser(id);
  if (!user) throw new Error(`User ${id} not found`);
  return user;  // TypeScript knows this is User (not null)
}

// ── Use Case 2: Filtering arrays ──────────────────────────────
// The isDefined helper from Chapter 9 — typed with NonNullable
function isDefined<T>(value: T | null | undefined): value is NonNullable<T> {
  return value !== null && value !== undefined;
```

```
}

const users: (User | null | undefined)[] = [
  { id: 1, name: 'Rahul' } as User,
  null,
  { id: 2, name: 'Priya' } as User,
  undefined,
  { id: 3, name: 'Ankit' } as User,
];

const validUsers: User[] = users.filter(isDefined);
// [Rahul, Priya, Ankit] — null and undefined removed — type is User[]

// ── Use Case 3: API response property —  always defined ──────
interface ApiResponse {
  user?:    User;
  error?:   string;
}

function extractUser(res: ApiResponse): NonNullable<ApiResponse['user']> {
  if (!res.user) throw new Error('No user in response');
  return res.user;  // User (not User | undefined)
}
```

11. ReturnType<T> and Parameters<T>

```
// ── ReturnType<T> — extract return type of a function ────────
function createProduct(name: string, price: number, category: string) {
  return {
    id:         Math.random(),
    name,
    price,
    category,
    createdAt:  new Date(),
    inStock:    true,
  };
}

// Without ReturnType — must define type manually and keep in sync:
// type Product = { id: number; name: string; ... }  — tedious
```

```
// With ReturnType — derived automatically from the function:
type Product = ReturnType<typeof createProduct>;
// { id: number; name: string; price: number; category: string; createdAt:
Date; inStock: boolean }

const p: Product = createProduct('iPhone', 79999, 'Electronics');
console.log(p.createdAt.toISOString()); // ✅ TypeScript knows createdAt is
Date

// ── Use Case: Matching function return types in tests ─────────
type ServiceResponse = ReturnType<typeof UserService.prototype.getUser>;
// Mock must return the SAME shape as the real service

// ── Parameters<T> — extract param types as tuple ──────────────
function sendNotification(
  userId: number,
  type:   'push' | 'email' | 'sms',
  message: string,
  options?: { priority: 'high' | 'normal'; retry: boolean }
): void {}

type NotifParams = Parameters<typeof sendNotification>;
// [userId: number, type: 'push' | 'email' | 'sms', message: string,
//  options?: { priority: 'high' | 'normal'; retry: boolean } | undefined]

// ── Use Case: Forwarding calls ────────────────────────────────
function queueNotification(...args: Parameters<typeof sendNotification>): void
{
  notificationQueue.push(args);
  // When queue processes, call: sendNotification(...args)
}

// ── Awaited<T> — unwrap Promise ───────────────────────────────
async function fetchUser(id: number): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  return res.json();
}

type FetchedUser = Awaited<ReturnType<typeof fetchUser>>;
// User — Promise<User> is unwrapped to just User

// ── ConstructorParameters<T> — for classes ────────────────────
class UserSession {
```

```
  constructor(
    public userId:  number,
    public token:   string,
    public expires: Date
  ) {}
}

type SessionArgs = ConstructorParameters<typeof UserSession>;
// [userId: number, token: string, expires: Date]

function createSession(...args: SessionArgs): UserSession {
  return new UserSession(...args);
}
```

12. Combining Utility Types

The real power of utility types comes from composing them. Most real-world code uses combinations of multiple utility types together.

```
// ── The master User type ──────────────────────────────────
interface User {
  readonly id:       number;
  name:              string;
  email:             string;
  password:          string;
  phone?:            string;
  address?:          string;
  role:              'admin' | 'user' | 'guest';
  readonly createdAt: Date;
  updatedAt:         Date;
}

// ── ALL derived from the single source of truth ──────────────

// 1. Creating a new user (no auto fields, password required)
type CreateUserDto = Omit<User, 'id' | 'createdAt' | 'updatedAt'>;

// 2. PATCH endpoint (everything optional, no auto fields)
type PatchUserDto = Partial<Omit<User, 'id' | 'createdAt' | 'updatedAt'>>;

// 3. Safe user for API response (no password)
```

```
type SafeUser = Omit<User, 'password'>;

// 4. Public profile (only id, name, role — minimal exposure)
type PublicUser = Pick<User, 'id' | 'name' | 'role'>;

// 5. Admin view — full user but password is readonly
type AdminUserView = Readonly<Omit<User, 'password'>>;

// 6. Login form fields
type LoginDto = Pick<User, 'email' | 'password'>;

// 7. Non-nullable user (after null check)
type DefiniteUser = NonNullable<User | null | undefined>;

// ── Another real pattern: form state types ──────────────────
interface ProductForm {
  name:        string;
  price:       number;
  description: string;
  category:    string;
  stock:       number;
}

// Form errors — same keys but values are error messages (string)
type FormErrors<T> = Partial<Record<keyof T, string>>;
// { name?: string; price?: string; description?: string; ... }

// Form touched state — same keys but values are boolean
type FormTouched<T> = Partial<Record<keyof T, boolean>>;

// Usage:
const errors: FormErrors<ProductForm>  = { price: 'Must be positive' };
const touched: FormTouched<ProductForm> = { name: true, price: true };

// ── Deep Partial (recursive) — a custom utility type ──────────
// Built-in Partial is shallow. For nested objects, use DeepPartial:
type DeepPartial<T> = {
  [P in keyof T]?: T[P] extends object ? DeepPartial<T[P]> : T[P];
};

interface Address {
  street: string;
```

```
  city:   string;
  state:  string;
  pincode: string;
}

interface FullUser {
  name:    string;
  address: Address;  // Nested object
}

// Normal Partial — address is optional but if provided must be complete
type ShallowPartial = Partial<FullUser>;
// { name?: string; address?: Address }  — still requires full Address

// DeepPartial — address AND its fields are optional
type DeepPartialUser = DeepPartial<FullUser>;
// { name?: string; address?: { street?: string; city?: string; ... } }
```

13. Template Literal Types (Bonus)

A powerful feature that works with string literal types — TypeScript can create new string types by combining string patterns.

```
// ── Template literal types ──────────────────────────────
type Greeting = `Hello, ${string}!`;
// Any string matching 'Hello, ...!' pattern

const g1: Greeting = 'Hello, Rahul!';   // ✅
const g2: Greeting = 'Hello, World!';   // ✅
const g3: Greeting = 'Hi there!';       // ❌ doesn't match pattern

// ── Real-world: Event names ─────────────────────────────
type EventName = `on${Capitalize<string>}`;
// 'onClick', 'onChange', 'onSubmit', 'onKeyDown' etc.

// ── Real-world: CSS property accessor ───────────────────
type CssProp = 'margin' | 'padding' | 'border';
type CssSide = 'Top' | 'Bottom' | 'Left' | 'Right';
type CssProperty = `${CssProp}${CssSide}`;
// 'marginTop' | 'marginBottom' | 'marginLeft' | 'marginRight'
// | 'paddingTop' | ... | 'borderTop' | ... (12 combinations!)
```

```
// ── Real-world: API routes ──────────────────────────────
type Resource = 'user' | 'product' | 'order';
type Action   = 'get' | 'create' | 'update' | 'delete';
type ApiAction = `${Action}${Capitalize<Resource>}`;
// 'getUser' | 'createUser' | 'updateUser' | 'deleteUser'
// | 'getProduct' | ... | 'getOrder' | ... (12 combinations)

// ── Built-in string manipulation types ──────────────────
type Upper = Uppercase<'hello'>;    // 'HELLO'
type Lower = Lowercase<'WORLD'>;    // 'world'
type Cap   = Capitalize<'rahul'>;   // 'Rahul'
type Uncap = Uncapitalize<'Rahul'>; // 'rahul'

// ── Real-world: getter/setter naming ────────────────────
type PropName = 'name' | 'email' | 'phone';
type Getter = `get${Capitalize<PropName>}`;
// 'getName' | 'getEmail' | 'getPhone'
type Setter = `set${Capitalize<PropName>}`;
// 'setName' | 'setEmail' | 'setPhone'
```

14. Complete Utility Types Reference

Utility Type	What it produces	Internal Implementation	Real-world use
`Partial<T>`	All props optional	`{ [P in keyof T]?: T[P] }`	PATCH endpoints, builders
`Required<T>`	All props required	`{ [P in keyof T]-?: T[P] }`	Form submission, config resolve
`Readonly<T>`	All props readonly	`{ readonly [P in keyof T]: T[P] }`	Immutable config, props
`Pick<T, K>`	Only keys in K	`{ [P in K]: T[P] }`	Public API, card summaries
`Omit<T, K>`	All keys except K	`Pick<T, Exclude<keyof T, K>>`	DTOs without auto-fields
`Record<K, V>`	Object with keys K, values V	`{ [P in K]: V }`	Lookups, translations, cache
`Exclude<T, U>`	T union minus U members	`T extends U ? never : T`	Filter union types
`Extract<T, U>`	T union keeping U matches	`T extends U ? T : never`	Get subset of union
`NonNullable<T>`	Remove null & undefined	`Exclude<T, null |`	Guaranteed non-null

		undefined>	values
ReturnType<T>	Return type of function T	infer R in return position	Derive type from function
Parameters<T>	Params of function T as tuple	infer P in params position	Forwarding, mocking
Awaited<T>	Unwrap Promise<T> to T	Recursive Promise unwrapping	Get resolved async type
ConstructorParameters	Constructor params as tuple	infer P from constructor	Factory functions
InstanceType<T>	Instance type of a class	infer I from constructor return	Work with class instances

15. Interview Questions — Chapter 11

Q: What is the difference between Partial<T> and Required<T>?

Answer: They are opposites.

Partial<T> makes ALL properties optional — adds '?' to every property.

Use for: PATCH/update endpoints, builder patterns, config overrides.

Internal: { [P in keyof T]?: T[P] }

Required<T> makes ALL optional properties required — removes '?' from every property.

Use for: form submission validation, after resolving defaults, asserting completeness.

Internal: { [P in keyof T]-?: T[P] } — the '-?' REMOVES the optional modifier.

They are exact inverses of each other: Required<Partial<T>> gives you back T.

Q: What is the difference between Pick<T,K> and Omit<T,K>? When do you choose each?

Answer: Both create a subset of T's properties:

Pick<T, K>: Creates a type with ONLY the specified keys K. Good when you want a FEW specific fields from a large type.

Pick<User, 'id' | 'name'> → { id: number; name: string } — only 2 fields

Omit<T, K>: Creates a type with ALL keys EXCEPT K. Good when you want MOST fields minus a few.

Omit<User, 'password' | 'createdAt'> → all User fields except those 2

Rule of thumb: If selecting 2 from 10 → use Pick. If removing 2 from 10 → use Omit.

Internally: Omit<T,K> = Pick<T, Exclude<keyof T, K>>

Q: What is the difference between Exclude<T,U> and Omit<T,K>?

Answer: They look similar but operate on different things:

Exclude<T, U>: Works on UNION TYPES. Removes members from a union.

Exclude<'a'|'b'|'c', 'a'> → 'b'|'c' — removes 'a' from the union

Omit<T, K>: Works on OBJECT TYPES. Removes properties from an interface/type.

Omit<User, 'password'> → User without password property

Memory trick: Exclude = for UNION types (flat). Omit = for OBJECT types (structured).

Q: What does Readonly<T> do? What is its limitation?

Answer: Readonly<T> adds the 'readonly' modifier to every property of T. Any attempt to reassign a property causes a compile error.

Main limitation: Readonly is SHALLOW — it only makes the top-level properties readonly, not nested objects or arrays.

Example: const order: Readonly<Order> = { items: ['a', 'b'] }

order.items = []; // ✕ ERROR — the reference is readonly

order.items.push('c'); // ⚠ TypeScript ALLOWS this! Shallow!

For deep immutability, use ReadonlyArray<T> for arrays, and define a custom DeepReadonly<T> recursive type for nested objects.

Q: What is Record<K, V> and why is it better than an index signature?

Answer: Record<K, V> creates an object type with keys of type K and values of type V.

Record<string, number> vs { [key: string]: number } — functionally similar for broad string keys.

But Record shines when K is a LITERAL UNION:

Record<'admin'|'user'|'guest', string[]>

→ TypeScript ensures ALL three keys are present — you CANNOT miss one!

→ With index signature, there's no such guarantee.

Use Record for: permission maps, translation tables, lookup tables with known keys, state maps.

Q: How would you create a type for form errors using utility types?

Answer: This is a common pattern in React/Angular forms:

Given: interface ProductForm { name: string; price: number; category: string }

Form errors type: type FormErrors<T> = Partial<Record<keyof T, string>>

→ keyof T gives: 'name' | 'price' | 'category'

→ Record<...> makes: { name: string; price: string; category: string }

→ Partial<...> makes all optional: { name?: string; price?: string; ... }

Usage: const errors: FormErrors<ProductForm> = { price: 'Must be positive' }

Only invalid fields have error messages — valid fields simply have no entry.

Q: What is ReturnType<T> and when would you use it in practice?

Answer: ReturnType<typeof fn> extracts the return type of a function.

Use it when: 1) You don't want to manually define a return type interface and risk getting out of sync. 2) You're working with a library function and need to type its return value. 3) You're writing tests and need to type mock return values to match the real service.

Example: function createUser() { return { id: 1, name: 'Rahul', createdAt: new Date() }; }

type User = ReturnType<typeof createUser>

→ { id: number; name: string; createdAt: Date } — TypeScript infers it!

Combined with Awaited for async functions:

type User = Awaited<ReturnType<typeof fetchUser>> — unwraps Promise<User> to User.

Rapid Fire — Chapter 11

Question	One-Line Answer

Partial internal implementation?	{ [P in keyof T]?: T[P] } — adds ? to all keys
Required internal implementation?	{ [P in keyof T]-?: T[P] } — removes ? from all keys
Is Readonly<T> deep or shallow?	Shallow — nested arrays/objects can still be mutated
Exclude works on union or object types?	Union types — removes matching members from a union
Omit works on union or object types?	Object types — removes keys from an interface
Extract<'a'|'b'|'c', 'a'|'b'> result?	'a'|'b' — only the intersection is kept
NonNullable<string|null|undefined> result?	string — null and undefined are removed
How to get param types of a function?	Parameters<typeof fn> — returns a tuple of param types
How to unwrap Promise<User> to User?	Awaited<Promise<User>> — unwraps to User
Combination for PATCH DTO?	Partial<Omit<T, 'id' | 'createdAt'>> — optional, no auto fields

Chapter 11 Key Takeaways

Utility types transform existing types — derive multiple types from one source of truth.

Partial<T>: all optional → PATCH/update. Required<T>: all required → form submission.
Readonly<T>: all immutable (shallow!) → configs, props.

Pick<T,K>: keep a FEW fields. Omit<T,K>: remove a FEW fields.
Pick when selecting 2 from 10. Omit when removing 2 from 10.

Record<K,V>: key→value object. Best with literal union keys — exhaustiveness guaranteed.

Exclude<T,U>: union minus U. Extract<T,U>: union keeping U. Both work on UNION types.
NonNullable<T>: removes null and undefined. Shorthand for Exclude<T, null|undefined>.

ReturnType, Parameters, Awaited, ConstructorParameters: function/class type extraction.

Combining utility types: Partial<Omit<User,'id'|'createdAt'>> = perfect PATCH DTO.

FormErrors<T> = Partial<Record<keyof T, string>> — a powerful reusable pattern.

Template literal types: create string patterns → useful for event names, routes, i18n keys.

End of Chapter 11

CHAPTER 12

Advanced Types

Mapped Types • Conditional Types • infer • Template Literals • Recursive Types • Variance

1. Why Advanced Types?

You've mastered the fundamentals. Now it's time to understand how TypeScript works at a deeper level. Advanced types let you write type-level logic — transforming, computing, and constraining types programmatically. These are the techniques used inside TypeScript's own standard library and in frameworks like React, Vue, and NestJS.

What This Chapter Covers

Mapped Types — iterate over keys of a type and transform them
Conditional Types — if/else at the type level: T extends U ? A : B
infer keyword — extract ('infer') a type from inside another type
Template Literals — build new string types from patterns
Recursive Types — types that reference themselves (trees, JSON, etc.)
Variance — covariance and contravariance explained simply

These are the tools that POWER every utility type (Partial, Pick, Omit...)
You will understand exactly HOW they work, not just how to USE them.

2. Mapped Types — Iterating Over Keys

A mapped type creates a new type by **iterating over the keys of an existing type** and transforming each property. The syntax is `{ [P in keyof T]: ... }` . All of TypeScript's built-in utility types are built with mapped types.

Real-World Analogy — The Photocopier

Imagine a fancy photocopier that can:
→ Copy every page of a document (iterating all keys)
→ Optionally make each page optional, readonly, or modified

Mapped types are that photocopier for TypeScript types.

They say: 'For EVERY property in this type, do THIS transformation.'

Partial<T> = 'For every key, make it optional'
Readonly<T> = 'For every key, make it readonly'
Custom mapped type = 'For every key, do whatever you want'

2.1 Basic Mapped Type Syntax

```
// ── Anatomy of a mapped type ──────────────────────────────
//                  key name  key source
//                     ↓         ↓
type MyMapped<T> = { [P in keyof T]: T[P] }
//                              ↑
//                        value type (same as original)

// This is basically a 'copy' of T — same as T itself.
// Real power comes from modifying the modifiers or value types.

// ── Add optional modifier to every key ────────────────────
type Optional<T> = { [P in keyof T]?: T[P] };
// Same as Partial<T> — just written manually

// ── Add readonly modifier to every key ────────────────────
type Immutable<T> = { readonly [P in keyof T]: T[P] };
// Same as Readonly<T>

// ── Remove optional modifier (make all required) ──────────
type Concrete<T> = { [P in keyof T]-?: T[P] };
// Same as Required<T>

// ── Remove readonly modifier ──────────────────────────────
type Mutable<T> = { -readonly [P in keyof T]: T[P] };
// Built-in doesn't have this! -readonly removes readonly.

interface Config {
  readonly host: string;
  readonly port: number;
}

type EditableConfig = Mutable<Config>;
// { host: string; port: number } — readonly removed!
```

```
const config: EditableConfig = { host: 'localhost', port: 3000 };
config.host = 'newhost'; // ✅ Now mutable
```

2.2 Transforming Value Types

```
// ── Change the type of every value ──────────────────────────

// Make every value nullable:
type Nullable<T> = { [P in keyof T]: T[P] | null };

interface User { id: number; name: string; email: string }
type NullableUser = Nullable<User>;
// { id: number | null; name: string | null; email: string | null }

// Make every value a Promise:
type Promisify<T> = { [P in keyof T]: Promise<T[P]> };

type AsyncUser = Promisify<User>;
// { id: Promise<number>; name: Promise<string>; email: Promise<string> }

// Make every value an array:
type Arrayify<T> = { [P in keyof T]: T[P][] };

type UserHistory = Arrayify<User>;
// { id: number[]; name: string[]; email: string[] }

// ── Real-world: Form state types ────────────────────────────
// All values become string (for form input binding):
type FormValues<T> = { [P in keyof T]: string };

// All values become error message or undefined:
type FormErrors<T> = { [P in keyof T]?: string };

// All values become 'dirty' flag:
type FormDirty<T> = { [P in keyof T]: boolean };

interface ProductForm {
  name:     string;
  price:    number;
  category: string;
}
```

```
const values:  FormValues<ProductForm>  = { name: '', price: '0', category:
'' };
const errors:  FormErrors<ProductForm>  = { price: 'Must be positive' };
const touched: FormDirty<ProductForm>   = { name: true, price: true, category:
false };
```

2.3 Key Remapping with 'as'

```
// ── Key remapping — change the KEY name in the output type ────
// TypeScript 4.1+: use 'as' clause to rename keys

type Getters<T> = {
  [P in keyof T as `get${Capitalize<string & P>}`]: () => T[P]
};
// For every key P in T, create 'getP' method that returns T[P]

interface User { name: string; age: number; email: string }

type UserGetters = Getters<User>;
// {
//   getName:  () => string;
//   getAge:   () => number;
//   getEmail: () => string;
// }

// ── Real-world: Event handler types ──────────────────────────
type EventHandlers<T> = {
  [P in keyof T as `on${Capitalize<string & P>}Change`]: (value: T[P]) => void
};

interface FormFields { name: string; age: number; email: string }

type FormEventHandlers = EventHandlers<FormFields>;
// {
//   onNameChange:  (value: string) => void;
//   onAgeChange:   (value: number) => void;
//   onEmailChange: (value: string) => void;
// }

// ── Filter keys with 'never' in remapping ────────────────────
// Remove keys where the value is a function:
type DataOnly<T> = {
```

```
  [P in keyof T as T[P] extends Function ? never : P]: T[P]
};

interface MixedClass {
  name:  string;
  age:   number;
  greet(): void;
  save():  Promise<void>;
}

type DataOnlyClass = DataOnly<MixedClass>;
// { name: string; age: number } — methods are filtered out!
```

> ⚙ **Real-World Scenario: NestJS / Angular Framework Internals**
> Frameworks use mapped types heavily. NestJS uses them to create DTO validation schemas from entity types. Angular uses them to create reactive form group types. Understanding mapped types lets you read and understand framework source code, write custom decorators, and build type-safe APIs from scratch.

3. Conditional Types — if/else at the Type Level

Conditional types allow TypeScript to **make decisions at the type level** — choosing between two types based on a condition. The syntax mirrors JavaScript's ternary operator: `T extends U ? TypeIfTrue : TypeIfFalse` .

Real-World Analogy — A Smart Sorting Machine

Imagine a sorting machine at a factory.

Every item passes through a sensor:

If item is 'small' → goes to Box A

If item is 'large' → goes to Box B

Conditional types do the same at the TYPE level:

If T extends string → produce StringResult

If T extends number → produce NumberResult

The 'machine' runs at COMPILE TIME — no runtime cost!

3.1 Basic Conditional Type Syntax

```
// ── Syntax: T extends U ? TrueType : FalseType ────────────────

// Is T a string?
type IsString<T> = T extends string ? true : false;

type A = IsString<string>;   // true
type B = IsString<number>;   // false
type C = IsString<'hello'>;  // true  (literals extend their base type)
type D = IsString<any>;      // boolean (any distributes both ways)

// ── Returning different types based on input ──────────────────
type TypeName<T> =
  T extends string  ? 'string'  :
  T extends number  ? 'number'  :
  T extends boolean ? 'boolean' :
  T extends null    ? 'null'    :
  T extends undefined ? 'undefined' :
  T extends Function ? 'function' :
  'object';

type T1 = TypeName<string>;    // 'string'
type T2 = TypeName<() => void>; // 'function'
type T3 = TypeName<number[]>;  // 'object'

// ── Real-world: Flatten array types ──────────────────────────
type Flatten<T> = T extends Array<infer Item> ? Item : T;
// If T is an array, return the element type; else return T as-is

type F1 = Flatten<string[]>;   // string
type F2 = Flatten<number[]>;   // number
type F3 = Flatten<string>;     // string (not an array — returns T itself)
type F4 = Flatten<User[]>;     // User
```

3.2 Distributive Conditional Types

```
// ── When T is a union, conditional types DISTRIBUTE over it ──
// TypeScript applies the condition to EACH member separately

type ToArray<T> = T extends any ? T[] : never;

type Result = ToArray<string | number | boolean>;
// Distributes: (string extends any ? string[] : never)
```

```
//             | (number extends any ? number[] : never)
//             | (boolean extends any ? boolean[] : never)
// = string[] | number[] | boolean[]

// ── NonNullable implemented with conditional type ──────────────
type MyNonNullable<T> = T extends null | undefined ? never : T;

type N = MyNonNullable<string | null | undefined | number>;
// Each member is checked:
// string    extends null|undefined? No  → string
// null      extends null|undefined? Yes → never
// undefined extends null|undefined? Yes → never
// number    extends null|undefined? No  → number
// Result: string | number (null and undefined become never = removed)

// ── Prevent distribution with [] wrapping ────────────────────
type IsNever<T> = [T] extends [never] ? true : false;
// Without [], T = never would make the conditional return never (not true)

type Check1 = IsNever<never>;   // true
type Check2 = IsNever<string>;  // false

// ── Real-world: Filter union by type ─────────────────────────
type FilterStrings<T> = T extends string ? T : never;

type Mixed = 'active' | 'pending' | 42 | true | 'completed';
type StringsOnly = FilterStrings<Mixed>;
// 'active' | 'pending' | 'completed'  — numbers and booleans removed
```

3.3 Real-World Conditional Types

```
// ── Unwrap Promise type ──────────────────────────────────────
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;

type R1 = UnwrapPromise<Promise<User>>;    // User
type R2 = UnwrapPromise<Promise<string>>;  // string
type R3 = UnwrapPromise<string>;           // string (not a Promise — returns
T)

// ── Get function return type ─────────────────────────────────
type GetReturn<T> = T extends (...args: any[]) => infer R ? R : never;
```

```
type R4 = GetReturn<() => string>;            // string
type R5 = GetReturn<(x: number) => boolean>; // boolean
type R6 = GetReturn<string>;                  // never (not a function)

// ── Make a type serialisable — recursively stringify values ──
type Stringify<T> =
  T extends string  ? T :
  T extends number  ? `${T}` :
  T extends boolean ? `${T}` :
  T extends object  ? { [K in keyof T]: Stringify<T[K]> } :
  string;

// ── API response conditional — different shapes per method ────
type ApiResult<T, Method extends 'GET' | 'POST' | 'DELETE'> =
  Method extends 'DELETE'
    ? { success: boolean; deletedId: number }
    : Method extends 'POST'
      ? { success: boolean; created: T }
      : { success: boolean; data: T };   // GET

type GetUser    = ApiResult<User, 'GET'>;    // { success, data: User }
type CreateUser = ApiResult<User, 'POST'>;   // { success, created: User }
type DeleteUser = ApiResult<User, 'DELETE'>; // { success, deletedId: number }
```

4. The 'infer' Keyword — Type Extraction

The `infer` keyword is used INSIDE conditional types to **capture and name** a type that TypeScript infers from a pattern match. Think of it as 'let TypeScript figure out what type is here, and give it this name'.

Real-World Analogy — The Detective's Extract

A detective examines a crime scene.

He doesn't know who the criminal is yet, but he knows the SHAPE of the evidence.

Detective: 'If I find a shoe print matching this pattern,

INFER the shoe size and store it as ShoeSizeType'

TypeScript: 'If T matches Promise<___>, INFER what ___ is and call it U'

infer = 'If this pattern matches, extract and name the unknown part'

4.1 infer Syntax and Basics

```
// ── Infer the type inside a Promise ──────────────────────────
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;
//                                        ↑↑↑↑↑
//                'If T is Promise<something>, call that something U'

type A = UnwrapPromise<Promise<string>>;  // string  (U = string)
type B = UnwrapPromise<Promise<User>>;    // User    (U = User)
type C = UnwrapPromise<Promise<number[]>>;// number[] (U = number[])
type D = UnwrapPromise<string>;           // string  (not a Promise → T
itself)

// ── Infer the element type of an array ───────────────────────
type ElementOf<T> = T extends (infer E)[] ? E : never;

type E1 = ElementOf<string[]>;   // string
type E2 = ElementOf<User[]>;     // User
type E3 = ElementOf<number[][]>; // number[]  (inner array type)
type E4 = ElementOf<string>;     // never  (not an array)

// ── Infer return type of a function ──────────────────────────
type RetType<T> = T extends (...args: any[]) => infer R ? R : never;

type R1 = RetType<() => string>;              // string
type R2 = RetType<(x: number) => User>;       // User
type R3 = RetType<(a: string, b: number) => boolean[]>; // boolean[]

// ── Infer first parameter type ───────────────────────────────
type FirstParam<T> = T extends (first: infer F, ...rest: any[]) => any ? F :
never;

type P1 = FirstParam<(name: string) => void>;         // string
type P2 = FirstParam<(id: number, name: string) => User>; // number
```

4.2 Advanced infer Patterns

```
// ── Infer deeply nested types ────────────────────────────────

// Unwrap multiple levels of Promise:
```

```
type DeepUnwrap<T> = T extends Promise<infer U> ? DeepUnwrap<U> : T;

type D1 = DeepUnwrap<Promise<string>>;                      // string
type D2 = DeepUnwrap<Promise<Promise<string>>>;             // string
type D3 = DeepUnwrap<Promise<Promise<Promise<User>>>>;      // User

// ── Infer tuple types ──────────────────────────────────────
type Head<T extends any[]> = T extends [infer H, ...any[]] ? H : never;
type Tail<T extends any[]> = T extends [any, ...infer T]    ? T : never;
type Last<T extends any[]> = T extends [...any[], infer L]  ? L : never;

type T1 = Head<[string, number, boolean]>;   // string
type T2 = Tail<[string, number, boolean]>;   // [number, boolean]
type T3 = Last<[string, number, boolean]>;   // boolean

// ── Infer object property type ─────────────────────────────
type PropType<T, K extends keyof T> = T extends { [key in K]: infer V } ? V :
never;

interface User { id: number; name: string; role: 'admin' | 'user' }

type IdType   = PropType<User, 'id'>;    // number
type NameType = PropType<User, 'name'>;  // string
type RoleType = PropType<User, 'role'>;  // 'admin' | 'user'

// ── Real-world: Extract route params from URL string ────────
// Template literal + infer = route param extraction
type RouteParam<T extends string> =
  T extends `${string}:${infer Param}/${string}` ? Param :
  T extends `${string}:${infer Param}` ? Param :
  never;

type P = RouteParam<'/users/:id/posts/:postId'>; // 'id' (first param found)
```

5. Template Literal Types — Advanced

We saw template literals briefly in Chapter 11. Here we go deep — combining template literals with mapped types, conditional types, and infer to build truly powerful type transformations.

```
// ── Template literal type fundamentals ────────────────────
type Lang   = 'en' | 'hi' | 'ta';
```

```
type Format = 'json' | 'xml' | 'csv';

type ApiRoute = `/api/${Lang}/data.${Format}`;
// '/api/en/data.json' | '/api/en/data.xml' | '/api/en/data.csv'
// | '/api/hi/data.json' | ...  (9 combinations automatically!)

// ── Build event listener method names ──────────────────────
type EventName = 'click' | 'focus' | 'blur' | 'change' | 'submit';
type Listener  = `on${Capitalize<EventName>}`;
// 'onClick' | 'onFocus' | 'onBlur' | 'onChange' | 'onSubmit'

type EventHandlerMap = { [E in Listener]?: (event: Event) => void };
// { onClick?: (event: Event) => void; onFocus?: ...; ... }

// ── CSS variable names with type safety ────────────────────
type ColorName = 'primary' | 'secondary' | 'danger' | 'success';
type ColorShade = '100' | '200' | '300' | '400' | '500' | '600';
type CssVar = `--color-${ColorName}-${ColorShade}`;
// '--color-primary-100' | '--color-primary-200' | ... (24 combinations)

// ── Extract parts from a string type with infer ────────────
type ExtractVerb<T extends string> =
  T extends `${infer Verb}_${string}` ? Verb : T;

type V1 = ExtractVerb<'click_button'>;    // 'click'
type V2 = ExtractVerb<'submit_form'>;     // 'submit'
type V3 = ExtractVerb<'hover'>;           // 'hover' (no _, returns T)

// ── SplitString type ───────────────────────────────────────
type Split<S extends string, D extends string> =
  S extends `${infer Head}${D}${infer Tail}`
    ? [Head, ...Split<Tail, D>]
    : [S];

type S1 = Split<'a.b.c', '.'>;     // ['a', 'b', 'c']
type S2 = Split<'en-US', '-'>;     // ['en', 'US']
type S3 = Split<'hello', '.'>;     // ['hello']

// ── Deep key access with dot notation ──────────────────────
type DeepKeyOf<T, Prefix extends string = ''> = {
  [K in keyof T & string]:
    T[K] extends object
```

```
      ? DeepKeyOf<T[K], `${Prefix}${K}.`>
      : `${Prefix}${K}`
}[keyof T & string];

interface Config {
  db: { host: string; port: number };
  app: { name: string; version: string };
}

type ConfigKeys = DeepKeyOf<Config>;
// 'db.host' | 'db.port' | 'app.name' | 'app.version'
```

6. Recursive Types

Recursive types reference themselves in their own definition. They are essential for modelling nested structures like trees, file systems, JSON data, and deeply nested config objects.

Real-World Analogy — A Family Tree

A Person can have children.

Each child is also a Person, who can have their own children.

Each of THOSE children is also a Person... and so on.

type Person = { name: string; children: Person[] }

The type references ITSELF — that's recursive. It models the nesting naturally, no matter how deep the tree goes.

```
// ── Basic recursive type ──────────────────────────────────
type NestedArray<T> = T | NestedArray<T>[];
// A value is either T, or an array of NestedArray<T>

const a: NestedArray<number> = 1;
const b: NestedArray<number> = [1, 2, 3];
const c: NestedArray<number> = [1, [2, 3], [4, [5, 6]]]; // deeply nested

// ── JSON type (fully typed JSON) ──────────────────────────
type JsonPrimitive = string | number | boolean | null;
type JsonArray     = JsonValue[];
type JsonObject    = { [key: string]: JsonValue };
```

```
type JsonValue     = JsonPrimitive | JsonArray | JsonObject;

// Any valid JSON can be typed as JsonValue:
const config: JsonValue = {
  name:     'my-app',
  version:  '1.0.0',
  features: ['auth', 'payments'],
  settings: { theme: 'dark', lang: 'en' },
  active:   true,
};

// ── Tree data structure ──────────────────────────────────────
interface TreeNode<T> {
  value:    T;
  children: TreeNode<T>[];  // Each child is also a TreeNode
}

// File system tree:
const fileTree: TreeNode<string> = {
  value: 'root',
  children: [
    { value: 'src', children: [
      { value: 'index.ts',  children: [] },
      { value: 'app.ts',    children: [] },
    ]},
    { value: 'package.json', children: [] },
    { value: 'tsconfig.json', children: [] },
  ]
};

// ── Category hierarchy (e-commerce) ──────────────────────────
interface Category {
  id:         number;
  name:       string;
  slug:       string;
  children:   Category[];  // Subcategories — recursive!
  parentId?:  number;
}

const electronics: Category = {
  id: 1, name: 'Electronics', slug: 'electronics',
  children: [
    { id: 2, name: 'Phones', slug: 'phones', parentId: 1,
```

```
      children: [
        { id: 5, name: 'Smartphones', slug: 'smartphones', parentId: 2,
children: [] },
      ]
    },
    { id: 3, name: 'Laptops', slug: 'laptops', parentId: 1, children: [] },
  ]
};
```

6.2 Recursive Utility Types

```
// ── DeepReadonly — recursively make everything readonly ────────
type DeepReadonly<T> =
  T extends (infer E)[]
    ? ReadonlyArray<DeepReadonly<E>>
    : T extends object
      ? { readonly [P in keyof T]: DeepReadonly<T[P]> }
      : T;

interface NestedConfig {
  db: { host: string; port: number; credentials: { user: string } };
  app: { name: string; flags: string[] };
}

type FrozenConfig = DeepReadonly<NestedConfig>;
// Every nested property is readonly — truly immutable

const config: FrozenConfig = {
  db:  { host: 'localhost', port: 5432, credentials: { user: 'admin' } },
  app: { name: 'myapp', flags: ['auth'] }
};

config.db.host = 'newhost';            // ✗ ERROR — readonly
config.db.credentials.user = 'hacked'; // ✗ ERROR — deeply readonly
config.app.flags.push('payments');     // ✗ ERROR — ReadonlyArray

// ── DeepPartial — recursively make everything optional ────────
type DeepPartial<T> =
  T extends object
    ? { [P in keyof T]?: DeepPartial<T[P]> }
    : T;
```

```
// DeepPartial<NestedConfig>:
// { db?: { host?: string; port?: number; credentials?: { user?:
string } }; ... }


// ── Path type — dot notation for deep access ──────────────────
type Paths<T, Prefix extends string = ''> =
  T extends object
    ? { [K in keyof T & string]: Paths<T[K], `${Prefix}${K}.`> }[keyof T &
string]
    : Prefix extends `${string}.` ? Prefix extends `${infer R}.` ? R : never :
never;
```

7. Variance — Covariance and Contravariance

Variance describes how TypeScript handles subtype relationships when types are composed. Understanding variance helps you understand why some type assignments work and others don't.

Simple Explanation

Covariant = allows using a MORE SPECIFIC type where a broader one is expected

Example: Dog can be used where Animal is expected ✅

Contravariant = requires a MORE GENERAL type where a specific one is expected

Example: A function accepting Animal can handle Dog calls ✅

Invariant = must match EXACTLY — no substitution allowed

```
// ── Covariance — return types are covariant ────────────────────
class Animal { name: string = ''; }
class Dog extends Animal { breed: string = ''; }

// Return type is COVARIANT: can return a subtype
type AnimalFactory = () => Animal;
type DogFactory    = () => Dog;

let makeAnimal: AnimalFactory;
let makeDog:    DogFactory    = () => new Dog();

makeAnimal = makeDog; // ✅ Dog is-a Animal — covariant return is safe
// Every time makeAnimal is called and returns a Dog,
// the caller expected an Animal — Dog satisfies that.
```

```
// ── Contravariance — parameter types are contravariant ────────
type AnimalHandler = (animal: Animal) => void;
type DogHandler    = (dog: Dog) => void;

let handleAnimal: AnimalHandler = (a) => console.log(a.name);
let handleDog:    DogHandler;

handleDog = handleAnimal; // ✅ Safe — a function that handles ANY animal
                          // can certainly handle a Dog
// handleAnimal = handleDog; // ❌ Unsafe — a Dog handler might access
                             // dog.breed which Animal doesn't have

// ── Practical: Why function params feel 'flipped' ────────────
// This is why: (Dog) => void  is NOT assignable to  (Animal) => void
// The parameter direction is REVERSED from what you might expect

// ── Readonly makes types covariant ───────────────────────────
// Mutable arrays are INVARIANT (neither assignable to the other)
const dogs: Dog[] = [new Dog()];
let animals: Animal[];
// animals = dogs; // ❌ Unsafe — someone could push(new Cat()) into animals

// Readonly arrays are COVARIANT (safer)
const readonlyDogs: ReadonlyArray<Dog> = [new Dog()];
let readonlyAnimals: ReadonlyArray<Animal>;
readonlyAnimals = readonlyDogs; // ✅ Safe — can only read, not push/mutate
```

8. Branded / Nominal Types

TypeScript uses structural typing — if two types have the same shape, they are interchangeable. But sometimes you want TWO TYPES WITH THE SAME STRUCTURE to be DISTINCT. Branded types (also called nominal types) solve this.

The Problem — Structural Typing Can Be Dangerous

```
type UserId  = number;
type OrderId = number;

function getOrder(orderId: OrderId): Order { ... }
```

const userId: UserId = 42;
getOrder(userId); // ✅ TypeScript ALLOWS this — both are just 'number'!
// But this is a bug! You passed a UserId where an OrderId was needed.

Branded types prevent this by making UserId and OrderId DISTINCT
even though both are numbers underneath.

```
// ── Branded type pattern ──────────────────────────────────
// Use an intersection with a unique 'brand' marker
type UserId  = number & { readonly __brand: 'UserId' };
type OrderId = number & { readonly __brand: 'OrderId' };
type ProductId = number & { readonly __brand: 'ProductId' };

// Brand constructor functions — the only way to create branded values
function userId(n: number):    UserId    { return n as UserId; }
function orderId(n: number):   OrderId   { return n as OrderId; }
function productId(n: number): ProductId { return n as ProductId; }

// Usage:
const uid: UserId    = userId(42);
const oid: OrderId   = orderId(99);
const pid: ProductId = productId(7);

function getOrder(id: OrderId): void { console.log(`Order: ${id}`); }
function getUser(id: UserId):   void { console.log(`User: ${id}`);  }

getOrder(oid);  // ✅ Correct
getOrder(uid);  // ❌ ERROR: UserId is not assignable to OrderId
getOrder(pid);  // ❌ ERROR: ProductId is not assignable to OrderId
getOrder(42);   // ❌ ERROR: plain number not assignable to OrderId

// ── Branded types for validated data ──────────────────────
type ValidatedEmail = string & { readonly __brand: 'ValidatedEmail' };
type HashedPassword = string & { readonly __brand: 'HashedPassword' };

function validateEmail(email: string): ValidatedEmail {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!regex.test(email)) throw new Error('Invalid email');
  return email as ValidatedEmail;
}
```

```
function hashPassword(password: string): HashedPassword {
  const hashed = Buffer.from(password).toString('base64');
  return hashed as HashedPassword;
}

// Functions requiring validated/hashed values:
function sendEmail(to: ValidatedEmail, subject: string): void {}
function createUser(email: ValidatedEmail, pwdHash: HashedPassword): void {}

const email = validateEmail('rahul@gmail.com'); // ValidatedEmail
const hash  = hashPassword('secret123');        // HashedPassword

sendEmail(email, 'Welcome!');        // ✅
sendEmail('rahul@gmail.com', 'Hi'); // ❌ ERROR: raw string not allowed
// You MUST call validateEmail() first — prevents passing unvalidated data
```

9. Satisfies Operator (TypeScript 4.9+)

The `satisfies` operator lets you check that a value matches a type **without widening** its type to that of the annotation. You get both type checking AND the benefit of the inferred type.

```
// ── The problem: 'as' annotation widens types ────────────────
type ColorMap = Record<string, string | [number, number, number]>;

// With type annotation — values become string | [number, number, number]
const colors1: ColorMap = {
  red:   [255, 0, 0],
  green: '#00ff00',
  blue:  [0, 0, 255],
};

// TypeScript loses specific type info:
colors1.red.map(x => x * 2); // ❌ ERROR — TS thinks it's string|[n,n,n]
                             // It forgot red is specifically a tuple!

// ── With satisfies — checks type but KEEPS the inferred type ──
const colors2 = {
  red:   [255, 0, 0],
  green: '#00ff00',
  blue:  [0, 0, 255],
} satisfies ColorMap;
```

```
// Now: TypeScript checks that the object satisfies ColorMap (validation)
// BUT it also knows the specific types of each property!
colors2.red.map(x => x * 2);         // ✅ TypeScript knows red is number[]
colors2.green.toUpperCase();          // ✅ TypeScript knows green is string
colors2.notExists;                   // ❌ ERROR — checked against ColorMap

// ── Real-world: Route configuration ──────────────────────────
type RouteConfig = Record<string, {
  path: string;
  component: string;
  auth?: boolean;
}>;

const routes = {
  home:     { path: '/',        component: 'HomePage' },
  login:    { path: '/login',   component: 'LoginPage' },
  dashboard:{ path: '/dash',    component: 'DashPage', auth: true },
  profile:  { path: '/profile', component: 'ProfilePage', auth: true },
} satisfies RouteConfig;

// TypeScript validates the shape (satisfies RouteConfig)
// AND preserves specific types of each route:
routes.home.path;                     // ✅ string
routes.dashboard.auth;               // ✅ boolean (not boolean | undefined)
routes.nonExistentRoute;             // ❌ ERROR — caught by satisfies
```

10. Complete Real-World Example — Type-Safe Query Builder

Let's combine mapped types, conditional types, template literals, and recursive types to build a type-safe database query builder.

```
// ═══ Types for a type-safe query builder ═══════════════════

// Brand IDs so they can't be mixed up:
type UserId    = number & { __brand: 'UserId' }
type ProductId = number & { __brand: 'ProductId' }

// Core entity types:
interface User {
```

```
  id:        UserId;
  name:      string;
  email:     string;
  role:      'admin' | 'user' | 'guest';
  createdAt: Date;
}

// Operators that can be used on different field types:
type StringOps  = 'eq' | 'ne' | 'like' | 'startsWith' | 'in';
type NumberOps  = 'eq' | 'ne' | 'gt' | 'gte' | 'lt' | 'lte' | 'in';
type DateOps    = 'eq' | 'ne' | 'gt' | 'gte' | 'lt' | 'lte';
type BooleanOps = 'eq';

// Map TypeScript types to their allowed query operators:
type OpsFor<T> =
  T extends string  ? StringOps  :
  T extends number  ? NumberOps  :
  T extends Date    ? DateOps    :
  T extends boolean ? BooleanOps :
  never;

// A filter for a single field:
type FieldFilter<T, V> = {
  [Op in OpsFor<V>]?: V extends any[] ? V : V;
};

// Build filter type for ALL fields of an entity:
type QueryFilter<T> = {
  [K in keyof T]?: {
    op:    OpsFor<T[K]>;
    value: T[K];
  }
};

// Column selection type:
type SelectColumns<T> = (keyof T)[];

// Sort order:
type SortOrder<T> = {
  field: keyof T;
  dir:   'asc' | 'desc';
};
```

```
// Full query options:
interface QueryOptions<T> {
  select?:  SelectColumns<T>;
  where?:   QueryFilter<T>;
  orderBy?: SortOrder<T>;
  limit?:   number;
  offset?:  number;
}

// Type-safe query function:
function query<T>(entity: string, options: QueryOptions<T>): T[] {
  // Build and execute SQL from typed options
  console.log(`Querying ${entity}:`, JSON.stringify(options, null, 2));
  return [];  // Return from DB
}

// ═══ Usage — all type-safe! ═══════════════════════════════
const users = query<User>('users', {
  select: ['id', 'name', 'email'],        // ✅ only User keys allowed
  where: {
    name:  { op: 'like', value: 'Rah%' }, // ✅ string → StringOps
    role:  { op: 'eq',   value: 'admin' },// ✅ literal string
  },
  orderBy: { field: 'createdAt', dir: 'desc' }, // ✅ valid User field
  limit: 10,
});

// Type errors caught at compile time:
query<User>('users', {
  select: ['nonExistent'],               // ❌ not a User field
  where: {
    name: { op: 'gt', value: 'Rahul' },  // ❌ 'gt' not valid for strings
  },
  orderBy: { field: 'fake', dir: 'asc' }, // ❌ not a User field
});
```

11. Interview Questions — Chapter 12

Q: What is a mapped type? Write one from scratch.

Answer: A mapped type creates a new type by iterating over the keys of an existing type and

transforming each property.

Syntax: { [P in keyof T]: TransformedType }

Example — make every value a getter function:

```
type Getters<T> = { [P in keyof T]: () => T[P] }
```

This transforms { name: string } into { name: () => string }

TypeScript's built-in Partial, Readonly, Record are all implemented as mapped types.

The +? adds optional, -? removes optional, readonly adds readonly, -readonly removes it.

Q: What is a conditional type? Give a real-world example.

Answer: A conditional type works like an if/else at the type level:

```
T extends U ? TrueType : FalseType
```

If T is assignable to U, the type resolves to TrueType; otherwise FalseType.

Example: type Flatten<T> = T extends Array<infer Item> ? Item : T

```
Flatten<string[]> → string   (it's an array, return element type)
Flatten<number>   → number   (not an array, return T itself)
```

Real-world: ApiResult<T, Method> — return different shapes for GET vs POST vs DELETE.

Q: What does the 'infer' keyword do?

Answer: 'infer' is used inside conditional types to capture and name an unknown type that TypeScript figures out from a pattern match.

Without infer: T extends Promise<???> — can't name or use the inner type.

With infer: T extends Promise<infer U> ? U : T — U captures the inner type.

Examples:

```
T extends Array<infer E> ? E : never        — gets array element type
T extends (...args: any) => infer R ? R : never — gets function return type
T extends Promise<infer U> ? U : T          — unwraps Promise
```

Infer only works in the 'true' branch of a conditional type.

Q: What are branded types? Why would you use them?

Answer: TypeScript uses structural typing — two types with the same shape are interchangeable. Branded types add a unique invisible 'brand' to make structurally identical types distinct.

Problem: type UserId = number; type OrderId = number;

getOrder(userId) — TypeScript allows this! Both are numbers.

Solution: type UserId = number & { __brand: 'UserId' }

type OrderId = number & { __brand: 'OrderId' }

Now UserId and OrderId are DISTINCT — assigning one to the other is an error.

Use for: database IDs, validated data (ValidatedEmail, HashedPassword), currency amounts (USD vs EUR), physical units (Metres vs Feet).

Q: What is the 'satisfies' operator and how does it differ from a type annotation?

Answer: The satisfies operator (TypeScript 4.9+) validates that a value matches a type without widening the inferred type.

Type annotation (: Type): TypeScript widens the value's type to match the annotation. You lose specificity.

const x: ColorMap = { red: [255,0,0] }

x.red is now string | number[] — TypeScript forgot it's specifically a tuple!

satisfies: Validates against the type but PRESERVES the inferred type.

const x = { red: [255,0,0] } satisfies ColorMap

x.red is still [number, number, number] — specific type preserved!

Use satisfies when you want both validation AND the benefits of type inference.

Q: What are distributive conditional types? How do you prevent distribution?

Answer: When T in a conditional type is a union, TypeScript DISTRIBUTES the condition across each union member separately.

Example: type ToArray<T> = T extends any ? T[] : never

ToArray<string | number> = string[] | number[]

TypeScript checks: (string extends any → string[]) | (number extends any → number[])

To PREVENT distribution, wrap T in a tuple: [T] extends [U] ? ... : ...

type IsNever<T> = [T] extends [never] ? true : false

Without [], IsNever<never> = never (distributes over empty union = nothing)

With [], IsNever<never> = true (correctly detects never)

Q: What is the difference between covariance and contravariance?

Answer: These describe how subtype relationships work when types are composed.

Covariant (return types): you can use a MORE SPECIFIC type where a broader type is expected.

() => Dog is assignable to () => Animal ✓

(Dog is-a Animal, so a function returning Dog satisfies 'returns Animal')

Contravariant (param types): you need a MORE GENERAL type.

(animal: Animal) => void is assignable to (dog: Dog) => void ✓

(but NOT the reverse — a Dog handler might use Dog-specific properties that a plain Animal doesn't have)

Practical effect: readonly arrays are covariant (Dog[] → Animal[] OK), mutable arrays are invariant (Dog[] → Animal[] unsafe — someone could push a Cat!).

Rapid Fire — Chapter 12

Question	One-Line Answer
Syntax for a mapped type?	{ [P in keyof T]: T[P] } — iterate and transform every key
How to remove optional with mapped type?	{ [P in keyof T]-?: T[P] } — the -? removes optional marker
How to remove readonly with mapped type?	{ -readonly [P in keyof T]: T[P] } — the - prefix removes it
Syntax for conditional type?	T extends U ? TrueType : FalseType
What does infer capture?	An unknown type from inside a pattern match in a conditional type
What does Flatten<string[]> return?	string — infers element type from the array wrapper

Branded types prevent what?	Accidentally passing one ID type where a different ID type is expected
satisfies vs : annotation?	satisfies validates WITHOUT widening; : annotation widens to annotated type
Recursive type example?	type Json = string | number | boolean | null | Json[] | { [k: string]: Json }
Covariant position in functions?	Return types — can return a subtype. Contravariant = parameter types

Chapter 12 Key Takeaways

Mapped types: iterate over keys and transform — { [P in keyof T]: ... }.

+?/-? adds/removes optional. readonly/-readonly adds/removes immutability.

Key remapping with 'as' renames keys; returning 'never' filters them out.

Conditional types: T extends U ? A : B — if/else at the type level.

Distributive: applied to each union member separately.

Prevent distribution: wrap in tuple [T] extends [U].

infer: capture and name an unknown type inside a conditional type.

T extends Promise<infer U> — U is the resolved type of the Promise.

Recursive types: types that reference themselves (JSON, trees, categories).

DeepReadonly<T> and DeepPartial<T> are recursively applied mapped types.

Branded types: add an invisible marker to prevent structural type confusion.

Perfect for IDs, validated data, and units of measurement.

satisfies: validate against a type while preserving the inferred type.

Variance: return types are covariant (can use subtype).

Parameter types are contravariant (need supertype).

Readonly arrays are covariant; mutable arrays are invariant.

End of Chapter 12

CHAPTER 13

Modules & Namespaces

ES Modules • import / export • Module Resolution • Declaration Files • Namespaces • Barrel Files

1. Why Modules? — The Problem They Solve

Before modules existed, all JavaScript ran in a single global scope. Every variable, function, and class was accessible from everywhere — and naming collisions were a constant nightmare. Modules solve this by giving each file its own private scope, with explicit control over what to export and what to keep internal.

Real-World Analogy — The Office Building

Imagine a company without offices — everyone works in one open hall.

Every paper, file, and conversation is visible to everyone.

A new employee named 'config' joins — but there are already 3 other 'configs'!

Modules are like giving each team their own OFFICE (file scope).

→ Each office has its own files and people (private to the module)

→ Teams can SHARE specific things through official channels (exports)

→ Other teams REQUEST what they need (imports)

→ No accidental name collisions between departments

Each .ts file = one office. export = what you share. import = what you request.

Module vs Script — A Critical Distinction

A TypeScript file is a MODULE if it has at least one import or export statement.

A TypeScript file is a SCRIPT if it has NO import or export statements.

Scripts: All variables are in GLOBAL scope — can conflict with other scripts.

Modules: All variables are in MODULE scope — isolated, no conflicts.

Modern TypeScript projects almost always use modules.

If you want an empty file to be a module (not a script):

export {} — empty export makes it a module.

2. ES Modules — export and import

2.1 Named Exports

```
// ── File: utils/math.ts ──────────────────────────────────

// Named export: export specific things by name
export function add(a: number, b: number): number {
  return a + b;
}

export function subtract(a: number, b: number): number {
  return a - b;
}

export const PI = 3.14159;

export interface MathConfig {
  precision: number;
  rounding: 'floor' | 'ceil' | 'round';
}

export type Operation = (a: number, b: number) => number;

// This is NOT exported — private to this module:
function internalHelper(n: number): number {
  return Math.abs(n);
}

// ── File: main.ts — importing named exports ────────────────
import { add, subtract, PI } from './utils/math';
import type { MathConfig, Operation } from './utils/math';
// 'import type' — only imports the TYPE, erased at runtime

console.log(add(5, 3));       // 8
console.log(subtract(10, 4)); // 6
console.log(PI);              // 3.14159

const config: MathConfig = { precision: 2, rounding: 'round' };
```

```
// ── Renaming imports with 'as' ──────────────────────────
import { add as sum, PI as piValue } from './utils/math';
console.log(sum(1, 2));   // 3
console.log(piValue);     // 3.14159
```

2.2 Default Exports

```
// ── File: services/UserService.ts — default export ───────

interface User { id: number; name: string; email: string }

class UserService {
  private users: User[] = [];

  async getAll(): Promise<User[]> { return this.users; }

  async getById(id: number): Promise<User | undefined> {
    return this.users.find(u => u.id === id);
  }

  async create(data: Omit<User, 'id'>): Promise<User> {
    const user: User = { ...data, id: Date.now() };
    this.users.push(user);
    return user;
  }
}

export default UserService;   // Default export — no name required on import

// ── File: main.ts — importing default export ──────────────
import UserService from './services/UserService';
// The import name can be ANYTHING — no curly braces needed
import MyUserService from './services/UserService'; // Also valid!

const service = new UserService();
const users   = await service.getAll();

// ── Mixing default and named exports ──────────────────────
// File: config.ts
export const VERSION = '1.0.0';          // named
export type  Env = 'dev' | 'prod';        // named type
export default { apiUrl: '/api', timeout: 5000 }; // default
```

```
// Importing both:
import defaultConfig, { VERSION, type Env } from './config';
console.log(defaultConfig.apiUrl, VERSION);
```

2.3 Re-Exporting (Barrel Files)

```
// ── Barrel file pattern — index.ts aggregates exports ────
// File structure:
//   src/
//     services/
//       UserService.ts
//       OrderService.ts
//       PaymentService.ts
//       index.ts   ← barrel file

// File: services/index.ts
export { default as UserService }    from './UserService';
export { default as OrderService }   from './OrderService';
export { default as PaymentService } from './PaymentService';
export type { User }   from './UserService';
export type { Order }  from './OrderService';

// ── Now consumers import from ONE place ──────────────────
// Without barrel file (messy):
import UserService    from './services/UserService';
import OrderService   from './services/OrderService';
import PaymentService from './services/PaymentService';

// With barrel file (clean):
import { UserService, OrderService, PaymentService } from './services';
// TypeScript resolves './services' to './services/index.ts'

// ── Re-export with rename ────────────────────────────────
export { add as sum } from './math';        // rename on re-export
export * from './validators';               // re-export everything
export * as MathUtils from './math';        // namespace re-export

// ── Real-world project barrel structure ──────────────────
// src/models/index.ts
export type { User }    from './User';
export type { Product } from './Product';
```

```
export type { Order }   from './Order';

// src/utils/index.ts
export { formatDate }   from './date';
export { formatCurrency } from './currency';
export { slugify }      from './string';

// src/index.ts (root barrel)
export * from './models';
export * from './utils';
export * from './services';
```

Real-World Scenario: Large-Scale Node.js / React Project

In a production app with hundreds of files, barrel files are essential. Every feature module (auth, orders, payments) has its own index.ts. Other modules import cleanly from 'features/auth' instead of hunting for specific file paths. Refactoring a file's location only breaks the barrel file — not every importer.

3. Module Resolution

When TypeScript sees an import, it needs to find the actual file. Module resolution is the algorithm TypeScript uses to locate files. Understanding it saves hours of debugging 'Cannot find module' errors.

3.1 Relative vs Non-Relative Imports

```
// ── Relative imports — start with . or .. ─────────────────
import { add }      from './math';        // same directory
import { User }     from '../models/User'; // parent directory
import { config }   from '../../config';  // two levels up

// TypeScript tries these extensions in order: .ts, .tsx, .d.ts
// So './math' will find './math.ts' or './math/index.ts'

// ── Non-relative imports — no . prefix ────────────────────
import React        from 'react';        // node_modules
import { Router }   from 'express';      // node_modules
import { format }   from 'date-fns';     // node_modules

// TypeScript looks in node_modules for these.
// Also needs @types/react, @types/express for type definitions.
```

```
// ── Path aliases — configured in tsconfig.json ──────────
// Without aliases (messy relative paths):
import { UserService } from '../../../services/UserService';
import { formatDate }  from '../../../../utils/date';

// With path aliases (clean absolute-style imports):
import { UserService } from '@/services/UserService';
import { formatDate }  from '@/utils/date';
```

3.2 tsconfig.json Module Settings

```
// ── tsconfig.json — key module-related settings ──────────
{
  "compilerOptions": {

    // 'module' — output module format
    "module": "NodeNext",    // Modern Node.js (recommended)
    // "module": "ESNext",   // Browser / bundler (Vite, webpack)
    // "module": "CommonJS", // Older Node.js (require/module.exports)

    // 'moduleResolution' — how TypeScript finds files
    "moduleResolution": "NodeNext",  // Modern, supports exports field
    // "moduleResolution": "Bundler",  // For Vite/webpack setups
    // "moduleResolution": "Node",     // Classic Node.js (older projects)

    // 'baseUrl' — root for non-relative imports
    "baseUrl": "./src",

    // 'paths' — path aliases
    "paths": {
      "@/*":          ["./src/*"],
      "@models/*":    ["./src/models/*"],
      "@services/*":  ["./src/services/*"],
      "@utils/*":     ["./src/utils/*"],
    },

    // 'rootDir' — where your source files are
    "rootDir": "./src",

    // 'outDir' — where compiled JS goes
    "outDir": "./dist",
```

```
    // 'esModuleInterop' — enables default imports from CommonJS
    "esModuleInterop": true,

    // 'resolveJsonModule' — allows importing JSON files
    "resolveJsonModule": true,
  }
}

// ── With path aliases configured, you can now do: ─────────
import { UserService }   from '@/services/UserService';
import { Product }       from '@models/Product';
import { formatCurrency } from '@utils/currency';
import data              from '@/data/config.json'; // JSON import
```

3.3 Module Resolution Algorithm (Node Strategy)

```
// When TypeScript resolves: import { x } from './foo'
//
// Step 1: Try file directly
//   ./foo.ts
//   ./foo.tsx
//   ./foo.d.ts
//
// Step 2: Try as a directory with index file
//   ./foo/index.ts
//   ./foo/index.tsx
//   ./foo/index.d.ts
//
// Step 3: Check package.json 'types' field (for packages)
//
// When TypeScript resolves: import { x } from 'express'
//
// Step 1: Look in node_modules/express
// Step 2: Look for node_modules/express/index.d.ts
// Step 3: Look for node_modules/@types/express

// ── Common 'Cannot find module' fixes ────────────────────
// Problem: 'Cannot find module react'
// Fix: npm install --save-dev @types/react

// Problem: 'Cannot find module ./utils'
// Fix: Check file exists as utils.ts or utils/index.ts
```

```
// Problem: Path aliases not working
// Fix: Set baseUrl AND paths in tsconfig, AND configure
//       bundler aliases (vite.config.ts, webpack.config.js)
```

4. Declaration Files (.d.ts)

Declaration files (`.d.ts`) contain ONLY type information — no runtime JavaScript. They describe the shape of existing JavaScript code so TypeScript can type-check it. Every popular JS library has either built-in declarations or community-provided `@types/...` packages.

Real-World Analogy — The Product Manual

A car engine exists (JavaScript code) — it runs and does things.

The product manual (declaration file) describes the engine:

→ What buttons do what

→ What values each gauge shows

→ What inputs it accepts

The manual is NOT the engine — it just DESCRIBES it.

TypeScript reads the manual (.d.ts) to understand the engine (JS code).

npm install lodash → the engine (JS code)

npm install @types/lodash → the product manual (.d.ts types)

4.1 Writing Declaration Files

```
// ── Scenario: you have a plain JS file with no types ────
// File: legacy/validator.js (JavaScript — no types)
// function isEmail(str) { return str.includes('@'); }
// function isPhone(str) { return /^[0-9]{10}$/.test(str); }
// module.exports = { isEmail, isPhone };

// ── Create: legacy/validator.d.ts ──────────────────────
// This file DESCRIBES the JS file above — no implementation
export declare function isEmail(str: string): boolean;
export declare function isPhone(str: string): boolean;

// ── Now TypeScript knows the types when you import it: ───
import { isEmail, isPhone } from './legacy/validator';
```

```
isEmail('test@gmail.com');  // ✅ TypeScript knows it takes string → boolean
isEmail(42);                // ❌ ERROR — caught by type system

// ── Declaration file for a module ──────────────────────
// File: types/analytics.d.ts
declare module 'my-analytics' {
  export interface TrackEvent {
    name: string;
    properties?: Record<string, string | number>;
  }

  export function track(event: TrackEvent): void;
  export function identify(userId: string, traits?: object): void;
  export function page(name: string): void;
}

// ── Declaration for a global variable (CDN script) ────────
// File: types/globals.d.ts
declare const __APP_VERSION__: string;
declare const __BUILD_DATE__: string;

declare interface Window {
  analytics: {
    track(event: string, properties?: object): void;
  };
}

// Usage — TypeScript now knows these globals exist:
console.log(__APP_VERSION__);       // ✅
window.analytics.track('pageview'); // ✅
```

4.2 @types Packages and lib.d.ts

```
// ── Installing type definitions for popular libraries ─────
// npm install --save-dev @types/node
// npm install --save-dev @types/express
// npm install --save-dev @types/jest
// npm install --save-dev @types/lodash

// Many modern libraries include their own types:
// axios, zod, prisma, next.js — no @types needed
```

```
// ── lib.d.ts — TypeScript's built-in declarations ──────────
// TypeScript ships with declaration files for browser APIs,
// DOM, Node.js, and ES built-ins. Controlled via tsconfig:

// tsconfig.json:
// { "compilerOptions": { "lib": ["ES2022", "DOM"] } }

// 'ES2022' includes: Array, Map, Set, Promise, structuredClone...
// 'DOM'    includes: document, window, fetch, HTMLElement...
// 'DOM.Iterable' — adds iteration to DOM collections

// ── Checking if a library has its own types ────────────────
// Check package.json of the library:
// "types": "./dist/index.d.ts"  ← has own types
// "typings": "./types/index.d.ts" ← also has own types

// If neither exists, look for @types/library-name on npm

// ── Triple-slash directives (legacy, still used) ───────────
/// <reference types="node" />    // Include Node.js types
/// <reference lib="dom" />       // Include DOM types
/// <reference path="./types.d.ts" /> // Include specific file

// Usually managed via tsconfig 'lib' and 'types' — prefer that.
```

4.3 Module Augmentation

```
// ── Extend an existing module's types ────────────────────
// Very useful for adding properties to third-party libraries

// File: types/express.d.ts
// Augment Express's Request interface:
declare module 'express' {
  interface Request {
    user?:      { id: number; email: string; role: string };
    requestId?: string;
    startTime?: number;
  }
}

// Now in your Express route handlers:
```

```
import { Request, Response } from 'express';

function authMiddleware(req: Request, res: Response, next: Function) {
  const token = req.headers.authorization;
  if (token) {
    req.user = { id: 1, email: 'r@g.com', role: 'admin' }; // ✅ TypeScript
knows!
    req.requestId = generateId();
  }
  next();
}

function userRoute(req: Request, res: Response) {
  if (!req.user) return res.status(401).send('Unauthorized');
  res.json({ user: req.user });  // ✅ req.user is typed!
}

// ── Augment global Window for analytics ──────────────────
// File: types/window.d.ts
declare global {
  interface Window {
    dataLayer: Record<string, unknown>[];
    gtag: (command: string, ...args: unknown[]) => void;
  }
}

export {}; // Must have export to make this a module, not a script

// Usage:
window.gtag('event', 'purchase', { value: 99.99 }); // ✅ Typed!
```

5. Namespaces

Namespaces (formerly called 'internal modules') are a TypeScript-specific way to organise code within a single file or across files by grouping related declarations under a named scope. They are **less common in modern TypeScript** (modules are preferred), but you will encounter them in older codebases and in some declaration files.

Namespace vs Module — When to Use Which

Namespace: Code organisation WITHIN a file or across files WITHOUT a module system.

Used in: declaration files, older projects, global scripts, ambient declarations.

Module (import/export): Code organisation WITH a module system (Node, webpack, Vite).
Used in: ALL modern projects.

Rule: If you're building a modern app — use MODULES (import/export).
If you're writing .d.ts files or maintaining old code — know namespaces.

5.1 Basic Namespace Syntax

```
// ── Basic namespace ──────────────────────────────────────
namespace Validation {
  // Private to namespace (no export)
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  // Public — exported from namespace
  export interface StringValidator {
    isValid(s: string): boolean;
  }

  export class EmailValidator implements StringValidator {
    isValid(s: string): boolean {
      return emailRegex.test(s);
    }
  }

  export class PhoneValidator implements StringValidator {
    isValid(s: string): boolean {
      return /^[6-9]\d{9}$/.test(s);
    }
  }
}

// Usage:
const emailV = new Validation.EmailValidator();
const phoneV = new Validation.PhoneValidator();

console.log(emailV.isValid('test@gmail.com')); // true
console.log(phoneV.isValid('9876543210'));      // true
console.log(emailV.isValid('not-an-email'));    // false

// ── Nested namespaces ────────────────────────────────────
```

```
namespace App {
  export namespace Models {
    export interface User { id: number; name: string }
    export interface Product { id: number; name: string; price: number }
  }

  export namespace Services {
    export class UserService {
      getUser(id: number): App.Models.User {
        return { id, name: 'Rahul' };
      }
    }
  }
}

const service = new App.Services.UserService();
const user:     App.Models.User = service.getUser(1);
```

5.2 Namespace in Declaration Files

```
// ── Namespaces are very common in .d.ts files ──────────────
// This is where you'll encounter them most in modern TS

// Example: jQuery's type declarations (simplified)
declare namespace jQuery {
  function ajax(url: string, settings?: AjaxSettings): JQueryXHR;
  function get(url: string): JQueryXHR;

  interface AjaxSettings {
    method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
    data?:   object;
    success?: (data: unknown) => void;
  }

  interface JQueryXHR {
    done(handler: (data: unknown) => void): this;
    fail(handler: (error: Error) => void):  this;
  }
}

// ── Alias long namespace paths with import ─────────────────
// Instead of App.Services.Database.MySQL.query() everywhere:
```

```
import DbQuery = App.Services.Database.MySQL;

// Now use short alias:
DbQuery.query('SELECT * FROM users');

// ── Merging namespaces ──────────────────────────────────
// Namespaces with the same name MERGE (like interfaces)
namespace Utils {
  export function formatDate(d: Date): string { return d.toISOString(); }
}

namespace Utils {
  export function formatMoney(n: number): string { return `₹${n}`; }
}

// Both are available:
Utils.formatDate(new Date());   // ✅
Utils.formatMoney(1000);        // ✅
```

6. Dynamic Imports

Dynamic imports (`import()`) let you load modules **at runtime** — not at the top of the file. They return a Promise and are perfect for code splitting, lazy loading, and conditional module loading.

```
// ── Static import (top of file, always loaded) ────────────
import { HeavyComponent } from './HeavyComponent'; // Always loaded

// ── Dynamic import (loaded on demand) ─────────────────────
// Returns Promise — load only when needed

// Basic syntax:
const module = await import('./utils/math');
console.log(module.add(1, 2)); // 3

// ── Real-world: Lazy loading a heavy library ──────────────
async function exportToPdf(data: ReportData): Promise<Blob> {
  // Only load pdfmake when user clicks 'Export to PDF'
  // Not loaded on initial page load — saves bundle size
  const { createPdf } = await import('pdfmake/build/pdfmake');
  const { vfs } = await import('pdfmake/build/vfs_fonts');
  createPdf.vfs = vfs;
```

```
  return createPdf(buildDocDef(data)).getBlob(blob => blob);
}

// ── Real-world: Conditional module based on environment ───
async function getDb() {
  if (process.env.NODE_ENV === 'test') {
    const { MockDatabase } = await import('./mocks/MockDatabase');
    return new MockDatabase();
  }
  const { Database } = await import('./Database');
  return new Database(process.env.DATABASE_URL!);
}

// ── Dynamic import with type safety ──────────────────────
type ChartModule = typeof import('./charts/LineChart');

async function loadChart(): Promise<ChartModule> {
  return import('./charts/LineChart');
}

const chartMod = await loadChart();
new chartMod.LineChart(canvas, data); // ✅ fully typed

// ── React lazy loading pattern ───────────────────────────
// React.lazy uses dynamic import internally:
// const Dashboard = React.lazy(() => import('./Dashboard'));
// TypeScript understands this pattern automatically.
```

7. import type — Type-Only Imports

TypeScript 3.8+ introduced `import type`. It imports ONLY type information, which is **completely erased at compile time** — the import has zero impact on the runtime JavaScript bundle.

```
// ── import type vs regular import ────────────────────────

// Regular import — may include runtime code
import { User } from './models/User';

// import type — GUARANTEED to be erased, zero runtime cost
import type { User } from './models/User';
import type { Product, Order } from './models';
```

```
// ── When MUST you use import type? ──────────────────────
// When the import is a TYPE and is used only as a type annotation:

import type { RequestHandler } from 'express';

const myMiddleware: RequestHandler = (req, res, next) => {
  // req, res, next are typed ✅
  next();
};
// RequestHandler is only used as a type annotation — import type is correct.

// ── Inline 'type' keyword in import ────────────────────
// TypeScript 4.5+ allows mixed imports:
import { type User, type Product, createUser } from './models';
// User and Product are type-only, createUser is a value import

// ── Why import type matters ────────────────────────────
// 1. Performance: smaller bundles — type-only imports erased
// 2. Circular dependencies: type-only imports don't count
//    as runtime circular deps — safer for large projects
// 3. Clarity: makes it obvious this is a type, not a value

// ── 'isolatedModules' tsconfig flag ────────────────────
// Many build tools (Vite, Babel, SWC) process files independently.
// They can't know if an import is a type or value.
// With: "isolatedModules": true in tsconfig
// TypeScript ERRORS if you import a type without 'import type'.
// This ensures compatibility with fast bundlers.
```

8. Real-World Project Module Architecture

Let's see how a production-grade TypeScript project organises its modules — combining everything we've learned.

```
// ═══ Project structure ═══════════════════════════════
// src/
//  ├── types/              ← shared type definitions
//  │   ├── index.ts        ← barrel
//  │   ├── api.types.ts
//  │   └── domain.types.ts
```

```
//  ├── models/              ← data models
//  │   ├── index.ts
//  │   ├── User.ts
//  │   ├── Product.ts
//  │   └── Order.ts
//  ├── services/            ← business logic
//  │   ├── index.ts
//  │   ├── UserService.ts
//  │   ├── OrderService.ts
//  │   └── PaymentService.ts
//  ├── utils/               ← helpers
//  │   ├── index.ts
//  │   ├── date.ts
//  │   └── currency.ts
//  ├── config/              ← configuration
//  │   └── index.ts
//  └── index.ts             ← app entry point

// ═══ types/api.types.ts ═══════════════════════════════
export interface ApiResponse<T> {
  data:       T;
  success:    boolean;
  message:    string;
  statusCode: number;
}

export interface PaginatedResponse<T> extends ApiResponse<T[]> {
  total:      number;
  page:       number;
  totalPages: number;
}

// ═══ models/User.ts ═══════════════════════════════════
export interface User {
  readonly id:  number;
  name:         string;
  email:        string;
  role:         'admin' | 'user' | 'guest';
  createdAt:    Date;
}

export type CreateUserDto = Omit<User, 'id' | 'createdAt'>;
export type UpdateUserDto = Partial<CreateUserDto>;
```

```
// ═══ services/UserService.ts ═══════════════════════════
import type { User, CreateUserDto, UpdateUserDto } from '@models/User';
import type { ApiResponse, PaginatedResponse }     from '@/types';

export class UserService {
  async getAll(page = 1, limit = 10): Promise<PaginatedResponse<User>> {
    const res = await fetch(`/api/users?page=${page}&limit=${limit}`);
    return res.json();
  }

  async getById(id: number): Promise<ApiResponse<User>> {
    const res = await fetch(`/api/users/${id}`);
    return res.json();
  }

  async create(dto: CreateUserDto): Promise<ApiResponse<User>> {
    const res = await fetch('/api/users', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(dto),
    });
    return res.json();
  }

  async update(id: number, dto: UpdateUserDto): Promise<ApiResponse<User>> {
    const res = await fetch(`/api/users/${id}`, {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(dto),
    });
    return res.json();
  }
}

// ═══ services/index.ts (barrel) ═══════════════════════
export { UserService }    from './UserService';
export { OrderService }   from './OrderService';
export { PaymentService } from './PaymentService';

// ═══ App entry: src/index.ts ═══════════════════════════
import { UserService }  from '@/services';
import { OrderService } from '@/services';
```

```
import type { User }     from '@/models';

const userService  = new UserService();
const orderService = new OrderService();

const { data: users } = await userService.getAll();
users.forEach((u: User) => console.log(u.name));
```

9. Named vs Default Exports — The Great Debate

	Named Exports { x }	Default Export (x)
Syntax	export function add() {}	export default function add() {}
Import syntax	import { add } from './math'	import add from './math'
Multiple per file	✅ Many named exports per file	❌ Only ONE default per file
Rename on import	import { add as sum }	import anyNameYouWant from '...'
Refactoring	✅ IDE auto-renames all usages	❌ Import name not tied to export
Tree-shaking	✅ Bundlers can drop unused exports	⚠ Harder — default is one blob
Barrel files	✅ Works perfectly	✅ Works with 'export { default }'
TypeScript support	✅ Best in class	✅ Works well
Recommended for	Utilities, types, constants	React components, classes, pages

The Community Verdict

Most modern TypeScript style guides (Google, Airbnb, many framework docs) recommend NAMED EXPORTS for:

→ Utility functions, helpers, constants, types, interfaces
→ Anything that might be re-exported or tree-shaken

DEFAULT EXPORTS are still widely used for:

→ React components (convention in the React community)
→ Next.js pages (required by the framework)
→ Class services when there's one per file

Bottom line: be CONSISTENT within your project/team.

Named exports are generally safer and more refactor-friendly.

10. Interview Questions — Chapter 13

Q: What is the difference between a TypeScript module and a script?

Answer: A TypeScript file is a MODULE if it contains at least one import or export statement. All declarations in a module are scoped to that file — not visible outside unless explicitly exported.

A TypeScript file is a SCRIPT if it has NO import or export statements. All declarations in a script are in GLOBAL scope — accessible everywhere and prone to naming conflicts.

To force a file with no imports/exports to be a module: add 'export {}' at the bottom.

Modern TypeScript projects should use modules exclusively. Scripts are for legacy or specific global declaration files.

Q: What are barrel files? Why are they useful?

Answer: A barrel file is an index.ts that re-exports everything from a directory, providing a single import point for consumers.

Without barrel: import UserService from '../../../services/UserService'; // messy paths

With barrel: import { UserService } from '../services'; // clean

Benefits: 1) Clean import paths for consumers. 2) Refactoring a file only breaks the barrel — not every importer. 3) Acts as a public API for a module — what's in the barrel is 'published', internals stay private.

Be careful: accidentally importing from a barrel that imports the entire app can cause circular dependency issues in large projects.

Q: What is 'import type' and why should you use it?

Answer: 'import type' is a TypeScript-specific import that only imports TYPE information. It is completely erased at compile time — zero impact on the runtime JavaScript bundle.

import type { User } from './User'; — type-only, erased at build

import { User } from './User'; — may include runtime code

Use it for: 1) Better bundle performance — type-only imports never end up in JS output. 2) Avoiding circular dependency issues — type-only imports don't create runtime circular deps. 3) Compatibility with build tools (Vite, Babel, SWC) that use 'isolatedModules' mode.

With 'isolatedModules: true' in tsconfig, TypeScript ERRORS if you import a type without 'import type'.

Q: What is a .d.ts declaration file? When would you create one?

Answer: A .d.ts file contains ONLY type declarations — no runtime JavaScript. It describes the shape of JavaScript code so TypeScript can type-check it.

You create one when: 1) Using a plain JavaScript library that has no built-in types and no @types package. 2) Building a library you want to publish — .d.ts files ship with it so consumers get type safety. 3) Declaring global variables injected by Webpack/Vite. 4) Adding properties to third-party module interfaces (module augmentation).

Most popular libraries either ship their own .d.ts or have @types/library-name on npm.

Q: What is module augmentation? Give a real-world example.

Answer: Module augmentation lets you ADD new declarations to an existing module — extending third-party library types without modifying them.

Real-world example: Express.js adds req.user after authentication middleware, but Express's types don't know about it.

```
declare module 'express' {
  interface Request {
    user?: { id: number; role: string };
  }
}
```

Now TypeScript knows req.user exists and what shape it has. Without this, accessing req.user would be an error.

Other use cases: adding gtag to Window, adding custom methods to Express Response, adding Prisma model extensions.

Q: What is dynamic import? When would you use it?

Answer: Dynamic import — import('./path') — loads a module at RUNTIME instead of at startup. It returns a Promise.

Use when: 1) Code splitting — lazy-load heavy pages/components only when needed. 2) Conditional loading — load a mock DB in tests, real DB in production. 3) Optional features — only load PDF generation when user clicks 'Export'. 4) Performance — reduce initial bundle size by deferring large libraries.

Example: const { jsPDF } = await import('jspdf') — only loads when user exports.

TypeScript fully types dynamic imports — the returned type is the module's type with all its exports.

Q: What is the difference between 'module' and 'moduleResolution' in tsconfig?

Answer: These are two distinct settings:

'module': Controls what FORMAT the compiled JavaScript uses for modules.

'CommonJS' → require() / module.exports (classic Node.js)

'ESNext' → import/export (browsers, modern bundlers)

'NodeNext' → ES modules for modern Node.js

'moduleResolution': Controls how TypeScript FINDS files when resolving imports.

'Node' → Classic Node.js resolution (node_modules lookup)

'NodeNext' → Modern Node with package.json exports field support

'Bundler' → For Vite/webpack — doesn't require file extensions

They usually go together: module: NodeNext + moduleResolution: NodeNext for modern Node.

module: ESNext + moduleResolution: Bundler for browser/bundler projects.

Rapid Fire — Chapter 13

Question	One-Line Answer
How to make a file a module with no imports?	Add 'export {}' — empty export forces module scope
What does a .d.ts file contain?	Type declarations only — no runtime JavaScript code
Named vs default export — which is more	Named exports — tied to a specific identifier, IDE

refactor-safe?	can auto-rename
What is a barrel file?	An index.ts that re-exports from a folder for clean imports
What does import type guarantee?	The import is completely erased from compiled JavaScript output
What is module augmentation?	Adding new declarations to an existing module using 'declare module'
What does dynamic import() return?	A Promise — module loads at runtime, not startup
Where does TypeScript look for @types?	node_modules/@types/library-name
What is 'isolatedModules' in tsconfig?	Forces all type imports to use 'import type' — required for Vite/Babel
Can namespaces merge like interfaces?	Yes — same-named namespaces automatically merge their exports

Chapter 13 Key Takeaways

A file is a MODULE if it has at least one import/export — otherwise it's a global script.

Named exports: { x } — multiple per file, refactor-safe, tree-shakeable. Preferred.
Default export: x — one per file, any name on import. Used for components/classes.

Barrel files (index.ts): re-export everything from a folder for clean single-point imports.

import type: erased at compile time. Use for type-only imports. Required with isolatedModules.

.d.ts files: type declarations for JS code. Created manually or via tsc --declaration.
@types/library: community type definitions for popular JS libraries.

Module augmentation: 'declare module' extends existing library types (Express, Window).

Module resolution: how TypeScript finds files.
'module' = output format (CommonJS, ESNext, NodeNext).
'moduleResolution' = file discovery algorithm.
Path aliases in tsconfig 'paths' for clean @/ imports.

Dynamic import(): lazy-load modules at runtime → code splitting + conditional loading.

Namespaces: legacy code organisation, still found in .d.ts files. Prefer modules for new code.

End of Chapter 13

CHAPTER 14

Decorators

Class Decorators • Method Decorators • Property Decorators • Parameter Decorators • NestJS • Reflect Metadata

1. What Are Decorators?

Decorators are a powerful meta-programming feature that let you add extra behaviour to classes, methods, properties, and parameters — by attaching a function (the decorator) to them using the @ syntax. They are heavily used in frameworks like NestJS, Angular, TypeORM, and class-validator.

Real-World Analogy — Stickers on a Notebook

Imagine a plain notebook (class or method).

You add stickers to it — 'URGENT', 'READ ONLY', 'LOGGING ENABLED'.

The stickers don't change what the notebook IS,

but they ADD instructions about how it should be treated.

Decorators work exactly like this:

```
@Injectable()    → 'This class can be injected by a DI container'
@Get('/users')   → 'This method handles GET /users requests'
@IsEmail()       → 'This property must be a valid email'
@Log()           → 'Log every call to this method'
```

The decorator ANNOTATES something and (optionally) modifies its behaviour.

Enabling Decorators — tsconfig.json

Decorators are an experimental feature. You must enable them in tsconfig.json:

Stage 3 Decorators (modern, recommended — TypeScript 5.0+):

```
{ "compilerOptions": { "experimentalDecorators": false } }
```

(Stage 3 decorators work by default in TS 5+ — no flag needed)

Legacy / Experimental Decorators (older, needed for Angular, NestJS currently):

```
{ "compilerOptions": {
```

```
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

This chapter covers BOTH the legacy and modern (Stage 3) syntax,
with practical focus on the patterns used in production frameworks.

2. Class Decorators

A class decorator is a function that receives the **constructor** of the class it decorates. It can observe, modify, or replace the class definition entirely.

2.1 Basic Class Decorator

```
// ── A decorator is just a FUNCTION ──────────────────────
// Class decorator receives the constructor as its argument
function Sealed(constructor: Function) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
  console.log(`${constructor.name} is now sealed`);
}

// Apply with @ syntax:
@Sealed
class BankAccount {
  constructor(public owner: string, private balance: number) {}
  deposit(amount: number) { this.balance += amount; }
}

// When class is defined, Sealed() runs immediately:
// Output: 'BankAccount is now sealed'

// ── Class decorator with arguments (decorator factory) ──
// A decorator factory is a function that RETURNS a decorator
function Component(config: { selector: string; template: string }) {
  return function (constructor: Function) {
    // Attach metadata to the constructor
    (constructor as any).selector = config.selector;
    (constructor as any).template = config.template;
```

```
    console.log(`Component registered: ${config.selector}`);
  };
}

@Component({
  selector: 'app-header',
  template: '<header>{{title}}</header>',
})
class HeaderComponent {
  title = 'My App';
}

// Access metadata:
console.log((HeaderComponent as any).selector); // 'app-header'
```

2.2 Class Decorator That Modifies the Class

```
// ── Decorator that adds a property/method to the class ───
function Timestamped<T extends { new(...args: any[]): {} }>(Base: T) {
  return class extends Base {
    createdAt = new Date();
    updatedAt = new Date();

    touch() {
      this.updatedAt = new Date();
    }
  };
}

@Timestamped
class User {
  constructor(public name: string, public email: string) {}
}

const user = new User('Rahul', 'r@g.com');
console.log(user.createdAt); // Date — added by decorator!
(user as any).touch();       // method from decorator

// ── Singleton decorator ──────────────────────────────────
function Singleton<T extends { new(...args: any[]): {} }>(constructor: T) {
  let instance: InstanceType<T>;
```

```
  return new Proxy(constructor, {
    construct(target, args) {
      if (!instance) {
        instance = new target(...args);
      }
      return instance;
    },
  });
}

@Singleton
class DatabaseConnection {
  private static count = 0;
  readonly id: number;
  constructor(public url: string) {
    this.id = ++DatabaseConnection.count;
  }
}

const db1 = new DatabaseConnection('mongodb://localhost');
const db2 = new DatabaseConnection('mongodb://localhost');
console.log(db1 === db2);  // true — same instance!
console.log(db1.id);       // 1
console.log(db2.id);       // 1 — same object
```

2.3 NestJS-style Class Decorators

```
// ── How NestJS @Controller, @Injectable work ──────────────
// These store metadata on the class for the DI framework

// Simplified implementation of @Injectable:
function Injectable(): ClassDecorator {
  return (target) => {
    Reflect.defineMetadata('injectable', true, target);
  };
}

// Simplified implementation of @Controller:
function Controller(prefix: string = ''): ClassDecorator {
  return (target) => {
    Reflect.defineMetadata('controller', true, target);
    Reflect.defineMetadata('prefix', prefix, target);
  };
```

```
}

// Usage (just like real NestJS):
@Injectable()
class UserService {
  findAll() { return []; }
}

@Controller('/users')
class UserController {
  constructor(private userService: UserService) {}

  getAll() {
    return this.userService.findAll();
  }
}

// The DI framework reads this metadata to:
// 1. Know UserService can be injected
// 2. Know UserController handles /users prefix
// 3. Auto-inject UserService into UserController constructor
```

3. Method Decorators

A method decorator is applied to a method on a class. It receives three arguments: the **target** (prototype), the **method name**, and the **property descriptor**. It can observe, modify, or replace the method implementation.

```
// ── Method decorator signature ──────────────────────────
// target:     prototype of the class (or constructor for static methods)
// key:        name of the method
// descriptor: PropertyDescriptor — contains the method itself

// ── @Log — logging decorator ────────────────────────────
function Log(target: any, key: string, descriptor: PropertyDescriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = function (...args: any[]) {
    console.log(`[LOG] Calling ${key} with args:`, args);
    const result = originalMethod.apply(this, args);
    console.log(`[LOG] ${key} returned:`, result);
    return result;
```

```
  };

  return descriptor;
}

class OrderService {
  @Log
  createOrder(userId: number, items: string[]): string {
    return `ORDER-${userId}-${Date.now()}`;
  }
}

const svc = new OrderService();
svc.createOrder(42, ['iPhone', 'AirPods']);
// [LOG] Calling createOrder with args: [42, ['iPhone', 'AirPods']]
// [LOG] createOrder returned: ORDER-42-1704000000000

// ── @Debounce — factory decorator with config ──────────────
function Debounce(delayMs: number) {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;
    let timer: ReturnType<typeof setTimeout>;

    descriptor.value = function (...args: any[]) {
      clearTimeout(timer);
      timer = setTimeout(() => original.apply(this, args), delayMs);
    };

    return descriptor;
  };
}

class SearchComponent {
  @Debounce(300)  // Only fires 300ms after user stops typing
  search(query: string): void {
    console.log(`Searching for: ${query}`);
    // API call here
  }
}

// ── @Memoize — cache method results ──────────────────────
function Memoize(target: any, key: string, descriptor: PropertyDescriptor) {
  const original = descriptor.value;
```

```
  const cache = new Map<string, unknown>();

  descriptor.value = function (...args: any[]) {
    const cacheKey = JSON.stringify(args);
    if (cache.has(cacheKey)) {
      console.log(`[CACHE HIT] ${key}(${cacheKey})`);
      return cache.get(cacheKey);
    }
    const result = original.apply(this, args);
    cache.set(cacheKey, result);
    return result;
  };

  return descriptor;
}

class PriceCalculator {
  @Memoize
  calculateTax(price: number, rate: number): number {
    console.log('Computing tax...');  // Only runs once per unique args
    return price * rate;
  }
}

const calc = new PriceCalculator();
calc.calculateTax(1000, 0.18);  // Computing tax... → 180
calc.calculateTax(1000, 0.18);  // [CACHE HIT] → 180 (no recompute)
calc.calculateTax(2000, 0.18);  // Computing tax... → 360 (new args)
```

3.2 NestJS-style HTTP Method Decorators

```
// ── How @Get, @Post, @Put work in NestJS ─────────────────

function Get(path: string = '') {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    Reflect.defineMetadata('method', 'GET', target, key);
    Reflect.defineMetadata('path',   path,  target, key);
    return descriptor;
  };
}

function Post(path: string = '') {
```

```
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    Reflect.defineMetadata('method', 'POST', target, key);
    Reflect.defineMetadata('path',   path,   target, key);
    return descriptor;
  };
}

function Delete(path: string = '') {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    Reflect.defineMetadata('method', 'DELETE', target, key);
    Reflect.defineMetadata('path',   path,     target, key);
    return descriptor;
  };
}

// Usage — just like NestJS:
@Controller('/users')
class UserController {
  @Get()                  // GET /users
  findAll() { return []; }

  @Get('/:id')            // GET /users/:id
  findOne() { return {}; }

  @Post()                 // POST /users
  create() { return {}; }

  @Delete('/:id')         // DELETE /users/:id
  remove() { return true; }
}

// ── @Authorize — guard decorator ──────────────────────────
function Authorize(...roles: string[]) {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;

    descriptor.value = function (this: any, ...args: any[]) {
      const user = this.currentUser; // injected by context
      if (!user || !roles.includes(user.role)) {
        throw new Error(`Access denied. Requires: ${roles.join(' or ')}`);
      }
      return original.apply(this, args);
    };
```

```
    return descriptor;
  };
}

class AdminController {
  @Authorize('admin')
  deleteUser(id: number) {
    console.log(`Deleting user ${id}`);
  }

  @Authorize('admin', 'moderator')
  banUser(id: number) {
    console.log(`Banning user ${id}`);
  }
}
```

4. Property Decorators

A property decorator is applied to a class **property**. It receives the target (prototype) and the property name. Note: property descriptors are **not passed** to property decorators — they are mainly used for **attaching metadata** or setting up accessor logic via Reflect.

```
// ── Property decorator signature ──────────────────────────
// target: prototype of the class
// key:    name of the property
// (no descriptor — use Reflect.metadata to store info)

// ── @Required — mark a property as required ──────────────
function Required(target: object, propertyKey: string) {
  const requiredProps: string[] =
    Reflect.getMetadata('required', target) || [];
  requiredProps.push(propertyKey);
  Reflect.defineMetadata('required', requiredProps, target);
}

// ── @Min / @Max — numeric validation ──────────────────────
function Min(minimum: number) {
  return function (target: object, propertyKey: string) {
    Reflect.defineMetadata(`min:${propertyKey}`, minimum, target);
  };
}
```

```
function Max(maximum: number) {
  return function (target: object, propertyKey: string) {
    Reflect.defineMetadata(`max:${propertyKey}`, maximum, target);
  };
}

// ── @IsEmail — format validation ──────────────────────────
function IsEmail(target: object, propertyKey: string) {
  Reflect.defineMetadata(`validate:${propertyKey}`, 'email', target);
}

// ── Apply on a DTO class ──────────────────────────────────
class CreateUserDto {
  @Required
  @IsEmail
  email: string = '';

  @Required
  name: string = '';

  @Min(18)
  @Max(120)
  age: number = 0;
}

// ── Validate function reads the metadata ──────────────────
function validate(obj: object): string[] {
  const errors: string[] = [];
  const proto = Object.getPrototypeOf(obj);
  const required: string[] = Reflect.getMetadata('required', proto) || [];

  for (const key of required) {
    const val = (obj as any)[key];
    if (!val) errors.push(`${key} is required`);
  }

  return errors;
}

const dto = new CreateUserDto();
dto.name = 'Rahul';
```

```
// email is empty

const errors = validate(dto);
console.log(errors); // ['email is required']
```

4.2 class-validator Style Decorators

```
// ── class-validator is a real npm library using these patterns ─
// npm install class-validator class-transformer

import { IsString, IsEmail, IsInt, Min, Max,
         IsOptional, Length, validate } from 'class-validator';

class RegisterDto {
  @IsString()
  @Length(2, 50, { message: 'Name must be 2-50 characters' })
  name: string;

  @IsEmail({}, { message: 'Invalid email format' })
  email: string;

  @IsString()
  @Length(8, 100, { message: 'Password must be at least 8 characters' })
  password: string;

  @IsInt()
  @Min(18, { message: 'Must be at least 18 years old' })
  @Max(120)
  age: number;

  @IsOptional()
  @IsString()
  phone?: string;
}

// Validate automatically:
const dto = new RegisterDto();
dto.name  = 'R';          // too short
dto.email = 'not-email';  // invalid
dto.age   = 15;           // too young

const errors = await validate(dto);
```

```
errors.forEach(e => console.log(Object.values(e.constraints ?? {})));
// ['Name must be 2-50 characters']
// ['Invalid email format']
// ['Must be at least 18 years old']
```

5. Parameter Decorators

Parameter decorators are applied to **function parameters**. They receive the target, the method name, and the **parameter index**. They are mainly used for metadata — most commonly in frameworks like NestJS for **dependency injection** and route parameter extraction.

```
// ── Parameter decorator signature ──────────────────────
// target:          prototype of the class
// methodName:      name of the method
// parameterIndex:  position of the parameter (0, 1, 2...)

// ── @Inject — DI parameter decorator ────────────────────
function Inject(token: string) {
  return function (target: any, methodName: string | undefined, paramIndex:
number) {
    const existingInjections =
      Reflect.getMetadata('inject', target, methodName ?? '') || [];
    existingInjections[paramIndex] = token;
    Reflect.defineMetadata('inject', existingInjections, target, methodName ??
'');
  };
}

class UserController {
  constructor(
    @Inject('UserService')  private userService: any,
    @Inject('LogService')   private logService:  any,
  ) {}
}

// NestJS reads these at startup to autowire dependencies.

// ── NestJS-style @Body, @Param, @Query ──────────────────
// These are parameter decorators that extract request data

function Body(key?: string) {
  return function (target: any, method: string, index: number) {
```

```
    const params = Reflect.getMetadata('params', target, method) || [];
    params[index] = { type: 'body', key };
    Reflect.defineMetadata('params', params, target, method);
  };
}

function Param(key: string) {
  return function (target: any, method: string, index: number) {
    const params = Reflect.getMetadata('params', target, method) || [];
    params[index] = { type: 'param', key };
    Reflect.defineMetadata('params', params, target, method);
  };
}

function Query(key: string) {
  return function (target: any, method: string, index: number) {
    const params = Reflect.getMetadata('params', target, method) || [];
    params[index] = { type: 'query', key };
    Reflect.defineMetadata('params', params, target, method);
  };
}

// Usage — just like real NestJS:
@Controller('/orders')
class OrderController {
  @Get('/:id')
  getOrder(@Param('id') id: string) {
    return { orderId: id };
  }

  @Post()
  createOrder(@Body() createDto: CreateOrderDto) {
    return { created: createDto };
  }

  @Get()
  listOrders(
    @Query('page') page: string,
    @Query('limit') limit: string,
  ) {
    return { page, limit };
  }
}
```

6. Accessor Decorators

Accessor decorators are applied to a `get` or `set` accessor. They receive the same arguments as method decorators — target, key, and descriptor — and can modify the getter/setter logic.

```
// ── Accessor decorator example ──────────────────────────
function Configurable(value: boolean) {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    descriptor.configurable = value;
    return descriptor;
  };
}

function Enumerable(value: boolean) {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    descriptor.enumerable = value;
    return descriptor;
  };
}

class Employee {
  constructor(
    private _firstName: string,
    private _lastName: string,
    private _salary: number
  ) {}

  @Configurable(false)  // Cannot be deleted from the object
  @Enumerable(true)     // Shows in for...in loops
  get fullName(): string {
    return `${this._firstName} ${this._lastName}`;
  }

  @Enumerable(false)    // Hidden in for...in loops (salary is private)
  get salary(): number {
    return this._salary;
  }
}

const emp = new Employee('Rahul', 'Sharma', 90000);
console.log(emp.fullName); // 'Rahul Sharma'
```

```
// ── @ReadOnly accessor decorator ──────────────────────────
function ReadOnly(target: any, key: string, descriptor: PropertyDescriptor) {
  descriptor.writable = false;
  return descriptor;
}
```

7. Decorator Execution Order

When multiple decorators are applied to the same thing, understanding their execution order is critical for avoiding bugs.

Execution Order Rules

1. PARAMETER decorators run first (innermost first → outermost)
2. METHOD / ACCESSOR / PROPERTY decorators (bottom-up per class)
3. CLASS decorators run last (bottom-up if multiple on same class)

For stacked decorators on ONE target:

```
@A       → A evaluates first (outer)
@B       → B evaluates second (inner)
method()  → B's function runs first, then A's function
```

Think of it like nested function calls: A(B(method))

Evaluation (factory calls): A, then B

Execution (returned functions): B runs first, then A

```
// ── Demonstrating decorator execution order ──────────────
function First() {
  console.log('First(): factory evaluated');
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    console.log('First(): decorator called');
    return descriptor;
  };
}

function Second() {
  console.log('Second(): factory evaluated');
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    console.log('Second(): decorator called');
```

```
    return descriptor;
  };
}

class Demo {
  @First()   // outer
  @Second()  // inner
  greet() { return 'Hello'; }
}

// Output:
// First():  factory evaluated   ← factory evaluated top-down
// Second(): factory evaluated
// Second(): decorator called    ← decorators applied bottom-up
// First():  decorator called

// ── Real-world example: order matters ──────────────────
class UserService {
  @Log         // runs AFTER Auth (outer wraps inner)
  @Authorize('admin')  // runs first (inner)
  deleteUser(id: number): void {
    console.log(`Deleting user ${id}`);
  }
}

// Call order: Authorize check → (if ok) Log → original method
// This is the correct order: check auth BEFORE logging
```

8. Reflect Metadata

Reflect Metadata is a library (`reflect-metadata`) that decorators use to attach and read metadata on classes, methods, and properties. It's the backbone of frameworks like NestJS, TypeORM, and Angular.

```
// ── Setup ──────────────────────────────────────────────
// npm install reflect-metadata
// import 'reflect-metadata'; // at the top of entry file
// tsconfig: { "emitDecoratorMetadata": true }

// ── Core Reflect.metadata API ──────────────────────────

// Set metadata on a class/method/property:
```

```
Reflect.defineMetadata(key, value, target);
Reflect.defineMetadata(key, value, target, propertyKey);

// Get metadata:
Reflect.getMetadata(key, target);
Reflect.getMetadata(key, target, propertyKey);

// Check if metadata exists:
Reflect.hasMetadata(key, target);

// Delete metadata:
Reflect.deleteMetadata(key, target);

// ── Real-world: Route registration system ────────────────
import 'reflect-metadata';

const ROUTES_METADATA = 'routes';

function Get(path: string) {
  return (target: any, key: string, descriptor: PropertyDescriptor) => {
    const routes = Reflect.getMetadata(ROUTES_METADATA, target.constructor) ||
[];
    routes.push({ method: 'GET', path, handlerKey: key });
    Reflect.defineMetadata(ROUTES_METADATA, routes, target.constructor);
  };
}

function Post(path: string) {
  return (target: any, key: string, descriptor: PropertyDescriptor) => {
    const routes = Reflect.getMetadata(ROUTES_METADATA, target.constructor) ||
[];
    routes.push({ method: 'POST', path, handlerKey: key });
    Reflect.defineMetadata(ROUTES_METADATA, routes, target.constructor);
  };
}

@Controller('/products')
class ProductController {
  @Get('/')         getAll()    { return []; }
  @Get('/:id')      getById()   { return {}; }
  @Post('/')        create()    { return {}; }
}
```

```
// A micro-framework reads all routes at startup:
function registerRoutes(ControllerClass: any, app: any) {
  const prefix = Reflect.getMetadata('prefix', ControllerClass) ?? '';
  const routes = Reflect.getMetadata(ROUTES_METADATA, ControllerClass) ?? [];
  const instance = new ControllerClass();

  routes.forEach(({ method, path, handlerKey }: any) => {
    const fullPath = prefix + path;
    console.log(`Registering: ${method} ${fullPath}`);
    app[method.toLowerCase()](fullPath, instance[handlerKey].bind(instance));
  });
}

// registerRoutes(ProductController, expressApp);
// Registers: GET /products/, GET /products/:id, POST /products/
```

8.2 emitDecoratorMetadata — Type Information

```
// ── emitDecoratorMetadata emits design:type, design:paramtypes ─
// TypeScript automatically stores constructor param types as metadata!

// tsconfig: { "emitDecoratorMetadata": true }

@Injectable()
class EmailService {
  send(to: string, body: string): void {
    console.log(`Sending to ${to}`);
  }
}

@Injectable()
class UserService {
  constructor(private emailService: EmailService) {}
  // TypeScript emits: design:paramtypes = [EmailService]
  // The DI container reads this to know what to inject!
}

// The DI framework does this automatically:
function resolveService(ServiceClass: any, registry: Map<any, any>): any {
  const paramTypes: any[] = Reflect.getMetadata('design:paramtypes',
ServiceClass) || [];
  // paramTypes = [EmailService] — TypeScript put this there!
```

```
  const dependencies = paramTypes.map(dep => {
    if (!registry.has(dep)) throw new Error(`Cannot resolve: ${dep.name}`);
    return registry.get(dep);
  });

  return new ServiceClass(...dependencies);
}

// Usage:
const registry = new Map();
registry.set(EmailService, new EmailService());

const userService = resolveService(UserService, registry);
// EmailService is automatically injected — no manual wiring!
```

9. TypeORM-Style Property Decorators

> 🗄 **Real-World Scenario: TypeORM — Database ORM Using Decorators**
> TypeORM is one of the most popular ORMs for Node.js. It uses decorators extensively to map TypeScript classes to database tables and their properties to columns. Understanding how these decorators work helps you use TypeORM confidently and debug schema issues.

```
// ── TypeORM entity decorators (real usage) ──────────────
import {
  Entity, Column, PrimaryGeneratedColumn,
  CreateDateColumn, UpdateDateColumn,
  ManyToOne, OneToMany, JoinColumn
} from 'typeorm';

@Entity('users')          // Maps to 'users' table in DB
class User {
  @PrimaryGeneratedColumn()  // Auto-increment ID
  id: number;

  @Column({ type: 'varchar', length: 100 })
  name: string;

  @Column({ type: 'varchar', unique: true })
  email: string;

  @Column({ type: 'varchar', select: false }) // Never selected by default
```

```
  passwordHash: string;

  @Column({ type: 'enum', enum: ['admin', 'user', 'guest'], default: 'user' })
  role: 'admin' | 'user' | 'guest';

  @Column({ default: true })
  isActive: boolean;

  @CreateDateColumn()   // Automatically set on insert
  createdAt: Date;

  @UpdateDateColumn()   // Automatically set on update
  updatedAt: Date;

  @OneToMany(() => Order, order => order.user)
  orders: Order[];
}

@Entity('orders')
class Order {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ type: 'decimal', precision: 10, scale: 2 })
  total: number;

  @Column({ type: 'enum', enum: ['pending', 'shipped', 'delivered'] })
  status: string;

  @ManyToOne(() => User, user => user.orders)
  @JoinColumn({ name: 'user_id' })
  user: User;

  @CreateDateColumn()
  createdAt: Date;
}

// TypeORM reads these decorators to:
// 1. Create/migrate the database schema
// 2. Map query results to typed class instances
// 3. Handle relationships (JOINs) automatically
```

10. Stage 3 Decorators (TypeScript 5.0+)

TypeScript 5.0 introduced support for Stage 3 (TC39 standard) decorators. They have a different syntax and API compared to the legacy experimental decorators, but offer more power and standardisation.

```
// ── Stage 3 decorator syntax (TS 5.0+) ──────────────────
// No experimentalDecorators flag needed
// Different API — receives a 'context' object

// Class decorator — Stage 3
function logged<T extends { new(...args: any[]): {} }>(Base: T, context:
ClassDecoratorContext) {
  context.addInitializer(function(this: any) {
    console.log(`${Base.name} instance created`);
  });
  return Base;
}

// Method decorator — Stage 3
function logMethod(method: Function, context: ClassMethodDecoratorContext) {
  const methodName = String(context.name);
  return function(this: any, ...args: any[]) {
    console.log(`Calling ${methodName} with:`, args);
    const result = method.apply(this, args);
    console.log(`${methodName} returned:`, result);
    return result;
  };
}

// Accessor decorator — Stage 3
function bounded(_: any, context: ClassAccessorDecoratorContext) {
  return {
    get(this: any) {
      return _.get.call(this).bind(this);
    }
  };
}

// Auto-accessor (Stage 3 feature)
class Person {
  accessor name: string; // Creates getter/setter automatically
```

```
  @logMethod
  greet() {
    return `Hello, ${this.name}!`;
  }
}

// ── Comparison: Legacy vs Stage 3 ──────────────────────
// Legacy (experimentalDecorators: true):
//   function Log(target, key, descriptor) { ... }
//   @Log method() {}

// Stage 3 (TS 5.0+, no flag):
//   function log(fn, context: ClassMethodDecoratorContext) { ... }
//   @log method() {}

// Key difference: Stage 3 gets a 'context' object with:
//   context.name    — name of the decorated element
//   context.kind    — 'class'|'method'|'getter'|'setter'|'field'|'accessor'
//   context.static  — whether it's static
//   context.private — whether it's private
//   context.addInitializer() — run code after class is defined
```

11. Complete Real-World Example — Mini NestJS

Let's build a mini framework demonstrating all decorator types working together — controllers, routes, DI, and validation.

```
// ═══ types.ts ═══════════════════════════════════════
import 'reflect-metadata';

// ═══ decorators.ts ══════════════════════════════════

// Class: @Injectable, @Controller
export function Injectable(): ClassDecorator {
  return target => Reflect.defineMetadata('injectable', true, target);
}

export function Controller(prefix = ''): ClassDecorator {
  return target => {
    Reflect.defineMetadata('prefix', prefix, target);
    Reflect.defineMetadata('controller', true, target);
```

```
  };
}

// Method: @Get, @Post, @Put, @Delete
function httpMethod(method: string, path: string) {
  return (target: any, key: string, descriptor: PropertyDescriptor) => {
    const routes = Reflect.getMetadata('routes', target.constructor) || [];
    routes.push({ method, path, handler: key });
    Reflect.defineMetadata('routes', routes, target.constructor);
    return descriptor;
  };
}

export const Get    = (p = '') => httpMethod('GET', p);
export const Post   = (p = '') => httpMethod('POST', p);
export const Put    = (p = '') => httpMethod('PUT', p);
export const Delete = (p = '') => httpMethod('DELETE', p);

// Method: @UseGuards
export function UseGuards(...roles: string[]) {
  return (target: any, key: string, descriptor: PropertyDescriptor) => {
    Reflect.defineMetadata('guards', roles, target, key);
    const original = descriptor.value;
    descriptor.value = function (this: any, req: any, ...args: any[]) {
      const user = req?.user;
      if (!user || !roles.includes(user.role)) {
        throw new Error(`403 Forbidden — requires: ${roles.join('/')}`);
      }
      return original.apply(this, [req, ...args]);
    };
    return descriptor;
  };
}

// Property: @IsRequired, @IsEmail
export function IsRequired(target: any, key: string) {
  const fields = Reflect.getMetadata('required_fields', target) || [];
  Reflect.defineMetadata('required_fields', [...fields, key], target);
}

export function IsEmail(target: any, key: string) {
  const fields = Reflect.getMetadata('email_fields', target) || [];
  Reflect.defineMetadata('email_fields', [...fields, key], target);
```

```
}

// ═══ user.dto.ts ═══════════════════════════════════
export class CreateUserDto {
  @IsRequired
  @IsEmail
  email: string = '';

  @IsRequired
  name: string = '';

  password: string = '';
}

// ═══ user.service.ts ═══════════════════════════
@Injectable()
export class UserService {
  private users: any[] = [];

  findAll()          { return this.users; }
  findById(id: number)  { return this.users.find(u => u.id === id); }
  create(dto: CreateUserDto) {
    const user = { ...dto, id: this.users.length + 1 };
    this.users.push(user);
    return user;
  }
}

// ═══ user.controller.ts ═══════════════════════
@Controller('/users')
export class UserController {
  constructor(private userService: UserService) {}

  @Get('/')          getAll()      { return this.userService.findAll(); }
  @Get('/:id')       getById(id: number) { return
this.userService.findById(id); }
  @Post('/')         create(dto: CreateUserDto) { return
this.userService.create(dto); }

  @Delete('/:id')
  @UseGuards('admin')
  delete(req: any, id: number) {
    return { deleted: id };
```

```
  }
}
```

12. Interview Questions — Chapter 14

Q: What is a decorator in TypeScript? What types exist?

Answer: A decorator is a function applied to a class, method, property, or parameter using the @ syntax. It runs at DEFINITION TIME (when the class is defined), not at call time.

Five types of decorators:

1. Class decorator — @Sealed, @Component, @Controller
2. Method decorator — @Log, @Debounce, @Get, @Post
3. Property decorator — @Required, @IsEmail, @Column
4. Parameter decorator — @Body, @Param, @Inject
5. Accessor decorator — @Enumerable, @ReadOnly (on get/set)

You must enable them in tsconfig: 'experimentalDecorators: true' for legacy decorators (used by NestJS, Angular). TypeScript 5+ supports Stage 3 decorators without a flag.

Q: What is a decorator factory? How does it differ from a plain decorator?

Answer: A plain decorator is a function applied directly: @Log

A decorator factory is a function that RETURNS a decorator, allowing you to pass configuration: @Log('info')

Plain decorator: function Log(target, key, descriptor) { ... }

Decorator factory: function Log(level: string) { return function(target, key, descriptor) { ... }; }

Use decorator factory when you need to customise behaviour per-use.

Examples: @Min(18), @Debounce(300), @Authorize('admin'), @Column({ unique: true })

The factory is called immediately when the class is evaluated (at definition time), and the returned inner function is the actual decorator that runs.

Q: What is the execution order when multiple decorators are stacked?

Answer: When multiple decorators are stacked on the same element:

1. Factory functions are evaluated TOP to BOTTOM (outer to inner).
2. The decorator functions returned by factories run BOTTOM to TOP (inner to outer).

```
@A  ← factory A evaluates first
@B  ← factory B evaluates second
method()
```

Execution: B's decorator runs first, A's decorator wraps around B.

For class members overall: parameter → method/property/accessor → class decorators.

Practical impact: @UseGuards should be BELOW @Log so auth runs before logging.

Q: What is Reflect Metadata and why is it needed for decorators?

Answer: Reflect Metadata is a library that provides a standard API for attaching metadata (key-value pairs) to classes, methods, and properties at design time.

Decorators alone can't easily store and retrieve information across the class. Reflect.defineMetadata() stores the data, Reflect.getMetadata() retrieves it.

With 'emitDecoratorMetadata: true' in tsconfig, TypeScript automatically stores:

design:type → type of a property

design:paramtypes → constructor/method parameter types

design:returntype → return type of a method

NestJS reads 'design:paramtypes' to automatically inject dependencies without you manually specifying what to inject — just declare constructor(private svc: MyService) and the framework does the rest.

Q: How does NestJS use decorators for dependency injection?

Answer: NestJS uses a combination of decorators and Reflect Metadata to implement dependency injection:

1. @Injectable() marks a class as injectable — stores metadata flag.
2. TypeScript's emitDecoratorMetadata stores constructor parameter types.
3. NestJS reads 'design:paramtypes' to discover what dependencies each service needs.
4. The DI container resolves each dependency recursively and injects them.

When you write: constructor(private userService: UserService)

NestJS sees: design:paramtypes = [UserService]

It then resolves UserService and passes it automatically.

This is why you never call 'new UserService()' in NestJS — the framework creates and injects it.

Q: What is the difference between legacy experimental decorators and Stage 3 decorators?

Answer: Two different decorator implementations exist:

Legacy (experimentalDecorators: true):

Function signature: (target, key, descriptor) => {}

Status: Non-standard, TypeScript-specific, used by Angular, NestJS, TypeORM

Requires: experimentalDecorators + emitDecoratorMetadata in tsconfig

Stage 3 (TypeScript 5.0+):

Function signature: (value, context: ClassMethodDecoratorContext) => {}

Status: TC39 standard — will be in JavaScript itself

Requires: No tsconfig flags — works by default in TS 5+

New: 'context' object with name, kind, static, private, addInitializer()

For now: use legacy decorators for NestJS/Angular projects (they haven't migrated yet). Use Stage 3 for new personal projects.

Rapid Fire — Chapter 14

Question	One-Line Answer
tsconfig flag for legacy decorators?	'experimentalDecorators: true' + 'emitDecoratorMetadata: true'
What does a class decorator receive?	The constructor function of the decorated class
What does a method decorator receive?	target (prototype), key (method name), PropertyDescriptor
What does a property decorator receive?	target (prototype) and key (property name) — no descriptor
What does a parameter decorator	target, method name, and the parameter index (0, 1,

receive?	2...)
What is a decorator factory?	A function that returns a decorator — used when decorators need config
Decorator execution order (stacked)?	Factories: top-to-bottom; Decorators: bottom-to-top (inner first)
What does emitDecoratorMetadata emit?	design:paramtypes — constructor param types for DI
Which real libraries use decorators heavily?	NestJS, Angular, TypeORM, class-validator, inversify
Stage 3 decorators differ how?	New context object API — TC39 standard, no tsconfig flag needed in TS 5+

Chapter 14 Key Takeaways

Decorators = functions that annotate and optionally modify classes, methods, properties, params.

Applied with @ syntax. Run at DEFINITION time, not call time.

Five types: @Class, @Method, @Property, @Parameter, @Accessor.

Decorator factory: function returning a decorator — used when config is needed.

Execution order: Factories evaluate top-down. Decorators apply bottom-up.

Reflect Metadata: library to store/retrieve metadata on class members.

emitDecoratorMetadata: TypeScript stores constructor param types → powers DI.

Class decorator: receives constructor. Can observe, modify, or replace the class.

Method decorator: receives target, key, descriptor. Can wrap/replace the method.

Property decorator: receives target and key. Main use: attach metadata.

Parameter decorator: receives target, method name, param index. Used for DI markers.

Real-world usage:

NestJS: @Controller, @Injectable, @Get, @Post, @Body, @Param

TypeORM: @Entity, @Column, @PrimaryGeneratedColumn, @ManyToOne

class-validator: @IsEmail, @IsString, @Min, @Max, @Length

Legacy vs Stage 3: Legacy for NestJS/Angular (current). Stage 3 = TC39 standard in TS 5+.

End of Chapter 14

CHAPTER 15

React + TypeScript

Component Props • useState • useEffect • Custom Hooks • Context • Events • Refs • Generics in React

1. Why TypeScript with React?

React and TypeScript are one of the most popular combinations in modern frontend development. TypeScript catches prop mistakes at compile time, gives you autocomplete for component APIs, and makes large React codebases dramatically easier to refactor and maintain.

What TypeScript Adds to React

Without TypeScript (plain JS React):

→ Pass wrong prop type? Runtime error or silent bug.

→ Forget a required prop? No warning until it crashes.

→ Rename a prop? Hunt through every file manually.

→ Use a hook wrongly? Silent undefined errors.

With TypeScript + React:

→ Wrong prop type? ✕ RED underline at compile time.

→ Missing required prop? ✕ Error before you even run the app.

→ Rename a prop? ✓ IDE renames every usage instantly.

→ Wrong hook usage? ✓ TypeScript catches it immediately.

TypeScript turns React from a 'guess and check' experience into a 'guided and safe' development experience.

Setup — Create React App or Vite with TypeScript

Create React App:

```
npx create-react-app my-app --template typescript
```

Vite (recommended — much faster):

```
npm create vite@latest my-app -- --template react-ts
```

Adding TypeScript to existing React project:

npm install --save-dev typescript @types/react @types/react-dom

Rename .jsx files to .tsx

Add tsconfig.json with jsx: 'react-jsx'

2. Typing Component Props

2.1 Function Component with Props Interface

```
// ── Defining and typing props ──────────────────────────────
import React from 'react';

// Step 1: Define an interface for props
interface ButtonProps {
  label:     string;                        // required string
  onClick:   () => void;                    // required callback
  variant?:  'primary' | 'secondary' | 'danger'; // optional literal
  disabled?: boolean;                       // optional boolean
  size?:     'sm' | 'md' | 'lg';            // optional literal
  icon?:     React.ReactNode;               // optional React content
}

// Step 2: Use the interface on the component
function Button({
  label,
  onClick,
  variant  = 'primary',
  disabled = false,
  size     = 'md',
  icon,
}: ButtonProps) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn btn-${variant} btn-${size}`}
    >
      {icon && <span className='icon'>{icon}</span>}
      {label}
    </button>
  );
```

```
}

// Usage — TypeScript validates all props:
<Button label='Submit' onClick={() => console.log('clicked')} />  // ✅
<Button label='Delete' onClick={handleDelete} variant='danger' /> // ✅
<Button onClick={() => {}} />     // ❌ ERROR: label is required
<Button label='Go' variant='x' onClick={fn} /> // ❌ ERROR: 'x' not valid
variant

// ── React.FC type (older approach — still valid) ──────────
const Button: React.FC<ButtonProps> = ({ label, onClick }) => {
  return <button onClick={onClick}>{label}</button>;
};
// Note: React.FC automatically adds 'children' prop (which is now optional).
// Many modern teams prefer the plain function approach above.
```

2.2 Children Prop Typing

```
// ── React.ReactNode — most flexible, accepts anything ────
interface CardProps {
  title:    string;
  children: React.ReactNode;  // JSX, string, number, null, array...
}

function Card({ title, children }: CardProps) {
  return (
    <div className='card'>
      <h2>{title}</h2>
      <div className='body'>{children}</div>
    </div>
  );
}

<Card title='My Card'>
  <p>Any JSX content here</p>  {/* ✅ ReactNode accepts anything */}
</Card>

// ── React.ReactElement — only JSX, not strings/numbers ───
interface LayoutProps {
  sidebar:  React.ReactElement;  // must be JSX element
  children: React.ReactNode;
}
```

```
// ── PropsWithChildren helper ──────────────────────────────
import { PropsWithChildren } from 'react';

// Instead of manually adding children:
// interface PanelProps { title: string; children: React.ReactNode }

// Use PropsWithChildren shorthand:
type PanelProps = PropsWithChildren<{ title: string }>;
// Expands to: { title: string; children?: React.ReactNode }

function Panel({ title, children }: PanelProps) {
  return <div><h3>{title}</h3>{children}</div>;
}

// ── Render prop pattern ───────────────────────────────────
interface ListProps<T> {
  items:      T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map((item, i) => (
        <li key={keyExtractor(item)}>{renderItem(item, i)}</li>
      ))}
    </ul>
  );
}

// Usage — TypeScript infers T from items:
<List
  items={users}
  keyExtractor={u => u.id}
  renderItem={u => <span>{u.name}</span>}
/>
```

2.3 Extending HTML Element Props

```
// ── Extend native HTML element props ────────────────────
```

```
// Pass through ALL button props + your custom ones
interface IconButtonProps extends
React.ButtonHTMLAttributes<HTMLButtonElement> {
  icon:   React.ReactNode;
  label?: string;
}

function IconButton({ icon, label, ...buttonProps }: IconButtonProps) {
  return (
    <button {...buttonProps}>  {/* spread ALL native button props */}
      {icon}
      {label && <span>{label}</span>}
    </button>
  );
}

// Now accepts ALL native button props + your custom ones:
<IconButton
  icon={<SearchIcon />}
  label='Search'
  onClick={handleSearch}      // ✅ native button prop
  disabled={isLoading}        // ✅ native button prop
  aria-label='Search button'  // ✅ ARIA attribute
  type='submit'               // ✅ native button prop
/>

// ── Extend input props ──────────────────────────────────
interface TextFieldProps extends React.InputHTMLAttributes<HTMLInputElement> {
  label:        string;
  error?:       string;
  helperText?:  string;
}

function TextField({ label, error, helperText, ...inputProps }:
TextFieldProps) {
  return (
    <div className={`field ${error ? 'error' : ''}`}>
      <label>{label}</label>
      <input {...inputProps} />
      {error && <span className='error'>{error}</span>}
      {helperText && <span className='helper'>{helperText}</span>}
    </div>
  );
```

```
}

<TextField
  label='Email'
  type='email'                // ✅ HTMLInputElement prop
  placeholder='you@email.com' // ✅ HTMLInputElement prop
  value={email}               // ✅ controlled input
  onChange={e => setEmail(e.target.value)}
  error={emailError}
/>
```

3. Typing useState

```
// ── TypeScript infers type from initial value ──────────────
const [count, setCount]   = useState(0);       // number
const [name, setName]     = useState('');      // string
const [active, setActive] = useState(false);   // boolean

// setCount expects number — TypeScript knows:
setCount(42);       // ✅
setCount('hello');  // ❌ ERROR: string not assignable to number

// ── Explicit generic when initial value is null/undefined ─
const [user, setUser] = useState<User | null>(null);
// Without <User | null>: TypeScript infers useState<null>
// which means you can NEVER set a User — wrong!

setUser({ id: 1, name: 'Rahul' } as User); // ✅
setUser(null);                             // ✅

// ── Typing arrays and objects ──────────────────────────────
const [users, setUsers]       = useState<User[]>([]);
const [errors, setErrors]     = useState<Record<string, string>>({});
const [selected, setSelected] = useState<number[]>([]);

// ── Typing with union state ────────────────────────────────
type FetchState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error';   error: string };
```

```
const [state, setState] = useState<FetchState<User[]>>({ status: 'idle' });

// Usage — TypeScript narrows in each case:
if (state.status === 'success') {
  state.data.forEach(u => console.log(u.name)); // ✅ data is User[]
}

if (state.status === 'error') {
  console.log(state.error);  // ✅ error is string
}

// ── Using useState with a function form (lazy initializer) ─
const [config, setConfig] = useState<AppConfig>(() => {
  // This function only runs once on mount
  const saved = localStorage.getItem('config');
  return saved ? JSON.parse(saved) : defaultConfig;
});
```

4. Typing useEffect and useCallback

```
// ── useEffect — no special typing needed usually ─────────
import { useEffect, useCallback, useMemo } from 'react';

useEffect(() => {
  // Setup code
  const subscription = subscribe(userId);

  // Cleanup function (optional) — TypeScript checks the return type:
  return () => {
    subscription.unsubscribe(); // ✅ void return or cleanup function
  };
}, [userId]); // dependency array

// ── Common mistake: async in useEffect ──────────────────
// ❌ WRONG — useEffect cannot return a Promise:
// useEffect(async () => { await fetchData(); }, []); // TypeScript warns!

// ✅ CORRECT — define async function inside, call it:
useEffect(() => {
  async function loadUser() {
```

```
    const user = await fetchUser(userId);
    setUser(user);
  }
  loadUser();
}, [userId]);

// ── useCallback typing ──────────────────────────────────
const handleSearch = useCallback((query: string): void => {
  setSearchQuery(query);
  performSearch(query);
}, [performSearch]); // TypeScript checks deps array

// With explicit return type:
const calculateTotal = useCallback(
  (items: CartItem[]): number => {
    return items.reduce((sum, item) => sum + item.price * item.qty, 0);
  },
  []
);

// ── useMemo typing ──────────────────────────────────────
const sortedUsers = useMemo(():User[] => {
  return [...users].sort((a, b) => a.name.localeCompare(b.name));
}, [users]); // sortedUsers: User[] — TypeScript infers from return

// ── useRef typing ───────────────────────────────────────
// For DOM elements — pass the element type:
const inputRef = useRef<HTMLInputElement>(null);
const divRef   = useRef<HTMLDivElement>(null);
const canvasRef = useRef<HTMLCanvasElement>(null);

// Access safely with optional chaining:
inputRef.current?.focus();  // null when not mounted

// For mutable values (not DOM — like timers):
const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
// Use ReturnType<typeof setTimeout> — works in both Node and browser

timerRef.current = setTimeout(() => {}, 1000);
clearTimeout(timerRef.current ?? undefined);
```

5. Typing Event Handlers

Event handling is one of the most common sources of TypeScript confusion in React. The key is using the correct React event type for each event.

```
// ── React synthetic event types ──────────────────────────
// Pattern: React.{EventName}Event<HTMLElementType>

// Click events:
function handleClick(e: React.MouseEvent<HTMLButtonElement>): void {
  e.preventDefault();
  console.log(e.currentTarget.textContent); // ✅ HTMLButtonElement methods
}

// Change events (inputs):
function handleChange(e: React.ChangeEvent<HTMLInputElement>): void {
  const value: string = e.target.value;  // ✅ string
  const name:  string = e.target.name;   // ✅ string
  setFormData(prev => ({ ...prev, [name]: value }));
}

// Select change:
function handleSelect(e: React.ChangeEvent<HTMLSelectElement>): void {
  setCategory(e.target.value);
}

// Textarea:
function handleTextArea(e: React.ChangeEvent<HTMLTextAreaElement>): void {
  setBio(e.target.value);
}

// Form submit:
function handleSubmit(e: React.FormEvent<HTMLFormElement>): void {
  e.preventDefault();
  submitForm(formData);
}

// Keyboard:
function handleKeyDown(e: React.KeyboardEvent<HTMLInputElement>): void {
  if (e.key === 'Enter') handleSearch(e.currentTarget.value);
  if (e.key === 'Escape') clearSearch();
}
```

```
// Focus / Blur:
function handleFocus(e: React.FocusEvent<HTMLInputElement>): void {
  e.target.select(); // select all text on focus
}

// Drag events:
function handleDrop(e: React.DragEvent<HTMLDivElement>): void {
  e.preventDefault();
  const files = Array.from(e.dataTransfer.files);
  handleFiles(files);
}

// ── Inline event handler types ──────────────────────────
// TypeScript can infer inline — no explicit type needed:
<button onClick={e => console.log(e.currentTarget.id)}>Click</button>
<input onChange={e => setValue(e.target.value)} />
<form onSubmit={e => { e.preventDefault(); submit(); }}>

// ── Custom event handler type alias ─────────────────────
// For reusable event handler types:
type InputChangeHandler    = React.ChangeEventHandler<HTMLInputElement>;
type ButtonClickHandler    = React.MouseEventHandler<HTMLButtonElement>;
type FormSubmitHandler     = React.FormEventHandler<HTMLFormElement>;

const onChange: InputChangeHandler = e => setValue(e.target.value);
```

6. Custom Hooks with TypeScript

Custom hooks are where TypeScript in React truly shines. A well-typed custom hook is self-documenting, prevents misuse, and gives consumers perfect autocomplete.

6.1 Basic Custom Hook

```
// ── useFetch — generic data fetching hook ───────────────
interface FetchState<T> {
  data:     T | null;
  loading:  boolean;
  error:    string | null;
  refetch:  () => void;
}
```

```
function useFetch<T>(url: string): FetchState<T> {
  const [data, setData]       = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState<string | null>(null);

  const fetchData = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const json: T = await res.json();
      setData(json);
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Unknown error');
    } finally {
      setLoading(false);
    }
  }, [url]);

  useEffect(() => { fetchData(); }, [fetchData]);

  return { data, loading, error, refetch: fetchData };
}

// Usage — T is inferred from explicit type parameter:
const { data: users, loading, error, refetch } =
useFetch<User[]>('/api/users');
// users:   User[] | null  ✅
// loading: boolean        ✅
// error:   string | null  ✅

if (loading) return <Spinner />;
if (error)   return <ErrorMsg message={error} />;
users?.map(u => <UserCard key={u.id} user={u} />);
```

6.2 useLocalStorage Hook

```
// ── useLocalStorage — typed persistent state ──────────────
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) =>
void] {
  const [storedValue, setStoredValue] = useState<T>(() => {
```

```
    try {
      const item = localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = useCallback((value: T) => {
    try {
      setStoredValue(value);
      localStorage.setItem(key, JSON.stringify(value));
    } catch {
      console.error('Failed to save to localStorage');
    }
  }, [key]);

  return [storedValue, setValue];
}

// Usage — T inferred from initialValue:
const [theme, setTheme]     = useLocalStorage('theme', 'light');
// theme: string, setTheme: (value: string) => void

const [user, setUser]       = useLocalStorage<User | null>('user', null);
// user: User | null, setUser: (value: User | null) => void

const [settings, setSettings] = useLocalStorage('settings', {
  notifications: true,
  darkMode: false,
  fontSize: 14,
});
// settings is { notifications: boolean; darkMode: boolean; fontSize: number }
```

6.3 useForm Hook

```
// ── useForm — generic typed form hook ────────────────────
interface UseFormOptions<T> {
  initialValues: T;
  validate?:     (values: T) => Partial<Record<keyof T, string>>;
  onSubmit:      (values: T) => void | Promise<void>;
}
```

```
interface UseFormReturn<T> {
  values:     T;
  errors:     Partial<Record<keyof T, string>>;
  touched:    Partial<Record<keyof T, boolean>>;
  isSubmitting: boolean;
  handleChange:  (field: keyof T, value: T[keyof T]) => void;
  handleBlur:    (field: keyof T) => void;
  handleSubmit:  (e: React.FormEvent) => void;
  reset:         () => void;
}

function useForm<T extends object>({
  initialValues, validate, onSubmit
}: UseFormOptions<T>): UseFormReturn<T> {
  const [values,       setValues]       = useState<T>(initialValues);
  const [errors,       setErrors]       = useState<Partial<Record<keyof T,
string>>>({});
  const [touched,      setTouched]      = useState<Partial<Record<keyof T,
boolean>>>({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = useCallback((field: keyof T, value: T[keyof T]) => {
    setValues(prev => ({ ...prev, [field]: value }));
    if (errors[field]) setErrors(prev => ({ ...prev, [field]: undefined }));
  }, [errors]);

  const handleBlur = useCallback((field: keyof T) => {
    setTouched(prev => ({ ...prev, [field]: true }));
  }, []);

  const handleSubmit = useCallback(async (e: React.FormEvent) => {
    e.preventDefault();
    if (validate) {
      const validationErrors = validate(values);
      if (Object.keys(validationErrors).length > 0) {
        setErrors(validationErrors);
        return;
      }
    }
    setIsSubmitting(true);
    try { await onSubmit(values); }
    finally { setIsSubmitting(false); }
  }, [values, validate, onSubmit]);
```

```
  const reset = useCallback(() => {
    setValues(initialValues);
    setErrors({});
    setTouched({});
  }, [initialValues]);

  return { values, errors, touched, isSubmitting, handleChange, handleBlur,
handleSubmit, reset };
}

// ── Usage ──────────────────────────────────────────────────
interface LoginForm { email: string; password: string }

function LoginPage() {
  const { values, errors, handleChange, handleSubmit, isSubmitting } =
    useForm<LoginForm>({
      initialValues: { email: '', password: '' },
      validate: (v) => {
        const e: Partial<Record<keyof LoginForm, string>> = {};
        if (!v.email)    e.email    = 'Email required';
        if (!v.password) e.password = 'Password required';
        return e;
      },
      onSubmit: async (v) => {
        await login(v.email, v.password);
      },
    });

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={values.email}
        onChange={e => handleChange('email', e.target.value)}
      />
      {errors.email && <span>{errors.email}</span>}
      <button type='submit' disabled={isSubmitting}>Login</button>
    </form>
  );
}
```

7. Context API with TypeScript

```
// ── Typing React Context ──────────────────────────────

// Step 1: Define the context value type
interface AuthContextType {
  user:     User | null;
  isLoading: boolean;
  login:    (email: string, password: string) => Promise<void>;
  logout:   () => void;
  isAdmin:  boolean;
}

// Step 2: Create context with a proper default or null
const AuthContext = React.createContext<AuthContextType | null>(null);

// Step 3: Provider component
function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user,      setUser]      = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  const login = useCallback(async (email: string, password: string) => {
    setIsLoading(true);
    const user = await authService.login(email, password);
    setUser(user);
    setIsLoading(false);
  }, []);

  const logout = useCallback(() => {
    authService.logout();
    setUser(null);
  }, []);

  const value: AuthContextType = {
    user,
    isLoading,
    login,
    logout,
    isAdmin: user?.role === 'admin' ?? false,
  };

  return <AuthContext.Provider
value={value}>{children}</AuthContext.Provider>;
```

```
}

// Step 4: Custom hook with null safety
function useAuth(): AuthContextType {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an <AuthProvider>');
  }
  return context;
  // Returns AuthContextType (not null) — consumers are safe!
}

// Usage — fully typed, no null checks needed:
function ProfilePage() {
  const { user, logout, isAdmin } = useAuth();
  // user: User | null — still need to check this one
  // logout: () => void — fully typed
  // isAdmin: boolean — fully typed

  if (!user) return <Navigate to='/login' />;
  return (
    <div>
      <h1>Welcome {user.name}</h1>   {/* ✅ user.name is safe here */}
      {isAdmin && <AdminPanel />}
      <button onClick={logout}>Logout</button>
    </div>
  );
}
```

8. useReducer with TypeScript

```
// ── useReducer — discriminated union for actions ─────────

// Define state type
interface CartState {
  items:    CartItem[];
  total:    number;
  discount: number;
}

interface CartItem {
```

```
  productId: number;
  name:      string;
  price:     number;
  quantity:  number;
}

// Define action types (discriminated union)
type CartAction =
  | { type: 'ADD_ITEM';      payload: CartItem }
  | { type: 'REMOVE_ITEM';   payload: { productId: number } }
  | { type: 'UPDATE_QTY';    payload: { productId: number; quantity:
number } }
  | { type: 'APPLY_DISCOUNT';payload: { code: string; percent: number } }
  | { type: 'CLEAR_CART' };

// Initial state
const initialState: CartState = { items: [], total: 0, discount: 0 };

// Reducer — TypeScript narrows action.type in each case
function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'ADD_ITEM': {
      // action.payload: CartItem — fully typed
      const existing = state.items.find(i => i.productId ===
action.payload.productId);
      if (existing) {
        const items = state.items.map(i =>
          i.productId === action.payload.productId
            ? { ...i, quantity: i.quantity + 1 }
            : i
        );
        return { ...state, items, total: calcTotal(items, state.discount) };
      }
      const items = [...state.items, { ...action.payload, quantity: 1 }];
      return { ...state, items, total: calcTotal(items, state.discount) };
    }

    case 'REMOVE_ITEM': {
      // action.payload: { productId: number }
      const items = state.items.filter(i => i.productId !==
action.payload.productId);
      return { ...state, items, total: calcTotal(items, state.discount) };
    }
```

```
    case 'UPDATE_QTY': {
      // action.payload: { productId: number; quantity: number }
      const items = state.items.map(i =>
        i.productId === action.payload.productId
          ? { ...i, quantity: action.payload.quantity }
          : i
      );
      return { ...state, items, total: calcTotal(items, state.discount) };
    }

    case 'APPLY_DISCOUNT':
      return { ...state, discount: action.payload.percent,
               total: calcTotal(state.items, action.payload.percent) };

    case 'CLEAR_CART':
      return initialState;

    default: {
      const _exhaustive: never = action;  // Catches unhandled actions!
      return state;
    }
  }
}

function calcTotal(items: CartItem[], discount: number): number {
  const subtotal = items.reduce((s, i) => s + i.price * i.quantity, 0);
  return subtotal * (1 - discount / 100);
}

// Usage:
function CartProvider({ children }: { children: React.ReactNode }) {
  const [cart, dispatch] = useReducer(cartReducer, initialState);

  return (
    <CartContext.Provider value={{ cart, dispatch }}>
      {children}
    </CartContext.Provider>
  );
}

// Dispatching — TypeScript validates the action shape:
dispatch({ type: 'ADD_ITEM', payload: item });     // ✅
dispatch({ type: 'CLEAR_CART' });                  // ✅ no payload
```

```
dispatch({ type: 'REMOVE_ITEM' });                // ✗ missing payload
dispatch({ type: 'UNKNOWN_ACTION' });             // ✗ not a valid type
```

9. Generics in React Components

```
// ── Generic component — works for any data type ──────────

// Generic Select component:
interface SelectProps<T> {
  options:      T[];
  value:        T | null;
  onChange:     (value: T) => void;
  getLabel:     (item: T) => string;
  getValue:     (item: T) => string | number;
  placeholder?: string;
}

function Select<T>({
  options, value, onChange, getLabel, getValue, placeholder
}: SelectProps<T>) {
  return (
    <select
      value={value ? String(getValue(value)) : ''}
      onChange={e => {
        const selected = options.find(o => String(getValue(o)) ===
e.target.value);
        if (selected) onChange(selected);
      }}
    >
      {placeholder && <option value=''>{placeholder}</option>}
      {options.map(opt => (
        <option key={String(getValue(opt))} value={String(getValue(opt))}>
          {getLabel(opt)}
        </option>
      ))}
    </select>
  );
}

// Usage — TypeScript infers T from 'options':
<Select<User>
  options={users}
```

```
  value={selectedUser}
  onChange={setSelectedUser}
  getLabel={u => u.name}
  getValue={u => u.id}
  placeholder='Select a user'
/>

<Select<Product>
  options={products}
  value={selectedProduct}
  onChange={setSelectedProduct}
  getLabel={p => `${p.name} — ₹${p.price}`}
  getValue={p => p.id}
/>

// ── Generic Table component ──────────────────────────────
interface Column<T> {
  key:      keyof T;
  header:   string;
  render?:  (value: T[keyof T], row: T) => React.ReactNode;
  width?:   string;
}

interface TableProps<T extends { id: number | string }> {
  data:       T[];
  columns:    Column<T>[];
  onRowClick?: (row: T) => void;
  isLoading?: boolean;
}

function DataTable<T extends { id: number | string }>({
  data, columns, onRowClick, isLoading
}: TableProps<T>) {
  if (isLoading) return <div>Loading...</div>;

  return (
    <table>
      <thead><tr>
        {columns.map(col => (
          <th key={String(col.key)} style={{ width:
col.width }}>{col.header}</th>
        ))}
      </tr></thead>
```

```
      <tbody>
        {data.map(row => (
          <tr key={row.id} onClick={() => onRowClick?.(row)}>
            {columns.map(col => (
              <td key={String(col.key)}>
                {col.render
                  ? col.render(row[col.key], row)
                  : String(row[col.key])}
              </td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

// Usage:
<DataTable<User>
  data={users}
  columns={[
    { key: 'name',  header: 'Name' },
    { key: 'email', header: 'Email' },
    { key: 'role',  header: 'Role',
      render: (val) => <Badge label={String(val)} />},
  ]}
  onRowClick={user => navigate(`/users/${user.id}`)}
/>
```

10. forwardRef and useImperativeHandle

```
// ── forwardRef — pass ref to child component ──────────────

// Define what methods/properties the ref exposes:
interface InputHandle {
  focus:    () => void;
  clear:    () => void;
  getValue: () => string;
}

// The component's own props:
interface SmartInputProps {
```

```
  label:        string;
  placeholder?: string;
  onChange?:    (value: string) => void;
}

// forwardRef with generics: forwardRef<RefType, PropsType>
const SmartInput = React.forwardRef<InputHandle, SmartInputProps>(
  function SmartInput({ label, placeholder, onChange }, ref) {
    const inputRef = useRef<HTMLInputElement>(null);
    const [value, setValue] = useState('');

    // Expose methods to parent via ref:
    useImperativeHandle(ref, () => ({
      focus:    () => inputRef.current?.focus(),
      clear:    () => { setValue(''); inputRef.current &&
(inputRef.current.value = ''); },
      getValue: () => value,
    }));

    return (
      <div>
        <label>{label}</label>
        <input
          ref={inputRef}
          value={value}
          placeholder={placeholder}
          onChange={e => {
            setValue(e.target.value);
            onChange?.(e.target.value);
          }}
        />
      </div>
    );
  }
);

// Usage — parent can call ref methods:
function SearchBar() {
  const inputRef = useRef<InputHandle>(null);

  const handleClear = () => {
    inputRef.current?.clear();     // ✅ typed — calls clear()
    inputRef.current?.focus();     // ✅ typed — calls focus()
```

```
  };

  return (
    <div>
      <SmartInput ref={inputRef} label='Search' />
      <button onClick={handleClear}>Clear</button>
    </div>
  );
}
```

11. Common Patterns and Best Practices

11.1 Discriminated Union Props

```
// ── Component with mutually exclusive props ──────────────
// Instead of having optional props that conflict:
// ✗ Bad: interface AlertProps { icon?: string; emoji?: string }
// Both can be passed at once — no enforcement

// ✓ Good: discriminated union enforces one or the other
type AlertProps =
  | { variant: 'icon';  icon:  React.ReactNode; message: string }
  | { variant: 'emoji'; emoji: string;          message: string }
  | { variant: 'plain';                         message: string };

function Alert(props: AlertProps) {
  return (
    <div className={`alert alert-${props.variant}`}>
      {props.variant === 'icon'  && props.icon}
      {props.variant === 'emoji' && <span>{props.emoji}</span>}
      <p>{props.message}</p>
    </div>
  );
}

<Alert variant='icon'  icon={<InfoIcon />} message='Info!' />    // ✓
<Alert variant='emoji' emoji='🎉'          message='Success!' /> // ✓
<Alert variant='plain'                     message='Plain' />    // ✓
<Alert variant='icon'  emoji='🎉' message='x' /> // ✗ ERROR: emoji invalid
with icon
```

11.2 Polymorphic Components

```
// ── Polymorphic component — render as any HTML element ───
type PolymorphicAsProp<E extends React.ElementType> = {
  as?: E;
};

type PolymorphicProps<E extends React.ElementType, P = {}> =
  P & PolymorphicAsProp<E> & Omit<React.ComponentPropsWithRef<E>, keyof P>;

interface TextOwnProps {
  size?: 'sm' | 'md' | 'lg' | 'xl';
  weight?: 'normal' | 'medium' | 'bold';
  color?:  string;
}

type TextProps<E extends React.ElementType = 'span'> =
  PolymorphicProps<E, TextOwnProps>;

function Text<E extends React.ElementType = 'span'>({
  as, size = 'md', weight = 'normal', color, children, ...rest
}: TextProps<E>) {
  const Component = as ?? 'span';
  return (
    <Component
      className={`text-${size} font-${weight}`}
      style={{ color }}
      {...rest}
    >
      {children}
    </Component>
  );
}

// Usage — 'as' changes the element type and available props:
<Text as='h1' size='xl' weight='bold'>Heading</Text>
<Text as='p'  size='md'>Paragraph text</Text>
<Text as='a'  href='/about' size='sm'>Link</Text>  // href is valid only for
'a'!
<Text as='button' onClick={fn} size='md'>Button</Text>
<Text as='h1' href='/x'>Bad</Text>  // ✗ ERROR: href not valid on h1
```

11.3 Type-Safe Environment Variables

```
// ── Typing process.env / import.meta.env ──────────────────

// For Vite projects — types/env.d.ts:
/// <reference types='vite/client' />

interface ImportMetaEnv {
  readonly VITE_API_URL:      string;
  readonly VITE_APP_NAME:     string;
  readonly VITE_MAX_RETRIES:  string; // always string in env
  readonly VITE_DEBUG?:       string; // optional
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}

// Now env variables are typed:
const apiUrl = import.meta.env.VITE_API_URL;      // string
const debug  = import.meta.env.VITE_DEBUG === 'true'; // boolean
const maxR   = Number(import.meta.env.VITE_MAX_RETRIES); // convert

// ── Type-safe config helper ──────────────────────────────
function getEnvVar(key: keyof ImportMetaEnv): string {
  const value = import.meta.env[key];
  if (!value) throw new Error(`Missing env variable: ${key}`);
  return value;
}

const apiUrl2 = getEnvVar('VITE_API_URL'); // ✅ typed + validated
const invalid  = getEnvVar('UNKNOWN_VAR'); // ❌ ERROR: not a valid key
```

12. Interview Questions — Chapter 15

Q: How do you type component props in React TypeScript?

Answer: Define an interface (or type) for the props and use it as the parameter type of the function component.

```
interface ButtonProps { label: string; onClick: () => void; disabled?: boolean }
function Button({ label, onClick, disabled = false }: ButtonProps) { ... }
```

For optional props use ?, for default values use destructuring defaults.

For children use 'children: React.ReactNode' or PropsWithChildren<T>.

For extending HTML element props use 'extends React.ButtonHTMLAttributes<HTMLButtonElement>'

and spread the rest props: const { custom, ...rest } = props; <button {...rest} />

Q: When do you need to explicitly type useState?

Answer: TypeScript infers the type from the initial value. You need to explicitly type when:

1. Initial value is null or undefined: useState<User | null>(null)
   Without <User|null>, TypeScript infers useState<null> and you can never set a User.

2. Initial value is an empty array: useState<User[]>([])
   Without it, TypeScript infers never[] which won't accept User objects.

3. Union state: useState<'idle'|'loading'|'success'>('idle')

4. Discriminated union state: useState<LoadingState | SuccessState | ErrorState>({status:'idle'})

Q: What React event types exist? How do you type onChange on an input?

Answer: React provides event types following the pattern:
React.{EventName}Event<HTMLElement>

onClick → React.MouseEvent<HTMLButtonElement>

onChange → React.ChangeEvent<HTMLInputElement>

onSubmit → React.FormEvent<HTMLFormElement>

onKeyDown → React.KeyboardEvent<HTMLInputElement>

onFocus → React.FocusEvent<HTMLInputElement>

onDrop → React.DragEvent<HTMLDivElement>

For onChange on input: (e: React.ChangeEvent<HTMLInputElement>) => void

Access value via: e.target.value

Inline event handlers are automatically typed by TypeScript — no explicit annotation needed.

Q: How do you type useRef in React?

Answer: useRef has two uses with different typing:

1. DOM element ref — pass the HTML element type, initial value null:
   const ref = useRef<HTMLInputElement>(null)
   ref.current is HTMLInputElement | null — use optional chaining: ref.current?.focus()

2. Mutable value (like a timer) — not a DOM ref, never null:
   const timer = useRef<ReturnType<typeof setTimeout> | null>(null)
   timer.current = setTimeout(...)

The key difference: DOM refs start as null and get assigned by React.
Mutable refs hold values you control — initialise them as needed.

Q: How do you create a type-safe Context in React?

Answer: Three steps:

1. Define the context value interface:
   interface ThemeContext { theme: 'light'|'dark'; toggle: () => void }

2. Create context with null default:
   const ThemeCtx = createContext<ThemeContext | null>(null)

3. Create a custom hook that throws if used outside provider:

```
function useTheme(): ThemeContext {
  const ctx = useContext(ThemeCtx);
  if (!ctx) throw new Error('useTheme must be inside ThemeProvider');
  return ctx;  // Returns ThemeContext (not ThemeContext | null)
}
```

The custom hook removes null from the return type — consumers never need to null-check.

Q: How do you write a generic React component?

Answer: Add a generic type parameter to the component function and use it in props:

```
function List<T extends { id: number }>({ items, renderItem }: {
```

```
  items: T[];
  renderItem: (item: T) => React.ReactNode;
}) {
  return <ul>{items.map(i => <li key={i.id}>{renderItem(i)}</li>)}</ul>;
}
```

TypeScript infers T from the items array. You can also specify explicitly: <List<User> ...>

Common use cases: generic DataTable, generic Select, generic List, generic Modal.

Q: What is forwardRef and how do you type it?

Answer: forwardRef allows a parent component to access a child component's DOM node or imperative methods via a ref.

Type: React.forwardRef<RefType, PropsType>

```
const Input = React.forwardRef<HTMLInputElement, InputProps>(
  ({ label, ...props }, ref) => (
    <div><label>{label}</label><input ref={ref} {...props} /></div>
  )
);
```

For exposing custom methods (not raw DOM), use useImperativeHandle:

```
interface InputHandle { focus(): void; clear(): void }
const Input = React.forwardRef<InputHandle, InputProps>((props, ref) => {
  useImperativeHandle(ref, () => ({ focus: ..., clear: ... }));
  ...
})
```

Rapid Fire — Chapter 15

Question	One-Line Answer
Type for any JSX/React content as prop?	React.ReactNode — most flexible, accepts JSX, strings, null
Type for onClick on a button?	React.MouseEvent<HTMLButtonElement>

Type for onChange on input?	React.ChangeEvent<HTMLInputElement>
useState<User | null>(null) — why?	Without it TS infers useState<null> and you can never set a User
Type for children that must be JSX (not text)?	React.ReactElement (not ReactNode)
How to type useRef for HTMLInputElement?	useRef<HTMLInputElement>(null) — current is HTMLInputElement | null
How to type useRef for a timer?	useRef<ReturnType<typeof setTimeout> | null>(null)
Why null default for createContext?	Avoids fake default values — custom hook throws if used outside provider
How to prevent useEffect receiving async fn?	Define async function inside effect, call it — never pass async directly
How to type all native div props + custom ones?	interface Props extends React.HTMLAttributes<HTMLDivElement> { custom: ... }

Chapter 15 Key Takeaways

Props: define interface, use in function params. Optional props use ?, defaults in destructuring.

Extend HTML props: extends React.ButtonHTMLAttributes<HTMLButtonElement> + spread rest.

useState: TypeScript infers from initial value. Explicitly type when initial value is null/[].

useRef: useRef<HTMLInputElement>(null) for DOM; useRef<ReturnType<typeof setTimeout>|null> for values.

useEffect: never pass async function directly — define async inside, call it.

Events: React.ChangeEvent<HTMLInputElement>, React.MouseEvent<HTMLButtonElement> etc.

Inline handlers are auto-typed — no annotation needed inside JSX.

Context: interface → createContext<T|null>(null) → custom hook that throws → consumers get T.

useReducer: discriminated union for actions → TypeScript narrows in each case.

Custom hooks: return tuple typed with 'as const' or explicit return type annotation.

Generic components: List<T>, Select<T>, Table<T> — T inferred from props.

forwardRef: React.forwardRef<RefType, PropsType> + useImperativeHandle for custom methods.

Discriminated union props: type variants with different required props — enforces at compile time.

End of Chapter 15

CHAPTER 16

tsconfig.json — Deep Dive

Compiler Options • Strict Mode • Module Settings • Path Aliases • Project References • Config Templates

1. What is tsconfig.json?

tsconfig.json is the configuration file for the TypeScript compiler. It controls everything — which files to compile, how strict the type checking should be, what output format to produce, and how modules are resolved. Understanding tsconfig.json is essential for every serious TypeScript project.

Real-World Analogy — The Blueprint for a Building

Imagine constructing a building.

The architect creates a BLUEPRINT that specifies:

→ Which rooms to build (files to include)

→ What materials to use (compiler options)

→ How strict the safety codes must be (strict settings)

→ Where the final building stands (outDir)

→ Which building codes to follow (target ES version)

tsconfig.json is that blueprint for TypeScript.

Without it, TypeScript uses defaults — which may not suit your project.

With a well-crafted tsconfig, TypeScript works exactly how you need it.

1.1 Basic Structure

```
// tsconfig.json — full structure overview
{
  "include":  ["src/**/*"],          // which files to compile
  "exclude":  ["node_modules", "dist"], // files to skip
  "extends":  "./tsconfig.base.json",  // inherit from another config
  "references": [                      // monorepo project references
    { "path": "./packages/core" }
  ],
  "compilerOptions": {
    // ...all the options...
  }
```

```
}

// include uses glob patterns:
// 'src'       → all .ts/.tsx files in src and subfolders
// 'src/**/*'  → same (explicit)
// '**/*.ts'   → all .ts files anywhere

// exclude always defaults to node_modules, outDir, and files
// you should add dist, coverage, and test files from main build
```

2. Type Checking Options — The Strict Family

The strict family of options controls how rigorously TypeScript checks your code. The umbrella strict: true enables all of them at once — always use it in new projects.

What strict: true Enables

strict: true turns on ALL of these simultaneously:

strictNullChecks — null/undefined are not assignable to other types
noImplicitAny — no silent 'any' from missing annotations
strictFunctionTypes — contravariant function parameter types
strictBindCallApply — typed bind/call/apply methods
strictPropertyInitialization — class props must be initialized
noImplicitThis — 'this' must be typed in functions
useUnknownInCatchVariables — catch(e) gives 'unknown' not 'any'
alwaysStrict — adds 'use strict' to all output files

Cost: a few extra type annotations.

Benefit: entire classes of runtime bugs caught at compile time.

2.1 strictNullChecks

```
// strictNullChecks: false (default without strict)
let name: string = null;       // allowed — dangerous silent bug
let age:  number = undefined;  // allowed — will crash at runtime

// strictNullChecks: true
let name: string = null;       // ERROR: null not assignable to string
```

```
let name: string | null = null; // correct — explicit union

// Forces null checks before using values:
function greet(user: User | null): string {
  // return user.name;           // ERROR: user possibly null
  if (!user) return 'Guest';
  return user.name;              // safe — TypeScript knows it is User
}

// Optional chaining + nullish coalescing become idiomatic:
const display = user?.profile?.displayName ?? user?.name ?? 'Unknown';
```

2.2 noImplicitAny

```
// noImplicitAny: false — TypeScript silently infers 'any'
function process(data) {          // data: any (implicit)
  return data.whatever.you.want;  // no errors — type safety lost
}

// noImplicitAny: true
function process(data) {          // ERROR: implicit any on parameter
  return data.whatever;
}

function process(data: InputData): Result { // correct
  return transform(data);
}

// Fires for: untyped params, empty arrays, destructuring without annotation
const fn  = (x) => x * 2;        // ERROR: x implicit any
const fn  = (x: number) => x * 2; // correct
const arr = [];                   // ERROR: never[] becomes any[]
const arr: string[] = [];          // correct
```

2.3 strictPropertyInitialization

```
// Class properties must be initialized in the constructor
class User {
  name:  string;   // ERROR: not definitely assigned in constructor
  email: string;   // ERROR
}
```

```
// Fix 1: Initialize in constructor
class User {
  name:  string;
  email: string;
  constructor(name: string, email: string) {
    this.name = name; this.email = email; // correct
  }
}

// Fix 2: Default value
class User { name: string = ''; email: string = ''; }

// Fix 3: Optional
class User { name?: string; email?: string; }

// Fix 4: Definite assignment assertion (use sparingly)
class UserWithORM {
  name!: string;   // '!' = 'I guarantee this will be set before use'
  // Used for ORMs like TypeORM that set properties via reflection
}
```

2.4 useUnknownInCatchVariables

```
// Before (catch variable was 'any' — unsafe):
try {
  await fetchData();
} catch (e) {
  console.log(e.message); // no error even if e is not an Error
}

// useUnknownInCatchVariables: true (part of strict)
// catch variable is now 'unknown' — forces you to check:
try {
  await fetchData();
} catch (e) {
  // e.message;            // ERROR: e is unknown
  if (e instanceof Error) {
    console.log(e.message); // safe
  } else {
    console.log(String(e)); // handle non-Error throws
  }
}
```

```
// Helper function (recommended pattern):
function toErrorMessage(e: unknown): string {
  if (e instanceof Error) return e.message;
  return String(e);
}

try { await riskyOp(); }
catch (e) { showError(toErrorMessage(e)); }
```

2.5 Additional Strictness Flags

```
// ── Beyond strict: true — extra safety options ──────────
{
  "compilerOptions": {
    "strict": true,

    // noUncheckedIndexedAccess: array[i] returns T | undefined
    // (not just T) — catches out-of-bounds access
    "noUncheckedIndexedAccess": true,

    // noImplicitOverride: must use 'override' keyword on overriding methods
    "noImplicitOverride": true,

    // noPropertyAccessFromIndexSignature: must use bracket notation
    // for index-signature properties
    "noPropertyAccessFromIndexSignature": true,

    // exactOptionalPropertyTypes: optional props cannot be set to undefined
    // explicitly — they can only be omitted
    "exactOptionalPropertyTypes": true,

    // noUnusedLocals: error on variables declared but never used
    "noUnusedLocals": true,

    // noUnusedParameters: error on function params never used
    "noUnusedParameters": true,

    // noImplicitReturns: all code paths must return a value
    "noImplicitReturns": true,

    // noFallthroughCasesInSwitch: switch cases must break/return
    "noFallthroughCasesInSwitch": true,
```

```
  }
}

// noUncheckedIndexedAccess example:
const users: User[] = getUsers();
const first = users[0];     // User | undefined (not just User!)
first.name;                 // ERROR: possibly undefined
first?.name;                // correct — optional chaining
if (first) first.name;      // correct — null check
```

3. Output Options

3.1 target — ES Output Version

```
// target controls which ES version TypeScript compiles TO
{
  "compilerOptions": {
    // Common target values:
    "target": "ES2022",    // recommended for modern Node.js (16+)
    // "target": "ES2017",  // async/await natively (good for Node 8+)
    // "target": "ES5",     // max browser compatibility (older projects)
    // "target": "ESNext",  // latest ES features (for bundlers)
  }
}

// What target affects:
// ES5:    classes → prototype chains, arrow fns → function, async → helpers
// ES2017: async/await compiled natively (no polyfill needed)
// ES2022: class fields, top-level await, at() method — all native

// Rule: Match your target to the MINIMUM runtime you support.
// Node 18 LTS → ES2022 is safe
// Old browsers → ES5 or ES2015
// Vite/webpack with babel → ESNext (bundler handles downcompilation)

// 'lib' controls which APIs TypeScript knows about (separate from target):
{
  "compilerOptions": {
    "target": "ES2022",
    "lib":    ["ES2022", "DOM", "DOM.Iterable"]
    // DOM     → window, document, fetch, HTMLElement...
    // ES2022  → Array.at(), Object.hasOwn(), structuredClone...
```

```
  }
}
```

3.2 outDir, rootDir, declaration

```
{
  "compilerOptions": {
    // Where compiled JS goes:
    "outDir": "./dist",

    // Root of source files (preserves folder structure in outDir):
    "rootDir": "./src",

    // Generate .d.ts declaration files alongside JS output:
    "declaration": true,

    // Generate source maps (maps compiled JS back to .ts source):
    "sourceMap": true,

    // Generate .d.ts.map files (for declaration source maps):
    "declarationMap": true,

    // Only emit declaration files — no JS output
    // Useful when bundler handles JS, you just want types:
    "emitDeclarationOnly": true,

    // Don't emit any files — just type-check:
    "noEmit": true,
    // Used when bundler (Vite, webpack) handles transpilation
    // and you want TypeScript only for type checking.
  }
}

// Typical Node.js library setup:
// src/index.ts  → dist/index.js + dist/index.d.ts
// src/utils.ts  → dist/utils.js + dist/utils.d.ts

// Typical Vite/bundler app setup:
// noEmit: true — Vite handles JS, tsc only checks types
```

3.3 removeComments, noEmitOnError

```
{
  "compilerOptions": {
    // Strip all comments from output JS:
    "removeComments": true,

    // Do not emit JS if there are type errors:
    "noEmitOnError": true,
    // Prevents deploying broken builds if tsc has errors

    // Preserve const enums as their literal values in output:
    "preserveConstEnums": true,

    // Inline source map into the JS file (not a separate .map file):
    "inlineSourceMap": true,

    // Inline source content into source maps:
    "inlineSources": true,
  }
}
```

4. Module Options

4.1 module and moduleResolution

```
{
  "compilerOptions": {
    // 'module' — output module format:
    "module": "NodeNext",    // modern Node.js (ESM with .js extensions)
    // "module": "CommonJS", // require() / module.exports for older Node
    // "module": "ESNext",   // import/export for bundlers
    // "module": "Preserve", // TS 5.4+: keep whatever the source uses

    // 'moduleResolution' — how TypeScript FINDS files:
    "moduleResolution": "NodeNext",  // modern Node with exports field
    // "moduleResolution": "Bundler", // for Vite/webpack (no extensions
needed)
    // "moduleResolution": "Node",    // classic Node (old projects)

    // Enable default imports from CommonJS modules:
    "esModuleInterop": true,
    // Without it:  import * as React from 'react'
    // With it:     import React from 'react'  (nicer)
```

```
    // Allow CommonJS-style require() imports:
    "allowSyntheticDefaultImports": true,

    // Allow importing JSON files:
    "resolveJsonModule": true,

    // Isolate each file for single-file transpilers (Vite, Babel, SWC):
    "isolatedModules": true,
    // Forces 'import type' for all type-only imports
  }
}

// ── Common project preset combos ──────────────────────────
// Node.js app/library:    module: NodeNext + moduleResolution: NodeNext
// Vite React app:         module: ESNext  + moduleResolution: Bundler
// Next.js app:            module: ESNext  + moduleResolution: Bundler
// Old Node app:           module: CommonJS + moduleResolution: Node
```

4.2 Path Aliases

```
// Path aliases let you use clean imports instead of relative paths

// Without aliases (messy — changes with every file moved):
import { UserService } from '../../../services/UserService';
import { formatDate }  from '../../../../utils/date';
import { Button }      from '../../components/ui/Button';

// With aliases (clean — stays the same no matter where the file is):
import { UserService } from '@/services/UserService';
import { formatDate }  from '@/utils/date';
import { Button }      from '@/components/ui/Button';

// tsconfig.json setup:
{
  "compilerOptions": {
    "baseUrl": ".",          // root of the project
    "paths": {
      "@/*":             ["./src/*"],
      "@components/*":   ["./src/components/*"],
      "@services/*":     ["./src/services/*"],
      "@utils/*":        ["./src/utils/*"],
```

```
      "@models/*":      ["./src/models/*"],
      "@hooks/*":       ["./src/hooks/*"],
      "@assets/*":      ["./src/assets/*"],
    }
  }
}

// IMPORTANT: tsconfig paths only tell TypeScript where to find types.
// Your BUNDLER also needs to know about aliases:

// Vite — vite.config.ts:
import { defineConfig } from 'vite';
import path from 'path';

export default defineConfig({
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') }
  }
});

// Webpack — webpack.config.js:
module.exports = {
  resolve: {
    alias: { '@': path.resolve(__dirname, 'src') }
  }
};

// Node.js (no bundler) — use tsconfig-paths:
// npm install tsconfig-paths
// node -r tsconfig-paths/register dist/index.js
```

5. JSX Options (React)

```
// For React projects, configure JSX transformation:
{
  "compilerOptions": {
    // jsx — how to transform JSX:
    "jsx": "react-jsx",    // modern React 17+ automatic transform
    // "jsx": "react",     // classic: import React from 'react' required
    // "jsx": "preserve",  // keep JSX as-is (let bundler handle it)
    // "jsx": "react-native", // for React Native projects
```

```
    // jsxImportSource — customise the JSX factory (e.g. for Preact):
    "jsxImportSource": "react",  // default
    // "jsxImportSource": "preact/compat", // for Preact
    // "jsxImportSource": "@emotion/react", // for Emotion CSS-in-JS
  }
}

// With react-jsx: you do NOT need 'import React from "react"' in every file.
// TypeScript (and Vite/webpack) handle the React import automatically.

// Files with JSX must use .tsx extension (not .ts).
// TypeScript will error on JSX in .ts files.
```

6. Decorator Options

```
// For NestJS, Angular, TypeORM — enable legacy decorators:
{
  "compilerOptions": {
    "experimentalDecorators":  true,  // enable decorator syntax
    "emitDecoratorMetadata":   true,  // emit design:paramtypes for DI
  }
}

// TypeScript 5.0+ supports Stage 3 decorators natively — no flags needed.
// But NestJS/Angular currently require experimentalDecorators: true.

// These two settings are independent:
// experimentalDecorators: enables @Decorator syntax
// emitDecoratorMetadata:  emits constructor parameter type metadata
//   → required for DI containers that use reflect-metadata
//   → requires 'import reflect-metadata' at app entry point
```

7. extends — Config Inheritance

Large projects need multiple tsconfig files for different purposes — one for the app, one for tests, one for builds. The extends field lets them share a base and override only what differs.

```
// ── tsconfig.base.json — shared settings ──────────────────
{
```

```
  "compilerOptions": {
    "target":              "ES2022",
    "lib":                 ["ES2022"],
    "module":              "NodeNext",
    "moduleResolution":    "NodeNext",
    "strict":              true,
    "noUnusedLocals":      true,
    "noUnusedParameters":  true,
    "noImplicitReturns":   true,
    "esModuleInterop":     true,
    "resolveJsonModule":   true,
    "declaration":         true,
    "sourceMap":           true
  }
}

// ── tsconfig.json — main app (extends base) ────────────────
{
  "extends": "./tsconfig.base.json",
  "compilerOptions": {
    "outDir":   "./dist",
    "rootDir":  "./src",
  },
  "include": ["src"]
}

// ── tsconfig.test.json — for Jest/Vitest ───────────────────
{
  "extends": "./tsconfig.base.json",
  "compilerOptions": {
    "noEmit":          true,  // tests don't produce output
    "noUnusedLocals":  false, // test files often have unused imports
    "types":           ["jest", "node"]
  },
  "include": ["src", "tests"]
}

// ── tsconfig.build.json — for production build ─────────────
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "noEmitOnError":   true,  // fail build if type errors
    "removeComments":  true,
```

```
  },
  "exclude": ["**/*.test.ts", "**/*.spec.ts", "tests/"]
}

// package.json scripts using different configs:
// "type-check": "tsc --noEmit"
// "build":      "tsc -p tsconfig.build.json"
// "test":       "jest --config jest.config.ts"
```

8. Project References (Monorepo)

Project references allow you to split a TypeScript project into smaller sub-projects that are built independently and can reference each other. This dramatically speeds up compilation in large codebases.

```
// ── Monorepo structure ──────────────────────────────
// packages/
//   core/          ← shared utilities
//     tsconfig.json
//   api/           ← backend (depends on core)
//     tsconfig.json
//   web/           ← frontend (depends on core)
//     tsconfig.json
// tsconfig.json    ← root (references all packages)

// ── packages/core/tsconfig.json ──────────────────────
{
  "compilerOptions": {
    "outDir":      "./dist",
    "rootDir":     "./src",
    "composite":   true,     // REQUIRED for project references
    "declaration": true,     // REQUIRED for project references
  },
  "include": ["src"]
}

// ── packages/api/tsconfig.json ───────────────────────
{
  "compilerOptions": {
    "outDir":    "./dist",
    "rootDir":   "./src",
    "composite": true,
```

```
  },
  "references": [
    { "path": "../core" }   // api depends on core
  ],
  "include": ["src"]
}

// ── Root tsconfig.json ──────────────────────────────────
{
  "references": [
    { "path": "./packages/core" },
    { "path": "./packages/api" },
    { "path": "./packages/web" }
  ],
  "files": []  // root doesn't compile anything directly
}

// Build commands:
// tsc --build packages/core   ← build just core
// tsc --build packages/api    ← build api (builds core if needed)
// tsc --build                 ← build everything in dependency order
// tsc --build --watch         ← incremental watch mode
// tsc --build --clean         ← clean all build outputs

// Benefits: incremental compilation — only rebuild what changed.
// On large codebases, this can go from 60s → 3s build times.
```

9. Useful Compiler Flags

9.1 skipLibCheck

```
// skipLibCheck: true — skip type-checking of .d.ts files
{
  "compilerOptions": {
    "skipLibCheck": true
  }
}

// Why use it: Some third-party type definitions have errors.
// If @types/some-library has a bug, your build fails for no reason.
// skipLibCheck skips checking ALL declaration files.
```

```
// Risk: You miss genuine type errors in your own .d.ts files.
// Recommendation: enable it — the community benefit outweighs the risk.
// Almost all starter templates (CRA, Vite, Next.js) enable it by default.
```

9.2 forceConsistentCasingInFileNames

```
// Prevents case-sensitivity bugs between macOS/Windows and Linux
{
  "compilerOptions": {
    "forceConsistentCasingInFileNames": true
  }
}

// macOS/Windows: import './UserService' == import './userservice' (same file)
// Linux (CI/CD): import './UserService' != import './userservice'
(different!)

// Without this: works on your Mac, crashes on Linux CI. Very common bug.
// With this: TypeScript errors if import casing doesn't match file name.
// ALWAYS enable this for any project that deploys to Linux.
```

9.3 allowJs and checkJs

```
// allowJs: true — TypeScript will compile .js files too
// checkJs: true — TypeScript will type-check .js files
{
  "compilerOptions": {
    "allowJs":  true,   // compile JS files alongside TS
    "checkJs":  true,   // type-check JS files (uses JSDoc types)
    "maxNodeModuleJsDepth": 1  // depth to search JS in node_modules
  }
}

// Useful for: gradual migration from JS to TS.
// Start with allowJs, add types gradually, rename files to .ts over time.

// JSDoc types in .js files (works with checkJs):
/**
 * @param {string} name
 * @returns {string}
 */
function greet(name) {
  return 'Hello ' + name;
```

```
}
```

9.4 Types and typeRoots

```
// Control which @types packages TypeScript auto-includes
{
  "compilerOptions": {
    // typeRoots: where to look for type definitions
    "typeRoots": ["./node_modules/@types", "./src/types"],

    // types: ONLY include these @types packages (not all of them)
    "types": ["node", "jest"]
    // Without 'types', TypeScript includes ALL @types/* packages.
    // With 'types', only the listed ones are included globally.
    // Prevents window/document types leaking into Node.js code etc.
  }
}

// Common use case — separate browser and Node contexts:
// tsconfig.node.json:    "types": ["node"]
// tsconfig.browser.json: "types": ["dom"]
// Prevents: accidentally using process.env in browser code
```

10. Complete Config Templates

10.1 Node.js Backend (Express/Fastify)

```
// tsconfig.json for a Node.js backend project
{
  "compilerOptions": {
    "target":                          "ES2022",
    "lib":                             ["ES2022"],
    "module":                          "NodeNext",
    "moduleResolution":                "NodeNext",
    "rootDir":                         "./src",
    "outDir":                          "./dist",
    "strict":                          true,
    "noUncheckedIndexedAccess":        true,
    "noImplicitReturns":               true,
    "noFallthroughCasesInSwitch":      true,
    "noUnusedLocals":                  true,
    "noUnusedParameters":              true,
```

```
    "esModuleInterop":                  true,
    "resolveJsonModule":                true,
    "declaration":                      true,
    "declarationMap":                   true,
    "sourceMap":                        true,
    "skipLibCheck":                     true,
    "forceConsistentCasingInFileNames":  true,
    "baseUrl":                          ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist"]
}
```

10.2 React + Vite Frontend

```
// tsconfig.json for a Vite React project
{
  "compilerOptions": {
    "target":                           "ES2022",
    "lib":                              ["ES2022", "DOM", "DOM.Iterable"],
    "module":                           "ESNext",
    "moduleResolution":                 "Bundler",
    "jsx":                              "react-jsx",
    "strict":                           true,
    "noUnusedLocals":                   true,
    "noUnusedParameters":               true,
    "noFallthroughCasesInSwitch":       true,
    "noImplicitReturns":                true,
    "esModuleInterop":                  true,
    "allowImportingTsExtensions":       true,
    "resolveJsonModule":                true,
    "isolatedModules":                  true,
    "noEmit":                           true,
    "skipLibCheck":                     true,
    "forceConsistentCasingInFileNames":  true,
    "baseUrl":                          ".",
    "paths": {
      "@/*":           ["./src/*"],
      "@components/*": ["./src/components/*"],
      "@hooks/*":      ["./src/hooks/*"],
```

```
      "@utils/*":      ["./src/utils/*"]
    }
  },
  "include": ["src"],
  "references": [{ "path": "./tsconfig.node.json" }]
}

// tsconfig.node.json (for vite.config.ts itself)
{
  "compilerOptions": {
    "composite":        true,
    "module":           "ESNext",
    "moduleResolution": "Bundler",
    "allowSyntheticDefaultImports": true
  },
  "include": ["vite.config.ts"]
}
```

10.3 NestJS Backend

```
// tsconfig.json for a NestJS project
{
  "compilerOptions": {
    "module":                          "CommonJS",
    "declaration":                     true,
    "removeComments":                  true,
    "emitDecoratorMetadata":           true,
    "experimentalDecorators":          true,
    "allowSyntheticDefaultImports":    true,
    "target":                          "ES2021",
    "lib":                             ["ES2021"],
    "sourceMap":                       true,
    "outDir":                          "./dist",
    "baseUrl":                         "./",
    "incremental":                     true,
    "strict":                          true,
    "noImplicitAny":                   true,
    "strictNullChecks":                true,
    "noUnusedLocals":                  true,
    "noUnusedParameters":              true,
    "noImplicitReturns":               true,
    "skipLibCheck":                    true,
    "forceConsistentCasingInFileNames":  true,
```

```
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

10.4 TypeScript Library / npm Package

```
// tsconfig.json for publishing a TypeScript library
{
  "compilerOptions": {
    "target":                          "ES2020",
    "lib":                             ["ES2020"],
    "module":                          "NodeNext",
    "moduleResolution":                "NodeNext",
    "rootDir":                         "./src",
    "outDir":                          "./dist",
    "declaration":                     true,
    "declarationMap":                  true,
    "sourceMap":                       true,
    "composite":                       true,
    "strict":                          true,
    "noUncheckedIndexedAccess":        true,
    "noUnusedLocals":                  true,
    "noUnusedParameters":              true,
    "noImplicitReturns":               true,
    "esModuleInterop":                 true,
    "skipLibCheck":                    true,
    "forceConsistentCasingInFileNames":  true
  },
  "include": ["src"],
  "exclude": ["node_modules", "dist", "**/*.test.ts", "**/*.spec.ts"]
}
```

11. tsc CLI Commands

```
// ── Common tsc command-line flags ──────────────────────────

// Type-check without emitting JS (use in CI/pre-commit):
tsc --noEmit
```

```
// Use a specific config file:
tsc -p tsconfig.build.json
tsc --project tsconfig.test.json

// Watch mode — recompile on file changes:
tsc --watch
tsc -w

// Build mode (project references):
tsc --build
tsc -b

// Build and watch:
tsc --build --watch

// Clean build outputs:
tsc --build --clean

// Show verbose output:
tsc --build --verbose

// Check TypeScript version:
tsc --version

// Show all compiler options with explanations:
tsc --help --all

// ── package.json scripts pattern ──────────────────────────
{
  "scripts": {
    "build":       "tsc -p tsconfig.build.json",
    "type-check":  "tsc --noEmit",
    "type-check:watch": "tsc --noEmit --watch",
    "clean":       "tsc --build --clean",
    "dev":         "ts-node-dev --respawn src/index.ts"
  }
}
```

12. Complete Options Reference Table

Option	Recommended	What it does
`strict`	`true`	Enables ALL strict checks — always set this
`strictNullChecks`	`true (via strict)`	null/undefined not assignable to other types
`noImplicitAny`	`true (via strict)`	No silent 'any' from missing annotations
`strictPropertyInitialization`	`true (via strict)`	Class properties must be initialized
`useUnknownInCatchVariables`	`true (via strict)`	Catch variables are 'unknown' not 'any'
`noUncheckedIndexedAccess`	`true`	array[i] returns T | undefined
`noUnusedLocals`	`true`	Error on declared but unused variables
`noUnusedParameters`	`true`	Error on declared but unused function params
`noImplicitReturns`	`true`	All code paths must return a value
`noFallthroughCasesInSwitch`	`true`	Switch cases must break or return
`target`	`ES2022`	ES output version — match your runtime
`module`	`NodeNext/ESNext`	Output module format
`moduleResolution`	`NodeNext/Bundler`	How TypeScript finds imported files
`esModuleInterop`	`true`	import React instead of import * as React
`resolveJsonModule`	`true`	Allow importing .json files
`isolatedModules`	`true`	Force import type — needed for Vite/Babel
`declaration`	`true (libs)`	Generate .d.ts declaration files
`sourceMap`	`true`	Generate .js.map source map files
`outDir`	`"./dist"`	Where compiled JS output goes
`rootDir`	`"./src"`	Root of source files
`baseUrl + paths`	`"@/*"`	Path aliases for clean imports
`skipLibCheck`	`true`	Skip type-checking of .d.ts files
`forceConsistentCasingInFileNames`	`true`	Prevent macOS/Windows vs Linux case bugs
`noEmit`	`true (Vite)`	Type-check only — bundler handles JS
`jsx`	`react-jsx`	JSX transformation mode for React
`experimentalDecorators`	`true (NestJS)`	Enable @Decorator syntax (legacy)
`emitDecoratorMetadata`	`true (NestJS)`	Emit design:paramtypes for DI containers
`composite`	`true (monorepo)`	Enable incremental project references

13. Interview Questions — Chapter 16

Q: What does strict: true enable in tsconfig.json?

Answer: strict: true is an umbrella flag that enables all strict type-checking options simultaneously.

It enables: strictNullChecks (null not assignable to other types), noImplicitAny (no silent any), strictFunctionTypes (contravariant parameters), strictBindCallApply (typed bind/call/apply), strictPropertyInitialization (class properties must be initialized), noImplicitThis (this must be typed), useUnknownInCatchVariables (catch gives unknown not any), and alwaysStrict (adds 'use strict' to output).

Always enable strict: true in new projects. The extra annotations are worth the safety.

Q: What is the difference between 'module' and 'moduleResolution' in tsconfig?

Answer: They control different aspects of the module system:

'module': the output FORMAT of the compiled JavaScript.

CommonJS → require() / module.exports (Node.js classic)

NodeNext → ES modules with .js extensions (modern Node)

ESNext → import/export (browsers and bundlers)

'moduleResolution': the ALGORITHM TypeScript uses to find imported files.

Node → classic Node.js algorithm (node_modules lookup)

NodeNext → modern Node with package.json exports field support

Bundler → for Vite/webpack — no file extension required

They usually pair together: NodeNext+NodeNext for modern Node, ESNext+Bundler for Vite.

Q: What are path aliases and how do you configure them?

Answer: Path aliases let you use short, absolute-style imports instead of relative paths.

Configure in tsconfig.json under compilerOptions:

'baseUrl': '.' (project root)

'paths': { '@/*': ['./src/*'] }

Then: import { Button } from '@/components/Button' instead of '../../../../components/Button'

IMPORTANT: tsconfig paths only teach TypeScript where to find types.

Your BUNDLER also needs the same aliases configured:

Vite: resolve.alias in vite.config.ts

Webpack: resolve.alias in webpack.config.js

Node.js (no bundler): tsconfig-paths package

Q: What is noUncheckedIndexedAccess and why is it useful?

Answer: When noUncheckedIndexedAccess: true, accessing an array by index returns T | undefined instead of just T.

Without it:

```
const users: User[] = [];
const first = users[0];   // TypeScript thinks: User (not User | undefined!)
first.name;               // No error — but crashes if array is empty
```

With it:

```
const first = users[0];  // TypeScript says: User | undefined
first.name;              // ERROR: possibly undefined
first?.name;             // correct — optional chaining
```

This catches a very common source of runtime crashes.

Consider enabling it even in projects with strict: true (it's not included in strict).

Q: What is the 'composite' option and when do you need it?

Answer: composite: true enables TypeScript's project references feature for monorepos.

It requires and enables: declaration: true (generates .d.ts files), declarationMap: true, and stores incremental build info.

You need it when: one TypeScript project depends on another TypeScript project (monorepo pattern). Each sub-project sets composite: true. The root tsconfig.json lists all projects in 'references'.

Benefit: incremental compilation — TypeScript only rebuilds packages that have changed. On large codebases this reduces build time from minutes to seconds.

Commands: tsc --build to build all, tsc --build --watch for watch mode.

Q: What does skipLibCheck do? Should you always enable it?

Answer: skipLibCheck: true tells TypeScript to skip type-checking of all .d.ts declaration files, including those from node_modules/@types.

Why enable it: Third-party type definitions sometimes have errors or conflicts between versions. Without skipLibCheck, your build fails due to bugs in OTHER people's code.

Example: @types/react and @types/node may have conflicting global types.

Recommendation: enable it for almost all projects. The community consensus (Vite, CRA, Next.js all enable it by default) is that the benefit — avoiding third-party type noise — outweighs the risk of missing genuine errors in your own .d.ts files.

Rapid Fire — Chapter 16

Question	One-Line Answer
What does strict: true enable?	All 8 strict sub-flags including strictNullChecks, noImplicitAny, etc.
catch (e) type with useUnknownInCatchVariables?	'unknown' — forces you to check with instanceof before using
noEmit: true use case?	Type-check only — Vite/webpack handles compilation, not tsc
What must pair with tsconfig paths?	Bundler alias config (Vite resolve.alias / webpack resolve.alias)
composite: true is required for?	Project references in monorepos — enables incremental builds
forceConsistentCasingInFileNames prevents?	Import casing bugs that work on macOS/Windows but fail on Linux CI
declaration: true generates?	.d.ts type declaration files alongside compiled JS
isolatedModules: true requires?	All type imports to use 'import type' — required for Vite/Babel/SWC
What is extends in tsconfig used for?	Inheriting from a base config — share settings across multiple configs
noUncheckedIndexedAccess makes arr[0] return?	T | undefined — forces null check before using array elements

Chapter 16 Key Takeaways

tsconfig.json = blueprint for the TypeScript compiler.

include/exclude: which files to compile. extends: inherit from base config.

strict: true — always enable. Turns on all 8 strict sub-checks.
noUncheckedIndexedAccess: not in strict — enable separately for extra safety.
noUnusedLocals/Parameters: great for keeping code clean.
noImplicitReturns + noFallthroughCasesInSwitch: prevent silent logic bugs.

target: ES output version — match your minimum runtime (ES2022 for Node 18).
module + moduleResolution: output format and file-finding algorithm.
  NodeNext+NodeNext for modern Node. ESNext+Bundler for Vite apps.

esModuleInterop: true → import React from 'react' (not import * as React).
resolveJsonModule: true → import data from './data.json'.
isolatedModules: true → required for Vite/Babel (forces import type).

Path aliases: baseUrl + paths in tsconfig PLUS bundler alias config.
skipLibCheck: true → skip errors in third-party .d.ts files.
forceConsistentCasingInFileNames: true → prevents macOS/Linux case bugs.

noEmit: true → type-check only, no JS output (standard for Vite projects).
declaration: true → generate .d.ts files (required for library projects).

extends: share a base config, override per-environment (app, test, build).
composite: true → project references for monorepo incremental builds.

End of Chapter 16

CHAPTER 17

Interview Q&A — Rapid Fire

100+ Questions • Beginner to Advanced • Concept Checks • Code Challenges • Real Answers

This final chapter is your interview preparation toolkit. It consolidates the most commonly asked TypeScript interview questions across all difficulty levels — from fresher screening calls to senior architect interviews. Every question comes with a thorough, interview-ready answer that you can study, adapt, and confidently deliver.

How to Use This Chapter

BEGINNER questions (Section 1-3): Asked at 0-1 year experience level.

INTERMEDIATE questions (Section 4-7): Asked at 1-3 year experience level.

ADVANCED questions (Section 8-11): Asked at 3+ year / senior level.

CODE CHALLENGES (Section 12): Fix the bug / predict the output format.

Study tip: For each answer, try to give it in your own words first,

then compare with the answer here. Practice explaining aloud.

Interviewers care as much about HOW you explain as WHAT you say.

SECTION 1 — Foundations (Fresher Level)

Q: 1. What is TypeScript? How is it different from JavaScript?

TypeScript is a statically typed superset of JavaScript developed by Microsoft. Every valid JavaScript program is also a valid TypeScript program — TypeScript adds optional static type annotations on top.

Key differences:

Type system: TypeScript has types; JavaScript is dynamically typed.

Compilation: TypeScript compiles to JavaScript; JS runs directly.

Error timing: TypeScript catches errors at compile time; JS at runtime.

IDE support: TypeScript gives richer autocomplete, refactoring, and navigation.

Interfaces: TypeScript has interfaces, enums, generics; JS does not.

TypeScript adds zero runtime overhead — the types are completely erased when compiled to JavaScript.

Q: 2. What is type inference? Give an example.

Type inference means TypeScript automatically figures out the type of a variable from its initial value — without you needing to write an explicit annotation.

```
const name = 'Rahul';   // TypeScript infers: string
const age  = 28;        // TypeScript infers: number
const arr  = [1, 2, 3]; // TypeScript infers: number[]

function add(a: number, b: number) {
  return a + b;  // return type inferred as: number
}
```

TypeScript uses inference as much as possible. You only need explicit annotations when inference cannot figure out the type (e.g. function parameters, or when the initial value is null/undefined).

Q: 3. What are the basic types in TypeScript?

Primitive types: string, number, boolean, null, undefined, symbol, bigint.

Special types:

```
any      — opt out of type checking (use sparingly)
unknown  — like any but safe — must check type before use
never    — represents values that never exist (unreachable code, infinite loop)
void     — function that returns nothing (or undefined)
```

Object types: object, Array<T> (or T[]), tuple ([string, number]), enum.

```
let name:    string  = 'Rahul';
let age:     number  = 28;
let active:  boolean = true;
let data:    unknown = fetch('/api');  // safe unknown
let nothing: void    = undefined;
```

Q: 4. What is the difference between 'any' and 'unknown'?

Both accept any value assigned to them, but they differ in how you can USE the value:

```
any:     You can do ANYTHING with it — call methods, access properties, pass anywhere.
         TypeScript completely turns off type checking. Dangerous — it spreads.

unknown: You CANNOT use it without first checking its type.
         Safe alternative to any. TypeScript forces you to narrow before use.

let a: any     = fetchData();
a.name.toUpperCase(); // no error — any disables all checks

let u: unknown = fetchData();
u.name;              // ERROR — must check first
if (typeof u === 'string') u.toUpperCase(); // safe
```

Rule: Use unknown when you receive data of uncertain type (API responses, catch blocks). Use any only as a last resort when migrating JavaScript code.

Q: 5. What is the difference between 'interface' and 'type'?

Both define the shape of an object, and in most cases they are interchangeable. Key differences:

Interface:

→ Declaration merging: two interfaces with the same name MERGE automatically.

→ Can be extended with 'extends' keyword (classes can implement them).

→ Cannot represent union types: interface X = A | B is invalid.

Type alias:

→ No declaration merging — redeclaring the same name is an error.

→ Can represent unions, intersections, primitives, tuples, mapped types.

→ More flexible for complex type computations.

Community preference: Use interface for object shapes (especially public APIs). Use type for unions, intersections, aliases for primitives, and complex generic types.

Q: 6. What is 'void' vs 'undefined' vs 'never' as return types?

void: a function does NOT return a meaningful value. The function may return undefined implicitly. Used for event handlers and side-effect functions.

```
function log(msg: string): void { console.log(msg); }
```

undefined: the function explicitly returns undefined. More specific than void.

```
function getNothing(): undefined { return undefined; }
```

never: the function NEVER returns at all — it either throws an error or runs forever.

```
function fail(msg: string): never { throw new Error(msg); }
function loop(): never { while(true) {} }
```

never is also what TypeScript infers for the default case of an exhaustive switch — it means 'this code is unreachable'. Use it to catch unhandled union cases.

Q: 7. What are union types and intersection types?

Union type (|): a value can be ONE OF several types.

```
type ID = string | number;
let id: ID = 'abc';   // string — valid
let id: ID = 42;      // number — valid
```

Intersection type (&): a value must satisfy ALL types simultaneously.

```
type Admin = User & { adminLevel: number };
// Admin has ALL User properties PLUS adminLevel
```

Real-world use:

Union → function parameters that accept multiple types (string | null)

Intersection → combining multiple interfaces (Serializable & Cacheable)

Narrowing: TypeScript uses type guards (typeof, instanceof, in) to narrow unions to specific types inside if/switch blocks.

Q: 8. What is 'readonly' and when should you use it?

readonly is a modifier that prevents a property from being reassigned after initialization.

```
interface Point { readonly x: number; readonly y: number; }
const p: Point = { x: 10, y: 20 };
```

```
p.x = 30; // ERROR: x is readonly
```

Use readonly for: configuration objects, entity IDs, immutable value objects, function parameters you should not mutate.

Important: readonly is SHALLOW — it prevents reassigning the property reference, but does not prevent mutating the object/array it points to. Use ReadonlyArray<T> or 'as const' for deep immutability.

```
const arr: readonly string[] = ['a', 'b'];
arr.push('c');    // ERROR — ReadonlyArray
arr[0] = 'z';     // ERROR — ReadonlyArray
```

SECTION 2 — Type System (Junior Level)

Q: 9. What is a literal type? Give a real-world use case.

A literal type restricts a variable to ONE specific value rather than any value of its type.

```
type Direction = 'north' | 'south' | 'east' | 'west';
type StatusCode = 200 | 201 | 400 | 401 | 404 | 500;
type True = true;
```

Real-world use case: API response status, button variants, theme names.

```
type ButtonVariant = 'primary' | 'secondary' | 'danger' | 'ghost';
function renderButton(variant: ButtonVariant) { ... }
renderButton('primary');   // valid
renderButton('purple');    // ERROR — not a valid variant
```

Combined with as const: const ROLES = ['admin', 'user', 'guest'] as const;

```
type Role = typeof ROLES[number]; // 'admin' | 'user' | 'guest'
```

Q: 10. What are type guards? Name four kinds.

Type guards are expressions that narrow a type to something more specific at runtime. TypeScript uses the result to narrow the type in the subsequent code block.

```
1. typeof: for primitives
   if (typeof x === 'string') { x.toUpperCase(); } // x is string here

2. instanceof: for class instances
   if (error instanceof NetworkError) { error.statusCode; }

3. 'in' operator: for property existence
   if ('adminLevel' in user) { user.adminLevel; } // narrowed to Admin

4. Custom type guard function: 'value is Type' predicate
   function isUser(x: unknown): x is User {
     return typeof x === 'object' && x !== null && 'id' in x && 'name' in x;
   }

5. Equality narrowing: === narrows literals
   if (status === 'success') { result.data; } // data property exists
```

Q: 11. What is a discriminated union? Why is it useful?

A discriminated union is a union of object types that all share a common literal property (the discriminant). TypeScript uses the discriminant to narrow to the exact member type.

```
type Shape =
  | { kind: 'circle';    radius: number }
  | { kind: 'rectangle'; width: number; height: number }
  | { kind: 'triangle';  base: number;  height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle':    return Math.PI * s.radius ** 2;  // s is circle here
    case 'rectangle': return s.width * s.height;       // s is rectangle
    case 'triangle':  return 0.5 * s.base * s.height;  // s is triangle
  }
}
```

Benefits: TypeScript narrows automatically — no casting needed. Adding a new union member causes a compile error in any switch that doesn't handle it (with the never exhaustive check pattern).

Q: 12. What is 'as const' and what does it do?

as const is a type assertion that makes TypeScript infer the MOST SPECIFIC (narrowest) type for an expression, and makes all properties readonly.

```
const config = { env: 'prod', port: 3000 } as const;
// Without as const: { env: string; port: number }
// With as const:    { readonly env: 'prod'; readonly port: 3000 }

const ROLES = ['admin', 'user', 'guest'] as const;
// Without: string[]
// With:    readonly ['admin', 'user', 'guest']
type Role = typeof ROLES[number]; // 'admin' | 'user' | 'guest'
```

Use as const for: configuration objects, lookup arrays, route definitions, anything where you want literal types rather than widened types.

Q: 13. What is the 'keyof' operator?

keyof T produces a union type of all the keys of type T.

```
interface User { id: number; name: string; email: string }
type UserKey = keyof User;  // 'id' | 'name' | 'email'

// Use with generics for type-safe property access:
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user: User = { id: 1, name: 'Rahul', email: 'r@g.com' };
getProperty(user, 'name');   // returns string — typed!
getProperty(user, 'id');     // returns number — typed!
getProperty(user, 'phone');  // ERROR — 'phone' not in User
```

keyof + typeof is a powerful combo: keyof typeof myObject gives the union of keys from a runtime value.

Q: 14. What is the 'typeof' type operator (at the type level)?

At the TYPE level, typeof extracts the TypeScript type of a variable or expression — not its runtime type string.

```
const user = { id: 1, name: 'Rahul', role: 'admin' as const };
type User = typeof user;
// { id: number; name: string; role: 'admin' }

function createUser(name: string, email: string) {
  return { id: Date.now(), name, email, createdAt: new Date() };
}
type User = ReturnType<typeof createUser>;
// { id: number; name: string; email: string; createdAt: Date }
```

typeof at type level is useful for: deriving types from values (config objects, functions), avoiding type duplication, keeping types in sync with implementation automatically.

Q: 15. What is the 'in' operator as a type guard?

The in operator checks if a property exists in an object — TypeScript uses this to narrow union types.

```
type Admin   = { name: string; adminLevel: number };
type RegularUser = { name: string; subscription: string };

function handleUser(user: Admin | RegularUser) {
  if ('adminLevel' in user) {
    // TypeScript narrows: user is Admin
    console.log(user.adminLevel);
  } else {
    // TypeScript narrows: user is RegularUser
    console.log(user.subscription);
  }
}
```

Best used when union members are object types distinguished by which properties they have (rather than by a literal discriminant field).

SECTION 3 — Generics (Junior–Mid Level)

Q: 16. What are generics? Why are they better than 'any'?

Generics are type parameters that let you write reusable code that works with multiple types while still maintaining full type safety.

```
// Without generics — uses any, loses types:
function identity(x: any): any { return x; }
const result = identity('hello'); // result: any — type lost!

// With generics — preserves types:
function identity<T>(x: T): T { return x; }
const result = identity('hello'); // result: string — type preserved!
const num    = identity(42);      // result: number
```

any says 'I don't care about the type'. Generics say 'I'll figure out the type when called and track it through'.

Real-world use: generic utility functions (map, filter, find), generic classes (Stack<T>, Repository<T>), generic API response wrappers (ApiResponse<T>).

Q: 17. What are generic constraints? Give an example.

Constraints limit what types can be used as a generic parameter using 'extends'.

```
// T can be ANYTHING — too flexible, can't call .length:
function getLength<T>(x: T): number { return x.length; } // ERROR

// Constrain T to types that have a length property:
function getLength<T extends { length: number }>(x: T): number {
  return x.length;  // safe — TypeScript knows T has length
}

getLength('hello');    // 5 — string has length
getLength([1, 2, 3]);  // 3 — array has length
getLength(42);         // ERROR — number has no length
```

Common constraints: T extends object, T extends string | number, T extends { id: number }, T extends keyof SomeType.

Q: 18. What are the built-in utility types? Name six and explain.

Utility types are TypeScript's built-in generic type transformers.

Partial<T>: All properties become optional. Use for PATCH/update endpoints.

Required<T>: All optional properties become required. Use for form submission.

Readonly<T>: All properties become readonly. Use for immutable configs.

Pick<T, K>: Keep only specified keys. Use for public API responses.

Omit<T, K>: Remove specified keys. Use for DTOs without auto-generated fields.

Record<K, V>: Key-value map. Use for lookup tables and translation maps.

Exclude<T, U>: Remove union members. Use for filtering union types.

NonNullable<T>: Remove null and undefined. Use after null checks.

ReturnType<T>: Extract function return type. Use to avoid duplicating types.

Parameters<T>: Extract function parameter types as tuple.

Q: 19. How do you make a generic function infer its type parameter?

TypeScript infers the generic type from the arguments you pass — you don't always need to specify it explicitly.

```
function wrap<T>(value: T): { data: T } {
  return { data: value };
}

const r1 = wrap('hello');  // TypeScript infers T = string, returns { data: string }
const r2 = wrap(42);       // Infers T = number, returns { data: number }
const r3 = wrap<User>({}); // Explicit — specify when inference fails
```

Inference works when the type parameter appears in a function parameter. If it only appears in the return type, you must specify it explicitly: useState<User | null>(null).

Q: 20. What is the 'infer' keyword in TypeScript?

infer is used inside conditional types to capture and name an unknown type that TypeScript figures out from a pattern match.

```
// Extract element type from an array:
```

```
type ElementOf<T> = T extends (infer E)[] ? E : never;
type E = ElementOf<string[]>;  // string

// Extract Promise resolved type:
type Unwrap<T> = T extends Promise<infer U> ? U : T;
type U = Unwrap<Promise<User>>;  // User

// Extract function return type (same as built-in ReturnType):
type GetReturn<T> = T extends (...args: any[]) => infer R ? R : never;
```

infer only works in the 'true branch' of a conditional type. Think of it as 'if T matches this pattern, infer and name the unknown part'.

SECTION 4 — Classes & OOP (Mid Level)

Q: 21. What are access modifiers in TypeScript classes?

TypeScript adds three access modifiers to control visibility of class members:

public (default): accessible from anywhere — inside the class, outside, and in subclasses.

private: only accessible inside THIS class. Subclasses cannot access it.

protected: accessible inside this class AND in any subclass. Not accessible from outside.

readonly: not an access modifier but a mutability modifier — property can only be set in the constructor.

Constructor shorthand: TypeScript lets you declare and assign properties in one line:

```
constructor(public name: string, private email: string) {}
// Equivalent to declaring the properties + this.name = name inside the body.
```

Q: 22. What is the difference between abstract class and interface?

Abstract class:

→ Can have BOTH concrete (implemented) and abstract (unimplemented) methods.

→ Can have a constructor, access modifiers, private/protected members.

→ Generates actual JavaScript code (a real class).

→ A class can extend ONLY ONE abstract class.

→ Use for IS-A relationships with shared implementation (Animal → Dog).

Interface:

→ Only declarations — no implementation, no constructor.

→ Completely erased at compile time — zero runtime footprint.

→ A class can implement MULTIPLE interfaces.

→ Use for CAN-DO contracts with no shared code (Serializable, Cacheable).

Rule: If you need shared implementation code, use abstract class. If you only need a type contract, use interface.

Q: 23. What is the 'override' keyword?

The override keyword (TypeScript 4.3+) explicitly marks a method as intentionally overriding a method from the parent class.

```
class Animal { describe(): string { return 'An animal'; } }
class Dog extends Animal {
  override describe(): string { return 'A dog'; } // explicit override
}
```

Without override: if the parent renames describe() to toString(), the Dog's describe() silently becomes a NEW method — no error. Subtle bug.

With override: TypeScript ERRORS if the parent class has no method called describe(). The child class is forced to update.

Enable enforcement with: 'noImplicitOverride: true' in tsconfig — TypeScript errors if you override without the override keyword.

Q: 24. What are static class members?

Static members belong to the CLASS itself, not to instances. You access them via ClassName.member, not via this.

```
class Counter {
  static count = 0;
  constructor() { Counter.count++; }
  static getCount() { return Counter.count; }
}
```

```
new Counter(); new Counter();
Counter.getCount(); // 2 — shared across all instances
```

Use static for: shared state counters, factory methods (Temperature.fromCelsius()), Singleton pattern (private constructor + static getInstance()), utility/helper methods that don't need instance data.

SECTION 5 — Advanced Types (Mid–Senior Level)

Q: 25. What are mapped types? Write one from scratch.

Mapped types create a new type by iterating over the keys of an existing type and transforming each property.

```
// Syntax: { [P in keyof T]: TransformedType }

// Make every property nullable:
type Nullable<T> = { [P in keyof T]: T[P] | null };

// Make every property a getter function:
type Getters<T> = { [P in keyof T]: () => T[P] };

// Make every property optional and string (for form values):
type FormValues<T> = { [P in keyof T]?: string };
```

Modifiers: +? adds optional, -? removes optional. readonly adds readonly, -readonly removes it.

All of TypeScript's built-in utility types (Partial, Readonly, Record) are implemented as mapped types internally.

Q: 26. What are conditional types? Give a practical example.

Conditional types work like if/else at the type level: T extends U ? TrueType : FalseType

```
// Is T an array? If yes, give me the element type. If no, give me T itself.
type Flatten<T> = T extends Array<infer Item> ? Item : T;
type F1 = Flatten<string[]>;  // string
type F2 = Flatten<number>;    // number (not an array, returns T)
```

```
// Different API response shapes per HTTP method:
type ApiResult<T, M extends 'GET' | 'POST' | 'DELETE'> =
  M extends 'DELETE' ? { success: boolean; deletedId: number } :
  M extends 'POST'   ? { success: boolean; created: T } :
  { success: boolean; data: T };  // GET
```

When T is a union, conditional types DISTRIBUTE — they apply to each member separately. Prevent with: [T] extends [U] ? ...

Q: 27. What are branded (nominal) types? Why are they needed?

TypeScript uses structural typing — two types with the same shape are interchangeable. Branded types add an invisible 'brand' to make structurally identical types distinct.

Problem: type UserId = number; type OrderId = number;

getOrder(userId) — TypeScript allows this! Both are number.

Solution:

```
type UserId  = number & { readonly __brand: 'UserId' };
type OrderId = number & { readonly __brand: 'OrderId' };

function getOrder(id: OrderId): void { ... }
const uid = 42 as UserId;
getOrder(uid);  // ERROR: UserId not assignable to OrderId
```

Use for: database IDs, validated emails, hashed passwords, physical units (Metres vs Feet), currency amounts (USD vs EUR).

Q: 28. What is the 'satisfies' operator?

satisfies (TypeScript 4.9+) validates that a value matches a type WITHOUT widening the inferred type to the annotation.

```
type ColorMap = Record<string, string | [number, number, number]>;

// With annotation: TypeScript widens to string | [number,number,number]
const c1: ColorMap = { red: [255, 0, 0] };
c1.red.map(x => x); // ERROR — TS forgot red is specifically a tuple!
```

```
// With satisfies: validates AND preserves inferred type
const c2 = { red: [255, 0, 0] } satisfies ColorMap;
c2.red.map(x => x); // Works! TS still knows red is number[]
c2.notExists;       // ERROR — caught by satisfies check
```

Use satisfies when you want both validation (against a broad type) AND the benefits of specific type inference.

Q: 29. What is a recursive type? Give an example.

A recursive type references itself in its own definition. Essential for modelling nested or tree-like structures.

```
// JSON type — fully typed:
type JsonValue =
  | string | number | boolean | null
  | JsonValue[]
  | { [key: string]: JsonValue };

// Tree node — each node can have children of the same type:
interface TreeNode<T> {
  value:    T;
  children: TreeNode<T>[];
}

// DeepPartial — recursively make everything optional:
type DeepPartial<T> = T extends object
  ? { [P in keyof T]?: DeepPartial<T[P]> }
  : T;
```

Common uses: JSON values, file system trees, category hierarchies, abstract syntax trees, nested menu structures.

Q: 30. What is variance in TypeScript? Explain covariance and contravariance.

Variance describes how subtype relationships are preserved when types are composed.

Covariant (return types): you can use a MORE SPECIFIC type where a broader type is expected.

() => Dog is assignable to () => Animal (Dog is-a Animal, return type is covariant)

Contravariant (parameter types): you need a MORE GENERAL type.

(animal: Animal) => void is assignable to (dog: Dog) => void

(but NOT the reverse — a Dog handler might use Dog-specific properties)

Practical: Readonly arrays are covariant (Dog[] assignable to Animal[]) but mutable arrays are invariant (Dog[] NOT assignable to Animal[] — someone could push a Cat!).

TypeScript uses 'strictFunctionTypes' to enforce proper contravariance for function parameters.

SECTION 6 — Modules & Declaration Files (Mid Level)

Q: 31. What is the difference between a TypeScript module and a script?

A file is a MODULE if it has at least one top-level import or export statement. All declarations are scoped to the module — no global pollution.

A file is a SCRIPT if it has no import/export. All declarations go into GLOBAL scope — they can conflict with other scripts and libraries.

In modern TypeScript: always use modules. To force an empty file to be a module: add 'export {}' at the bottom.

Why it matters: accidentally writing a script file when you mean a module means your variables are globally visible, causing hard-to-debug naming conflicts.

Q: 32. What is 'import type' and why should you use it?

import type imports only TYPE information — it is completely erased from compiled JavaScript output, leaving zero runtime footprint.

```
import type { User } from './User';  // erased at compile time
import { createUser } from './User'; // may include runtime code
```

Use import type when the import is only used as a type annotation — not as a value.

Why it matters:

1. Smaller bundles — type-only imports never appear in JS output.
2. Circular dependencies — type-only imports don't create runtime circular deps.
3. isolatedModules compatibility — required for Vite, Babel, SWC which process each file independently.

With 'isolatedModules: true' in tsconfig, TypeScript errors if you import a type without 'import type'.

Q: 33. What is a .d.ts declaration file? When would you write one?

A .d.ts file contains ONLY type declarations — no runtime JavaScript. It describes the shape of JavaScript code so TypeScript can type-check usages of it.

You write one when:

1. Using a plain JavaScript library with no built-in types and no @types package.
2. Publishing a TypeScript library — tsc generates .d.ts files for consumers.
3. Declaring global variables injected by build tools (webpack DefinePlugin, Vite env).
4. Module augmentation — adding properties to third-party library types.

Example:

```
// types/globals.d.ts
declare const __APP_VERSION__: string;
declare module 'my-lib' { export function helper(): void; }
```

Q: 34. What is module augmentation? Give a practical example.

Module augmentation lets you ADD declarations to an existing module's type — without modifying the library source.

Most common use case: adding req.user to Express after auth middleware.

```
// types/express.d.ts
declare module 'express' {
  interface Request {
    user?: { id: number; email: string; role: string };
  }
}
```

```
// Now in route handlers — TypeScript knows req.user:
app.get('/profile', (req, res) => {
  if (!req.user) return res.status(401).send('Unauthorised');
  res.json(req.user.email); // fully typed
});
```

Other uses: adding gtag/analytics to window, adding custom methods to Prisma client, extending NestJS request object.

SECTION 7 — Decorators (Mid–Senior Level)

Q: 35. What is a decorator? What are the five kinds?

A decorator is a function applied to a class, method, property, or parameter using @ syntax. It runs at class DEFINITION time (not call time) and can observe or modify the target.

Five kinds:

@Class — receives the constructor. Can modify/replace the class.

@Method — receives target, method name, PropertyDescriptor. Can wrap the method.

@Property — receives target and property name. Used for metadata.

@Parameter — receives target, method name, parameter index. Used for DI markers.

@Accessor — receives target, accessor name, PropertyDescriptor. For get/set.

Enable with: 'experimentalDecorators: true' in tsconfig (legacy — needed for NestJS/Angular). TypeScript 5.0+ supports Stage 3 decorators without any flag.

Q: 36. What is Reflect Metadata? How does NestJS use it for DI?

Reflect Metadata is a library (npm: reflect-metadata) providing a standard API to attach and retrieve key-value metadata on classes and class members.

NestJS DI workflow:

1. @Injectable() stores metadata: Reflect.defineMetadata('injectable', true, target).
2. emitDecoratorMetadata: true makes TypeScript emit constructor parameter types automatically as 'design:paramtypes'.
3. NestJS DI container reads 'design:paramtypes' to discover what each service needs.
4. Container creates each dependency recursively and injects them.

This is why: constructor(private userService: UserService) works in NestJS without you manually creating UserService — the framework reads the type metadata and handles it.

Q: 37. What is the execution order of stacked decorators?

When multiple decorators are stacked on the same element:

```
@A   // outer — factory A evaluates FIRST
@B   // inner — factory B evaluates SECOND
method()
```

Factory evaluation: top to bottom (A then B).

Decorator application: bottom to top (B's function runs first, A's wraps around it).

Think of it as: A(B(method)) — B is applied first, then A wraps the result.

Overall class decoration order: parameter decorators → method/property/accessor decorators → class decorator.

Practical: @UseGuards should be below @Log so auth runs before logging.

SECTION 8 — React + TypeScript (Mid Level)

Q: 38. How do you type React component props?

Define an interface for props and use it as the component's parameter type:

```
interface ButtonProps {
  label:    string;
  onClick:  () => void;
  variant?: 'primary' | 'danger';
  disabled?: boolean;
}
function Button({ label, onClick, variant = 'primary' }: ButtonProps) { ... }
```

For children: use 'children: React.ReactNode' or PropsWithChildren<T>.

For extending HTML elements: 'extends React.ButtonHTMLAttributes<HTMLButtonElement>' and spread rest props.

For generic components: function List<T>({ items }: { items: T[] }) { ... }

Q: 39. When do you need to explicitly type useState?

TypeScript infers from initial value — explicit typing needed when:

1. Initial is null: useState<User | null>(null) — otherwise infers useState<null>
2. Initial is empty []: useState<User[]>([]) — otherwise infers never[]
3. Union state: useState<'idle' | 'loading' | 'success'>('idle')
4. Discriminated union: useState<LoadState | SuccessState>({ status: 'idle' })

If you don't type useState<User | null>(null), TypeScript infers the state as null and you can NEVER set a User value — a very common mistake.

Q: 40. What are the React event types? How do you type onChange?

React provides typed synthetic events following: React.{EventName}Event<HTMLElement>

```
onClick  → React.MouseEvent<HTMLButtonElement>
onChange → React.ChangeEvent<HTMLInputElement>
onSubmit → React.FormEvent<HTMLFormElement>
onKeyDown → React.KeyboardEvent<HTMLInputElement>
onFocus  → React.FocusEvent<HTMLInputElement>
onDrop   → React.DragEvent<HTMLDivElement>

function handleChange(e: React.ChangeEvent<HTMLInputElement>): void {
  setValue(e.target.value); // e.target.value is string
}
```

Inline handlers are automatically typed — no annotation needed inside JSX attributes.

Q: 41. How do you type React Context?

Four steps:

1. Define context type: interface ThemeCtx { theme: 'light'|'dark'; toggle: () => void }

```
2. Create with null:    const ThemeContext = createContext<ThemeCtx | null>(null)
3. Provider component:  <ThemeContext.Provider value={value}>
4. Safe custom hook:    function useTheme(): ThemeCtx {
     const ctx = useContext(ThemeContext);
     if (!ctx) throw new Error('useTheme must be inside ThemeProvider');
     return ctx; // returns ThemeCtx (not ThemeCtx | null)
   }
```

The custom hook removes null from the return type — consumers never need null checks. This pattern is called 'assert-on-use' and is the standard TypeScript Context pattern.

Q: 42. How do you type useReducer?

Define the state type and a discriminated union for actions:

```
type Action =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'SET'; payload: number };

function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'INCREMENT':  return state + 1;
    case 'DECREMENT':  return state - 1;
    case 'SET':        return action.payload; // payload typed!
    default: {
      const _: never = action; // exhaustive check
      return state;
    }
  }
}

const [count, dispatch] = useReducer(reducer, 0);
dispatch({ type: 'SET', payload: 42 }); // typed
dispatch({ type: 'INVALID' });          // ERROR
```

SECTION 9 — tsconfig & Tooling (Mid–Senior Level)

Q: 43. What does strict: true enable?

strict: true is an umbrella that enables all eight strict sub-flags:

```
strictNullChecks          → null/undefined not assignable to other types
noImplicitAny             → no silent any from missing annotations
strictFunctionTypes       → contravariant function parameters
strictBindCallApply       → typed bind/call/apply
strictPropertyInitialization → class properties must be initialized
noImplicitThis            → this must be typed in non-class functions
useUnknownInCatchVariables → catch(e) gives unknown, not any
alwaysStrict              → emits 'use strict' in all output files
```

Always enable strict: true in new projects. Catching these issues at compile time is far cheaper than debugging at runtime.

Q: 44. What is the difference between 'module' and 'moduleResolution'?

'module': the OUTPUT FORMAT TypeScript compiles your import/export statements to.

```
CommonJS → require() / module.exports  (old Node)
NodeNext → ES modules with .js extensions  (modern Node)
ESNext   → import/export  (browsers/bundlers)
```

'moduleResolution': the ALGORITHM TypeScript uses to FIND files when resolving imports.

```
Node     → classic node_modules lookup
NodeNext → modern Node with package.json exports field
Bundler  → for Vite/webpack, no extensions required
```

They usually pair: NodeNext+NodeNext for modern Node apps. ESNext+Bundler for Vite React apps.

Q: 45. What is 'noUncheckedIndexedAccess' and why is it not in strict?

noUncheckedIndexedAccess: true makes array index access return T | undefined instead of T.

```
const users: User[] = [];
```

```
const first = users[0]; // without flag: User; with flag: User | undefined
first.name;             // with flag: ERROR — possibly undefined
first?.name;            // correct with optional chaining
```

It's not included in strict: true because it's quite noisy — every array access needs optional chaining or a guard. The TypeScript team considered it too disruptive for the strict bundle.

Recommendation: enable it in new projects — the extra safety is worth it. It catches a very common source of undefined runtime errors.

Q: 46. What is 'composite: true' and when do you need it?

composite: true enables TypeScript's project references for monorepos. It allows one TS project to reference another and enables incremental compilation.

Requirements when composite: true: declaration must be true, files must be deterministic.

Setup: each sub-package gets composite: true in its tsconfig. The root tsconfig.json uses 'references' to list all packages. Then: tsc --build rebuilds only what changed.

Benefit: on large codebases, full rebuild goes from 60 seconds to 3 seconds because TypeScript caches and only recompiles packages whose source changed.

Commands: tsc --build, tsc --build --watch, tsc --build --clean.

SECTION 10 — Tricky Concepts (Senior Level)

Q: 47. What is the 'never' type? Name three places it appears.

never represents values that NEVER exist — code that is unreachable or values that can never be produced.

1. Function that never returns (throws or infinite loop):

```
function fail(msg: string): never { throw new Error(msg); }
```

2. Exhaustive switch check — after handling all union cases:

```
const _exhaustive: never = action; // error if a case is missed
```

3. Union narrowing — when all possibilities are excluded:

```
type StrOrNum = string | number;
if (typeof x === 'string') { ... }
else if (typeof x === 'number') { ... }
else { x; } // x is never here — all cases handled
```

4. Conditional types — members that don't match become never:

```
type NonNullable<T> = T extends null | undefined ? never : T
// null and undefined become never, effectively removing them from the union
```

Q: 48. Explain the difference between structural and nominal typing.

TypeScript uses STRUCTURAL typing (duck typing) — if two types have the same structure/shape, they are considered compatible, regardless of their declared names.

```
interface Point { x: number; y: number; }
interface Coordinate { x: number; y: number; }
const p: Point = { x: 1, y: 2 };
const c: Coordinate = p;  // Works! Same structure.
```

NOMINAL typing (used by Java, C#): types are compatible only if they have the SAME NAME/declaration, regardless of structure.

TypeScript has STRUCTURAL typing but you can simulate nominal typing with BRANDED TYPES:

```
type UserId  = number & { __brand: 'UserId' };
type OrderId = number & { __brand: 'OrderId' };
// Now they are structurally different despite both being numbers
```

Q: 49. What is Control Flow Analysis (CFA) in TypeScript?

Control Flow Analysis is TypeScript's ability to track the TYPE of a variable through code execution paths — automatically narrowing or widening based on conditions, assignments, and control flow.

```
function process(value: string | number | null) {
  if (value === null) return; // after return, value cannot be null
  // here: value is string | number
  if (typeof value === 'string') {
```

```
    value.toUpperCase(); // here: value is string
  } else {
    value.toFixed(2);    // here: value is number
  }
}
```

CFA also tracks through: assignments (value = 'hello' → type becomes string), loops, throw statements, and function calls. TypeScript remembers the type at every point in the code.

Q: 50. What is declaration merging? Give an example.

Declaration merging happens when TypeScript merges multiple declarations with the same name into one combined definition. Interfaces, namespaces, and functions all support merging.

Interface merging — the most common:

```
interface User { id: number; name: string; }
interface User { email: string; }
// Merged result: { id: number; name: string; email: string }
```

Real-world use: module augmentation in .d.ts files uses interface merging to add properties to third-party types.

```
declare module 'express' {
  interface Request { user?: AuthUser; }  // merges with Express's Request
}
```

Function + namespace merging: can add static properties to a function by declaring a namespace with the same name.

Q: 51. What is the difference between type widening and type narrowing?

Type WIDENING: TypeScript infers a BROADER type than the literal value assigned.

```
let x = 'hello';  // TypeScript widens: string (not literal 'hello')
let y = 42;       // TypeScript widens: number (not literal 42)
```

Prevent widening with 'as const': const x = 'hello' as const; // type is 'hello' (literal)

Type NARROWING: TypeScript REFINES a broad type to a specific type based on runtime checks.

```
function process(x: string | number) {
```

```
  if (typeof x === 'string') {
    // x is narrowed from string|number to string
    x.toUpperCase();
  }
}
```

Widening happens at inference time. Narrowing happens during type-checking inside conditionals, instanceof checks, type guards, and discriminated union switches.

Q: 52. Why can't you use a type alias in 'implements' or 'extends'? (Actually you can — explain when.)

Clarification: you CAN use type aliases in implements and extends — with important constraints.

```
class can 'implements' a type alias IF the alias resolves to an object type:
  type Printable = { print(): void };
  class Document implements Printable { print() { ... } } // valid!

class CANNOT 'extends' a type alias (extends requires a class or abstract class):
  type Base = { name: string };
  class Child extends Base { }  // ERROR: can only extend a class

Interface CAN 'extends' other interfaces or type aliases that resolve to object types:
  type HasId = { id: number };
  interface User extends HasId { name: string; } // valid!
```

The distinction: extends for classes means 'inherit implementation'. implements means 'satisfy this contract'. Type aliases can be contracts but not implementations.

SECTION 11 — Quick Concept Checks (All Levels)

Question	Answer
What does '!' (non-null assertion) do?	Tells TypeScript 'I know this isn't null' — removes null/undefined. Use sparingly.
What is '?.' (optional chaining)?	Accesses property safely — returns undefined instead of throwing if null.

What is '??' (nullish coalescing)?	Returns right side ONLY if left is null or undefined (not falsy like 0 or '').
What is the difference between '==' and '===' for types?	=== narrows both null AND undefined together; == null catches both.
What is 'as' in TypeScript?	Type assertion — tells TS to treat value as specific type. Bypasses checking.
What is 'unknown' vs 'any'?	Both accept anything; unknown requires type check before use. any = unsafe.
Can interfaces have methods?	Yes — declare with: methodName(param: type): returnType
What is 'declare' keyword for?	Tells TypeScript something exists without providing implementation.
What is 'namespace' used for?	Grouping related declarations — common in .d.ts files. Prefer modules in apps.
What is 'type narrowing' vs 'type assertion'?	Narrowing: safe, TS proves type from checks. Assertion: you force TS to believe you.
Does TypeScript have runtime overhead?	No — all types erased at compile. Zero runtime cost.
What is 'abstract' in TS?	Cannot be instantiated directly — must be subclassed. Methods can be abstract too.
What is 'override' keyword for?	Explicitly mark method as overriding parent. TS errors if parent method doesn't exist.
What does 'extends' do in generics?	Constrains a type parameter: <T extends object> means T must be an object.
What is 'Exclude' vs 'Omit'?	Exclude: remove from UNION. Omit: remove keys from OBJECT TYPE.
What is 'Extract' vs 'Pick'?	Extract: keep matching UNION members. Pick: keep matching OBJECT keys.
What is 'Awaited<T>'?	Unwraps Promise<T> to T. Awaited<Promise<User>> = User.
What is 'ConstructorParameters<T>'?	Extracts constructor param types as a tuple. Like Parameters<T> but for constructors.
Can enums be string-based?	Yes — enum Direction { North = 'NORTH', South = 'SOUTH' }
What is 'const enum'?	Enum values are inlined at compile time — no object generated at runtime.

SECTION 12 — Code Challenges

Challenge 1: Spot the Error

```
// What is wrong with this code? How do you fix it?

interface Config {
  readonly host: string;
  readonly port: number;
  tags: string[];
}

const config: Readonly<Config> = {
  host: 'localhost',
  port: 3000,
  tags: ['prod', 'v2'],
};

config.host = 'newhost';      // Line A
config.tags = ['qa'];         // Line B
config.tags.push('hotfix');   // Line C  ← is this an error?

// ANSWER:
// Line A: ERROR — host is readonly
// Line B: ERROR — tags reference is readonly
// Line C: ALLOWED! Readonly<T> is shallow — it prevents reassigning
//          the 'tags' reference, but NOT mutating the array contents.
//          config.tags.push() mutates the array in place — TypeScript allows
it.
//
// Fix: use ReadonlyArray<string> for deep immutability:
interface Config {
  readonly host: string;
  readonly port: number;
  readonly tags: ReadonlyArray<string>; // or: readonly string[]
}
// Now config.tags.push('hotfix') → ERROR
```

Challenge 2: What Does This Return?

```
// Predict the type of each result:

type Flatten<T> = T extends (infer E)[] ? E : T;
```

```
type A = Flatten<string[]>;          // ?
type B = Flatten<number[][]>;        // ?
type C = Flatten<boolean>;           // ?
type D = Flatten<(string | number)[]>; // ?

// ANSWERS:
// A = string       (string[] → infers E = string)
// B = number[]     (number[][] → infers E = number[]; one level only)
// C = boolean      (not an array → returns T itself)
// D = string | number  (element type of the mixed array)

// ── Second challenge ──────────────────────────────
type IsNever<T> = [T] extends [never] ? true : false;

type X = IsNever<never>;   // ?
type Y = IsNever<string>;  // ?
type Z = IsNever<never | string>; // ?

// ANSWERS:
// X = true   ([never] extends [never] → true)
// Y = false  ([string] extends [never] → false)
// Z = false  (never | string = string, so [string] extends [never] → false)
```

Challenge 3: Fix the Generic Constraint

```
// This code has a TypeScript error. Find and fix it.

function getFirstItem<T>(arr: T[]): T {
  return arr[0]; // What's wrong here with noUncheckedIndexedAccess?
}

// PROBLEM: With noUncheckedIndexedAccess: true,
// arr[0] returns T | undefined, but the function promises T.

// FIX 1: Update return type to reflect reality
function getFirstItem<T>(arr: T[]): T | undefined {
  return arr[0];
}

// FIX 2: Throw if empty
function getFirstItem<T>(arr: T[]): T {
  if (arr.length === 0) throw new Error('Array is empty');
```

```
  return arr[0]!;  // non-null assertion — safe because we checked length
}

// FIX 3: Require non-empty array (tuple constraint)
function getFirstItem<T>(arr: [T, ...T[]]): T {
  return arr[0];  // TypeScript knows arr has at least one element
}
```

Challenge 4: Complete the Utility Type

```
// Implement a DeepReadonly<T> utility type that makes
// every property at every level readonly.

// START:
type DeepReadonly<T> = ???

// TEST CASES:
interface Config {
  db: { host: string; port: number };
  app: { name: string; flags: string[] };
}

const config: DeepReadonly<Config> = {
  db: { host: 'localhost', port: 5432 },
  app: { name: 'myapp', flags: ['a', 'b'] }
};

config.db.host = 'x';            // should ERROR
config.app.flags.push('c');      // should ERROR

// ANSWER:
type DeepReadonly<T> =
  T extends (infer E)[]
    ? ReadonlyArray<DeepReadonly<E>>
    : T extends object
      ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
      : T;

// Explanation:
// 1. If T is an array → make it ReadonlyArray with deeply readonly elements
// 2. If T is an object → readonly all keys and recurse into values
// 3. If T is a primitive (string/number/boolean) → return as-is
```

Challenge 5: Design the Types

```
// A payment system needs these types. Design them.
// Requirements:
// 1. A payment can be in one of 3 states: pending, succeeded, failed
// 2. succeeded has: paymentId, amount, paidAt (Date)
// 3. failed has: reason string, failedAt (Date), canRetry boolean
// 4. pending has: createdAt (Date)
// 5. A function processResult that accepts any state and handles all three

// ANSWER:
type PaymentState =
  | { status: 'pending';   createdAt: Date }
  | { status: 'succeeded'; paymentId: string; amount: number; paidAt: Date }
  | { status: 'failed';    reason: string; failedAt: Date; canRetry:
boolean };

function processResult(payment: PaymentState): string {
  switch (payment.status) {
    case 'pending':
      return `Pending since ${payment.createdAt.toLocaleDateString()}`;

    case 'succeeded':
      return `Paid ₹${payment.amount} at ${payment.paidAt.toISOString()}`;

    case 'failed':
      const retry = payment.canRetry ? 'Retry allowed' : 'No retry';
      return `Failed: ${payment.reason}. ${retry}.`;

    default: {
      const _: never = payment; // exhaustive check — add new case if needed
      return 'Unknown state';
    }
  }
}
```

SECTION 13 — Final 30 Rapid Fire

#	Question	Answer

53	Type of catch variable with useUnknownInCatchVariables?	unknown — must narrow with instanceof before using
54	Does TypeScript have runtime overhead?	No — all types are erased at compile. Zero runtime cost.
55	What does '!' after a value do?	Non-null assertion — removes null/undefined from type. Risky.
56	Difference between unknown and any?	any: use freely. unknown: must type-check before use.
57	What is 'as const' used for?	Narrow inferred types to literals and make properties readonly.
58	Can you extend multiple interfaces?	Yes — interface A extends B, C, D { }
59	Can a class implement multiple interfaces?	Yes — class User implements Serializable, Cacheable { }
60	What does Readonly<T> NOT protect against?	Mutating nested objects/arrays — it is shallow, not deep.
61	What is the exhaustive never check pattern?	const _: never = x; in default case — errors if a union case is missed.
62	What is a type predicate?	function fn(x: any): x is User { } — custom type guard.
63	What does emitDecoratorMetadata do?	Emits design:paramtypes — TypeScript stores constructor param types for DI.
64	What is isolatedModules: true for?	Forces import type — required for Vite/Babel/SWC single-file transpilers.
65	What does forceConsistentCasingInFileNames prevent?	Case-sensitivity bugs: works on Mac/Windows, breaks on Linux CI.
66	What is declaration: true used for?	Generate .d.ts type files alongside JS — required for npm libraries.
67	What is the difference between Pick and Extract?	Pick removes object KEYS. Extract keeps UNION MEMBERS that match.
68	What is Awaited<T>?	Unwraps Promise<T> → T. Works recursively for nested Promises.
69	What is Parameters<T>?	Extracts function parameter types as a tuple type.
70	What does composite: true enable?	Project references for monorepo incremental compilation.
71	What is the 'in' operator in type guards?	Checks if property exists — TypeScript narrows to the type that has it.
72	What is the satisfies operator?	Validates against a type WITHOUT widening — keeps specific inferred type.

73	What is structural typing?	Two types are compatible if their shapes match — regardless of name.
74	What is declaration merging?	Same-name interfaces automatically merge their members.
75	What is Control Flow Analysis?	TypeScript tracks variable types through conditionals, assignments, loops.
76	What is 'noImplicitReturns' for?	Errors if any code path in a function doesn't return a value.
77	What is 'skipLibCheck: true' for?	Skip type errors in third-party .d.ts files — prevents noisy false positives.
78	When does TypeScript emit JavaScript?	When noEmit is false (default). Vite projects set noEmit: true.
79	What is 'module augmentation'?	Adding declarations to existing modules via 'declare module' in .d.ts files.
80	What is 'esModuleInterop: true' for?	Allows 'import React from react' instead of 'import * as React from react'.
81	What is resolveJsonModule?	Allows TypeScript to import .json files with full type inference.
82	Can 'never' be assigned to any type?	Yes — never is assignable to everything (bottom type). Nothing is assignable TO never.

Final Advice for TypeScript Interviews

CONCEPT questions: explain in simple analogies first, then technical detail.

CODE questions: think aloud — interviewers value your reasoning process.

EDGE cases: mention null handling, generic constraints, edge inputs.

Most asked topics in order of frequency:

1. any vs unknown vs never
2. interface vs type alias
3. Generics — basics and constraints
4. Utility types — Partial, Pick, Omit, Record
5. Type narrowing — typeof, instanceof, discriminated unions
6. strictNullChecks and null safety
7. Mapped and conditional types (for senior roles)
8. React + TypeScript patterns (for frontend roles)
9. Decorators and DI (for backend/NestJS roles)
10. tsconfig options — strict, module, paths

Remember: TypeScript is a tool for better JavaScript.

The best answer always ties types back to preventing real runtime bugs.

End of Chapter 17 — End of Book

TypeScript: The Complete Guide

Chapters 1–17 • 17 Deep-Dive Chapters • 100+ Interview Questions • Real-World Examples